

CS8165.001 Complete Lecture Readers

Readable commentary for each lecture in the course's primary language. This document is designed to make the raw slides human-readable while keeping links back to the extracted source chunks.

Contents

Lecture 01 - Introduction

Lecture 02 - Rendering Pipeline

Lecture 03 - Geometric Transformations

Lecture 04 - Geometric Projection

Lecture 05 - Clipping

Lecture 06 - Rasterization

Lecture 07 - Visibility Determination

Lecture 08 - Local Illumination

Lecture 09 - Texturing

Lecture 10 - Shadows

Lecture 01 - Introduction

This lecture is the orientation layer for the whole course. It explains why interactive computer graphics is a pipeline problem: a scene must be represented, processed under time constraints, sampled into an image, and displayed fast enough for interaction.

Source chunk	course_text_parts/03_lectures/01_introduction.txt
Extracted slide pages	51
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Course Learning Outcomes
Source cue	- Learn to program computer graphics applications using OpenGL / - Learn to realize efficient computer graphics applications on the GPU / - Implement your own computer graphics project / [Image Credits: Wikipedia User Mimigu] [Crassin et al., PG2011]

Professor-style explanation	The slide 'Course Learning Outcomes' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. On the slide, the concrete items are: - Learn to program computer graphics applications using OpenGL / - Learn to realize efficient computer graphics applications on the GPU / - Implement your own computer graphics project / [Image Credits: Wikipedia User Mimigu] [Crassin et al., PG2011]. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Course Learning Outcomes. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. Concrete items shown: - Learn to program computer graphics applications using OpenGL / - Learn to realize efficient computer graphics applications on the GPU / - Implement your own computer graphics project / [Image Credits: Wikipedia User Mimigu] [Crassin et al., PG2011]
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Course Learning Outcomes' to an exercise, project task, or exam-preparation action?

Page 3	Real-Time Rendering Contest Results 1/3
Source cue	Students: Frank Richter, Manuel Herbert Guntzel
Professor-style explanation	The slide 'Real-Time Rendering Contest Results 1/3' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. On the slide, the concrete items are: Students: Frank Richter, Manuel Herbert Guntzel. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Real-Time Rendering Contest Results 1/3. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. Concrete items shown: Students: Frank Richter, Manuel Herbert Guntzel
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.

Check yourself	Can you name the graphics capability or idea represented by 'Real-Time Rendering Contest Results 1/3' and why it mattered historically?
Page 4	Real-Time Rendering Contest Results 2/3
Source cue	Students: Andreas Rottach, Matthias Englert
Professor-style explanation	The slide 'Real-Time Rendering Contest Results 2/3' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. On the slide, the concrete items are: Students: Andreas Rottach, Matthias Englert. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Real-Time Rendering Contest Results 2/3. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. Concrete items shown: Students: Andreas Rottach, Matthias Englert
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Real-Time Rendering Contest Results 2/3' and why it mattered historically?
Page 5	Real-Time Rendering Contest Results 3/3
Source cue	Students: Jan Eric Haßler, Kai Viktor Freissler
Professor-style explanation	The slide 'Real-Time Rendering Contest Results 3/3' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. On the slide, the concrete items are: Students: Jan Eric Haßler, Kai Viktor Freissler. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Real-Time Rendering Contest Results 3/3. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. Concrete items shown: Students: Jan Eric Haßler, Kai Viktor Freissler

Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Real-Time Rendering Contest Results 3/3' and why it mattered historically?
Page 6	1.1 Course Organization
Source cue	Details about the lecture course
Professor-style explanation	The slide '1.1 Course Organization' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. On the slide, the concrete items are: Details about the lecture course. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about 1.1 Course Organization. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. Concrete items shown: Details about the lecture course
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect '1.1 Course Organization' to an exercise, project task, or exam-preparation action?
Page 7	Teachers
Source cue	- Lectures / timo.ropinski@uni-ulm.de / Office hours upon request / - Exercises / Poonam Aditya Sawant
Professor-style explanation	The slide 'Teachers' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. On the slide, the concrete items are: - Lectures / timo.ropinski@uni-ulm.de / Office hours upon request / - Exercises / Poonam Aditya Sawant. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Teachers. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. Concrete items shown: - Lectures / timo.ropinski@uni-ulm.de / Office hours upon request / - Exercises / Poonam Aditya Sawant

Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Teachers' to an exercise, project task, or exam-preparation action?
Page 8	Course Format
Source cue	- Lectures / - Tuesday (14-16) and every other Wednesdays (12-14) / - Room O28/1002 / - Specific dates/modalities in Moodle (password: ICG26SUMMER) / - Lecture slides
Professor-style explanation	The slide 'Course Format' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. On the slide, the concrete items are: - Lectures / - Tuesday (14-16) and every other Wednesdays (12-14) / - Room O28/1002 / - Specific dates/modalities in Moodle (password: ICG26SUMMER) / - Lecture slides. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Course Format. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. Concrete items shown: - Lectures / - Tuesday (14-16) and every other Wednesdays (12-14) / - Room O28/1002 / - Specific dates/modalities in Moodle (password: ICG26SUMMER) / - Lecture slides
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Course Format' to an exercise, project task, or exam-preparation action?
Page 9	Course Format
Source cue	- Exercises / - Every other Wednesday (12-14) / - Room O28/1002 / - Programming in C/C++ (introduction available upon request) / - Enroll in Moodle

Professor-style explanation	The slide 'Course Format' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. On the slide, the concrete items are: - Exercises / - Every other Wednesday (12-14) / - Room O28/1002 / - Programming in C/C++ (introduction available upon request) / - Enroll in Moodle. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Course Format. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. Concrete items shown: - Exercises / - Every other Wednesday (12-14) / - Room O28/1002 / - Programming in C/C++ (introduction available upon request) / - Enroll in Moodle
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Course Format' to an exercise, project task, or exam-preparation action?

Page 10	Exercises
Source cue	- Exercise / - Exercise 1 - Rendering Pipeline / - Exercise 2 - Primitive Types and Shaders / - Exercise 3 - Geometric Transformations / - Exercise 4 - Projection and Clipping
Professor-style explanation	The slide 'Exercises' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. On the slide, the concrete items are: - Exercise / - Exercise 1 - Rendering Pipeline / - Exercise 2 - Primitive Types and Shaders / - Exercise 3 - Geometric Transformations / - Exercise 4 - Projection and Clipping. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Exercises. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. Concrete items shown: - Exercise / - Exercise 1 - Rendering Pipeline / - Exercise 2 - Primitive Types and Shaders / - Exercise 3 - Geometric Transformations / - Exercise 4 - Projection and Clipping
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.

Check yourself	Can you connect 'Exercises' to an exercise, project task, or exam-preparation action?
Page 11	Successful Course Completion
Source cue	- Pass the oral/written exam / - Dates to be announced / - "Notenbonus" possible / - 50% success rate of exercise solutions for n-1 sheets / - Only running solutions are marked
Professor-style explanation	The slide 'Successful Course Completion' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. On the slide, the concrete items are: - Pass the oral/written exam / - Dates to be announced / - "Notenbonus" possible / - 50% success rate of exercise solutions for n-1 sheets / - Only running solutions are marked. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Successful Course Completion. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. Concrete items shown: - Pass the oral/written exam / - Dates to be announced / - "Notenbonus" possible / - 50% success rate of exercise solutions for n-1 sheets / - Only running solutions are marked
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Successful Course Completion' to an exercise, project task, or exam-preparation action?
Page 12	Course Overview
Source cue	Background / Journey along the / Rendering Pipeline / Lighting Effects / Extrahierte Tabellen:
Professor-style explanation	The slide 'Course Overview' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. On the slide, the concrete items are: Background / Journey along the / Rendering Pipeline / Lighting Effects / Extrahierte Tabellen. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.

Technical commentary	This slide is about Course Overview. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. Concrete items shown: Background / Journey along the / Rendering Pipeline / Lighting Effects / Extrahierte Tabellen:
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Course Overview' to an exercise, project task, or exam-preparation action?

Page 13	Literature
Source cue	- P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics / (4th Edition), AK Peters 2016. / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013.
Professor-style explanation	The slide 'Literature' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. On the slide, the concrete items are: - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics / (4th Edition), AK Peters 2016. / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide.
Technical commentary	This slide is about Literature. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics / (4th Edition), AK Peters 2016. / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature' points to for deeper study?

Page 14	Relevant Chapters and Link to Lecture Course
Source cue	- For self study

Professor-style explanation	The slide 'Relevant Chapters and Link to Lecture Course' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. On the slide, the concrete items are: - For self study. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about Relevant Chapters and Link to Lecture Course. The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. Concrete items shown: - For self study
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect 'Relevant Chapters and Link to Lecture Course' to an exercise, project task, or exam-preparation action?

Page 15	(1.1 Course Organization)
Source cue	1.2 Computer Graphics Overview / 1.3 Pixel-based Representations / 1.4 3D Models / 1.5 Algorithmic Paradigms
Professor-style explanation	The slide '(1.1 Course Organization)' describes the practical objects of the course: lectures, exercise sheets, programming tasks, project work, teachers, dates, tools, or submission structure. These objects are part of the learning system around the graphics content. The lecture introduces a concept, the exercise turns it into code or a calculation, and the project combines several such concepts into a working renderer. On the slide, the concrete items are: 1.2 Computer Graphics Overview / 1.3 Pixel-based Representations / 1.4 3D Models / 1.5 Algorithmic Paradigms. The important relation is between topic, practice format, and skill. A rendering concept that appears in an exercise is not just background vocabulary; it becomes something that can be recognized in C/C++ code, OpenGL calls, shader inputs, or debugging situations.
Technical commentary	This slide is about (1.1 Course Organization). The slide connects lecture theory with exercise work, programming practice, and assessment expectations. The named dates, exercises, or course components indicate where the concept will reappear. Concrete items shown: 1.2 Computer Graphics Overview / 1.3 Pixel-based Representations / 1.4 3D Models / 1.5 Algorithmic Paradigms
Why it matters	Course logistics often reveal which topics are practiced, assessed, or expected in code.
Check yourself	Can you connect '(1.1 Course Organization)' to an exercise, project task, or exam-preparation action?

Page 16	1.2 Computer Graphics Overview
Source cue	Relation of the field with respect to other disciplines

Professor-style explanation	The slide '1.2 Computer Graphics Overview' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Relation of the field with respect to other disciplines. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about 1.2 Computer Graphics Overview. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Relation of the field with respect to other disciplines
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '1.2 Computer Graphics Overview' into a causal sentence instead of repeating the slide title?

Page 17	Computer Graphics History
Source cue	- Origins in art / - Perspective, color, etc. / - Computer graphics milestones [Reichardt, 1968] / - 1960 - "Computer Graphics", William Fetter (Boeing) / - 1962 - Spacewars, Steve Russel (MIT) Computer
Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. On the slide, the concrete items are: - Origins in art / - Perspective, color, etc. / - Computer graphics milestones [Reichardt, 1968] / - 1960 - "Computer Graphics", William Fetter (Boeing) / - 1962 - Spacewars, Steve Russel (MIT) Computer. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. Concrete items shown: - Origins in art / - Perspective, color, etc. / - Computer graphics milestones [Reichardt, 1968] / - 1960 - "Computer Graphics", William Fetter (Boeing) / - 1962 - Spacewars, Steve Russel (MIT) Computer
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?

Page 18	Computer Graphics History
---------	---------------------------

Source cue	Sutherland, Utah (1970) Wireframe
Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. On the slide, the concrete items are: Sutherland, Utah (1970) Wireframe. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. Concrete items shown: Sutherland, Utah (1970) Wireframe
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?

Page 19	Computer Graphics History
Source cue	Sutherland, Utah (1970) Turner Whitted, Siggraph (1979)
Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. On the slide, the concrete items are: Sutherland, Utah (1970) Turner Whitted, Siggraph (1979). This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. Concrete items shown: Sutherland, Utah (1970) Turner Whitted, Siggraph (1979)
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?

Page 20	Computer Graphics History
Source cue	The Abyss (1989) Terminator 2 (1991) Jurassic Park (1993)
Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. On the slide, the concrete items are: The Abyss (1989) Terminator 2 (1991) Jurassic Park (1993). This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. Concrete items shown: The Abyss (1989) Terminator 2 (1991) Jurassic Park (1993)
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?
Page 21	Computer Graphics History
Source cue	Ikea (2014)
Professor-style explanation	The slide 'Computer Graphics History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. On the slide, the concrete items are: Ikea (2014). This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Computer Graphics History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. Concrete items shown: Ikea (2014)
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Computer Graphics History' and why it mattered historically?

Page 22	Related Disciplines
Source cue	- Human-Computer Interaction / - Virtual and Augmented Reality / - Visualization / - Image Processing / - Computer Vision
Professor-style explanation	The slide 'Related Disciplines' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Human-Computer Interaction / - Virtual and Augmented Reality / - Visualization / - Image Processing / - Computer Vision. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Related Disciplines. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Human-Computer Interaction / - Virtual and Augmented Reality / - Visualization / - Image Processing / - Computer Vision
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Related Disciplines' into a causal sentence instead of repeating the slide title?
Page 23	Applications
Source cue	- Computer games / - Animation movies / - Special effects in movies / - CAD/CAM [CryEngine 5.7, CryTek 2022] / - Simulation
Professor-style explanation	The slide 'Applications' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Computer games / - Animation movies / - Special effects in movies / - CAD/CAM [CryEngine 5.7, CryTek 2022] / - Simulation. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Applications. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Computer games / - Animation movies / - Special effects in movies / - CAD/CAM [CryEngine 5.7, CryTek 2022] / - Simulation
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Applications' into a causal sentence instead of repeating the slide title?
Page 24	1.3 Pixel-based Representations
Source cue	What is a pixel?

Professor-style explanation	The slide '1.3 Pixel-based Representations' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. On the slide, the concrete items are: What is a pixel?. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.
Technical commentary	This slide is about 1.3 Pixel-based Representations. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. Concrete items shown: What is a pixel?
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in '1.3 Pixel-based Representations'?

Page 25	Raster Images
Source cue	["Bibliothek" by press office Ulm University]
Professor-style explanation	The slide 'Raster Images' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. On the slide, the concrete items are: ["Bibliothek" by press office Ulm University]. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.
Technical commentary	This slide is about Raster Images. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. Concrete items shown: ["Bibliothek" by press office Ulm University]
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'Raster Images'?

Page 26	Pixel
Source cue	- Name derived from Word for Picture Element / - Pixel is a sample with associated sample values (=pixel components) / - Color / - Transparency (=alpha value) / - Depth (i.e., distance from viewer (=z value))

Professor-style explanation	The slide 'Pixel' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Name derived from Word for Picture Element / - Pixel is a sample with associated sample values (=pixel components) / - Color / - Transparency (=alpha value) / - Depth (i.e., distance from viewer (=z value)). The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Pixel. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Name derived from Word for Picture Element / - Pixel is a sample with associated sample values (=pixel components) / - Color / - Transparency (=alpha value) / - Depth (i.e., distance from viewer (=z value))
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Pixel' works per object, per image region, per ray, or per fragment?
Page 27	Color
Source cue	- In real world color is given / by color spectrum Day Light Light Bulb / Fluorescent Tube White LED / - In computer graphics color is represented through color models / - RGB color model (additive)
Professor-style explanation	The slide 'Color' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - In real world color is given / by color spectrum Day Light Light Bulb / Fluorescent Tube White LED / - In computer graphics color is represented through color models / - RGB color model (additive). The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Color. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - In real world color is given / by color spectrum Day Light Light Bulb / Fluorescent Tube White LED / - In computer graphics color is represented through color models / - RGB color model (additive)

Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Color'?
Page 28	LCD Raster Screen
Source cue	- Liquid crystal displays exist since the 90s and enable flat screens / - Liquid crystals affect the polarization of light / - Based on polarization, subpixel visibility is regulated
Professor-style explanation	The slide 'LCD Raster Screen' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Liquid crystal displays exist since the 90s and enable flat screens / - Liquid crystals affect the polarization of light / - Based on polarization, subpixel visibility is regulated. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about LCD Raster Screen. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Liquid crystal displays exist since the 90s and enable flat screens / - Liquid crystals affect the polarization of light / - Based on polarization, subpixel visibility is regulated
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'LCD Raster Screen'?
Page 29	Raster Image Sizes
Source cue	- Assume a raster image with 1024x768 pixels and 24-bit color depth / - How many different colors can be represented? / - $2^{24} = 2^8 \times 2^8 \times 2^8 = 16.777.216$ / - How much memory does the image need? / - $1024 \times 768 \times 24 = 18,874,368$ bits = 2,359,296 bytes = 2.25 MiB

Professor-style explanation	The slide 'Raster Image Sizes' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. On the slide, the concrete items are: - Assume a raster image with 1024x768 pixels and 24-bit color depth / - How many different colors can be represented? / - $2^{24} = 2^8 \times 2^8 \times 2^8 = 16,777,216$ / - How much memory does the image need? / - $1024 \times 768 \times 24 = 18,874,368$ bits = 2,359,296 bytes = 2.25 MiB. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.
Technical commentary	This slide is about Raster Image Sizes. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. Concrete items shown: - Assume a raster image with 1024x768 pixels and 24-bit color depth / - How many different colors can be represented? / - $2^{24} = 2^8 \times 2^8 \times 2^8 = 16,777,216$ / - How much memory does the image need? / - $1024 \times 768 \times 24 = 18,874,368$ bits = 2,359,296 bytes = 2.25 MiB
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'Raster Image Sizes'?

Page 30	1.4 3D Models
Source cue	Modeling with triangles and other geometric primitives
Professor-style explanation	The slide '1.4 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: Modeling with triangles and other geometric primitives. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about 1.4 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: Modeling with triangles and other geometric primitives
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by '1.4 3D Models' before rendering begins?

Page 31	3D Model Types
---------	----------------

Source cue	- Models specify the structures to be displayed / - Three commonly used model types exist / - Implicit surfaces / - Polygonal models / - Volumetric models
Professor-style explanation	The slide '3D Model Types' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Models specify the structures to be displayed / - Three commonly used model types exist / - Implicit surfaces / - Polygonal models / - Volumetric models. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about 3D Model Types. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Models specify the structures to be displayed / - Three commonly used model types exist / - Implicit surfaces / - Polygonal models / - Volumetric models
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by '3D Model Types' before rendering begins?

Page 32	Implicit Surfaces
Source cue	- Set of all points that fulfill $f(x, y, z) = 0$ / - All points with $f(x, y, z) < 0$ define solid bound by $f(x, y, z) = 0$ / - Display possibilities / - Search for null points / - Approximate with polygons
Professor-style explanation	The slide 'Implicit Surfaces' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Set of all points that fulfill $f(x, y, z) = 0$ / - All points with $f(x, y, z) < 0$ define solid bound by $f(x, y, z) = 0$ / - Display possibilities / - Search for null points / - Approximate with polygons. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Implicit Surfaces. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Set of all points that fulfill $f(x, y, z) = 0$ / - All points with $f(x, y, z) < 0$ define solid bound by $f(x, y, z) = 0$ / - Display possibilities / - Search for null points / - Approximate with polygons
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Implicit Surfaces' into a causal sentence instead of repeating the slide title?
Page 33	Quadrics
Source cue	- Surfaces represented by quadric equations / Ellipsoid Single Hyperboloid Double Hyperboloid / $x^2 y^2 z^2$ / $f(x, y, z) = + - 1$ / $\alpha^2 \beta^2 \gamma^2$
Professor-style explanation	The slide 'Quadrics' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Surfaces represented by quadric equations / Ellipsoid Single Hyperboloid Double Hyperboloid / $x^2 y^2 z^2$ / $f(x, y, z) = + - 1$ / $\alpha^2 \beta^2 \gamma^2$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Quadrics. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Surfaces represented by quadric equations / Ellipsoid Single Hyperboloid Double Hyperboloid / $x^2 y^2 z^2$ / $f(x, y, z) = + - 1$ / $\alpha^2 \beta^2 \gamma^2$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Quadrics' into a causal sentence instead of repeating the slide title?
Page 34	Object Representation
Source cue	Point Cloud (3D Scan) Binary Data (MRT) Signed Distance Fields (SDF) Meshes
Professor-style explanation	The slide 'Object Representation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Point Cloud (3D Scan) Binary Data (MRT) Signed Distance Fields (SDF) Meshes. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Object Representation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Point Cloud (3D Scan) Binary Data (MRT) Signed Distance Fields (SDF) Meshes
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Object Representation' into a causal sentence instead of repeating the slide title?

Page 35	Polygonal Models 1/3
Source cue	- Surface representation is approximated by polygons / - Polygons are made up of vertices and edges
Professor-style explanation	The slide 'Polygonal Models 1/3' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Surface representation is approximated by polygons / - Polygons are made up of vertices and edges. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Polygonal Models 1/3. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Surface representation is approximated by polygons / - Polygons are made up of vertices and edges
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 1/3' before rendering begins?
Page 36	Polygonal Models 1/4
Source cue	- Surface representation is approximated by polygons (mostly triangles/quads) / - Polygons are made up of vertices and edges / Valence / Degree Adjacent Edges 1-Ring Neighborhood
Professor-style explanation	The slide 'Polygonal Models 1/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Surface representation is approximated by polygons (mostly triangles/quads) / - Polygons are made up of vertices and edges / Valence / Degree Adjacent Edges 1-Ring Neighborhood. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Polygonal Models 1/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Surface representation is approximated by polygons (mostly triangles/quads) / - Polygons are made up of vertices and edges / Valence / Degree Adjacent Edges 1-Ring Neighborhood
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.

Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 1/4' before rendering begins?
Page 37	Polygonal Models 2/4
Source cue	- Surface representation are often approximated by triangular polygons / - Triangles consist of three vertices and three edges / - Display achieved through rendering by raster conversion of triangles
Professor-style explanation	The slide 'Polygonal Models 2/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Surface representation are often approximated by triangular polygons / - Triangles consist of three vertices and three edges / - Display achieved through rendering by raster conversion of triangles. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Polygonal Models 2/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Surface representation are often approximated by triangular polygons / - Triangles consist of three vertices and three edges / - Display achieved through rendering by raster conversion of triangles
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 2/4' before rendering begins?
Page 38	Polygonal Models 3/4
Source cue	- Various attributes are associated with vertices of a polygonal model / - Colors / - Material properties / - Textures / - ...
Professor-style explanation	The slide 'Polygonal Models 3/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Various attributes are associated with vertices of a polygonal model / - Colors / - Material properties / - Textures / - . This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Polygonal Models 3/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Various attributes are associated with vertices of a polygonal model / - Colors / - Material properties / - Textures / - ...

Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 3/4' before rendering begins?
Page 39	Polygonal Models 4/4
Source cue	- Surface normal required in many cases / - Normal is calculated through cross product: $\mathbf{a} \times \mathbf{b} = a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x$ / - Example use cases
Professor-style explanation	The slide 'Polygonal Models 4/4' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Surface normal required in many cases / - Normal is calculated through cross product: $\mathbf{a} \times \mathbf{b} = a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x$ / - Example use cases. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Polygonal Models 4/4. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Surface normal required in many cases / - Normal is calculated through cross product: $\mathbf{a} \times \mathbf{b} = a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x$ / - Example use cases
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Polygonal Models 4/4' before rendering begins?
Page 40	Volumetric Models
Source cue	- Space filling object representation through voxels (volume elements) / - Display possibilities / - Rendering of extracted surfaces / - Direct rendering through specialized algorithms
Professor-style explanation	The slide 'Volumetric Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Space filling object representation through voxels (volume elements) / - Display possibilities / - Rendering of extracted surfaces / - Direct rendering through specialized algorithms. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.

Technical commentary	This slide is about Volumetric Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Space filling object representation through voxels (volume elements) / - Display possibilities / - Rendering of extracted surfaces / - Direct rendering through specialized algorithms
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Volumetric Models' before rendering begins?
Page 41	Model Generation
Source cue	- Polygonal models are generated through 3D scanning or 3D modeling / (e.g., LIDAR, Blender, Cinema 4D, Maya) / - Volumetric models are generated through imaging or simulation / (e.g., CT, MRI, PET, CFD, seismic surveys)
Professor-style explanation	The slide 'Model Generation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Polygonal models are generated through 3D scanning or 3D modeling / (e.g., LIDAR, Blender, Cinema 4D, Maya) / - Volumetric models are generated through imaging or simulation / (e.g., CT, MRI, PET, CFD, seismic surveys). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Model Generation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Polygonal models are generated through 3D scanning or 3D modeling / (e.g., LIDAR, Blender, Cinema 4D, Maya) / - Volumetric models are generated through imaging or simulation / (e.g., CT, MRI, PET, CFD, seismic surveys)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Model Generation' into a causal sentence instead of repeating the slide title?
Page 42	Coordinate Systems
Source cue	- Three main coordinate system exist / - Model coordinate system - model specified in own coordinate system / - World coordinate system - models arranged in common coordinate system / - Eye/camera coordinate system - model in relation to camera / - To transfer between coordinate systems,

Professor-style explanation	The slide 'Coordinate Systems' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Three main coordinate system exist / - Model coordinate system - model specified in own coordinate system / - World coordinate system - models arranged in common coordinate system / - Eye/camera coordinate system - model in relation to camera / To transfer between coordinate systems, the central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Coordinate Systems. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Three main coordinate system exist / - Model coordinate system - model specified in own coordinate system / - World coordinate system - models arranged in common coordinate system / - Eye/camera coordinate system - model in relation to camera / - To transfer between coordinate systems,
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Coordinate Systems'?
Page 43	Rendering
Source cue	- Rendering is the process to generate raster images from 3D models / - During rendering several steps are necessary / - Coordinate system transformation / - Visibility determination / - Illumination
Professor-style explanation	The slide 'Rendering' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Rendering is the process to generate raster images from 3D models / - During rendering several steps are necessary / - Coordinate system transformation / - Visibility determination / - Illumination. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Rendering. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Rendering is the process to generate raster images from 3D models / - During rendering several steps are necessary / - Coordinate system transformation / - Visibility determination / - Illumination
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Rendering'?

Page 44	1.5 Algorithmic Paradigms
Source cue	Frequently found algorithm strategies
Professor-style explanation	The slide '1.5 Algorithmic Paradigms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Frequently found algorithm strategies. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about 1.5 Algorithmic Paradigms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Frequently found algorithm strategies
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '1.5 Algorithmic Paradigms' into a causal sentence instead of repeating the slide title?

Page 45	Photorealistic Rendering
Source cue	- Goal: simulate reality / [Embree Ray Tracing Website - http://embree.github.io/] [Maisch & Ropinski, Eurographics 2017] [Hendrik vann Jenssen]
Professor-style explanation	The slide 'Photorealistic Rendering' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Goal: simulate reality / [Embree Ray Tracing Website - http://embree.github.io/] [Maisch & Ropinski, Eurographics 2017] [Hendrik vann Jenssen]. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about Photorealistic Rendering. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Goal: simulate reality / [Embree Ray Tracing Website - http://embree.github.io/] [Maisch & Ropinski, Eurographics 2017] [Hendrik vann Jenssen]
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Photorealistic Rendering' works per object, per image region, per ray, or per fragment?

Page 46	Non-Photorealistic Rendering (NPR)
Source cue	- Goal: abstract reality / [Suggestive contour line drawings, DeCarlo 2003]
Professor-style explanation	The slide 'Non-Photorealistic Rendering (NPR)' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Goal: abstract reality / [Suggestive contour line drawings, DeCarlo 2003]. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Non-Photorealistic Rendering (NPR). The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Goal: abstract reality / [Suggestive contour line drawings, DeCarlo 2003]
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Non-Photorealistic Rendering (NPR)'?

Page 47	Image-Based Algorithms
Source cue	- Number of computations direct proportional to number of pixels / for all Pixels do { / for all SceneObjects do { / ... / - Benefits
Professor-style explanation	The slide 'Image-Based Algorithms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Number of computations direct proportional to number of pixels / for all Pixels do { / for all SceneObjects do { / ... / - Benefits. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Image-Based Algorithms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Number of computations direct proportional to number of pixels / for all Pixels do { / for all SceneObjects do { / ... / - Benefits
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Image-Based Algorithms' into a causal sentence instead of repeating the slide title?

Page 48	Object-Based Algorithms
Source cue	- Number of computations direct proportional to number of 3D models / for all SceneObjects do { / for all Pixels do { / ... / - Benefits
Professor-style explanation	The slide 'Object-Based Algorithms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Number of computations direct proportional to number of 3D models / for all SceneObjects do { / for all Pixels do { / ... / - Benefits. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Object-Based Algorithms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Number of computations direct proportional to number of 3D models / for all SceneObjects do { / for all Pixels do { / ... / - Benefits
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Object-Based Algorithms' into a causal sentence instead of repeating the slide title?

Page 49	are assigned
Source cue	- Raster images are formed of pixels to which colors, transparencies and depth / are assigned / polygonal models / - Most frequent building blocks of polygonal models are triangles, / which are formed of vertices and edges

Professor-style explanation	The slide 'are assigned' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Raster images are formed of pixels to which colors, transparencies and depth / are assigned / polygonal models / - Most frequent building blocks of polygonal models are triangles, / which are formed of vertices and edges. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about are assigned. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Raster images are formed of pixels to which colors, transparencies and depth / are assigned / polygonal models / - Most frequent building blocks of polygonal models are triangles, / which are formed of vertices and edges
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'are assigned'?

Page 50	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?

Page 51	Peters 2016. (Chapters 1, 3 & 21)
---------	-----------------------------------

Source cue	- Text Books / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK / Peters 2016. (Chapters 1, 3 & 21) / - Crassin, Cyril, Fabrice Neyret, Miguel Sainz, Simon Green, and Elmar Eisemann. "Interactive / indirect illumination using voxel cone tracing." In Computer Graphics Forum, vol. 30, no. 7, pp.
Professor-style explanation	The slide 'Peters 2016. (Chapters 1, 3 & 21)' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Text Books / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK / Peters 2016. (Chapters 1, 3 & 21) / - Crassin, Cyril, Fabrice Neyret, Miguel Sainz, Simon Green, and Elmar Eisemann. "Interactive / indirect illumination using voxel cone tracing." In Computer Graphics Forum, vol. 30, no. 7, pp. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Peters 2016. (Chapters 1, 3 & 21). The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Text Books / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK / Peters 2016. (Chapters 1, 3 & 21) / - Crassin, Cyril, Fabrice Neyret, Miguel Sainz, Simon Green, and Elmar Eisemann. "Interactive / indirect illumination using voxel cone tracing." In Computer Graphics Forum, vol. 30, no. 7, pp.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Peters 2016. (Chapters 1, 3 & 21)'?

01.1 Course Organization

Source location: Slides around 6-15

Commentary

The organizational slides matter because they explain how the theory and programming parts fit together. The exercises are not separate from the lecture; they are the practical version of the same pipeline ideas. When the course says that slides are generally not self-explanatory, that is a warning: the bullet points are anchors, not a textbook.

Treat every exercise sheet as an applied checkpoint. Rendering pipeline, primitive types, shaders, transformations, projection, clipping, and rasterization are introduced in the lectures and then turned into OpenGL tasks. If a topic appears both in a lecture and an exercise, it is more likely to be exam relevant.

Mental model

Lecture slides give the vocabulary; exercises force you to connect that vocabulary to code and visible output.

Check yourself

Can you list the five exercise themes and connect each one to a later lecture chapter?

01.2 Computer Graphics Overview

Source location: Section 1.2

Commentary

Computer graphics is about generating images from descriptions. The description can be pixel-based, object-based, physically motivated, artistic, or algorithmic. The common thread is that the computer must decide what color belongs at image samples.

Interactive graphics adds a time constraint. The image is not computed once and admired; it must respond to camera motion, user input, animation, and changing state. That is why real-time rendering often uses approximations and specialized GPU stages.

Mental model

Graphics is controlled image generation. Interactive graphics is controlled image generation under a strict time budget.

Check yourself

What changes when a renderer must be interactive rather than offline?

01.3 Pixel-Based Representations

Source location: Section 1.3

Commentary

A pixel-based representation stores an image directly as samples on a grid. It is easy to display and edit locally, but it does not know the underlying 3D scene. If you zoom into a raster image, you reveal the sampling grid, not more geometry.

This is the first place where sampling becomes important. Resolution, color depth, aliasing, and frame time are not cosmetic details. They decide what information can be represented and how much data must be processed per second.

Mental model

A raster image is already the answer. It contains color samples, not the geometric reasons behind them.

Check yourself

Why can a pixel image be easy to display but hard to reinterpret as a 3D scene?

01.4 3D Models

Source location: Section 1.4

Commentary

A 3D model is a structured description of objects before they become pixels. It may contain vertices, primitives, topology, normals, materials, textures, and transformations. Unlike a pixel image, it can be viewed from different camera positions.

The course mostly follows object-based rendering: start from model data, transform it, project it, rasterize it, shade it, test visibility, and finally update pixels. Later chapters explain these steps one by one.

Mental model

A model is not an image. It is a cause from which many possible images can be generated.

Check yourself

Which model attributes can influence final color besides vertex positions?

01.5 Algorithmic Paradigms

Source location: Section 1.5

Commentary

The lecture contrasts ways to generate images. Rasterization works from objects toward pixels and is the dominant real-time pipeline. Ray-based methods work from image samples into the scene and are powerful for visibility and lighting but can be more expensive.

Do not memorize these paradigms as names only. Ask what direction information flows. Does the algorithm start from geometry and find covered pixels, or from pixels/rays and find visible geometry? That direction explains the strengths and weaknesses.

Mental model

Rasterization asks, 'Which samples does this primitive cover?' Ray casting asks, 'What does this sample see?'

Check yourself

Which paradigm fits OpenGL's standard real-time pipeline most directly?

End-of-lecture summary

If you remember only one thing from Lecture 01, remember this: This lecture is the orientation layer for the whole course. It explains why interactive computer graphics is a pipeline problem: a scene must be represented, processed under time constraints, sampled into an image, and displayed fast enough for interaction.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 02 - Rendering Pipeline

This lecture turns the vague idea of rendering into a concrete sequence. It is the most important debugging map in the course: if an image is wrong, the pipeline tells you where the error could have entered.

Source chunk	course_text_parts/03_lectures/02_rendering-pipeline.txt
Extracted slide pages	94
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Object-based Rendering
Source cue	- Image generation done through rasterization / - Scene objects formed by 3D models (primitives) / are transformed through successive coordinate systems / - Primitives are rasterized into fragments, which are pixel predecessors / - A subset of the fragments becomes pixels

Professor-style explanation	The slide 'Object-based Rendering' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: - Image generation done through rasterization / - Scene objects formed by 3D models (primitives) / are transformed through successive coordinate systems / - Primitives are rasterized into fragments, which are pixel predecessors / - A subset of the fragments becomes pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.
Technical commentary	This slide is about Object-based Rendering. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: - Image generation done through rasterization / - Scene objects formed by 3D models (primitives) / are transformed through successive coordinate systems / - Primitives are rasterized into fragments, which are pixel predecessors / - A subset of the fragments becomes pixels
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'Object-based Rendering' in the rendering pipeline?
Page 3	2.1 Rendering Process
Source cue	2.2 OpenGL Overview / 2.3 OpenGL Rendering / 2.4 OpenGL Rendering Pipeline / 2.5 Fragment Tests and Operations / 2.6 Framebuffer
Professor-style explanation	The slide '2.1 Rendering Process' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: 2.2 OpenGL Overview / 2.3 OpenGL Rendering / 2.4 OpenGL Rendering Pipeline / 2.5 Fragment Tests and Operations / 2.6 Framebuffer. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.

Technical commentary	This slide is about 2.1 Rendering Process. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: 2.2 OpenGL Overview / 2.3 OpenGL Rendering / 2.4 OpenGL Rendering Pipeline / 2.5 Fragment Tests and Operations / 2.6 Framebuffer
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for '2.1 Rendering Process' in the rendering pipeline?
Page 4	2.1 Rendering Process
Source cue	From 3D models over fragments to pixels
Professor-style explanation	The slide '2.1 Rendering Process' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: From 3D models over fragments to pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.
Technical commentary	This slide is about 2.1 Rendering Process. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: From 3D models over fragments to pixels
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for '2.1 Rendering Process' in the rendering pipeline?
Page 5	Geometry-based Rendering 1/2
Source cue	CPU Application GPU Rendering Display Hardware / Rendering / 3D Model Framebuffer / Process / Video

Professor-style explanation	The slide 'Geometry-based Rendering 1/2' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: CPU Application GPU Rendering Display Hardware / Rendering / 3D Model Framebuffer / Process / Video. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.
Technical commentary	This slide is about Geometry-based Rendering 1/2. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: CPU Application GPU Rendering Display Hardware / Rendering / 3D Model Framebuffer / Process / Video
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'Geometry-based Rendering 1/2' in the rendering pipeline?

Page 6	Geometry-based Rendering 2/2
Source cue	- The rendering process is realized through the rendering pipeline / - Renders 3D models to 2D images / - Conceptually, the rendering pipeline consists of two GPU stages / - Geometry stage - processes geometric primitives / - Rasterization stage - processes fragments and pixels
Professor-style explanation	The slide 'Geometry-based Rendering 2/2' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: - The rendering process is realized through the rendering pipeline / - Renders 3D models to 2D images / - Conceptually, the rendering pipeline consists of two GPU stages / - Geometry stage - processes geometric primitives / - Rasterization stage - processes fragments and pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.

Technical commentary	This slide is about Geometry-based Rendering 2/2. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: - The rendering process is realized through the rendering pipeline / - Renders 3D models to 2D images / - Conceptually, the rendering pipeline consists of two GPU stages / - Geometry stage - processes geometric primitives / - Rasterization stage - processes fragments and pixels
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'Geometry-based Rendering 2/2' in the rendering pipeline?

Page 7	Application Stage
Source cue	- Specification of the model elements and the scene / as well as interaction capabilities / - Scene modeling: arrangement and attribution of the scene objects / - Calculation of animation paths / - Collision detection
Professor-style explanation	The slide 'Application Stage' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Specification of the model elements and the scene / as well as interaction capabilities / - Scene modeling: arrangement and attribution of the scene objects / - Calculation of animation paths / - Collision detection. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Application Stage. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Specification of the model elements and the scene / as well as interaction capabilities / - Scene modeling: arrangement and attribution of the scene objects / - Calculation of animation paths / - Collision detection
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Application Stage' into a causal sentence instead of repeating the slide title?

Page 8	Geometry Stage
Source cue	x x / - Geometric 3D transformation / Model Coordinate System World Coordinate System / - Alignment of individual scene objects in the overall scene / - Basic transformations: translation, scaling, rotation

Professor-style explanation	The slide 'Geometry Stage' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: $x \times x$ / - Geometric 3D transformation / Model Coordinate System World Coordinate System / - Alignment of individual scene objects in the overall scene / - Basic transformations: translation, scaling, rotation. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Geometry Stage. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $x \times x$ / - Geometric 3D transformation / Model Coordinate System World Coordinate System / - Alignment of individual scene objects in the overall scene / - Basic transformations: translation, scaling, rotation
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Geometry Stage'?
Page 9	Rasterization Stage
Source cue	- Conversion of primitives into fragments (rasterization) / - Discretization takes place in relation to a target grid / - Rasterization algorithms optimized for certain primitives / (e.g., lines, triangles) / - After rasterization per-fragment tests are performed
Professor-style explanation	The slide 'Rasterization Stage' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Conversion of primitives into fragments (rasterization) / - Discretization takes place in relation to a target grid / - Rasterization algorithms optimized for certain primitives / (e.g., lines, triangles) / - After rasterization per-fragment tests are performed. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Rasterization Stage. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Conversion of primitives into fragments (rasterization) / - Discretization takes place in relation to a target grid / - Rasterization algorithms optimized for certain primitives / (e.g., lines, triangles) / - After rasterization per-fragment tests are performed
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization Stage'?

Page 10	2.2 OpenGL Overview
Source cue	Library for real-time computer graphics
Professor-style explanation	The slide '2.2 OpenGL Overview' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Library for real-time computer graphics. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 2.2 OpenGL Overview. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Library for real-time computer graphics
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '2.2 OpenGL Overview'?

Page 11	OpenGL History
Source cue	- Unified graphics software interface was missing / as knowledge channeled into proprietary software packages / - HP's starbase / - SGI's Graphics Library (GL) / - Some standardization efforts existed

Professor-style explanation	The slide 'OpenGL History' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. On the slide, the concrete items are: - Unified graphics software interface was missing / as knowledge channeled into proprietary software packages / - HP's starbase / - SGI's Graphics Library (GL) / - Some standardization efforts existed. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about OpenGL History. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. Concrete items shown: - Unified graphics software interface was missing / as knowledge channeled into proprietary software packages / - HP's starbase / - SGI's Graphics Library (GL) / - Some standardization efforts existed
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'OpenGL History' and why it mattered historically?
Page 12	OpenGL
Source cue	- OpenGL (Open Graphics Library) / - Platform independent 2D and 3D graphics API / - API specification is provided by the Khronos Group / - Technical implementation / - Rendering pipeline consisting of static and programmable pipeline stages
Professor-style explanation	The slide 'OpenGL' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: - OpenGL (Open Graphics Library) / - Platform independent 2D and 3D graphics API / - API specification is provided by the Khronos Group / - Technical implementation / - Rendering pipeline consisting of static and programmable pipeline stages. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.

Technical commentary	This slide is about OpenGL. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: - OpenGL (Open Graphics Library) / - Platform independent 2D and 3D graphics API / - API specification is provided by the Khronos Group / - Technical implementation / - Rendering pipeline consisting of static and programmable pipeline stages
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'OpenGL' in the rendering pipeline?

Page 13	OpenGL vs. Vulkan
Source cue	- How about Vulkan? / - More modern than OpenGL / - Designed for high performance and low-level GPU control / - Better suited for complex, large-scale rendering engines / - Vulkan also harder to learn and requires more setup/code
Professor-style explanation	The slide 'OpenGL vs. Vulkan' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - How about Vulkan? / - More modern than OpenGL / - Designed for high performance and low-level GPU control / - Better suited for complex, large-scale rendering engines / - Vulkan also harder to learn and requires more setup/code. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL vs. Vulkan. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - How about Vulkan? / - More modern than OpenGL / - Designed for high performance and low-level GPU control / - Better suited for complex, large-scale rendering engines / - Vulkan also harder to learn and requires more setup/code
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL vs. Vulkan'?

Page 14	OpenGL Libraries
Source cue	- Different tastes of OpenGL / - OpenGL / - OpenGL ES - limited version for embedded systems / - WebGL - Javascript connection for Web Development / (functionality similar to OpenGL ES)

Professor-style explanation	The slide 'OpenGL Libraries' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Different tastes of OpenGL / - OpenGL / - OpenGL ES - limited version for embedded systems / - WebGL - Javascript connection for Web Development / (functionality similar to OpenGL ES). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Libraries. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Different tastes of OpenGL / - OpenGL / - OpenGL ES - limited version for embedded systems / - WebGL - Javascript connection for Web Development / (functionality similar to OpenGL ES)
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Libraries'?

Page 15	OpenGL Functionality
Source cue	- Functional API / - Objects, scenes and images are described by a sequence of commands / (functional approach, low-level) / - Contrast: scene or object description (declarative approach, high-level) / - OpenGL context
Professor-style explanation	The slide 'OpenGL Functionality' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Functional API / - Objects, scenes and images are described by a sequence of commands / (functional approach, low-level) / - Contrast: scene or object description (declarative approach, high-level) / - OpenGL context. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about OpenGL Functionality. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Functional API / - Objects, scenes and images are described by a sequence of commands / (functional approach, low-level) / - Contrast: scene or object description (declarative approach, high-level) / - OpenGL context
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Functionality'?

Page 16	OpenGL Context
Source cue	- Context is created and modified by the application, / and contains all the state variables of the OpenGL engine / - Any number of contexts per application possible, / but at most one active context per application / - Context is generally tied to an output window
Professor-style explanation	The slide 'OpenGL Context' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Context is created and modified by the application, / and contains all the state variables of the OpenGL engine / - Any number of contexts per application possible, / but at most one active context per application / - Context is generally tied to an output window. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Context. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Context is created and modified by the application, / and contains all the state variables of the OpenGL engine / - Any number of contexts per application possible, / but at most one active context per application / - Context is generally tied to an output window
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Context'?

Page 17	OpenGL Rendering Process
Source cue	- Initiation from the developer's perspective / - Creation of the output window / - Creation of the context / - Specification of the scene geometry / - Uploading of the scene geometry to the GPU

Professor-style explanation	The slide 'OpenGL Rendering Process' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: - Initiation from the developer's perspective / - Creation of the output window / - Creation of the context / - Specification of the scene geometry / - Uploading of the scene geometry to the GPU. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.
Technical commentary	This slide is about OpenGL Rendering Process. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: - Initiation from the developer's perspective / - Creation of the output window / - Creation of the context / - Specification of the scene geometry / - Uploading of the scene geometry to the GPU
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'OpenGL Rendering Process' in the rendering pipeline?

Page 18	Window Creation with GLFW 1/2
Source cue	<pre>#include <GLFW/glfw3.h> / #include <stdlib.h> / #include <stdio.h> / static void error_callback(int error, const char* description) { / fputs(description, stderr);</pre>
Professor-style explanation	The slide 'Window Creation with GLFW 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: <code>#include <GLFW/glfw3.h> / #include <stdlib.h> / #include <stdio.h> / static void error_callback(int error, const char* description) { / fputs(description, stderr)</code> . The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Window Creation with GLFW 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: <code>#include <GLFW/glfw3.h> / #include <stdlib.h> / #include <stdio.h> / static void error_callback(int error, const char* description) { / fputs(description, stderr);</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Window Creation with GLFW 1/2' into a causal sentence instead of repeating the slide title?
Page 19	Window Creation with GLFW 2/2
Source cue	<pre>int main(void){ / GLFWwindow* window; / glfwSetErrorCallback(error_callback); / if (!glfwInit()) exit(EXIT_FAILURE); / window = glfwCreateWindow(640, 480, "Simple example", NULL, NULL);</pre>
Professor-style explanation	The slide 'Window Creation with GLFW 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: <pre>int main(void){ / GLFWwindow* window; / glfwSetErrorCallback(error_callback); / if (!glfwInit()) exit(EXIT_FAILURE); / window = glfwCreateWindow(640, 480, "Simple example", NULL, NULL).</pre> The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Window Creation with GLFW 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: <pre>int main(void){ / GLFWwindow* window; / glfwSetErrorCallback(error_callback); / if (!glfwInit()) exit(EXIT_FAILURE); / window = glfwCreateWindow(640, 480, "Simple example", NULL, NULL);</pre>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Window Creation with GLFW 2/2' into a causal sentence instead of repeating the slide title?
Page 20	OpenGL Naming Conventions
Source cue	- Function-Prefix "gl" / - glClear, ... / - Constant-Prefix "GL" / - GL_POLYGON, GL_LINES, ... / - Parameter types are explicitly encoded in function names
Professor-style explanation	The slide 'OpenGL Naming Conventions' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Function-Prefix "gl" / - glClear, ... / - Constant-Prefix "GL" / - GL_POLYGON, GL_LINES, ... / - Parameter types are explicitly encoded in function names. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about OpenGL Naming Conventions. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Function-Prefix "gl" / - glClear, ... / - Constant-Prefix "GL" / - GL_POLYGON, GL_LINES, ... / - Parameter types are explicitly encoded in function names
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Naming Conventions'?

Page 21	2.3 OpenGL Rendering
Source cue	Transfer of 3D models to the GPU
Professor-style explanation	The slide '2.3 OpenGL Rendering' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Transfer of 3D models to the GPU. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 2.3 OpenGL Rendering. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Transfer of 3D models to the GPU
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '2.3 OpenGL Rendering'?

Page 22	OpenGL Rendering (until V2.1)
Source cue	- Rendering is done through Immediate Mode / void renderScene() { / glColor3f(0.0f, 1.0f, 0.0f); // drawing color becomes green / glBegin(GL_POLYGON); // begin primitive / glVertex2f(-0.5f,-0.5f); // 1. vertex

Professor-style explanation	The slide 'OpenGL Rendering (until V2.1)' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Rendering is done through Immediate Mode / <code>void renderScene() { / glColor3f(0.0f, 1.0f, 0.0f); // drawing color becomes green / glBegin(GL_POLYGON); // begin primitive / glVertex2f(-0.5f,-0.5f); // 1. vertex</code> . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Rendering (until V2.1). The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Rendering is done through Immediate Mode / <code>void renderScene() { / glColor3f(0.0f, 1.0f, 0.0f); // drawing color becomes green / glBegin(GL_POLYGON); // begin primitive / glVertex2f(-0.5f,-0.5f); // 1. vertex</code>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Rendering (until V2.1)'?
Page 23	OpenGL Rendering (since V3.0)
Source cue	- Using Vertex Array Objects (VAOs) / - Vertex specification in CPU memory / - Vertex data transfer from CPU to GPU / - Memory layout description of the vertex data / - Number of vertices, number of components (x, y, z, ...)
Professor-style explanation	The slide 'OpenGL Rendering (since V3.0)' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Using Vertex Array Objects (VAOs) / - Vertex specification in CPU memory / - Vertex data transfer from CPU to GPU / - Memory layout description of the vertex data / - Number of vertices, number of components (x, y, z, ...). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about OpenGL Rendering (since V3.0). The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Using Vertex Array Objects (VAOs) / - Vertex specification in CPU memory / - Vertex data transfer from CPU to GPU / - Memory layout description of the vertex data / - Number of vertices, number of components (x, y, z, ...)
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Rendering (since V3.0)'?

Page 24	1. Vertex Specification
Source cue	- Vertex data is specified as a C array // vertices of two triangles / GLfloat vertices[6][2] = { // 6 vertices with two components each (x,y) / {0.0, 1.0}, {1.0, 1.0}, {1.0, 0.0}, // 1. triangle / {0.5, 0.5}, {1.0, 0.3}, {0.7, 1.0} // 2. triangle
Professor-style explanation	The slide '1. Vertex Specification' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Vertex data is specified as a C array // vertices of two triangles / GLfloat vertices[6][2] = { // 6 vertices with two components each (x,y) / {0.0, 1.0}, {1.0, 1.0}, {1.0, 0.0}, // 1. triangle / {0.5, 0.5}, {1.0, 0.3}, {0.7, 1.0} // 2. triangle. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 1. Vertex Specification. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Vertex data is specified as a C array // vertices of two triangles / GLfloat vertices[6][2] = { // 6 vertices with two components each (x,y) / {0.0, 1.0}, {1.0, 1.0}, {1.0, 0.0}, // 1. triangle / {0.5, 0.5}, {1.0, 0.3}, {0.7, 1.0} // 2. triangle
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '1. Vertex Specification'?

Page 25	2. Vertex Data Transfer
Source cue	- VBO allocation and data transfer via / - glBufferData(GL_ARRAY_BUFFER, size, data, GL_STATIC_DRAW); / - data - address in main memory (i.e., vertices or &vertices[0]) / - size - size of data in number of bytes / (i.e. sizeof(vertices) or numVertices*sizeof(GLfloat)*2)

Professor-style explanation	The slide '2. Vertex Data Transfer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - VBO allocation and data transfer via / - <code>glBufferData(GL_ARRAY_BUFFER, size, data, GL_STATIC_DRAW);</code> / - data - address in main memory (i.e., vertices or <code>&vertices[0]</code>) / - size - size of data in number of bytes / (i.e. <code>sizeof(vertices)</code> or <code>numVertices*sizeof(GLfloat)*2</code>). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 2. Vertex Data Transfer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - VBO allocation and data transfer via / - <code>glBufferData(GL_ARRAY_BUFFER, size, data, GL_STATIC_DRAW);</code> / - data - address in main memory (i.e., vertices or <code>&vertices[0]</code>) / - size - size of data in number of bytes / (i.e. <code>sizeof(vertices)</code> or <code>numVertices*sizeof(GLfloat)*2</code>)
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '2. Vertex Data Transfer'?

Page 26	3. Memory Layout Description
Source cue	- Memory layout is described by the following function / - <code>glVertexAttribPointer(attribIndex, size, type, normalized, stride, offset);</code> / - attribIndex - attribute number (0 for position, later more ...) / - size - number of components per vertex (2 for xy, 3 for xyz) / - type - base type (typically <code>GL_FLOAT</code> or <code>GL_DOUBLE</code>)
Professor-style explanation	The slide '3. Memory Layout Description' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. On the slide, the concrete items are: - Memory layout is described by the following function / - <code>glVertexAttribPointer(attribIndex, size, type, normalized, stride, offset);</code> / - attribIndex - attribute number (0 for position, later more ...) / - size - number of components per vertex (2 for xy, 3 for xyz) / - type - base type (typically <code>GL_FLOAT</code> or <code>GL_DOUBLE</code>). The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.

Technical commentary	This slide is about 3. Memory Layout Description. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. Concrete items shown: - Memory layout is described by the following function / - <code>glVertexAttribPointer(attribIndex, size, type, normalized, stride, offset);</code> / - <code>attribIndex</code> - attribute number (0 for position, later more ...) / - <code>size</code> - number of components per vertex (2 for xy, 3 for xyz) / - <code>type</code> - base type (typically <code>GL_FLOAT</code> or <code>GL_DOUBLE</code>)
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in '3. Memory Layout Description'?

Page 27	4. VAO Drawing 1/2
Source cue	- Drawing is done with the command / - <code>glDrawArrays(mode, first, count);</code> / - <code>mode</code> - specifies the primitive type (e.g., <code>GL_TRIANGLES</code> or <code>GL_LINES</code>) / - From all vertices, the next <code>count</code> vertices are drawn from index <code>first</code> / - Actual vertex data no longer needs to be specified
Professor-style explanation	The slide '4. VAO Drawing 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Drawing is done with the command / - <code>glDrawArrays(mode, first, count);</code> / - <code>mode</code> - specifies the primitive type (e.g., <code>GL_TRIANGLES</code> or <code>GL_LINES</code>) / - From all vertices, the next <code>count</code> vertices are drawn from index <code>first</code> / - Actual vertex data no longer needs to be specified. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 4. VAO Drawing 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Drawing is done with the command / - <code>glDrawArrays(mode, first, count);</code> / - <code>mode</code> - specifies the primitive type (e.g., <code>GL_TRIANGLES</code> or <code>GL_LINES</code>) / - From all vertices, the next <code>count</code> vertices are drawn from index <code>first</code> / - Actual vertex data no longer needs to be specified
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '4. VAO Drawing 1/2'?

Page 28	4. VAO Drawing 2/2
Source cue	- There is a small number of primitive types / - Points / - Lines, Line Strips, Line Loops / - Triangles, Triangle Strips, Triangle Fans, Patches / v2 v2 v2

Professor-style explanation	The slide '4. VAO Drawing 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - There is a small number of primitive types / - Points / - Lines, Line Strips, Line Loops / - Triangles, Triangle Strips, Triangle Fans, Patches / v2 v2 v2. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 4. VAO Drawing 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - There is a small number of primitive types / - Points / - Lines, Line Strips, Line Loops / - Triangles, Triangle Strips, Triangle Fans, Patches / v2 v2 v2
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '4. VAO Drawing 2/2'?

Page 29	5. GPU Memory Release
Source cue	- After usage VAO and VBO should be deleted from GPU memory / - <code>glDeleteBuffers(...);</code> / - <code>glDeleteVertexArrays(...);</code>
Professor-style explanation	The slide '5. GPU Memory Release' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. On the slide, the concrete items are: - After usage VAO and VBO should be deleted from GPU memory / - <code>glDeleteBuffers(...);</code> / - <code>glDeleteVertexArrays(...)</code> . The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.
Technical commentary	This slide is about 5. GPU Memory Release. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. Concrete items shown: - After usage VAO and VBO should be deleted from GPU memory / - <code>glDeleteBuffers(...);</code> / - <code>glDeleteVertexArrays(...);</code>
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in '5. GPU Memory Release'?

Page 30	Rendering Initialization 1/2
---------	------------------------------

Source cue	<pre>static void init_render_scene(void) { / static const GLfloat vertices[6][2] = { / {0.0f, 1.0f}, {1.0f, 1.0f}, {1.0f, 0.0f}, /* 1st triangle */ / {0.5f, 0.5f}, {1.0f, 0.3f}, {0.7f, 1.0f} /* 2nd triangle */ / static const char* vs_src =</pre>
Professor-style explanation	The slide 'Rendering Initialization 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: <code>static void init_render_scene(void) { / static const GLfloat vertices[6][2] = { / {0.0f, 1.0f}, {1.0f, 1.0f}, {1.0f, 0.0f}, /* 1st triangle */ / {0.5f, 0.5f}, {1.0f, 0.3f}, {0.7f, 1.0f} /* 2nd triangle */ / static const char* vs_src =</code>. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Rendering Initialization 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: <code>static void init_render_scene(void) { / static const GLfloat vertices[6][2] = { / {0.0f, 1.0f}, {1.0f, 1.0f}, {1.0f, 0.0f}, /* 1st triangle */ / {0.5f, 0.5f}, {1.0f, 0.3f}, {0.7f, 1.0f} /* 2nd triangle */ / static const char* vs_src =</code>
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Rendering Initialization 1/2'?
Page 31	Rendering Initialization 2/2
Source cue	<pre>program = create_program(vs_src, fs_src); / glGenVertexArrays(1, &vao); / glGenBuffers(1, &vbo); / glBindVertexArray(vao); / glBindBuffer(GL_ARRAY_BUFFER, vbo);</pre>
Professor-style explanation	The slide 'Rendering Initialization 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: <code>program = create_program(vs_src, fs_src); / glGenVertexArrays(1, &vao); / glGenBuffers(1, &vbo); / glBindVertexArray(vao); / glBindBuffer(GL_ARRAY_BUFFER, vbo)</code>. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Rendering Initialization 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: <code>program = create_program(vs_src, fs_src); / glGenVertexArrays(1, &vao); / glGenBuffers(1, &vbo); / glBindVertexArray(vao); / glBindBuffer(GL_ARRAY_BUFFER, vbo);</code>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Rendering Initialization 2/2'?

Page 32	Rendering
Source cue	<code>static void render_scene(void) { / glClear(GL_COLOR_BUFFER_BIT); / glUseProgram(program); / glBindVertexArray(vao); / glDrawArrays(GL_TRIANGLES, 0, 6);</code>
Professor-style explanation	The slide 'Rendering' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: <code>static void render_scene(void) { / glClear(GL_COLOR_BUFFER_BIT); / glUseProgram(program); / glBindVertexArray(vao); / glDrawArrays(GL_TRIANGLES, 0, 6).</code> The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Rendering. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: <code>static void render_scene(void) { / glClear(GL_COLOR_BUFFER_BIT); / glUseProgram(program); / glBindVertexArray(vao); / glDrawArrays(GL_TRIANGLES, 0, 6);</code>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Rendering'?

Page 33	Additional Attributes
Source cue	- Additional attributes are necessary for drawing / - Color, normals, texture coordinates, ... / - There are two ways to assign these attributes to the vertex data / - Attributes are bound to the same VAO in separate VBO / - Attributes are stored interleaved with vertex data,

Professor-style explanation	The slide 'Additional Attributes' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Additional attributes are necessary for drawing / - Color, normals, texture coordinates, ... / - There are two ways to assign these attributes to the vertex data / - Attributes are bound to the same VAO in separate VBO / - Attributes are stored interleaved with vertex data,. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Additional Attributes. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Additional attributes are necessary for drawing / - Color, normals, texture coordinates, ... / - There are two ways to assign these attributes to the vertex data / - Attributes are bound to the same VAO in separate VBO / - Attributes are stored interleaved with vertex data,
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Additional Attributes'?

Page 34	2.4 OpenGL Rendering Pipeline
Source cue	Programmable and static stages
Professor-style explanation	The slide '2.4 OpenGL Rendering Pipeline' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: Programmable and static stages. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.
Technical commentary	This slide is about 2.4 OpenGL Rendering Pipeline. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: Programmable and static stages
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.

Check yourself	Can you name the input and output representation for '2.4 OpenGL Rendering Pipeline' in the rendering pipeline?
Page 35	The Rendering Pipeline 1/2
Source cue	Scene Description Raster Image / Geometry Fragment / Rasterization / Processing Processing / Vertices Primitives Fragments Pixels
Professor-style explanation	The slide 'The Rendering Pipeline 1/2' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: Scene Description Raster Image / Geometry Fragment / Rasterization / Processing Processing / Vertices Primitives Fragments Pixels. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.
Technical commentary	This slide is about The Rendering Pipeline 1/2. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: Scene Description Raster Image / Geometry Fragment / Rasterization / Processing Processing / Vertices Primitives Fragments Pixels
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'The Rendering Pipeline 1/2' in the rendering pipeline?
Page 36	The Rendering Pipeline 2/2
Source cue	- Sequence of fixed and programmable stages Programmable / Vertex Processing / - Process is influenced by context and scene objects / Context Fixed Function / - Programmable levels are programmed
Professor-style explanation	The slide 'The Rendering Pipeline 2/2' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: - Sequence of fixed and programmable stages Programmable / Vertex Processing / - Process is influenced by context and scene objects / Context Fixed Function / - Programmable levels are programmed. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.

Technical commentary	This slide is about The Rendering Pipeline 2/2. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: - Sequence of fixed and programmable stages Programmable / Vertex Processing / - Process is influenced by context and scene objects / Context Fixed Function / - Programmable levels are programmed
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'The Rendering Pipeline 2/2' in the rendering pipeline?
Page 37	Shader Programming - History 1/2
Source cue	- There is special graphics hardware (dedicated) / and general graphics hardware (general purpose) / - Special graphics hardware / - Volume Rendering: VolumePro / - Game consoles: Custom NVIDIA chips in XBox
Professor-style explanation	The slide 'Shader Programming - History 1/2' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. On the slide, the concrete items are: - There is special graphics hardware (dedicated) / and general graphics hardware (general purpose) / - Special graphics hardware / - Volume Rendering: VolumePro / - Game consoles: Custom NVIDIA chips in XBox. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Shader Programming - History 1/2. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. Concrete items shown: - There is special graphics hardware (dedicated) / and general graphics hardware (general purpose) / - Special graphics hardware / - Volume Rendering: VolumePro / - Game consoles: Custom NVIDIA chips in XBox
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Shader Programming - History 1/2' and why it mattered historically?
Page 38	Shader Programming - History 2/2
Source cue	- Various vendor-dependent extensions are introduced / to provide extensibility for general graphics hardware / - Texture shaders and register combiners allowed modification / through the application of textures / - Applicability is very limited by the possibilities of graphics hardware

Professor-style explanation	The slide 'Shader Programming - History 2/2' explains the historical background behind the graphics concept. Computer graphics did not appear as one finished pipeline; it developed from older ideas such as artistic perspective, color representation, display hardware, interactive systems, and increasingly programmable rendering algorithms. The historical objects on the slide are people, systems, dates, or milestones, and each milestone marks a capability that later became normal in graphics software. On the slide, the concrete items are: - Various vendor-dependent extensions are introduced / to provide extensibility for general graphics hardware / - Texture shaders and register combiners allowed modification / through the application of textures / - Applicability is very limited by the possibilities of graphics hardware. This history matters because it shows why the course combines mathematics, image representation, hardware acceleration, and interaction. Modern real-time rendering is the result of these threads converging into a pipeline that can generate images fast enough for user input.
Technical commentary	This slide is about Shader Programming - History 2/2. The slide places the technical topic into the historical development of computer graphics, showing how artistic perspective, display technology, interaction, and rendering algorithms evolved together. Concrete items shown: - Various vendor-dependent extensions are introduced / to provide extensibility for general graphics hardware / - Texture shaders and register combiners allowed modification / through the application of textures / - Applicability is very limited by the possibilities of graphics hardware
Why it matters	Historical slides explain why the current pipeline exists and which older problems led to modern graphics concepts.
Check yourself	Can you name the graphics capability or idea represented by 'Shader Programming - History 2/2' and why it mattered historically?
Page 39	Assembler Shader
Source cue	- First OpenGL extensions support shader programming in assembler code / !! ARBvp1.0 / ATTRIB iPos = vertex.position ; / ATTRIB iNormal = vertex.normal ; / ...
Professor-style explanation	The slide 'Assembler Shader' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - First OpenGL extensions support shader programming in assembler code / !! ARBvp1.0 / ATTRIB iPos = vertex.position ; / ATTRIB iNormal = vertex.normal ; / . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Assembler Shader. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - First OpenGL extensions support shader programming in assembler code / <code>!! ARBvp1.0 / ATTRIB iPos = vertex.position ; / ATTRIB iNormal = vertex.normal ; / ...</code>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Assembler Shader'?

Page 40	High-Level Shading Languages 1/2
Source cue	- Assembler code is hard to read and hard to maintain / - High-level languages for shading are specified in C-like syntax / - Source code is translated by the compiler into appropriate ASM statements / - Using shaders after initial binding automatically when rendering geometry / - Interface to OpenGL by passing appropriate variables or changing the OpenGL
Professor-style explanation	The slide 'High-Level Shading Languages 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Assembler code is hard to read and hard to maintain / - High-level languages for shading are specified in C-like syntax / - Source code is translated by the compiler into appropriate ASM statements / - Using shaders after initial binding automatically when rendering geometry / - Interface to OpenGL by passing appropriate variables or changing the OpenGL. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about High-Level Shading Languages 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Assembler code is hard to read and hard to maintain / - High-level languages for shading are specified in C-like syntax / - Source code is translated by the compiler into appropriate ASM statements / - Using shaders after initial binding automatically when rendering geometry / - Interface to OpenGL by passing appropriate variables or changing the OpenGL
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'High-Level Shading Languages 1/2'?
Page 41	High-Level Shading Languages 2/2

Source cue	- Existing high level languages f or shader development / - CG - C for Graphics by NVIDIA / - CG was introduced in 2003 by NVIDIA as a platform independent solution / (in practice rather semi-platform independent) / - HLSL - High Level Shading Language from Microsoft
Professor-style explanation	The slide 'High-Level Shading Languages 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Existing high level languages f or shader development / - CG - C for Graphics by NVIDIA / - CG was introduced in 2003 by NVIDIA as a platform independent solution / (in practice rather semi-platform independent) / - HLSL - High Level Shading Language from Microsoft. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about High-Level Shading Languages 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Existing high level languages f or shader development / - CG - C for Graphics by NVIDIA / - CG was introduced in 2003 by NVIDIA as a platform independent solution / (in practice rather semi-platform independent) / - HLSL - High Level Shading Language from Microsoft
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'High-Level Shading Languages 2/2'?
Page 42	Shader Types
Source cue	- Pipeline concept has been kept in place / and shaders are responsible for different stages / - First, geometry level programming was done by vertex shaders / - Vertex shaders allow modifiability of vertex transformations and lighting / - After that, there was an extension for image-based algorithms by fragment

Professor-style explanation	The slide 'Shader Types' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Pipeline concept has been kept in place / and shaders are responsible for different stages / - First, geometry level programming was done by vertex shaders / - Vertex shaders allow modifiability of vertex transformations and lighting / - After that, there was an extension for image-based algorithms by fragment. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Shader Types. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Pipeline concept has been kept in place / and shaders are responsible for different stages / - First, geometry level programming was done by vertex shaders / - Vertex shaders allow modifiability of vertex transformations and lighting / - After that, there was an extension for image-based algorithms by fragment
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Types'?
Page 43	Vertex Shader Concepts
Source cue	- Vertex shaders enable the programmable modification of processed geometry / - Possibilities of a vertex shader / Programmable / - Geometric transformation of vertices and normals / Vertex Processing
Professor-style explanation	The slide 'Vertex Shader Concepts' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Vertex shaders enable the programmable modification of processed geometry / - Possibilities of a vertex shader / Programmable / - Geometric transformation of vertices and normals / Vertex Processing. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Vertex Shader Concepts. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Vertex shaders enable the programmable modification of processed geometry / - Possibilities of a vertex shader / Programmable / - Geometric transformation of vertices and normals / Vertex Processing
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Concepts'?
Page 44	Vertex Shader Examples 1/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Vertex Shader Examples 1/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Vertex Shader Examples 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Examples 1/2'?
Page 45	Vertex Shader Examples 2/2
Source cue	- Morphing / - Shadow volume calculation for shadow effects / [ID Software's Doom III]
Professor-style explanation	The slide 'Vertex Shader Examples 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Morphing / - Shadow volume calculation for shadow effects / [ID Software's Doom III]. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Vertex Shader Examples 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Morphing / - Shadow volume calculation for shadow effects / [ID Software's Doom III]
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Examples 2/2'?

Page 46	Vertex Shader Coordinate Systems 1/2
Source cue	- Vertex shaders are executed for each vertex / - Vertex shaders calculate coordinates in clipping coordinates as output / - Vertex shaders can not / - Access adjacent vertices / - Change the number of vertices (i.e., generate or discard vertices)
Professor-style explanation	The slide 'Vertex Shader Coordinate Systems 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Vertex shaders are executed for each vertex / - Vertex shaders calculate coordinates in clipping coordinates as output / - Vertex shaders can not / - Access adjacent vertices / - Change the number of vertices (i.e., generate or discard vertices). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Vertex Shader Coordinate Systems 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Vertex shaders are executed for each vertex / - Vertex shaders calculate coordinates in clipping coordinates as output / - Vertex shaders can not / - Access adjacent vertices / - Change the number of vertices (i.e., generate or discard vertices)
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Coordinate Systems 1/2'?
Page 47	Vertex Shader Coordinate Systems 2/2
Source cue	Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform

Professor-style explanation	The slide 'Vertex Shader Coordinate Systems 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Vertex Shader Coordinate Systems 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Coordinate Systems 2/2'?

Page 48	Local Space
Source cue	- Coordinates of your object relative to its local origin / - 3D vertex positions
Professor-style explanation	The slide 'Local Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Coordinates of your object relative to its local origin / - 3D vertex positions. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Local Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Coordinates of your object relative to its local origin / - 3D vertex positions
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Local Space'?

Page 49	World Space
---------	-------------

Source cue	- Coordinates in respect of a larger world / - Relative to some global origin of the world / - Together with many other objects
Professor-style explanation	The slide 'World Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Coordinates in respect of a larger world / - Relative to some global origin of the world / - Together with many other objects. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about World Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Coordinates in respect of a larger world / - Relative to some global origin of the world / - Together with many other objects
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'World Space'?
Page 50	View Space
Source cue	- Coordinates as seen from the camera
Professor-style explanation	The slide 'View Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Coordinates as seen from the camera. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about View Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Coordinates as seen from the camera
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'View Space'?
Page 51	Clip Space

Source cue	- Clip coordinates range from -1 to 1 / - Projection (Orthographic, Perspective)
Professor-style explanation	The slide 'Clip Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Clip coordinates range from -1 to 1 / - Projection (Orthographic, Perspective). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Clip Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Clip coordinates range from -1 to 1 / - Projection (Orthographic, Perspective)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Clip Space'?
Page 52	Screen Space
Source cue	- Transform to image coordinates with glViewport / - Results are sent to the rasterizer
Professor-style explanation	The slide 'Screen Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Transform to image coordinates with glViewport / - Results are sent to the rasterizer. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Screen Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Transform to image coordinates with glViewport / - Results are sent to the rasterizer
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Screen Space'?
Page 53	Coordinate Systems

Source cue	Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform
Professor-style explanation	The slide 'Coordinate Systems' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Coordinate Systems. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Local Space World Space View Space Clip Space Screen Space / Viewport / M M M / model view project Transform
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Coordinate Systems'?

Page 54	Vertex Shader Variables
Source cue	- Input types / - Uniforms: read only in VS (& FS) // Intrinsic vertex attributes / out gl_PerVertex { / - For example: light position or light color / vec4 gl_Position;
Professor-style explanation	The slide 'Vertex Shader Variables' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Input types / - Uniforms: read only in VS (& FS) // Intrinsic vertex attributes / out gl_PerVertex { / - For example: light position or light color / vec4 gl_Position. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Vertex Shader Variables. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Input types / - Uniforms: read only in VS (& FS) // Intrinsic vertex attributes / out gl_PerVertex { / - For example: light position or light color / vec4 gl_Position;

Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Vertex Shader Variables'?
Page 55	Fixed Function Vertex Postprocessing
Source cue	- Clipping / - Discard invisible geometries / Programmable / - Perspective Division / Vertex Processing
Professor-style explanation	The slide 'Fixed Function Vertex Postprocessing' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Clipping / - Discard invisible geometries / Programmable / - Perspective Division / Vertex Processing. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Fixed Function Vertex Postprocessing. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Clipping / - Discard invisible geometries / Programmable / - Perspective Division / Vertex Processing
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Fixed Function Vertex Postprocessing'?
Page 56	Fixed Function Rasterization
Source cue	- Geometry is converted to raster coordinates / Programmable / Vertex Processing / Fixed Function / Primitives Fragments Vertex Postprocessing
Professor-style explanation	The slide 'Fixed Function Rasterization' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Geometry is converted to raster coordinates / Programmable / Vertex Processing / Fixed Function / Primitives Fragments Vertex Postprocessing. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Fixed Function Rasterization. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Geometry is converted to raster coordinates / Programmable / Vertex Processing / Fixed Function / Primitives Fragments Vertex Postprocessing
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Fixed Function Rasterization'?
Page 57	Fragment Shader Concept 1/2
Source cue	- Fragment shaders set the color of a pixel / - Possibilities of a fragment shader / Programmable / - Glare of textures / Vertex Processing
Professor-style explanation	The slide 'Fragment Shader Concept 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Fragment shaders set the color of a pixel / - Possibilities of a fragment shader / Programmable / - Glare of textures / Vertex Processing. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Fragment Shader Concept 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Fragment shaders set the color of a pixel / - Possibilities of a fragment shader / Programmable / - Glare of textures / Vertex Processing
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Fragment Shader Concept 1/2'?
Page 58	Fragment Shader Concept 2/2
Source cue	- Fragment shaders are executed for fragments (= pixel predecessors) / - Fragment shaders calculate the color and depth value of the current fragment / - No information about adjacent fragments available / - The number of fragments can be changed / - Fragment shaders can not generate fragments

Professor-style explanation	The slide 'Fragment Shader Concept 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Fragment shaders are executed for fragments (= pixel predecessors) / - Fragment shaders calculate the color and depth value of the current fragment / - No information about adjacent fragments available / - The number of fragments can be changed / - Fragment shaders can not generate fragments. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Fragment Shader Concept 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Fragment shaders are executed for fragments (= pixel predecessors) / - Fragment shaders calculate the color and depth value of the current fragment / - No information about adjacent fragments available / - The number of fragments can be changed / - Fragment shaders can not generate fragments
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Fragment Shader Concept 2/2'?
Page 59	Shader Example
Source cue	layout (std140) uniform Matrices { Vertex Shader / mat4 projModelViewMatrix; / mat3 normalMatrix; /** Fragment Shader / }; * Set fragment color to red. / in vec3 position;
Professor-style explanation	The slide 'Shader Example' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: layout (std140) uniform Matrices { Vertex Shader / mat4 projModelViewMatrix; / mat3 normalMatrix; /** Fragment Shader / }; * Set fragment color to red. / in vec3 position. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Shader Example. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: layout (std140) uniform Matrices { Vertex Shader / mat4 projModelViewMatrix; / mat3 normalMatrix; /** Fragment Shader / }; * Set fragment color to red. / in vec3 position;
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Example'?

Page 60	GLSL Shader Functions
Source cue	- Trigonometry functions / - radians, degrees, sin, cos, tan, asin, acos, atan / - Power calculations / - pow, exp2, log2, sqrt, inversesqrt / - General
Professor-style explanation	The slide 'GLSL Shader Functions' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Trigonometry functions / - radians, degrees, sin, cos, tan, asin, acos, atan / - Power calculations / - pow, exp2, log2, sqrt, inversesqrt / - General. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about GLSL Shader Functions. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Trigonometry functions / - radians, degrees, sin, cos, tan, asin, acos, atan / - Power calculations / - pow, exp2, log2, sqrt, inversesqrt / - General
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'GLSL Shader Functions'?

Page 61	Shader Usage
Source cue	- One or multiple shaders are attached to a shader program / Vertex Shader / glCreateShader glCreateProgram / Fragment Shader / glShaderSource glAttachShader glCreateShader

Professor-style explanation	The slide 'Shader Usage' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - One or multiple shaders are attached to a shader program / Vertex Shader / <code>glCreateShader glCreateProgram / Fragment Shader / glShaderSource glAttachShader glCreateShader</code> . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Shader Usage. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - One or multiple shaders are attached to a shader program / Vertex Shader / <code>glCreateShader glCreateProgram / Fragment Shader / glShaderSource glAttachShader glCreateShader</code>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Usage'?

Page 62	Binding Shaders
Source cue	<code>v = glCreateShader(GL_VERTEX_SHADER); // vertex shader / char* vs = readTextFile("vertex-shader.vert"); / const char* vv = vs; / glShaderSource(v, 1, &vv, NULL); / free(vs);</code>
Professor-style explanation	The slide 'Binding Shaders' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: <code>v = glCreateShader(GL_VERTEX_SHADER); // vertex shader / char* vs = readTextFile("vertex-shader.vert"); / const char* vv = vs; / glShaderSource(v, 1, &vv, NULL); / free(vs)</code> . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Binding Shaders. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: <code>v = glCreateShader(GL_VERTEX_SHADER); // vertex shader / char* vs = readTextFile("vertex-shader.vert"); / const char* vv = vs; / glShaderSource(v, 1, &vv, NULL); / free(vs);</code>

Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Binding Shaders'?
Page 63	Setting Uniforms
Source cue	- Uniforms are used to communicate between CPU and GPU / - Data is bound to uniform locations / - Must be updated after each linking process / <code>GLint loc0 = glGetUniformLocation(p, "isoValue_"); / glUniform1f(loc0, 0.45f);</code>
Professor-style explanation	The slide 'Setting Uniforms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Uniforms are used to communicate between CPU and GPU / - Data is bound to uniform locations / - Must be updated after each linking process / <code>GLint loc0 = glGetUniformLocation(p, "isoValue_"); / glUniform1f(loc0, 0.45f)</code> . The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Setting Uniforms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Uniforms are used to communicate between CPU and GPU / - Data is bound to uniform locations / - Must be updated after each linking process / <code>GLint loc0 = glGetUniformLocation(p, "isoValue_"); / glUniform1f(loc0, 0.45f);</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Setting Uniforms' into a causal sentence instead of repeating the slide title?
Page 64	Shader Performance
Source cue	- Guidelines for efficient shaders / - Since fragment shaders are executed for a large number of fragments, / they should contain as few instructions as possible / - There should be as little as possible a change of the active shaders, / because they have their own state and, accordingly, the pipeline may need

Professor-style explanation	The slide 'Shader Performance' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Guidelines for efficient shaders / - Since fragment shaders are executed for a large number of fragments, / they should contain as few instructions as possible / - There should be as little as possible a change of the active shaders, / because they have their own state and, accordingly, the pipeline may need. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Shader Performance. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Guidelines for efficient shaders / - Since fragment shaders are executed for a large number of fragments, / they should contain as few instructions as possible / - There should be as little as possible a change of the active shaders, / because they have their own state and, accordingly, the pipeline may need
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Performance'?

Page 65	Shader Development Environments 1/2
Source cue	- FX Composer, NVPerfHUD (NVIDIA) / - HLSL Shader IDE with performance analytics / www.developer.nvidia.com/page/tools.html / - RenderMonkey (ATI) / - HLSL, GLSL Shader IDE with performance analytics
Professor-style explanation	The slide 'Shader Development Environments 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - FX Composer, NVPerfHUD (NVIDIA) / - HLSL Shader IDE with performance analytics / www.developer.nvidia.com/page/tools.html / - RenderMonkey (ATI) / - HLSL, GLSL Shader IDE with performance analytics. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Shader Development Environments 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - FX Composer, NVPerfHUD (NVIDIA) / - HLSL Shader IDE with performance analytics / www.developer.nvidia.com/page/tools.html / - RenderMonkey (ATI) / - HLSL, GLSL Shader IDE with performance analytics
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Development Environments 1/2'?

Page 66	Shader Development Environments 2/2
Source cue	- Shdr online GLSL editor: https://shdr.bkcore.com/
Professor-style explanation	The slide 'Shader Development Environments 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Shdr online GLSL editor: https://shdr.bkcore.com/ . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Shader Development Environments 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Shdr online GLSL editor: https://shdr.bkcore.com/
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shader Development Environments 2/2'?
Page 67	2.5 Fragment Tests and Operations
Source cue	Towards the finishing line to the pixel

Professor-style explanation	The slide '2.5 Fragment Tests and Operations' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Towards the finishing line to the pixel. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 2.5 Fragment Tests and Operations. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Towards the finishing line to the pixel
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '2.5 Fragment Tests and Operations'?

Page 68	Fragment Tests and Operations 1/2
Source cue	- After fragments have been generated, they must go through the per-fragment / tests and operations / Programmable / - Only if all these tests are successful, / Vertex Processing
Professor-style explanation	The slide 'Fragment Tests and Operations 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - After fragments have been generated, they must go through the per-fragment / tests and operations / Programmable / - Only if all these tests are successful, / Vertex Processing. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Fragment Tests and Operations 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - After fragments have been generated, they must go through the per-fragment / tests and operations / Programmable / - Only if all these tests are successful, / Vertex Processing
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Fragment Tests and Operations 1/2' into a causal sentence instead of repeating the slide title?

Page 69	Fragment Tests and Operations 2/2
---------	-----------------------------------

Source cue	Fragment Pixel Ownership Scissor Alpha / Data Test Test Test / Depth Stencil / Framebuffer Dithering Blending / Test Test
Professor-style explanation	The slide 'Fragment Tests and Operations 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Fragment Pixel Ownership Scissor Alpha / Data Test Test Test / Depth Stencil / Framebuffer Dithering Blending / Test Test. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Fragment Tests and Operations 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Fragment Pixel Ownership Scissor Alpha / Data Test Test Test / Depth Stencil / Framebuffer Dithering Blending / Test Test
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Fragment Tests and Operations 2/2'?

Page 70	Alpha Test
Source cue	- If framebuffer is an RGBA buffer, the alpha test can be performed / - The test must first be activated / glEnable(GL_ALPHA_TEST); / - The comparison function indicates which fragments survive the test / glAlphaFunc(func, value);
Professor-style explanation	The slide 'Alpha Test' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - If framebuffer is an RGBA buffer, the alpha test can be performed / - The test must first be activated / glEnable(GL_ALPHA_TEST); / - The comparison function indicates which fragments survive the test / glAlphaFunc(func, value). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Alpha Test. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - If framebuffer is an RGBA buffer, the alpha test can be performed / - The test must first be activated / glEnable(GL_ALPHA_TEST); / - The comparison function indicates which fragments survive the test / glAlphaFunc(func, value);
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Alpha Test'?

Page 71	Stencil Test
Source cue	- Stencil Buffer / - Is a frame buffer component / - Is (indirectly) filled by drawing operations / - No special stencil drawing operations / - Stencil operation
Professor-style explanation	The slide 'Stencil Test' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Stencil Buffer / - Is a frame buffer component / - Is (indirectly) filled by drawing operations / - No special stencil drawing operations / - Stencil operation. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Stencil Test. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Stencil Buffer / - Is a frame buffer component / - Is (indirectly) filled by drawing operations / - No special stencil drawing operations / - Stencil operation
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Stencil Test'?

Page 72	Depth Test
Source cue	- If the framebuffer has a depth buffer, the depth test can be performed / - The test must first be activated / glEnable(GL_DEPTH_TEST); / - The comparison is made with the depth value already in the buffer / - The comparison function indicates which fragments survive the test

Professor-style explanation	The slide 'Depth Test' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - If the framebuffer has a depth buffer, the depth test can be performed / - The test must first be activated / <code>glEnable(GL_DEPTH_TEST);</code> / - The comparison is made with the depth value already in the buffer / - The comparison function indicates which fragments survive the test. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Depth Test. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - If the framebuffer has a depth buffer, the depth test can be performed / - The test must first be activated / <code>glEnable(GL_DEPTH_TEST);</code> / - The comparison is made with the depth value already in the buffer / - The comparison function indicates which fragments survive the test
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Test'?
Page 73	Image Composition and Mixture
Source cue	- Alpha values of the pixels / - control the combination of pixel values / - are specified as color components of color pixels / - $P = [R, G, B, A]$, with $R, G, B, A \in [0,1]$ / - Applications are manifold
Professor-style explanation	The slide 'Image Composition and Mixture' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Alpha values of the pixels / - control the combination of pixel values / - are specified as color components of color pixels / - $P = [R, G, B, A]$, with $R, G, B, A \in [0,1]$ / - Applications are manifold. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Image Composition and Mixture. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Alpha values of the pixels / - control the combination of pixel values / - are specified as color components of color pixels / - $P = [R, G, B, A]$, with $R, G, B, A \in [0,1]$ / - Applications are manifold

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Image Composition and Mixture' into a causal sentence instead of repeating the slide title?
Page 74	Blending Application
Source cue	- Example application for transparency: drawing with semi-transparent brush
Professor-style explanation	The slide 'Blending Application' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Example application for transparency: drawing with semi-transparent brush. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Blending Application. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Example application for transparency: drawing with semi-transparent brush
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Blending Application' into a causal sentence instead of repeating the slide title?
Page 75	Blending Process 1/2
Source cue	- Blending is an operation called per fragment / - Fragments are modified after rasterization, / but before writing to the framebuffer with a blending function / - Combination of a source fragment to be written with the destination pixel / in the frame buffer by using a blending function
Professor-style explanation	The slide 'Blending Process 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Blending is an operation called per fragment / - Fragments are modified after rasterization, / but before writing to the framebuffer with a blending function / - Combination of a source fragment to be written with the destination pixel / in the frame buffer by using a blending function. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Blending Process 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Blending is an operation called per fragment / - Fragments are modified after rasterization, / but before writing to the framebuffer with a blending function / - Combination of a source fragment to be written with the destination pixel / in the frame buffer by using a blending function
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Blending Process 1/2'?

Page 76	Blending Process 2/2
Source cue	- Source factor: F / - Destination factor: F / - Source fragment: [P , P , P , P] / s,r s,g s,b s,a / - Destination pixel: [P , P , P , P]
Professor-style explanation	The slide 'Blending Process 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Source factor: F / - Destination factor: F / - Source fragment: [P , P , P , P] / s,r s,g s,b s,a / - Destination pixel: [P , P , P , P]. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Blending Process 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Source factor: F / - Destination factor: F / - Source fragment: [P , P , P , P] / s,r s,g s,b s,a / - Destination pixel: [P , P , P , P]
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Blending Process 2/2' into a causal sentence instead of repeating the slide title?

Page 77	Applying Blending
Source cue	- Switch on (or switch off) blending / - glEnable(GL_BLEND); / - glDisable(GL_BLEND); / - Defining Blending Factors / - glBlendFunc(srcFac, destFac);
Professor-style explanation	The slide 'Applying Blending' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Switch on (or switch off) blending / - glEnable(GL_BLEND); / - glDisable(GL_BLEND); / - Defining Blending Factors / - glBlendFunc(srcFac, destFac). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Applying Blending. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Switch on (or switch off) blending / - glEnable(GL_BLEND); / - glDisable(GL_BLEND); / - Defining Blending Factors / - glBlendFunc(srcFac, destFac);
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Applying Blending' into a causal sentence instead of repeating the slide title?

Page 78	Blending Examples
Source cue	- Example 1: Copy source pixels and overwrite destination pixels / - glBlendFunc(GL_ONE, GL_ZERO); / - P = [P , P , P , P] / blend s,r s,g s,b s,a / - Example 2: Alpha-based mixing of source pixels and destination pixels
Professor-style explanation	The slide 'Blending Examples' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Example 1: Copy source pixels and overwrite destination pixels / - glBlendFunc(GL_ONE, GL_ZERO); / - P = [P , P , P , P] / blend s,r s,g s,b s,a / - Example 2: Alpha-based mixing of source pixels and destination pixels. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Blending Examples. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Example 1: Copy source pixels and overwrite destination pixels / - glBlendFunc(GL_ONE, GL_ZERO); / - P = [P , P , P , P] / blend s,r s,g s,b s,a / - Example 2: Alpha-based mixing of source pixels and destination pixels
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Blending Examples' into a causal sentence instead of repeating the slide title?
Page 79	2.6 Framebuffer
Source cue	Contains everything that is (in)visible on the screen

Professor-style explanation	The slide '2.6 Framebuffer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Contains everything that is (in)visible on the screen. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 2.6 Framebuffer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Contains everything that is (in)visible on the screen
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '2.6 Framebuffer'?

Page 80	Viewport
Source cue	- Rectangular area in pixel grid, set explicitly / - Rendering operations are limited to this area / - All primitives that are completely or partially outside / are clipped at the viewport area (clipping)
Professor-style explanation	The slide 'Viewport' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Rectangular area in pixel grid, set explicitly / - Rendering operations are limited to this area / - All primitives that are completely or partially outside / are clipped at the viewport area (clipping). The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Viewport. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Rectangular area in pixel grid, set explicitly / - Rendering operations are limited to this area / - All primitives that are completely or partially outside / are clipped at the viewport area (clipping)
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Viewport'?

Page 81	Framebuffer
---------	-------------

Source cue	- Context-bound raster with a resolution and structure / specified by the application / - Physical realization on the GPU / - Logical subdivision / - Color memory
Professor-style explanation	The slide 'Framebuffer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Context-bound raster with a resolution and structure / specified by the application / - Physical realization on the GPU / - Logical subdivision / - Color memory. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Framebuffer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Context-bound raster with a resolution and structure / specified by the application / - Physical realization on the GPU / - Logical subdivision / - Color memory
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Framebuffer'?
Page 82	OpenGL Framebuffers 1/3
Source cue	- Typical OpenGL framebuffer configuration / 8bit Stencil Buffer / 32bit Depth Buffer / = 32bit Color Buffer
Professor-style explanation	The slide 'OpenGL Framebuffers 1/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Typical OpenGL framebuffer configuration / 8bit Stencil Buffer / 32bit Depth Buffer / = 32bit Color Buffer. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Framebuffers 1/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Typical OpenGL framebuffer configuration / 8bit Stencil Buffer / 32bit Depth Buffer / = 32bit Color Buffer

Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Framebuffers 1/3'?
Page 83	OpenGL Framebuffers 2/3
Source cue	- All buffers of a frame buffer have the same grid size / - Number of bitplanes per buffer can be configured individually / - Individual buffers are directly supported by the hardware / - Buffer identifier / - GL_COLOR_BUFFER_BIT
Professor-style explanation	The slide 'OpenGL Framebuffers 2/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - All buffers of a frame buffer have the same grid size / - Number of bitplanes per buffer can be configured individually / - Individual buffers are directly supported by the hardware / - Buffer identifier / - GL_COLOR_BUFFER_BIT. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Framebuffers 2/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - All buffers of a frame buffer have the same grid size / - Number of bitplanes per buffer can be configured individually / - Individual buffers are directly supported by the hardware / - Buffer identifier / - GL_COLOR_BUFFER_BIT
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Framebuffers 2/3'?
Page 84	OpenGL Framebuffers 3/3
Source cue	- Delete a buffer / glClear(GLbitfield mask); / - Set the initialization color for the color buffer / glClearColor(1.0,1.0,1.0,1.0); / glClear(GL_COLOR_BUFFER_BIT);

Professor-style explanation	The slide 'OpenGL Framebuffers 3/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Delete a buffer / <code>glClear(GLbitfield mask);</code> / - Set the initialization color for the color buffer / <code>glClearColor(1.0,1.0,1.0,1.0);</code> / <code>glClear(GL_COLOR_BUFFER_BIT)</code> . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Framebuffers 3/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Delete a buffer / <code>glClear(GLbitfield mask);</code> / - Set the initialization color for the color buffer / <code>glClearColor(1.0,1.0,1.0,1.0);</code> / <code>glClear(GL_COLOR_BUFFER_BIT)</code> ;
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Framebuffers 3/3'?

Page 85	Double Buffering
Source cue	- Use of two color buffers to avoid flicker effects / - Image buildup takes place in the background (back buffer) / - Visible image is independent of this (front buffer) / Rendering / Process
Professor-style explanation	The slide 'Double Buffering' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Use of two color buffers to avoid flicker effects / - Image buildup takes place in the background (back buffer) / - Visible image is independent of this (front buffer) / Rendering / Process. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Double Buffering. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Use of two color buffers to avoid flicker effects / - Image buildup takes place in the background (back buffer) / - Visible image is independent of this (front buffer) / Rendering / Process

Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Double Buffering'?
Page 86	2.7 Pixel-based Rendering
Source cue	Direct pixel processing without involving 3D models
Professor-style explanation	The slide '2.7 Pixel-based Rendering' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. On the slide, the concrete items are: Direct pixel processing without involving 3D models. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.
Technical commentary	This slide is about 2.7 Pixel-based Rendering. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. Concrete items shown: Direct pixel processing without involving 3D models
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in '2.7 Pixel-based Rendering'?
Page 87	Pixel-based Rendering Operations
Source cue	- Generally / - Read and write operations for various image data formats, / usually as an external library (e.g., DevIL) / - Conversions between image data types / - In OpenGL done with framebuffer objects
Professor-style explanation	The slide 'Pixel-based Rendering Operations' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. On the slide, the concrete items are: - Generally / - Read and write operations for various image data formats, / usually as an external library (e.g., DevIL) / - Conversions between image data types / - In OpenGL done with framebuffer objects. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.

Technical commentary	This slide is about Pixel-based Rendering Operations. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. Concrete items shown: - Generally / - Read and write operations for various image data formats, / usually as an external library (e.g., DevIL) / - Conversions between image data types / - In OpenGL done with framebuffer objects
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'Pixel-based Rendering Operations'?

Page 88	Image Operations
Source cue	- Rendering image data with OpenGL / Copy Pixel Data / Draw/Read / Pixel / Geometry
Professor-style explanation	The slide 'Image Operations' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Rendering image data with OpenGL / Copy Pixel Data / Draw/Read / Pixel / Geometry. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Image Operations. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Rendering image data with OpenGL / Copy Pixel Data / Draw/Read / Pixel / Geometry
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Image Operations'?

Page 89	Write Pixels into Framebuffer
Source cue	- glDrawPixels(x, y, width, height, format, type, pixels); / - Example / // 64x64 array with rgb components / GLubyte img[64][64][3]; / void draw() {

Professor-style explanation	The slide 'Write Pixels into Framebuffer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - <code>glDrawPixels(x, y, width, height, format, type, pixels);</code> / - Example <code>// 64x64 array with rgb components / GLubyte img[64][64][3]; / void draw() {</code> . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Write Pixels into Framebuffer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - <code>glDrawPixels(x, y, width, height, format, type, pixels);</code> / - Example <code>// 64x64 array with rgb components / GLubyte img[64][64][3]; / void draw() {</code>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Write Pixels into Framebuffer'?

Page 90	Read Pixels from Framebuffer
Source cue	- <code>glReadPixels(x, y, width, height, format, type, pixels);</code> / - Example <code>/ GLubyte* img = 0; / int width, height; // canvas size / void snapshot() {</code>
Professor-style explanation	The slide 'Read Pixels from Framebuffer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - <code>glReadPixels(x, y, width, height, format, type, pixels);</code> / - Example <code>/ GLubyte* img = 0; / int width, height; // canvas size / void snapshot() {</code> . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Read Pixels from Framebuffer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - <code>glReadPixels(x, y, width, height, format, type, pixels);</code> / - Example <code>/ GLubyte* img = 0; / int width, height; // canvas size / void snapshot() {</code>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.

Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Read Pixels from Framebuffer'?
Page 91	Formats & Data Types
Source cue	- Format information for pixel array operations / - GL_RGB - red-green-blue values of the frame buffer (FB) / - GL_RED - red values of the FB / - GL_ALPHA - transparency values of the FB / - GL_RGBA - red-Green-Blue-Alpha values of the FB
Professor-style explanation	The slide 'Formats & Data Types' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Format information for pixel array operations / - GL_RGB - red-green-blue values of the frame buffer (FB) / - GL_RED - red values of the FB / - GL_ALPHA - transparency values of the FB / - GL_RGBA - red-Green-Blue-Alpha values of the FB. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Formats & Data Types. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Format information for pixel array operations / - GL_RGB - red-green-blue values of the frame buffer (FB) / - GL_RED - red values of the FB / - GL_ALPHA - transparency values of the FB / - GL_RGBA - red-Green-Blue-Alpha values of the FB
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Formats & Data Types'?
Page 92	coordinate systems in the geometry stage
Source cue	- Rendering pipeline consist of geometry stage and rasterization stage / - Geometric primitives (mostly triangles) are transformed through different / coordinate systems in the geometry stage / - Rasterization results in fragments, which are pixel predecessors / - Fragments have to endure a sequence of tests in the rasterization stage, in

Professor-style explanation	The slide 'coordinate systems in the geometry stage' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: - Rendering pipeline consist of geometry stage and rasterization stage / - Geometric primitives (mostly triangles) are transformed through different / coordinate systems in the geometry stage / - Rasterization results in fragments, which are pixel predecessors / - Fragments have to endure a sequence of tests in the rasterization stage, in. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.
Technical commentary	This slide is about coordinate systems in the geometry stage. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: - Rendering pipeline consist of geometry stage and rasterization stage / - Geometric primitives (mostly triangles) are transformed through different / coordinate systems in the geometry stage / - Rasterization results in fragments, which are pixel predecessors / - Fragments have to endure a sequence of tests in the rasterization stage, in
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'coordinate systems in the geometry stage' in the rendering pipeline?

Page 93	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.

Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 94	Guide: The Official Guide to Learning OpenGL
Source cue	- Text Books / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer
Professor-style explanation	The slide 'Guide: The Official Guide to Learning OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Text Books / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Guide: The Official Guide to Learning OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Text Books / - D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming / Guide: The Official Guide to Learning OpenGL / (8th Edition), Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Guide: The Official Guide to Learning OpenGL'?

02.1 Rendering Process

Source location: Section 2.1

Commentary

Object-based rendering starts with scene objects represented as primitives. The pipeline transforms vertices, assembles primitives, rasterizes them into fragments, and lets only some fragments become pixels. The important detail is that the representation changes at every stage.

The application stage prepares data and state. The geometry stage handles positions and primitives. The rasterization stage creates fragments and applies fragment operations. This separation helps you explain both theory questions and black-screen OpenGL bugs.

Mental model

Vertices become primitives, primitives become fragments, and fragments compete to become pixel updates.

Check yourself

What is the difference between a fragment and a final pixel?

02.2 OpenGL Overview

Source location: Section 2.2

Commentary

OpenGL is an API for controlling a graphics pipeline. It is not a renderer by itself in the sense of a single command that understands your scene. You create a context, provide buffers, set state, compile shaders, bind objects, and issue draw calls.

Modern OpenGL is stateful and shader-based. Many errors happen because the programmer assumes that a later call carries more information than it really does. The GPU uses the currently bound objects and current state at draw time.

Mental model

OpenGL draw calls consume the state machine as it exists right now.

Check yourself

Why can binding the wrong vertex array or shader program produce a valid draw call with wrong output?

02.3 OpenGL Rendering

Source location: Section 2.3

Commentary

Older OpenGL exposed a fixed-function style where many transformations and lighting choices were configured through predefined calls. Modern OpenGL expects you to write shader programs and explicitly manage data flow.

This is not just historical trivia. It explains why shader code, vertex attributes, uniforms, and buffer objects are central in the exercises. If the shader expects an attribute and the vertex layout does not provide it, the pipeline has no magic fallback.

Mental model

Modern OpenGL makes the data path explicit; that gives control but also creates responsibility.

Check yourself

Which data usually goes into vertex attributes, and which data usually goes into uniforms?

02.4 OpenGL Rendering Pipeline

Source location: Section 2.4

Commentary

The vertex shader runs per vertex and usually outputs clip-space position plus attributes for later interpolation. Primitive assembly groups vertices into points, lines, or triangles. Clipping handles geometry outside the view volume. Rasterization creates fragments.

The fragment shader computes candidate fragment output, often using interpolated attributes, uniforms, and textures. After that, tests and operations such as depth testing, stencil testing, blending, and masking decide what reaches the framebuffer.

Mental model

Programmable shaders compute values; fixed pipeline stages decide coverage, interpolation, tests, and output rules.

Check yourself

If your geometry appears in wireframe but colors are wrong, which stages become suspicious?

02.5 Fragment Tests, Framebuffer, and Pixel-Based Rendering

Source location: Sections 2.5-2.7

Commentary

Fragment tests are the gatekeepers after shading. A fragment can have a perfectly valid color and still fail because depth, stencil, scissor, masking, or blending state prevents a visible update. This is why final pixels are a subset of generated fragments.

The framebuffer stores the final render targets: color buffers, depth buffers, stencil buffers, or custom attachments. Pixel-based rendering starts from image data instead of geometric primitives, which is useful for image processing but conceptually different from the object-to-pixel pipeline.

Mental model

The framebuffer is the destination; fragment operations are the rules for writing into it.

Check yourself

Why can two fragments with different colors produce only one visible pixel color?

End-of-lecture summary

If you remember only one thing from Lecture 02, remember this: This lecture turns the vague idea of rendering into a concrete sequence. It is the most important debugging map in the course: if an image is wrong, the pipeline tells you where the error could have entered.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 03 - Geometric Transformations

This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Source chunk	course_text_parts/03_lectures/03_geometric-transformations.txt
Extracted slide pages	54
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Transforming 3D Models
Source cue	- 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through

Professor-style explanation	The slide 'Transforming 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Transforming 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Transforming 3D Models' before rendering begins?

Page 3	3.1 Mathematical Foundations
Source cue	3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL
Professor-style explanation	The slide '3.1 Mathematical Foundations' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: 3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 3.1 Mathematical Foundations. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: 3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL

Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '3.1 Mathematical Foundations'?
Page 4	3.1 Mathematical Foundations
Source cue	Vector and Matrix Calculus
Professor-style explanation	The slide '3.1 Mathematical Foundations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Vector and Matrix Calculus. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3.1 Mathematical Foundations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Vector and Matrix Calculus
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3.1 Mathematical Foundations'?
Page 5	Vertices as Vectors
Source cue	- Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of $\mathbb{R}^2$, $\mathbb{R}^3$, $\mathbb{R}^4$ / - In computer graphics mostly $\mathbb{R}^3$ or $\mathbb{R}^4$ vectors used / - Vertices of a polygon are represented as column vectors / $x \ v$
Professor-style explanation	The slide 'Vertices as Vectors' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of $\mathbb{R}^2$, $\mathbb{R}^3$, $\mathbb{R}^4$ / - In computer graphics mostly $\mathbb{R}^3$ or $\mathbb{R}^4$ vectors used / - Vertices of a polygon are represented as column vectors / $x \ v$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Vertices as Vectors. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of $\mathbb{R}^2$, $\mathbb{R}^3$, $\mathbb{R}^4$ / - In computer graphics mostly $\mathbb{R}^3$ or $\mathbb{R}^4$ vectors used / - Vertices of a polygon are represented as column vectors $x \cdot v$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Vertices as Vectors'?

Page 6	Transformations as Matrices
Source cue	- Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $a \ b \ c \ d / e \ f \ g \ h / M =$
Professor-style explanation	The slide 'Transformations as Matrices' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $a \ b \ c \ d / e \ f \ g \ h / M =$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Transformations as Matrices. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $a \ b \ c \ d / e \ f \ g \ h / M =$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Transformations as Matrices'?

Page 7	Matrix Vector Multiplication
Source cue	- Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $m \times n$ matrix A and $\mathbb{R}^m$ vector B / - Result: $\mathbb{R}^n$ vector C , whereby is B transformed through A / - Example for 4×4 matrix A and $\mathbb{R}^4$ vector B :

Professor-style explanation	The slide 'Matrix Vector Multiplication' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $m \times n$ matrix A and R_m vector B / - Result: R_n vector C, whereby is B transformed through A / - Example for 4×4 matrix A and R_4 vector B. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Matrix Vector Multiplication. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $m \times n$ matrix A and R_m vector B / - Result: R_n vector C, whereby is B transformed through A / - Example for 4×4 matrix A and R_4 vector B:
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Matrix Vector Multiplication'?
Page 8	3.2 Affine Transformations
Source cue	Translation, Rotation, Scaling, and more
Professor-style explanation	The slide '3.2 Affine Transformations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Translation, Rotation, Scaling, and more. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3.2 Affine Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Translation, Rotation, Scaling, and more
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.

Check yourself	Can you state the coordinate space before and after '3.2 Affine Transformations'?
Page 9	Translations - Moving 3D Models
Source cue	- Matrix representation of a translation / $\begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & d \\ 0 & 0 & 1 & d \end{bmatrix} / T(d, d, d) = / x y z$
Professor-style explanation	The slide 'Translations - Moving 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Matrix representation of a translation / $\begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & d \\ 0 & 0 & 1 & d \end{bmatrix} / T(d, d, d) = / x y z$. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Translations - Moving 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Matrix representation of a translation / $\begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & d \\ 0 & 0 & 1 & d \end{bmatrix} / T(d, d, d) = / x y z$
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Translations - Moving 3D Models' before rendering begins?
Page 10	Translation Properties
Source cue	- Lengths and angles are preserved / - Identity: $T(0, 0, 0) = I$ (identity matrix) / - Changing order of translations: $T(d_1, d_1, d_1) \cdot T(d_2, d_2, d_2) = T(d_2, d_2, d_2) \cdot T(d_1, d_1, d_1) / x y z x y z x y z x y z$
Professor-style explanation	The slide 'Translation Properties' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Lengths and angles are preserved / - Identity: $T(0, 0, 0) = I$ (identity matrix) / - Changing order of translations: $T(d_1, d_1, d_1) \cdot T(d_2, d_2, d_2) = T(d_2, d_2, d_2) \cdot T(d_1, d_1, d_1) / x y z x y z x y z x y z$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Translation Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Lengths and angles are preserved / - Identity: $T(0, 0, 0) = I$ (identity matrix) / - Changing order of translations: $T(d_1, d_1, d_1) \cdot T(d_2, d_2, d_2) = T(d_2, d_2, d_2) \cdot T(d_1, d_1, d_1)$ / $x y z x y z x y z x y z$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Translation Properties'?

Page 11	Scaling - Resizing 3D Models
Source cue	- Matrix representation of a scaling / $s \ 0 \ 0 \ 0 / 0 \ s \ 0 \ 0 / S \ s, s, s = / x y z$
Professor-style explanation	The slide 'Scaling - Resizing 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Matrix representation of a scaling / $s \ 0 \ 0 \ 0 / 0 \ s \ 0 \ 0 / S \ s, s, s = / x y z$. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Scaling - Resizing 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Matrix representation of a scaling / $s \ 0 \ 0 \ 0 / 0 \ s \ 0 \ 0 / S \ s, s, s = / x y z$
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Scaling - Resizing 3D Models' before rendering begins?

Page 12	Scaling Properties
Source cue	- Lengths are not preserved, angles only for uniform scalings / - Identity: $S(1, 1, 1) = I$ (identity matrix) / - Changing order of scalings: $S(s_1, s_1, s_1) \cdot S(s_2, s_2, s_2) = S(s_2, s_2, s_2) \cdot S(s_1, s_1, s_1)$ / $x y z x y z x y z x y z$

Professor-style explanation	The slide 'Scaling Properties' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Lengths are not preserved, angles only for uniform scalings / - Identity: $S(1, 1, 1) = I$ (identity matrix) / - Changing order of scalings: $S(s_1, s_1, s_1) \cdot S(s_2, s_2, s_2) = S(s_2, s_2, s_2) \cdot S(s_1, s_1, s_1)$ / $x y z x y z x y z x y z$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Scaling Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Lengths are not preserved, angles only for uniform scalings / - Identity: $S(1, 1, 1) = I$ (identity matrix) / - Changing order of scalings: $S(s_1, s_1, s_1) \cdot S(s_2, s_2, s_2) = S(s_2, s_2, s_2) \cdot S(s_1, s_1, s_1)$ / $x y z x y z x y z x y z$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Scaling Properties'?
Page 13	Rotation - Changing Orientation of 3D Models 1/2
Source cue	- Matrix representation of a rotation around x-, y- and z-axis / $1 \ 0 \ 0 \ 0 / 0 \ \cos \theta \ -\sin \theta \ 0 / R \ \theta = / 0 \ \sin \theta \ \cos \theta \ 0$
Professor-style explanation	The slide 'Rotation - Changing Orientation of 3D Models 1/2' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Matrix representation of a rotation around x-, y- and z-axis / $1 \ 0 \ 0 \ 0 / 0 \ \cos \theta \ -\sin \theta \ 0 / R \ \theta = / 0 \ \sin \theta \ \cos \theta \ 0$. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Rotation - Changing Orientation of 3D Models 1/2. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Matrix representation of a rotation around x-, y- and z-axis / $1 \ 0 \ 0 \ 0 / 0 \ \cos \theta \ -\sin \theta \ 0 / R \ \theta = / 0 \ \sin \theta \ \cos \theta \ 0$
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.

Check yourself	Can you name the geometric objects and attributes represented by 'Rotation - Changing Orientation of 3D Models 1/2' before rendering begins?
Page 14	Rotation - Changing Orientation of 3D Models 2/2
Source cue	- Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x
Professor-style explanation	The slide 'Rotation - Changing Orientation of 3D Models 2/2' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Rotation - Changing Orientation of 3D Models 2/2. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Rotation - Changing Orientation of 3D Models 2/2' before rendering begins?
Page 15	Rotation Properties
Source cue	- Lengths and angles are preserved / - Identity: $R_0 = R_0 = R_0 = I$ (identity matrix) / x y z / - Changing order of rotations around same axis: / - $R(\theta) \cdot R(\phi) = R(\phi) \cdot R(\theta)$

Professor-style explanation	The slide 'Rotation Properties' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Lengths and angles are preserved / - Identity: $R_0 = R_0 = R_0 = I$ (identity matrix) / x y z / - Changing order of rotations around same axis: / - $R(\theta) \cdot R(\phi) = R(\phi) \cdot R(\theta)$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Rotation Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Lengths and angles are preserved / - Identity: $R_0 = R_0 = R_0 = I$ (identity matrix) / x y z / - Changing order of rotations around same axis: / - $R(\theta) \cdot R(\phi) = R(\phi) \cdot R(\theta)$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Rotation Properties'?
Page 16	Mirroring - Reflecting 3D Models on a Plane
Source cue	- Matrix representation of mirroring along x-y plane / $1\ 0\ 0\ 0 / 0\ 1\ 0\ 0 / M = / 0\ 0\ -1\ 0$
Professor-style explanation	The slide 'Mirroring - Reflecting 3D Models on a Plane' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Matrix representation of mirroring along x-y plane / $1\ 0\ 0\ 0 / 0\ 1\ 0\ 0 / M = / 0\ 0\ -1\ 0$. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Mirroring - Reflecting 3D Models on a Plane. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Matrix representation of mirroring along x-y plane / $1\ 0\ 0\ 0 / 0\ 1\ 0\ 0 / M = / 0\ 0\ -1\ 0$
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Mirroring - Reflecting 3D Models on a Plane' before rendering begins?
Page 17	Shearing - Skewing 3D Models

Source cue	- Matrix representation of a shearing / $1 \ h \ 0 \ / \ xy \ xz \ / \ h \ 1 \ h \ 0 \ / \ yx \ yz$
Professor-style explanation	The slide 'Shearing - Skewing 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Matrix representation of a shearing / $1 \ h \ 0 \ / \ xy \ xz \ / \ h \ 1 \ h \ 0 \ / \ yx \ yz$. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Shearing - Skewing 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Matrix representation of a shearing / $1 \ h \ 0 \ / \ xy \ xz \ / \ h \ 1 \ h \ 0 \ / \ yx \ yz$
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Shearing - Skewing 3D Models' before rendering begins?
Page 18	3.3 Composition of Transformations
Source cue	Concatenation of several transformation steps
Professor-style explanation	The slide '3.3 Composition of Transformations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Concatenation of several transformation steps. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3.3 Composition of Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Concatenation of several transformation steps
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3.3 Composition of Transformations'?
Page 19	Composition of Transformations

Source cue	- Composition of transformations T is done / by multiplying the respective transformation matrices / $(T \cdot T \cdot \dots \cdot T \cdot T) \cdot v = (T \cdot (T \cdot (\dots (T \cdot (T \cdot v)))))) / 1 \ 2 \ n-1 \ n \ 1 \ 2 \ n-1 \ n$ / - Interpretation: apply first transformation T , then T , ..., and finally T
Professor-style explanation	The slide 'Composition of Transformations' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Composition of transformations T is done / by multiplying the respective transformation matrices / $(T \cdot T \cdot \dots \cdot T \cdot T) \cdot v = (T \cdot (T \cdot (\dots (T \cdot (T \cdot v)))))) / 1 \ 2 \ n-1 \ n \ 1 \ 2 \ n-1 \ n$ / - Interpretation: apply first transformation T , then T , ..., and finally T . The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Composition of Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Composition of transformations T is done / by multiplying the respective transformation matrices / $(T \cdot T \cdot \dots \cdot T \cdot T) \cdot v = (T \cdot (T \cdot (\dots (T \cdot (T \cdot v)))))) / 1 \ 2 \ n-1 \ n \ 1 \ 2 \ n-1 \ n$ / - Interpretation: apply first transformation T , then T , ..., and finally T
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Composition of Transformations'?

Page 20	Reference Points 1/2
Source cue	- Rotation and scaling happen in relation to origin (reference point)
Professor-style explanation	The slide 'Reference Points 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Rotation and scaling happen in relation to origin (reference point). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Reference Points 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Rotation and scaling happen in relation to origin (reference point)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Reference Points 1/2' into a causal sentence instead of repeating the slide title?
Page 21	Reference Points 2/2
Source cue	- In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point M to the origin O / - Rotate (or scale) the 3D model / - Move the 3D model back to M
Professor-style explanation	The slide 'Reference Points 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point M to the origin O / - Rotate (or scale) the 3D model / - Move the 3D model back to M. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Reference Points 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point M to the origin O / - Rotate (or scale) the 3D model / - Move the 3D model back to M
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Reference Points 2/2'?
Page 22	Rotation Around Arbitrary Axis
Source cue	- Rotation around any axis / - Axis given by center of rotation P and direction $U = (u_x, u_y, u_z)$ / - Rotation by θ degrees / - Construction of transformation as transformation sequence

Professor-style explanation	The slide 'Rotation Around Arbitrary Axis' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Rotation around any axis - Axis given by center of rotation P and direction $U = (u_x, u_y, u_z)$ - Rotation by θ degrees - Construction of transformation as transformation sequence. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Rotation Around Arbitrary Axis. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Rotation around any axis - Axis given by center of rotation P and direction $U = (u_x, u_y, u_z)$ - Rotation by θ degrees - Construction of transformation as transformation sequence
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Rotation Around Arbitrary Axis'?

Page 23	Reminder: Vector Calculus
Source cue	- Length of a vector is expressed as follows / $\ v\ = \sqrt{v_x^2 + v_y^2 + v_z^2}$ / - Vectors of length $\ v\ = 1$ are normalized and called unit vectors / - Arbitrary vectors can be normalized as follows
Professor-style explanation	The slide 'Reminder: Vector Calculus' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Length of a vector is expressed as follows / $\ v\ = \sqrt{v_x^2 + v_y^2 + v_z^2}$ / - Vectors of length $\ v\ = 1$ are normalized and called unit vectors / - Arbitrary vectors can be normalized as follows. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Reminder: Vector Calculus. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Length of a vector is expressed as follows / $\ v\ = \sqrt{v_x^2 + v_y^2 + v_z^2}$ / - Vectors of length $\ v\ = 1$ are normalized and called unit vectors / - Arbitrary vectors can be normalized as follows

Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Reminder: Vector Calculus'?
Page 24	Reminder: Cross Product (= vector product)
Source cue	- Given two vectors u and v in R^3 / - Cross product $u \times v$ of u and v is a vector defined as follows / $u \times v = u \cdot v - u \cdot v / x \times y \times z \times y / u \times v = u \times v = u \cdot v - u \cdot v$
Professor-style explanation	The slide 'Reminder: Cross Product (= vector product)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Given two vectors u and v in R^3 / - Cross product $u \times v$ of u and v is a vector defined as follows / $u \times v = u \cdot v - u \cdot v / x \times y \times z \times y / u \times v = u \times v = u \cdot v - u \cdot v$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Reminder: Cross Product (= vector product). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Given two vectors u and v in R^3 / - Cross product $u \times v$ of u and v is a vector defined as follows / $u \times v = u \cdot v - u \cdot v / x \times y \times z \times y / u \times v = u \times v = u \cdot v - u \cdot v$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Reminder: Cross Product (= vector product)' into a causal sentence instead of repeating the slide title?
Page 25	Example - Transforming Directed Line Segment 1/7
Source cue	- Example transformation task / - Given line segments $P_1 P_2$ and $P_1 P_3$ / 1 2 1 3 / - Transform it into the yz -plane so that $P_1 P_2$ lies on the z -axis / 1 2
Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 1/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Example transformation task / - Given line segments $P_1 P_2$ and $P_1 P_3$ / 1 2 1 3 / - Transform it into the yz -plane so that $P_1 P_2$ lies on the z -axis / 1 2. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Example - Transforming Directed Line Segment 1/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Example transformation task / - Given line segments $P_1 P_2$ and $P_1 P_3$ / - Transform it into the yz -plane so that $P_1 P_2$ lies on the z -axis / 1 2
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 1/7'?
Page 26	Example - Transforming Directed Line Segment 2/7
Source cue	- Approach 1: Determine rotation angles for rotation sequence / - Move $P = x, y, z$ into the origin / 1 1 1 1 / - Rotate around the y -axis so that $P_1 P_2$ lies in the yz -plane / 1 2
Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 2/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Approach 1: Determine rotation angles for rotation sequence / - Move $P = x, y, z$ into the origin / 1 1 1 1 / - Rotate around the y -axis so that $P_1 P_2$ lies in the yz -plane / 1 2. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 2/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Approach 1: Determine rotation angles for rotation sequence / - Move $P = x, y, z$ into the origin / 1 1 1 1 / - Rotate around the y -axis so that $P_1 P_2$ lies in the yz -plane / 1 2
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 2/7'?
Page 27	Example - Transforming Directed Line Segment 3/7
Source cue	- Step 2 - Rotation of $P' P'$ around y -axis into yz -plane / 1 2 / - Rotation $R(\theta - 90^\circ)$ / - Rotation angle - $(90^\circ - \theta) = \theta - 90^\circ$ / - Matrix ingredients

Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 3/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Step 2 - Rotation of $P' P'$ around y-axis into yz-plane / 1 2 / - Rotation $R(\theta - 90^\circ)$ / - Rotation angle - $(90^\circ - \theta) = \theta - 90^\circ$ / - Matrix ingredients. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 3/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Step 2 - Rotation of $P' P'$ around y-axis into yz-plane / 1 2 / - Rotation $R(\theta - 90^\circ)$ / - Rotation angle - $(90^\circ - \theta) = \theta - 90^\circ$ / - Matrix ingredients
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 3/7'?

Page 28	Example - Transforming Directed Line Segment 4/7
Source cue	- Step 3 - Rotation of $P'' P''$ around x-axis onto z-axis / 1 2 / - Rotation $R(\phi)$ / - Matrix ingredients / z''
Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 4/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Step 3 - Rotation of $P'' P''$ around x-axis onto z-axis / 1 2 / - Rotation $R(\phi)$ / - Matrix ingredients / z'' . The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 4/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Step 3 - Rotation of $P'' P''$ around x-axis onto z-axis / 1 2 / - Rotation $R(\phi)$ / - Matrix ingredients / z''
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.

Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 4/7'?
Page 29	Example - Transforming Directed Line Segment 5/7
Source cue	- Step 4 - Rotation of $P''' P'''$ into yz-plane / 1 3 / - Rotation $R(\alpha)$ / $y \blacksquare \blacksquare \blacksquare$ / - Matrix ingredients
Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 5/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Step 4 - Rotation of $P''' P'''$ into yz-plane / 1 3 / - Rotation $R(\alpha)$ / $y \blacksquare \blacksquare \blacksquare$ / - Matrix ingredients. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 5/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Step 4 - Rotation of $P''' P'''$ into yz-plane / 1 3 / - Rotation $R(\alpha)$ / $y \blacksquare \blacksquare \blacksquare$ / - Matrix ingredients
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 5/7'?
Page 30	Example - Transforming Directed Line Segment 6/7
Source cue	- Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / (R on x-axis, R on y-axis, R on z-axis)

Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 6/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / (R on x-axis, R on y-axis, R on z-axis). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Transforming Directed Line Segment 6/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / (R on x-axis, R on y-axis, R on z-axis)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 6/7'?
Page 31	Example - Transforming Directed Line Segment 7/7
Source cue	- Consideration 2: Formulate R as unit vector / orthogonal to the plane spanned by P_1, P_2, P_3 / $P_1 \times P_2 \times P_3$ / - $R = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & -1 \end{bmatrix}$
Professor-style explanation	The slide 'Example - Transforming Directed Line Segment 7/7' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Consideration 2: Formulate R as unit vector / orthogonal to the plane spanned by P_1, P_2, P_3 / $P_1 \times P_2 \times P_3$ / - $R = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & -1 \end{bmatrix}$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Example - Transforming Directed Line Segment 7/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Consideration 2: Formulate R as unit vector / orthogonal to the plane spanned by P_1, P_2 , and P_3 / $P_1 \times P_2$ / $R = \frac{1}{\ P_1 \times P_2\ } P_1 \times P_2$ / $T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Transforming Directed Line Segment 7/7'?

Page 32	Example - Direction of Flight Transformation 1/3
Source cue	- Example transformation task / - Given 3D model centered in origin of x, y, z -coordinate system / p / - Transform it such that / - it is centered in arbitrary point P , and
Professor-style explanation	The slide 'Example - Direction of Flight Transformation 1/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Example transformation task / - Given 3D model centered in origin of x, y, z -coordinate system / p / - Transform it such that / - it is centered in arbitrary point P , and. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Direction of Flight Transformation 1/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Example transformation task / - Given 3D model centered in origin of x, y, z -coordinate system / p / - Transform it such that / - it is centered in arbitrary point P , and
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Direction of Flight Transformation 1/3'?

Page 33	Example - Direction of Flight Transformation 2/3
Source cue	r_1, r_2, r_3 / $1x, 2x, 3x$ / - Step 1 - create orthogonal rotation matrix $R = \begin{bmatrix} r_1 & r_2 & r_3 \\ 1y & 2y & 3y \\ r_4 & r_5 & r_6 \end{bmatrix}$

Professor-style explanation	The slide 'Example - Direction of Flight Transformation 2/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: $r \ r \ r / 1x \ 2x \ 3x /$ - Step 1 - create orthogonal rotation matrix $R = r \ r \ r / 1y \ 2y \ 3y / r \ r \ r$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Direction of Flight Transformation 2/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $r \ r \ r / 1x \ 2x \ 3x /$ - Step 1 - create orthogonal rotation matrix $R = r \ r \ r / 1y \ 2y \ 3y / r \ r \ r$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Direction of Flight Transformation 2/3'?

Page 34	Example - Direction of Flight Transformation 3/3
Source cue	- Transformation M is described by a translation and an orthogonal matrix $/ r \ r \ r \ 0 / 1x \ 2x \ 3x / r \ r \ r \ 0 / 1y \ 2y \ 3y$
Professor-style explanation	The slide 'Example - Direction of Flight Transformation 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Transformation M is described by a translation and an orthogonal matrix $/ r \ r \ r \ 0 / 1x \ 2x \ 3x / r \ r \ r \ 0 / 1y \ 2y \ 3y$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example - Direction of Flight Transformation 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Transformation M is described by a translation and an orthogonal matrix $/ r \ r \ r \ 0 / 1x \ 2x \ 3x / r \ r \ r \ 0 / 1y \ 2y \ 3y$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example - Direction of Flight Transformation 3/3'?

Page 35	3.4 Coordinate System Change
Source cue	Transformation as change of frame of reference
Professor-style explanation	The slide '3.4 Coordinate System Change' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Transformation as change of frame of reference. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3.4 Coordinate System Change. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Transformation as change of frame of reference
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3.4 Coordinate System Change'?

Page 36	Coordinate System Change
Source cue	- Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2
Professor-style explanation	The slide 'Coordinate System Change' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Coordinate System Change. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.

Check yourself	Can you state the coordinate space before and after 'Coordinate System Change'?
Page 37	Local Coordinate Systems
Source cue	- Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS)
Professor-style explanation	The slide 'Local Coordinate Systems' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Local Coordinate Systems. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS)
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Local Coordinate Systems'?
Page 38	Example 1 - Coordinate System Change 1/2
Source cue	x(1) / - Example transformation task y y(5) / - Rotation and scaling of an object around its center M / y(1) / and subsequent translation to a point Q

Professor-style explanation	The slide 'Example 1 - Coordinate System Change 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: $x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center M / $y(1)$ / and subsequent translation to a point Q . The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example 1 - Coordinate System Change 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center M / $y(1)$ / and subsequent translation to a point Q
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 1 - Coordinate System Change 1/2'?

Page 39	Example 1 - Coordinate System Change 2/2
Source cue	$y(5) / y(2) y(3)$
Professor-style explanation	The slide 'Example 1 - Coordinate System Change 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: $y(5) / y(2) y(3)$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example 1 - Coordinate System Change 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $y(5) / y(2) y(3)$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 1 - Coordinate System Change 2/2'?

Page 40	Example 2 - Hierarchical Coordinate Systems 1/2
Source cue	- To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system
Professor-style explanation	The slide 'Example 2 - Hierarchical Coordinate Systems 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Example 2 - Hierarchical Coordinate Systems 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 2 - Hierarchical Coordinate Systems 1/2'?
Page 41	Example 2 - Hierarchical Coordinate Systems 1/2
Source cue	- Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation $R(\alpha)$ / y-tr
Professor-style explanation	The slide 'Example 2 - Hierarchical Coordinate Systems 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation $R(\alpha)$ / y-tr. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Example 2 - Hierarchical Coordinate Systems 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation R (α) / y-tr
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 2 - Hierarchical Coordinate Systems 1/2'?
Page 42	3.5 Transformations in OpenGL
Source cue	Implementation with GLM und GLSL
Professor-style explanation	The slide '3.5 Transformations in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Implementation with GLM und GLSL. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 3.5 Transformations in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Implementation with GLM und GLSL
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '3.5 Transformations in OpenGL'?
Page 43	Geometric Transformations in OpenGL
Source cue	- Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements

Professor-style explanation	The slide 'Geometric Transformations in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Geometric Transformations in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Geometric Transformations in OpenGL'?
Page 44	OpenGL Transformation Pipeline
Source cue	- Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline
Professor-style explanation	The slide 'OpenGL Transformation Pipeline' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about OpenGL Transformation Pipeline. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Transformation Pipeline'?

Page 45	Example - Translation and Scaling
Source cue	- Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized
Professor-style explanation	The slide 'Example - Translation and Scaling' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Example - Translation and Scaling. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example - Translation and Scaling' into a causal sentence instead of repeating the slide title?

Page 46	Matrix Specification
Source cue	- Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] =;

Professor-style explanation	The slide 'Matrix Specification' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] = . The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Matrix Specification. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] =;
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Matrix Specification'?

Page 47	Example - Portico Modeling 1/5
Source cue	- Columned hall consists of floor space, H·0.1 / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is H,
Professor-style explanation	The slide 'Example - Portico Modeling 1/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Columned hall consists of floor space, H·0.1 / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is H,. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Example - Portico Modeling 1/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Columned hall consists of floor space, H·0.1 / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is H,
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.

Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 1/5' before rendering begins?
Page 48	Example - Portico Modeling 2/5
Source cue	- Drawing a cylinder closed at top and bottom / - Each column should "stand" in the y-axis / - Each column should have a lid surface at the top and bottom / - Disk primitive is embedded in the xy-plane / - Cylinder primitive consists only of lateral surface
Professor-style explanation	The slide 'Example - Portico Modeling 2/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Drawing a cylinder closed at top and bottom / - Each column should "stand" in the y-axis / - Each column should have a lid surface at the top and bottom / - Disk primitive is embedded in the xy-plane / - Cylinder primitive consists only of lateral surface. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Example - Portico Modeling 2/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Drawing a cylinder closed at top and bottom / - Each column should "stand" in the y-axis / - Each column should have a lid surface at the top and bottom / - Disk primitive is embedded in the xy-plane / - Cylinder primitive consists only of lateral surface
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 2/5' before rendering begins?
Page 49	Example - Portico Modeling 3/5
Source cue	- Drawing a single column / - Composition of 5 cylinder objects / - Rotation into y-axis to match orientation / - Positioning of the parts by translation along the y-axis / - Incremental translation around the partial column height

Professor-style explanation	The slide 'Example - Portico Modeling 3/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Drawing a single column / - Composition of 5 cylinder objects / - Rotation into y-axis to match orientation / - Positioning of the parts by translation along the y-axis / - Incremental translation around the partial column height. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Example - Portico Modeling 3/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Drawing a single column / - Composition of 5 cylinder objects / - Rotation into y-axis to match orientation / - Positioning of the parts by translation along the y-axis / - Incremental translation around the partial column height
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 3/5' before rendering begins?

Page 50	Example - Portico Modeling 4/5
Source cue	- Drawing the group of columns / - Columns are evenly distributed on a circle with radius R / - Number N, radius R of the columns as well as floor radius R adjustable
Professor-style explanation	The slide 'Example - Portico Modeling 4/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Drawing the group of columns / - Columns are evenly distributed on a circle with radius R / - Number N, radius R of the columns as well as floor radius R adjustable. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Example - Portico Modeling 4/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Drawing the group of columns / - Columns are evenly distributed on a circle with radius R / - Number N, radius R of the columns as well as floor radius R adjustable
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.

Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 4/5' before rendering begins?
Page 51	Example - Portico Modeling 5/5
Source cue	- Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof
Professor-style explanation	The slide 'Example - Portico Modeling 5/5' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Example - Portico Modeling 5/5. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Example - Portico Modeling 5/5' before rendering begins?
Page 52	by applying affine transformations to their vertices
Source cue	- 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication
Professor-style explanation	The slide 'by applying affine transformations to their vertices' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about by applying affine transformations to their vertices. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'by applying affine transformations to their vertices'?

Page 53	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?

Page 54	(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)
Source cue	- Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer

Professor-style explanation	The slide '(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)' into a causal sentence instead of repeating the slide title?

03.1 Mathematical Foundations

Source location: Section 3.1

Commentary

The lecture uses vectors and matrices to express geometric operations. A vector can represent a point position, a direction, an offset, or an attribute depending on context. A matrix expresses a linear map or, with homogeneous coordinates, an affine transformation.

The central exam habit is to name the coordinate space. A point in model coordinates and a light direction in view coordinates cannot be mixed safely. Many visual bugs are coordinate-space bugs.

Mental model

A numeric vector is incomplete until you know what coordinate system it belongs to.

Check yourself

Why is a normal vector not transformed exactly like a point under all transformations?

03.2 Affine Transformations

Source location: Section 3.2

Commentary

Affine transformations include translation, rotation, scaling, shearing, and combinations of these. Homogeneous coordinates allow translation to be represented in matrix form together with other

transformations.

Translation moves positions but not pure directions. Rotation changes orientation while preserving lengths if it is a proper rotation. Scaling can change lengths and can also affect normals. These distinctions matter later in lighting.

Mental model

Use 4D homogeneous coordinates so that a single matrix pipeline can move 3D points through the scene.

Check yourself

What does the homogeneous w component let a transformation matrix express?

03.3 Composition of Transformations

Source location: Section 3.3

Commentary

Composition means applying several transformations in sequence. The order matters because matrix multiplication is generally not commutative. Rotating an object around its own origin and then translating it is different from translating it and then rotating around the world origin.

When you read a formula such as $\text{projection} * \text{view} * \text{model} * \text{position}$, read it from right to left for the point: first model, then view, then projection. The resulting matrix chain is compact, but the conceptual steps remain separate.

Mental model

Matrix chains are compressed stories of movement through coordinate spaces.

Check yourself

Why can swapping model and view matrices destroy the intended camera/object relation?

03.4 Coordinate System Change

Source location: Section 3.4

Commentary

A transformation can be read actively as moving an object, or passively as changing the coordinate frame used to describe it. Both interpretations are valid, but mixing them carelessly causes sign and order errors.

The view matrix is a classic example: conceptually you position a camera, but the pipeline usually transforms the world by the inverse of the camera transform so that the camera becomes the origin of view space.

Mental model

Moving the camera one way is equivalent to moving the world the opposite way for rendering.

Check yourself

Why is the view transform often related to the inverse camera transform?

03.5 Transformations in OpenGL

Source location: Section 3.5

Commentary

Modern OpenGL does not automatically manage the old matrix stacks. You usually compute model, view, and projection matrices in application code and pass them to shaders as uniforms.

The vertex shader is where the final clip-space position is normally computed. That means a wrong matrix uniform, wrong multiplication order, or wrong convention can make geometry vanish even though buffers and draw calls are correct.

Mental model

The shader is the place where abstract transformation math becomes GPU execution.

Check yourself

Which matrix would you inspect first if an object follows the camera instead of staying in the world?

End-of-lecture summary

If you remember only one thing from Lecture 03, remember this: This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 04 - Geometric Projection

This lecture explains how a 3D view becomes a 2D image. Projection is where camera geometry, homogeneous coordinates, clipping planes, depth precision, and viewport mapping meet.

Source chunk	course_text_parts/03_lectures/04_geometric-projection.txt
Extracted slide pages	61
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Projecting 3D Models
Source cue	- To compute the 2D representation of a 3D model on the screen, / it needs to be projected, whereby it loses its z-coordinates / - Depending on the projection type, the object might get skewed / (perspective shortening) / - Projection can also be expressed as a matrix multiplication,

Professor-style explanation	The slide 'Projecting 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - To compute the 2D representation of a 3D model on the screen, / it needs to be projected, whereby it loses its z-coordinates / - Depending on the projection type, the object might get skewed / (perspective shortening) / - Projection can also be expressed as a matrix multiplication,. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Projecting 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - To compute the 2D representation of a 3D model on the screen, / it needs to be projected, whereby it loses its z-coordinates / - Depending on the projection type, the object might get skewed / (perspective shortening) / - Projection can also be expressed as a matrix multiplication,
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Projecting 3D Models' before rendering begins?

Page 3	4.1 Linear Perspective
Source cue	4.2 Planar Projections / 4.3 Camera Modeling / 4.4 Specifying Projections in OpenGL / 4.5 Perspective Projection Derivation / 4.6 Orthographic Projection Derivation
Professor-style explanation	The slide '4.1 Linear Perspective' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: 4.2 Planar Projections / 4.3 Camera Modeling / 4.4 Specifying Projections in OpenGL / 4.5 Perspective Projection Derivation / 4.6 Orthographic Projection Derivation. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 4.1 Linear Perspective. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: 4.2 Planar Projections / 4.3 Camera Modeling / 4.4 Specifying Projections in OpenGL / 4.5 Perspective Projection Derivation / 4.6 Orthographic Projection Derivation

Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '4.1 Linear Perspective'?
Page 4	4.1 Linear Perspective
Source cue	Discovery of perspective distortion in art
Professor-style explanation	The slide '4.1 Linear Perspective' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: Discovery of perspective distortion in art. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about 4.1 Linear Perspective. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: Discovery of perspective distortion in art
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how '4.1 Linear Perspective' changes positions before rasterization?
Page 5	Linear Perspective
Source cue	- What is wrong in this image? / William Hogarth: / False Perspective, 1754 / Satire on False Perspective / William Hogarth (1753)
Professor-style explanation	The slide 'Linear Perspective' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - What is wrong in this image? / William Hogarth: / False Perspective, 1754 / Satire on False Perspective / William Hogarth (1753). The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Linear Perspective. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - What is wrong in this image? / William Hogarth: / False Perspective, 1754 / Satire on False Perspective / William Hogarth (1753)
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Linear Perspective' changes positions before rasterization?

Page 6	Drawing by Projecting
Source cue	- Projection: method for imaging objects using parallel or central beams / - Perspective: / "The science which teaches how to represent tri-dimensional objects / on a bi-dimensional surface, so that the perspective image coincides with / the one which is given by direct vision."
Professor-style explanation	The slide 'Drawing by Projecting' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Projection: method for imaging objects using parallel or central beams / - Perspective: / "The science which teaches how to represent tri-dimensional objects / on a bi-dimensional surface, so that the perspective image coincides with / the one which is given by direct vision.". The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Drawing by Projecting. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Projection: method for imaging objects using parallel or central beams / - Perspective: / "The science which teaches how to represent tri-dimensional objects / on a bi-dimensional surface, so that the perspective image coincides with / the one which is given by direct vision."
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Drawing by Projecting' changes positions before rasterization?

Page 7	Projection in the Arts 1/5
Source cue	- Before the Renaissance, pictures often had a religious / and symbolic character / - Perspective was not understood: / accordingly, they seem 2-dimensional / Fresco from an Egyptian grave

Professor-style explanation	The slide 'Projection in the Arts 1/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Before the Renaissance, pictures often had a religious / and symbolic character / - Perspective was not understood: / accordingly, they seem 2-dimensional / Fresco from an Egyptian grave. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection in the Arts 1/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Before the Renaissance, pictures often had a religious / and symbolic character / - Perspective was not understood: / accordingly, they seem 2-dimensional / Fresco from an Egyptian grave
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 1/5' changes positions before rasterization?

Page 8	Projection in the Arts 2/5
Source cue	- Techniques to simulate space: / - Shaded round volumetric shapes / - Spatial depth through converging lines / Giotto di Bondone / (1266-1337, Florence)
Professor-style explanation	The slide 'Projection in the Arts 2/5' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Techniques to simulate space: / - Shaded round volumetric shapes / - Spatial depth through converging lines / Giotto di Bondone / (1266-1337, Florence). The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Projection in the Arts 2/5. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Techniques to simulate space: / - Shaded round volumetric shapes / - Spatial depth through converging lines / Giotto di Bondone / (1266-1337, Florence)

Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Projection in the Arts 2/5' works per object, per image region, per ray, or per fragment?
Page 9	Projection in the Arts 3/5
Source cue	- Renaissance (1400-1700) / - New emphasis on the importance of personal views and interpretation of / the world, power of observation, especially of nature / (astronomy, anatomy, botany, ...) / - Universe as a clockwork:
Professor-style explanation	The slide 'Projection in the Arts 3/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Renaissance (1400-1700) / - New emphasis on the importance of personal views and interpretation of / the world, power of observation, especially of nature / (astronomy, anatomy, botany, ...) / - Universe as a clockwork. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection in the Arts 3/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Renaissance (1400-1700) / - New emphasis on the importance of personal views and interpretation of / the world, power of observation, especially of nature / (astronomy, anatomy, botany, ...) / - Universe as a clockwork:
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 3/5' changes positions before rasterization?
Page 10	Projection in the Arts 4/5
Source cue	- Examples from the Renaissance

Professor-style explanation	The slide 'Projection in the Arts 4/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Examples from the Renaissance. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection in the Arts 4/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Examples from the Renaissance
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 4/5' changes positions before rasterization?

Page 11	Projection in the Arts 4/5
Source cue	- Examples from the Renaissance / The Last Supper / Leonardo da Vinci (1498)
Professor-style explanation	The slide 'Projection in the Arts 4/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Examples from the Renaissance / The Last Supper / Leonardo da Vinci (1498). The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection in the Arts 4/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Examples from the Renaissance / The Last Supper / Leonardo da Vinci (1498)
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 4/5' changes positions before rasterization?

Page 12	Projection in the Arts 5/5
Source cue	- "The Invention of the Painting" (1830) by Karl Friedrich Schinkel (1781-1841): / - An image based on a legend by Pliny the Elder / - Tracing the shadow contours on a drawing surface
Professor-style explanation	The slide 'Projection in the Arts 5/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - "The Invention of the Painting" (1830) by Karl Friedrich Schinkel (1781-1841): / - An image based on a legend by Pliny the Elder / - Tracing the shadow contours on a drawing surface. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection in the Arts 5/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - "The Invention of the Painting" (1830) by Karl Friedrich Schinkel (1781-1841): / - An image based on a legend by Pliny the Elder / - Tracing the shadow contours on a drawing surface
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 5/5' changes positions before rasterization?
Page 13	Projection in the Arts 6/5
Source cue	- M. C. Escher (1898 - 1972)
Professor-style explanation	The slide 'Projection in the Arts 6/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - M. C. Escher (1898 - 1972). The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Projection in the Arts 6/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - M. C. Escher (1898 - 1972)
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 6/5' changes positions before rasterization?

Page 14	Projection Technics 1/3
Source cue	- Filippo Brunelleschi (1377-1446): / Systematic method for obtaining / a perspective projection
Professor-style explanation	The slide 'Projection Technics 1/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Filippo Brunelleschi (1377-1446): / Systematic method for obtaining / a perspective projection. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection Technics 1/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Filippo Brunelleschi (1377-1446): / Systematic method for obtaining / a perspective projection
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Technics 1/3' changes positions before rasterization?
Page 15	Projection Technics 1/3
Source cue	The Delivery of the Keys / Perugino (1482)

Professor-style explanation	The slide 'Projection Technics 1/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: The Delivery of the Keys / Perugino (1482). The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection Technics 1/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: The Delivery of the Keys / Perugino (1482)
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Technics 1/3' changes positions before rasterization?

Page 16	Projection Technics 2/3
Source cue	- Formal description of the perspective projection / - Leone Batista Alberti (1404-1472): / First essay on the subject of "La Pittura" (1435) / - "A painting [the projection plane] is the intersection of a visual pyramid [view / frustum] at a given distance, with a fixed center [center of projection] and a
Professor-style explanation	The slide 'Projection Technics 2/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Formal description of the perspective projection / - Leone Batista Alberti (1404-1472): / First essay on the subject of "La Pittura" (1435) / - "A painting [the projection plane] is the intersection of a visual pyramid [view / frustum] at a given distance, with a fixed center [center of projection] and a. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Projection Technics 2/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Formal description of the perspective projection / - Leone Batista Alberti (1404-1472): / First essay on the subject of "La Pittura" (1435) / - "A painting [the projection plane] is the intersection of a visual pyramid [view / frustum] at a given distance, with a fixed center [center of projection] and a
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Technics 2/3' changes positions before rasterization?

Page 17	Projection Technics 3/3
Source cue	- Albrecht Dürer (1471-1528): / Geometric and mechanical / description of the concept / of similar triangles / - Construction of several
Professor-style explanation	The slide 'Projection Technics 3/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Albrecht Dürer (1471-1528): / Geometric and mechanical / description of the concept / of similar triangles / - Construction of several. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Projection Technics 3/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Albrecht Dürer (1471-1528): / Geometric and mechanical / description of the concept / of similar triangles / - Construction of several
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Projection Technics 3/3'?

Page 18	Projection Technics 3/3
Source cue	- Camera Obscura

Professor-style explanation	The slide 'Projection Technics 3/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Camera Obscura. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection Technics 3/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Camera Obscura
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Technics 3/3' changes positions before rasterization?

Page 19	4.2 Planar Projections
Source cue	Perspective and parallel projections
Professor-style explanation	The slide '4.2 Planar Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: Perspective and parallel projections. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about 4.2 Planar Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: Perspective and parallel projections
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how '4.2 Planar Projections' changes positions before rasterization?

Page 20	Geometric Projections
---------	-----------------------

Source cue	- Method of mapping objects onto a screen using projectors / - Planar geometric projections / - Projection beams are straight lines / - Projection surface is a plane / (image plane, picture plane,
Professor-style explanation	The slide 'Geometric Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Method of mapping objects onto a screen using projectors / - Planar geometric projections / - Projection beams are straight lines / - Projection surface is a plane / (image plane, picture plane,. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Geometric Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Method of mapping objects onto a screen using projectors / - Planar geometric projections / - Projection beams are straight lines / - Projection surface is a plane / (image plane, picture plane,
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Geometric Projections' changes positions before rasterization?

Page 21	Projection Types 1/2
Source cue	- Perspective projection (= central projection) / - Established by projection center / (COP, center of projection) / - Perspective reduction: The size of the projection / of an object decreases with increasing distance
Professor-style explanation	The slide 'Projection Types 1/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Perspective projection (= central projection) / - Established by projection center / (COP, center of projection) / - Perspective reduction: The size of the projection / of an object decreases with increasing distance. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Projection Types 1/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Perspective projection (= central projection) / - Established by projection center / (COP, center of projection) / - Perspective reduction: The size of the projection / of an object decreases with increasing distance
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Types 1/2' changes positions before rasterization?

Page 22	Projection Types 2/2
Source cue	Planar geometric Projections / Parallel Perspective / Orthographic Oblique 1-Point / 2-Point / Top Axonometric
Professor-style explanation	The slide 'Projection Types 2/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: Planar geometric Projections / Parallel Perspective / Orthographic Oblique 1-Point / 2-Point / Top Axonometric. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection Types 2/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: Planar geometric Projections / Parallel Perspective / Orthographic Oblique 1-Point / 2-Point / Top Axonometric
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Types 2/2' changes positions before rasterization?

Page 23	Orthographic Projections
Source cue	- Orthographic projections often used for / construction drawings of machines and parts / - Main views: Front view, top view and side view / - Advantages / - Accurate length and angle measurements possible

Professor-style explanation	The slide 'Orthographic Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Orthographic projections often used for / construction drawings of machines and parts / - Main views: Front view, top view and side view / - Advantages / - Accurate length and angle measurements possible. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Orthographic Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Orthographic projections often used for / construction drawings of machines and parts / - Main views: Front view, top view and side view / - Advantages / - Accurate length and angle measurements possible
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Orthographic Projections' changes positions before rasterization?

Page 24	Axonometric Projections
Source cue	- Projection beams perpendicular to the projection plane / Isometric / - Projection plane not perpendicular to a coordinate axis / - Visible effects / - Several perpendicular surfaces visible
Professor-style explanation	The slide 'Axonometric Projections' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Projection beams perpendicular to the projection plane / Isometric / - Projection plane not perpendicular to a coordinate axis / - Visible effects / - Several perpendicular surfaces visible. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Axonometric Projections. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Projection beams perpendicular to the projection plane / Isometric / - Projection plane not perpendicular to a coordinate axis / - Visible effects / - Several perpendicular surfaces visible
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Axonometric Projections'?
Page 25	Isometric Projections
Source cue	- Applications in illustrations, furniture design, ... / - Advantages / - Communicate spatial impression in a single view / - Same length ratios along the main axes / - Disadvantages
Professor-style explanation	The slide 'Isometric Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Applications in illustrations, furniture design, ... / - Advantages / - Communicate spatial impression in a single view / - Same length ratios along the main axes / - Disadvantages. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Isometric Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Applications in illustrations, furniture design, ... / - Advantages / - Communicate spatial impression in a single view / - Same length ratios along the main axes / - Disadvantages
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Isometric Projections' changes positions before rasterization?
Page 26	Oblique Projection 1/2
Source cue	- Projection rays not perpendicular to the projection plane / - Common: $\alpha = 30^\circ$ / - Cavalier projection $\alpha = 45^\circ$, i.e., $\tan(\alpha) = 1$ / a' projectors

Professor-style explanation	The slide 'Oblique Projection 1/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Projection rays not perpendicular to the projection plane / - Common: $\alpha = 30^\circ$ / - Cavalier projection $\alpha = a'$ / - $\alpha = 45^\circ$, i.e., $\tan(\alpha) = 1 / a'$ projectors. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Oblique Projection 1/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Projection rays not perpendicular to the projection plane / - Common: $\alpha = 30^\circ$ / - Cavalier projection $\alpha = a'$ / - $\alpha = 45^\circ$, i.e., $\tan(\alpha) = 1 / a'$ projectors
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Oblique Projection 1/2' works per object, per image region, per ray, or per fragment?

Page 27	Oblique Projection 2/2
Source cue	- Advantages / - Can accurately represent the shape of a boundary surface / of an object for accurate measurements / - Missing foreshortening simplifies comparison of sizes / - Gives a good impression of the 3D appearance
Professor-style explanation	The slide 'Oblique Projection 2/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Advantages / - Can accurately represent the shape of a boundary surface / of an object for accurate measurements / - Missing foreshortening simplifies comparison of sizes / - Gives a good impression of the 3D appearance. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Oblique Projection 2/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Advantages / - Can accurately represent the shape of a boundary surface / of an object for accurate measurements / - Missing foreshortening simplifies comparison of sizes / - Gives a good impression of the 3D appearance

Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Oblique Projection 2/2' changes positions before rasterization?
Page 28	Properties of Parallel Projections
Source cue	- Assumption: Object surface of greatest interest lies parallel to a main plane / - Direction of projection: DOP (direction of projection) / - Normal of the projection plane: VPN (view plane normal) / - Top view (orthographic) / - VPN Main axis, DOP VPN
Professor-style explanation	The slide 'Properties of Parallel Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Assumption: Object surface of greatest interest lies parallel to a main plane / - Direction of projection: DOP (direction of projection) / - Normal of the projection plane: VPN (view plane normal) / - Top view (orthographic) / - VPN Main axis, DOP VPN. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Properties of Parallel Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Assumption: Object surface of greatest interest lies parallel to a main plane / - Direction of projection: DOP (direction of projection) / - Normal of the projection plane: VPN (view plane normal) / - Top view (orthographic) / - VPN Main axis, DOP VPN
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Properties of Parallel Projections' changes positions before rasterization?
Page 29	Perspective Projections 1/3
Source cue	- Properties / - Communicate realistic views of three-dimensional objects / and give a spatial impression / - Distort objects in perspective view / - Scaling of objects is lost

Professor-style explanation	The slide 'Perspective Projections 1/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Properties / - Communicate realistic views of three-dimensional objects / and give a spatial impression / - Distort objects in perspective view / - Scaling of objects is lost. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Perspective Projections 1/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Properties / - Communicate realistic views of three-dimensional objects / and give a spatial impression / - Distort objects in perspective view / - Scaling of objects is lost
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Perspective Projections 1/3' changes positions before rasterization?

Page 30	Perspective Projections 2/3
Source cue	- Vanishing points / - For rectangular objects (e.g., axis parallel cubes) whose surface normals / are parallel to the x-, y-, or z-axis, the number of vanishing points is equal / to the number of coordinate axes intersected by the projection plane / 1-Point Perspective 2-Point Perspective 3-Point Perspective
Professor-style explanation	The slide 'Perspective Projections 2/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Vanishing points / - For rectangular objects (e.g., axis parallel cubes) whose surface normals / are parallel to the x-, y-, or z-axis, the number of vanishing points is equal / to the number of coordinate axes intersected by the projection plane / 1-Point Perspective 2-Point Perspective 3-Point Perspective. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Perspective Projections 2/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Vanishing points / - For rectangular objects (e.g., axis parallel cubes) whose surface normals / are parallel to the x-, y-, or z-axis, the number of vanishing points is equal / to the number of coordinate axes intersected by the projection plane / 1-Point Perspective 2-Point Perspective 3-Point Perspective
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Perspective Projections 2/3'?

Page 31	Perspective Projections 3/3
Source cue	- What happens if the cube is rotated so that its surface normals are not / parallel to the coordinate axes? / - Although the projection plane intersects only one coordinate axis, in this case / you get 3 vanishing points
Professor-style explanation	The slide 'Perspective Projections 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - What happens if the cube is rotated so that its surface normals are not / parallel to the coordinate axes? / - Although the projection plane intersects only one coordinate axis, in this case / you get 3 vanishing points. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Perspective Projections 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - What happens if the cube is rotated so that its surface normals are not / parallel to the coordinate axes? / - Although the projection plane intersects only one coordinate axis, in this case / you get 3 vanishing points
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Perspective Projections 3/3'?
Page 32	4.3 Camera Modeling
Source cue	Orientation of the camera and definition of the view volume

Professor-style explanation	The slide '4.3 Camera Modeling' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: Orientation of the camera and definition of the view volume. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about 4.3 Camera Modeling. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: Orientation of the camera and definition of the view volume
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by '4.3 Camera Modeling' before rendering begins?

Page 33	Camera Modeling
Source cue	- Application of planar geometric projections, represented by 4×4 matrices, / to project a 3D scene onto a projection screen, i.e., the canvas / - Basic camera parameters / - Camera position / - Camera orientation
Professor-style explanation	The slide 'Camera Modeling' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Application of planar geometric projections, represented by 4×4 matrices, / to project a 3D scene onto a projection screen, i.e., the canvas / - Basic camera parameters / - Camera position / - Camera orientation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Camera Modeling. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Application of planar geometric projections, represented by 4×4 matrices, / to project a 3D scene onto a projection screen, i.e., the canvas / - Basic camera parameters / - Camera position / - Camera orientation
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Camera Modeling' before rendering begins?

Page 34	View Volume
---------	-------------

Source cue	- Objects that are within the view volume are visible for the camera / - Conical viewing volume / - Clipping the scene objects against a conical viewing volume is expensive / (solving quadratic equations) / - Frustum - rectangular viewing volume
Professor-style explanation	The slide 'View Volume' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Objects that are within the view volume are visible for the camera / - Conical viewing volume / - Clipping the scene objects against a conical viewing volume is expensive / (solving quadratic equations) / - Frustum - rectangular viewing volume. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about View Volume. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Objects that are within the view volume are visible for the camera / - Conical viewing volume / - Clipping the scene objects against a conical viewing volume is expensive / (solving quadratic equations) / - Frustum - rectangular viewing volume
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'View Volume'?

Page 35	Projection Specification
Source cue	- Specified by Height Angle, Aspect Ratio, / Near Plane Distance and Far Plane Distance / - Examples of Aspect Ratio / - NTSC 4 : 3, HDTV 16 : 9 / - Cinema 2.35 : 1, 2.39 : 1, 2.40 : 1, or 2.55 : 1
Professor-style explanation	The slide 'Projection Specification' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Specified by Height Angle, Aspect Ratio, / Near Plane Distance and Far Plane Distance / - Examples of Aspect Ratio / - NTSC 4 : 3, HDTV 16 : 9 / - Cinema 2.35 : 1, 2.39 : 1, 2.40 : 1, or 2.55 : 1. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Projection Specification. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Specified by Height Angle, Aspect Ratio, / Near Plane Distance and Far Plane Distance / - Examples of Aspect Ratio / - NTSC 4 : 3, HDTV 16 : 9 / - Cinema 2.35 : 1, 2.39 : 1, 2.40 : 1, or 2.55 : 1
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Specification' changes positions before rasterization?

Page 36	View Angle
Source cue	- Parallel projection: no viewing angle as rays are parallel / - Perspective projection: / viewing angle determines perspective distortion in the image / - Specification / - Through the viewing angle in height (field of view in y, fovy)
Professor-style explanation	The slide 'View Angle' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Parallel projection: no viewing angle as rays are parallel / - Perspective projection: / viewing angle determines perspective distortion in the image / - Specification / - Through the viewing angle in height (field of view in y, fovy). The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about View Angle. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Parallel projection: no viewing angle as rays are parallel / - Perspective projection: / viewing angle determines perspective distortion in the image / - Specification / - Through the viewing angle in height (field of view in y, fovy)
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'View Angle' works per object, per image region, per ray, or per fragment?

Page 37	Clipping Planes
Source cue	- Position of the planes is determined by the distance along the look vector: / near distance and far distance / - Near plane hides objects that are behind or too close to the camera / - Far plane hides objects that are too far away (and may be hard to spot) / $\Rightarrow$ Reduction of rendering overhead

Professor-style explanation	The slide 'Clipping Planes' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Position of the planes is determined by the distance along the look vector: / near distance and far distance / - Near plane hides objects that are behind or too close to the camera / - Far plane hides objects that are too far away (and may be hard to spot) / $\Rightarrow$ Reduction of rendering overhead. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Clipping Planes. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Position of the planes is determined by the distance along the look vector: / near distance and far distance / - Near plane hides objects that are behind or too close to the camera / - Far plane hides objects that are too far away (and may be hard to spot) / $\Rightarrow$ Reduction of rendering overhead
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Clipping Planes'?

Page 38	4.4 Specifying Projections in OpenGL
Source cue	Functions to describe the view volume
Professor-style explanation	The slide '4.4 Specifying Projections in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Functions to describe the view volume. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 4.4 Specifying Projections in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Functions to describe the view volume
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '4.4 Specifying Projections in OpenGL'?

Page 39	Projection Matrix Generation 1/3
Source cue	- Convenient generation of perspective projection matrix with GLM / - glm::perspective(fovy, aspect, near, far) / h -n -f / aspect ■ / fovy fovy
Professor-style explanation	The slide 'Projection Matrix Generation 1/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Convenient generation of perspective projection matrix with GLM / - glm::perspective(fovy, aspect, near, far) / h -n -f / aspect ■ / fovy fovy. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Projection Matrix Generation 1/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Convenient generation of perspective projection matrix with GLM / - glm::perspective(fovy, aspect, near, far) / h -n -f / aspect ■ / fovy fovy
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Projection Matrix Generation 1/3'?
Page 40	Projection Matrix Generation 2/3
Source cue	- Alternative generation of perspective projection matrix with GLM / - glm::frustum(left, right, bottom, top, near, far) / (r, t, -n) / z = -f (far plane) / (l, b, -n)
Professor-style explanation	The slide 'Projection Matrix Generation 2/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Alternative generation of perspective projection matrix with GLM / - glm::frustum(left, right, bottom, top, near, far) / (r, t, -n) / z = -f (far plane) / (l, b, -n). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Projection Matrix Generation 2/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Alternative generation of perspective projection matrix with GLM / - glm::frustum(left, right, bottom, top, near, far) / (r, t, -n) / z = -f (far plane) / (l, b, -n)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Projection Matrix Generation 2/3'?

Page 41	Projection Matrix Generation 3/3
Source cue	- Implementation of glm::perspective / detail::tmat4x4<T> perspective(T const & fovy, T const & aspect, / T const & near, T const & far) { / T const & l, r, b, t; / t = n * tan(fovy * M_PI / 360.0);
Professor-style explanation	The slide 'Projection Matrix Generation 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Implementation of glm::perspective / detail::tmat4x4<T> perspective(T const & fovy, T const & aspect, / T const & near, T const & far) { / T const & l, r, b, t; / t = n * tan(fovy * M_PI / 360.0). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Projection Matrix Generation 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Implementation of glm::perspective / detail::tmat4x4<T> perspective(T const & fovy, T const & aspect, / T const & near, T const & far) { / T const & l, r, b, t; / t = n * tan(fovy * M_PI / 360.0);
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Projection Matrix Generation 3/3'?

Page 42	Orthographic Matrix Specification
Source cue	- Convenient generation of parallel projection matrix with GLM / - glm::ortho(left, right, bottom, top, near, far) / (r, t, -f) / z = -f (far plane) / z = -n (near plane)

Professor-style explanation	The slide 'Orthographic Matrix Specification' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Convenient generation of parallel projection matrix with GLM / - <code>glm::ortho(left, right, bottom, top, near, far) / (r, t, -f) / z = -f (far plane) / z = -n (near plane)</code> . The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Orthographic Matrix Specification. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Convenient generation of parallel projection matrix with GLM / - <code>glm::ortho(left, right, bottom, top, near, far) / (r, t, -f) / z = -f (far plane) / z = -n (near plane)</code>
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Orthographic Matrix Specification'?

Page 43	4.5 Orthographic Projection Derivation
Source cue	Step-wise transformation of a cuboid into a cube
Professor-style explanation	The slide '4.5 Orthographic Projection Derivation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Step-wise transformation of a cuboid into a cube. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 4.5 Orthographic Projection Derivation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Step-wise transformation of a cuboid into a cube
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '4.5 Orthographic Projection Derivation'?

Page 44	Canonical View Volume
---------	-----------------------

Source cue	- Canonical view volume - simplest possible view volume / - Cube centered at the origin / - Cube faces $x = \pm 1, y = \pm 1, z = \pm 1$ / - Defined in a left-handed coordinate system (LHS) / (1, 1, 1)
Professor-style explanation	The slide 'Canonical View Volume' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Canonical view volume - simplest possible view volume / - Cube centered at the origin / - Cube faces $x = \pm 1, y = \pm 1, z = \pm 1$ / - Defined in a left-handed coordinate system (LHS) / (1, 1, 1). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Canonical View Volume. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Canonical view volume - simplest possible view volume / - Cube centered at the origin / - Cube faces $x = \pm 1, y = \pm 1, z = \pm 1$ / - Defined in a left-handed coordinate system (LHS) / (1, 1, 1)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Canonical View Volume'?
Page 45	Orthographic Projection 1/4
Source cue	- Transform right-handed view volume specified as $[l, r] \times [b, t] \times [-n, -f]$ / into left-handed canonical view volume - 1, 1 $\times$ - 1, 1 $\times$ - 1, 1 / and then create the projected image by neglecting z-values / (r, t, -f) / (1, 1, 1)
Professor-style explanation	The slide 'Orthographic Projection 1/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Transform right-handed view volume specified as $[l, r] \times [b, t] \times [-n, -f]$ / into left-handed canonical view volume - 1, 1 $\times$ - 1, 1 $\times$ - 1, 1 / and then create the projected image by neglecting z-values / (r, t, -f) / (1, 1, 1). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Orthographic Projection 1/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Transform right-handed view volume specified as $[l, r] \times [b, t] \times [-n, -f]$ / into left-handed canonical view volume $-1, 1 \times -1, 1 \times -1, 1$ / and then create the projected image by neglecting z-values / $(r, t, -f) / (1, 1, 1)$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Orthographic Projection 1/4'?

Page 46	Orthographic Projection 2/4
Source cue	- Let $p(x, y, z)$ be a model point and $q(x', y', z')$ its transformed point / - The desired pixel (x, y) results from orthographic projection of q / p / onto the plane $z = -1$: $x = x'$ and $y = y' / p$
Professor-style explanation	The slide 'Orthographic Projection 2/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Let $p(x, y, z)$ be a model point and $q(x', y', z')$ its transformed point / - The desired pixel (x, y) results from orthographic projection of q / p / onto the plane $z = -1$: $x = x'$ and $y = y' / p$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Orthographic Projection 2/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Let $p(x, y, z)$ be a model point and $q(x', y', z')$ its transformed point / - The desired pixel (x, y) results from orthographic projection of q / p / onto the plane $z = -1$: $x = x'$ and $y = y' / p$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Orthographic Projection 2/4'?

Page 47	Orthographic Projection 3/4
Source cue	- Step 2, scale cuboid from step 1 into cube / - Original cuboid bounds are $[r-l, r-l, t-b, t-b, f-n, f-n] / - , x-, x-, / 2 2 2 2 2 2$

Professor-style explanation	The slide 'Orthographic Projection 3/4' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Step 2, scale cuboid from step 1 into cube / - Original cuboid bounds are $r - l$ $r - l$ $t - b$ $t - b$ $f - n$ $f - n$ $/ - , x - , x - , / 2 2 2 2 2 2$. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Orthographic Projection 3/4. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Step 2, scale cuboid from step 1 into cube / - Original cuboid bounds are $r - l$ $r - l$ $t - b$ $t - b$ $f - n$ $f - n$ $/ - , x - , x - , / 2 2 2 2 2 2$
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Orthographic Projection 3/4' changes positions before rasterization?

Page 48	Orthographic Projection 4/4
Source cue	- Multiplication of all orthographic projection matrices results in $/ 2 r + l / 0$ $0 - / r - l r - l / 2 t + b$
Professor-style explanation	The slide 'Orthographic Projection 4/4' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Multiplication of all orthographic projection matrices results in $/ 2 r + l / 0$ $0 - / r - l r - l / 2 t + b$. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Orthographic Projection 4/4. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Multiplication of all orthographic projection matrices results in $/ 2 r + l / 0$ $0 - / r - l r - l / 2 t + b$
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.

Check yourself	Can you explain how 'Orthographic Projection 4/4' changes positions before rasterization?
Page 49	4.6 Perspective Projection Derivation
Source cue	Step-wise transformation of a frustum into a cube
Professor-style explanation	The slide '4.6 Perspective Projection Derivation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Step-wise transformation of a frustum into a cube. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 4.6 Perspective Projection Derivation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Step-wise transformation of a frustum into a cube
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '4.6 Perspective Projection Derivation'?
Page 50	Perspective Projection
Source cue	- Transform view volume created by <code>glm::frustum</code> or <code>glm::perspective</code> into / canonical view volume / and then create the projected image by neglecting z-values / (1, 1, 1) / (-1,-1,-1)
Professor-style explanation	The slide 'Perspective Projection' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Transform view volume created by <code>glm::frustum</code> or <code>glm::perspective</code> into / canonical view volume / and then create the projected image by neglecting z-values / (1, 1, 1) / (-1,-1,-1). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Perspective Projection. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Transform view volume created by glm::frustum or glm::perspective into / canonical view volume / and then create the projected image by neglecting z-values / (1, 1, 1) / (-1,-1,-1)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Perspective Projection'?

Page 51	Transformation of z-Values 1/2
Source cue	- After the perspective projection, z-values shall be mapped as follows / - All z-values lying on the near plane shall have a value of -1 / - All z-values lying on the far plane shall have a value of 1 / - The following matrix P allows modification of the depth values / and assures the flip (left-handed to right-handed conversion)
Professor-style explanation	The slide 'Transformation of z-Values 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - After the perspective projection, z-values shall be mapped as follows / - All z-values lying on the near plane shall have a value of -1 / - All z-values lying on the far plane shall have a value of 1 / - The following matrix P allows modification of the depth values / and assures the flip (left-handed to right-handed conversion). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Transformation of z-Values 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - After the perspective projection, z-values shall be mapped as follows / - All z-values lying on the near plane shall have a value of -1 / - All z-values lying on the far plane shall have a value of 1 / - The following matrix P allows modification of the depth values / and assures the flip (left-handed to right-handed conversion)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Transformation of z-Values 1/2'?

Page 52	Transformation of z-Values 2/2
Source cue	- Choosing values for a and b in P / - a and b must fulfill the following / $-an + b / P \cdot 0 \ 0 \ -n \ 1 \ T = 0 \ 0 \ -an + b \ n \ 0 \ 0 = 0 \ 0 \ -1 / \div w \ n$

Professor-style explanation	The slide 'Transformation of z-Values 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Choosing values for a and b in P / - a and b must fulfill the following $\begin{bmatrix} -a & b \\ P & 0 \end{bmatrix} \begin{bmatrix} 0 & 0 \\ -n & 1 \end{bmatrix} T = \begin{bmatrix} 0 & 0 \\ -a & b \end{bmatrix} \begin{bmatrix} n & T \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ -1 & \div w \end{bmatrix} n$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Transformation of z-Values 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Choosing values for a and b in P / - a and b must fulfill the following $\begin{bmatrix} -a & b \\ P & 0 \end{bmatrix} \begin{bmatrix} 0 & 0 \\ -n & 1 \end{bmatrix} T = \begin{bmatrix} 0 & 0 \\ -a & b \end{bmatrix} \begin{bmatrix} n & T \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ -1 & \div w \end{bmatrix} n$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Transformation of z-Values 2/2'?

Page 53	Transformation of x- and y-Values 1/2
Source cue	- Derivation so far leaves us with the following matrix / which only changes z-coordinates $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$ / - $P = f+n \cdot 2nf$
Professor-style explanation	The slide 'Transformation of x- and y-Values 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Derivation so far leaves us with the following matrix / which only changes z-coordinates $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$ / - $P = f+n \cdot 2nf$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Transformation of x- and y-Values 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Derivation so far leaves us with the following matrix / which only changes z-coordinates $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$ / - $P = f+n \cdot 2nf$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Transformation of x- and y-Values 1/2'?

Page 54	Transformation of x- and y-Values 2/2
Source cue	- To transform p to the coordinates (1,0,-1), the following equation must hold / $\alpha \tan / \alpha n \cdot \tan / n \cdot \tan 2 / 2 2 1$
Professor-style explanation	The slide 'Transformation of x- and y-Values 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - To transform p to the coordinates (1,0,-1), the following equation must hold / $\alpha \tan / \alpha n \cdot \tan / n \cdot \tan 2 / 2 2 1$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Transformation of x- and y-Values 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - To transform p to the coordinates (1,0,-1), the following equation must hold / $\alpha \tan / \alpha n \cdot \tan / n \cdot \tan 2 / 2 2 1$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Transformation of x- and y-Values 2/2'?
Page 55	4.7 Viewport Transformation
Source cue	Mapping the normalized device coordinates to the screen
Professor-style explanation	The slide '4.7 Viewport Transformation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Mapping the normalized device coordinates to the screen. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 4.7 Viewport Transformation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Mapping the normalized device coordinates to the screen
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.

Check yourself	Can you state the coordinate space before and after '4.7 Viewport Transformation'?
Page 56	Screen Coordinate Mapping
Source cue	- Mapping of the viewing plane on a drawing surface / - Define the pixel rectangle to be mapped to / - Initially, this rectangle is the size of the entire canvas / - <code>glViewport(GLint x, GLint y, GLsizei width, GLsizei height)</code> / - When window size changes, adjust the aspect ratio to avoid distortion
Professor-style explanation	The slide 'Screen Coordinate Mapping' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Mapping of the viewing plane on a drawing surface / - Define the pixel rectangle to be mapped to / - Initially, this rectangle is the size of the entire canvas / - <code>glViewport(GLint x, GLint y, GLsizei width, GLsizei height)</code> / - When window size changes, adjust the aspect ratio to avoid distortion. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Screen Coordinate Mapping. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Mapping of the viewing plane on a drawing surface / - Define the pixel rectangle to be mapped to / - Initially, this rectangle is the size of the entire canvas / - <code>glViewport(GLint x, GLint y, GLsizei width, GLsizei height)</code> / - When window size changes, adjust the aspect ratio to avoid distortion
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Screen Coordinate Mapping'?
Page 57	Viewport Transformation 1/2
Source cue	- Specification of the viewport parameters by <code>glm::perspective</code> / Projection / <code>glm::frustum</code> / transformation / - <code>glViewport(x, y, w, h)</code> <code>glm::ortho</code>

Professor-style explanation	The slide 'Viewport Transformation 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Specification of the viewport parameters by <code>glm::perspective / Projection / glm::frustum / transformation / - glViewport(x, y, w, h) glm::ortho</code> . The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Viewport Transformation 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Specification of the viewport parameters by <code>glm::perspective / Projection / glm::frustum / transformation / - glViewport(x, y, w, h) glm::ortho</code>
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Viewport Transformation 1/2'?

Page 58	Viewport Transformation 2/2
Source cue	- Viewport transformation is a 2D transformation defined as $p = M \cdot p$ with $\begin{matrix} / & s & s & N & / & \Delta x & \Delta y & / & - & M = & T & x, & y & \cdot & S & S, & S & \cdot & T & -x, & -y & / & s & S_{min} & S_{min} & N_{min} & N_{min} \end{matrix}$
Professor-style explanation	The slide 'Viewport Transformation 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Viewport transformation is a 2D transformation defined as $p = M \cdot p$ with $\begin{matrix} / & s & s & N & / & \Delta x & \Delta y & / & - & M = & T & x, & y & \cdot & S & S, & S & \cdot & T & -x, & -y & / & s & S_{min} & S_{min} & N_{min} & N_{min} \end{matrix}$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Viewport Transformation 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Viewport transformation is a 2D transformation defined as $p = M \cdot p$ with $\begin{matrix} / & s & s & N & / & \Delta x & \Delta y & / & - & M = & T & x, & y & \cdot & S & S, & S & \cdot & T & -x, & -y & / & s & S_{min} & S_{min} & N_{min} & N_{min} \end{matrix}$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.

Check yourself	Can you state the coordinate space before and after 'Viewport Transformation 2/2'?
Page 59	volume, we can differentiate different projection types
Source cue	- Projection has its origins in the arts / - Based on the deformation of the initial view volume to the canonical view / volume, we can differentiate different projection types / - In OpenGL two main projection types exist / - Perspective projection results in perspective shortening
Professor-style explanation	The slide 'volume, we can differentiate different projection types' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Projection has its origins in the arts / - Based on the deformation of the initial view volume to the canonical view / volume, we can differentiate different projection types / - In OpenGL two main projection types exist / - Perspective projection results in perspective shortening. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about volume, we can differentiate different projection types. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Projection has its origins in the arts / - Based on the deformation of the initial view volume to the canonical view / volume, we can differentiate different projection types / - In OpenGL two main projection types exist / - Perspective projection results in perspective shortening
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'volume, we can differentiate different projection types'?
Page 60	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.

Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?

Page 61	- Text Books
Source cue	- J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice (3rd / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics
Professor-style explanation	The slide '- Text Books' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice (3rd / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about - Text Books. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice (3rd / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '- Text Books' into a causal sentence instead of repeating the slide title?

04.1 Linear Perspective and Planar Projections

Source location: Sections 4.1-4.2

Commentary

Perspective projection models the visual effect that farther objects appear smaller. Orthographic projection removes this distance-based size change and keeps parallel lines parallel. Both are useful, but they communicate different spatial relationships.

Planar projection means mapping points onto an image plane. The lecture's art examples are not decoration; they show that projection is a geometric rule for constructing an image from a viewpoint.

Mental model

Projection is the rule that turns 3D positions into image-plane positions.

Check yourself

What visual cue does perspective projection add that orthographic projection lacks?

04.2 Camera Modeling

Source location: Section 4.3

Commentary

A virtual camera is defined by position, orientation, projection type, viewing volume, and image/viewport settings. The view transform places the world relative to the camera. The projection transform maps that view into clip coordinates.

Near and far clipping planes are part of the camera model. They decide what range of depths is represented and strongly affect depth buffer precision. A careless near/far setup can create z-fighting even if the scene geometry is correct.

Mental model

A camera is not only an eye position; it is also a volume and a mapping rule.

Check yourself

Why can changing the near plane affect depth artifacts?

04.3 Specifying Projections in OpenGL

Source location: Section 4.4

Commentary

In OpenGL, projection is usually encoded in a projection matrix sent to a shader. The matrix maps view-space coordinates to clip space. Clipping and the perspective divide then lead toward normalized device coordinates.

Aspect ratio and field of view must match the intended viewport. If the aspect ratio is wrong, objects appear stretched. If the field of view is extreme, the image can look distorted even though the math is functioning.

Mental model

The projection matrix is the camera lens encoded as linear algebra in homogeneous coordinates.

Check yourself

What symptom suggests that the projection aspect ratio does not match the window?

04.4 Orthographic and Perspective Derivations

Source location: Sections 4.5-4.6

Commentary

The derivations are there to explain where the matrix entries come from. Orthographic projection maps a box-shaped view volume into normalized coordinates. Perspective projection maps a frustum and uses the homogeneous component for the later divide.

You do not need to treat these matrices as random formulas. Each part maps a coordinate range into a standard range. The perspective matrix additionally arranges w so that the perspective divide creates foreshortening.

Mental model

Projection matrices normalize a viewing volume; perspective also prepares the divide by w .

Check yourself

What does the perspective divide do to x and y coordinates?

04.5 Viewport Transformation

Source location: Section 4.7

Commentary

After normalized device coordinates, the viewport transform maps the normalized range to actual window coordinates. This is the last geometric mapping before rasterization uses the target grid.

Separating projection from viewport mapping helps debugging. A wrong projection can produce wrong normalized coordinates; a wrong viewport can map otherwise valid coordinates into the wrong part of the window.

Mental model

Projection decides normalized position; viewport decides where that position lands on the screen.

Check yourself

Why is resizing a window related to both viewport and projection settings?

End-of-lecture summary

If you remember only one thing from Lecture 04, remember this: This lecture explains how a 3D view becomes a 2D image. Projection is where camera geometry, homogeneous coordinates, clipping planes, depth precision, and viewport mapping meet.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 05 - Clipping

This lecture focuses on algorithms that remove or trim geometry against boundaries. Clipping is a geometric operation before rasterization, not a visibility test between overlapping objects.

Source chunk	course_text_parts/03_lectures/05_clipping.txt
Extracted slide pages	56
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Clipping Projected Geometry 1/2
Source cue	- Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized

Professor-style explanation	The slide 'Clipping Projected Geometry 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Clipping Projected Geometry 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Now that the geometry has been transformed to its representation on the / screen, parts outside screen must be clipped away / - Clipping ensures that following costly computations / (e.g., rasterization, illumination, shading) are only applied to visible parts / - Since clipping is performed for all geometry, algorithms are highly optimized
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Clipping Projected Geometry 1/2'?

Page 3	Clipping Projected Geometry 2/2
Source cue	- Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping
Professor-style explanation	The slide 'Clipping Projected Geometry 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Clipping Projected Geometry 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Analytical clipping / - Clipping before raster conversion, clipped primitives are raster-converted / - Compute time savings if clipped primitive is much smaller than unclipped / - OpenGL: clipping against canonical view volume / - Pixel-based clipping
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Clipping Projected Geometry 2/2'?

Page 4	5.1 Cohen-Sutherland Line Clipping
Source cue	5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping
Professor-style explanation	The slide '5.1 Cohen-Sutherland Line Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: 5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.1 Cohen-Sutherland Line Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: 5.2 Cyrus-Beck Line Clipping / 5.3 Sutherland-Hodgman Polygon Clipping / 5.4 Weiler-Atherton Polygon Clipping / 5.5 Greiner-Hormann Polygon Clipping
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.1 Cohen-Sutherland Line Clipping'?

Page 5	5.1 Cohen-Sutherland Line Clipping
Source cue	Clip lines against rectangle

Professor-style explanation	The slide '5.1 Cohen-Sutherland Line Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: Clip lines against rectangle. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.1 Cohen-Sutherland Line Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: Clip lines against rectangle
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.1 Cohen-Sutherland Line Clipping'?

Page 6	Brute-Force Line Clipping 1/2
Source cue	- Check each edge of the rectangle for intersection with line / - Intersection calculation for the line defined by the edge, and the line / - If intersection lies within edge and within line, clipping necessary
Professor-style explanation	The slide 'Brute-Force Line Clipping 1/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Check each edge of the rectangle for intersection with line / - Intersection calculation for the line defined by the edge, and the line / - If intersection lies within edge and within line, clipping necessary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Brute-Force Line Clipping 1/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Check each edge of the rectangle for intersection with line / - Intersection calculation for the line defined by the edge, and the line / - If intersection lies within edge and within line, clipping necessary
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Brute-Force Line Clipping 1/2'?

Page 7	Brute-Force Line Clipping 2/2
Source cue	- Using the parameter representation for lines $x = x_0 + t \cdot (x_1 - x_0)$ and $y = y_0 + t \cdot (y_1 - y_0)$, $p = p_0 + t \cdot (p_1 - p_0)$, $p / 0 \neq 0$ $y - y_1$

Professor-style explanation	The slide 'Brute-Force Line Clipping 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Using the parameter representation for lines $x - x_0 = t(x_1 - x_0)$ / $y - y_0 = t(y_1 - y_0)$ / $p = p_0 + t(p_1 - p_0)$. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Brute-Force Line Clipping 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Using the parameter representation for lines $x - x_0 = t(x_1 - x_0)$ / $y - y_0 = t(y_1 - y_0)$ / $p = p_0 + t(p_1 - p_0)$.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Brute-Force Line Clipping 2/2'?

Page 8	Endpoint Analysis 1/2
Source cue	- Idea: Use end point analysis to create new lines / - Clipping against a rectangle yields a line / - Clipping against a convex polygon results in a line / - Endpoint analysis / - Line segment $s = (p_0, p_1)$, $p = (x, y)$ and $p = (x, y)$
Professor-style explanation	The slide 'Endpoint Analysis 1/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Idea: Use end point analysis to create new lines / - Clipping against a rectangle yields a line / - Clipping against a convex polygon results in a line / - Endpoint analysis / - Line segment $s = (p_0, p_1)$, $p = (x, y)$ and $p = (x, y)$. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Endpoint Analysis 1/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Idea: Use end point analysis to create new lines / - Clipping against a rectangle yields a line / - Clipping against a convex polygon results in a line / - Endpoint analysis / - Line segment $s = (p_0, p_1)$, $p = (x, y)$ and $p = (x, y)$
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Endpoint Analysis 1/2'?

Page 9	Endpoint Analysis 2/2
Source cue	- Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of x / min max / and above y and below y , they are inside / min max
Professor-style explanation	The slide 'Endpoint Analysis 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of x / min max / and above y and below y , they are inside / min max. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Endpoint Analysis 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Clipping lines against a clip rectangle by reduction to endpoint analysis / - If both endpoints are to the right of x and to the left of x / min max / and above y and below y , they are inside / min max
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Endpoint Analysis 2/2'?
Page 10	Binary Coding of Half-Planes 1/3
Source cue	- Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode
Professor-style explanation	The slide 'Binary Coding of Half-Planes 1/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Binary Coding of Half-Planes 1/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Classification of the plane in nine regions / - Classify a point by one bit each / for belonging to each of the 4 half-planes / 1001 1000 1010 / - 4-bit outcode
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Binary Coding of Half-Planes 1/3' into a causal sentence instead of repeating the slide title?
Page 11	Binary Coding of Half-Planes 2/3
Source cue	- Endpoint classification / - Line segment $s = (p, p) / 01$ / - Outcodes: / 1001 1000 1010
Professor-style explanation	The slide 'Binary Coding of Half-Planes 2/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Endpoint classification / - Line segment $s = (p, p) / 01$ / - Outcodes: / 1001 1000 1010. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Binary Coding of Half-Planes 2/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Endpoint classification / - Line segment $s = (p, p) / 01$ / - Outcodes: / 1001 1000 1010
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Binary Coding of Half-Planes 2/3'?
Page 12	Binary Coding of Half-Planes 3/3
Source cue	- Clipping, if $(oc0 \& oc1) = 0$: / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip s at this clip edge
Professor-style explanation	The slide 'Binary Coding of Half-Planes 3/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Clipping, if $(oc0 \& oc1) = 0$: / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip s at this clip edge. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about Binary Coding of Half-Planes 3/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Clipping, if (oc0 & oc1) = 0: / - Choose an order for clipping (e.g., top, bottom, right, left) / - Select clip edge that is crossed by the line / - Check done on basis of bit codes, which differ in at least 1 bit / - Clip s at this clip edge
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Binary Coding of Half-Planes 3/3'?

Page 13	Pseudo Code
Source cue	<pre>void cohensutherlandLineClipper(Line s, Rectangle r) { int oc0 = outcode(s.p0, r); int oc1 = outcode(s.p1, r); bool done = false; while (!done) { int outcode(Point p, Rectangle r) {</pre>
Professor-style explanation	The slide 'Pseudo Code' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: <pre>void cohensutherlandLineClipper(Line s, Rectangle r) { int oc0 = outcode(s.p0, r); int oc1 = outcode(s.p1, r); bool done = false; while (!done) { int outcode(Point p, Rectangle r) {</pre> The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Pseudo Code. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: <pre>void cohensutherlandLineClipper(Line s, Rectangle r) { int oc0 = outcode(s.p0, r); int oc1 = outcode(s.p1, r); bool done = false; while (!done) { int outcode(Point p, Rectangle r) {</pre>
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pseudo Code'?

Page 14	Example
Source cue	1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110

Professor-style explanation	The slide 'Example' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: 1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Example. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: 1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example' into a causal sentence instead of repeating the slide title?
Page 15	Evaluation
Source cue	- Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside
Professor-style explanation	The slide 'Evaluation' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Evaluation. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Occasionally performs several (unnecessary) clip operations on a line as test / and clippings are done in a fixed edge order / - Especially efficient when / - almost all lines are outside / - almost all lines are inside
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Evaluation'?
Page 16	5.2 Cyrus-Beck Line Clipping
Source cue	Clip lines against convex polygon

Professor-style explanation	The slide '5.2 Cyrus-Beck Line Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: Clip lines against convex polygon. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.2 Cyrus-Beck Line Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: Clip lines against convex polygon
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.2 Cyrus-Beck Line Clipping'?

Page 17	Cyrus-Beck Line Clipping
Source cue	- Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky
Professor-style explanation	The slide 'Cyrus-Beck Line Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Cyrus-Beck Line Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Introduced in 1978 by Cyrus and Beck / - Algorithm for clipping lines against a convex polygon in the plane / - Expandable to clipping lines against a convex polyhedron in space / - Fundamentally different approach than Cohen-Sutherland / - Further development in 1984 by Liang and Barsky
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.

Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Cyrus-Beck Line Clipping'?
Page 18	Parametric Line Representation
Source cue	- Parametric line representation written as follows / $p \cdot t = p + t \cdot p - p$, $0 \leq t \leq 1$ / 0 1 0 / - Proceeding / - Find the 4 t's for the 4 clip edges
Professor-style explanation	The slide 'Parametric Line Representation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Parametric line representation written as follows / $p \cdot t = p + t \cdot p - p$, $0 \leq t \leq 1$ / 0 1 0 / - Proceeding / - Find the 4 t's for the 4 clip edges. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Parametric Line Representation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Parametric line representation written as follows / $p \cdot t = p + t \cdot p - p$, $0 \leq t \leq 1$ / 0 1 0 / - Proceeding / - Find the 4 t's for the 4 clip edges
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Parametric Line Representation'?
Page 19	Intersection Point Calculation 1/2
Source cue	- Consider edge E and its outward pointing edge normal N / - Through dot product $N \cdot (p(t) - p)$ it can be decided / in which half-space a line point $p(t)$ lies / $p \cdot N \blacksquare (p(t) - p) < 0$ / E E
Professor-style explanation	The slide 'Intersection Point Calculation 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Consider edge E and its outward pointing edge normal N / - Through dot product $N \cdot (p(t) - p)$ it can be decided / in which half-space a line point $p(t)$ lies / $p \cdot N \blacksquare (p(t) - p) < 0$ / E E. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Intersection Point Calculation 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Consider edge E and its outward pointing edge normal N / - Through dot product $N \cdot (p(t) - p)$ it can be decided / in which half-space a line point $p(t)$ lies / $p \cdot N \blacksquare (p(t) - p) < 0 / E$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Intersection Point Calculation 1/2'?

Page 20	Intersection Point Calculation 2/2
Source cue	- Determine intersection by calculating t for $N \cdot p \cdot t - p = 0$ / - Substitution of p t and regrouping gives $N \cdot p - p / 0 E$
Professor-style explanation	The slide 'Intersection Point Calculation 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Determine intersection by calculating t for $N \cdot p \cdot t - p = 0$ / - Substitution of p t and regrouping gives $N \cdot p - p / 0 E$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Intersection Point Calculation 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Determine intersection by calculating t for $N \cdot p \cdot t - p = 0$ / - Substitution of p t and regrouping gives $N \cdot p - p / 0 E$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Point Calculation 2/2' into a causal sentence instead of repeating the slide title?

Page 21	Determine Relevant t-Values
Source cue	- Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / s PE / PL PL PL / p s
Professor-style explanation	The slide 'Determine Relevant t-Values' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / s PE / PL PL PL / p s. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Determine Relevant t-Values. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Classification of intersection points as potentially entering (PE) / or potentially leaving (PL) with respect to interior of half-plane edge / s PE / PL PL PL / p s
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Determine Relevant t-Values' into a causal sentence instead of repeating the slide title?
Page 22	Pseudo Code
Source cue	<code>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</code>
Professor-style explanation	The slide 'Pseudo Code' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: <code>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</code> . The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Pseudo Code. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: <code>void cyrusBeckLineClipper(Line s, Rectangle r) { / precalculate N for edges of r and select P / i E / D = s.p1 - s.p0 / if (s.p0 == s.p1) {</code>
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pseudo Code'?
Page 23	Axis-Parallel Clipping Region
Source cue	- When clipping region is axis parallel and given by $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$, / min max min max / following values occur / $N \cdot p - p / 0 E$

Professor-style explanation	The slide 'Axis-Parallel Clipping Region' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - When clipping region is axis parallel and given by $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$, / min max min max / following values occur / $N \cdot p - p / 0$ E. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Axis-Parallel Clipping Region. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - When clipping region is axis parallel and given by $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$, / min max min max / following values occur / $N \cdot p - p / 0$ E
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Axis-Parallel Clipping Region'?

Page 24	Comparison with Cohen-Sutherland
Source cue	- Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected
Professor-style explanation	The slide 'Comparison with Cohen-Sutherland' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Comparison with Cohen-Sutherland. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Algorithm of Cohen-Sutherland / - Sometimes performs useless clip operations, / as clipping and testing are done in a fixed order / - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise / operations) and the majority of line segments are trivially accepted or rejected

Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Comparison with Cohen-Sutherland'?
Page 25	5.3 Sutherland-Hodgman Polygon Clipping
Source cue	A pipelined version of the scalar product algorithm extended to clip polygons
Professor-style explanation	The slide '5.3 Sutherland-Hodgman Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: A pipelined version of the scalar product algorithm extended to clip polygons. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.3 Sutherland-Hodgman Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: A pipelined version of the scalar product algorithm extended to clip polygons
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.3 Sutherland-Hodgman Polygon Clipping'?
Page 26	Sutherland-Hodgman Polygon Clipping 1/2
Source cue	- Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping
Professor-style explanation	The slide 'Sutherland-Hodgman Polygon Clipping 1/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about Sutherland-Hodgman Polygon Clipping 1/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Background / - General polygon clipping (arbitrary polygon against convex polygon) / - Originally developed for visibility detection / (apply before rasterization to eliminate non-visible polygon pieces) / - Polygon clipping is more complex than line clipping
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Sutherland-Hodgman Polygon Clipping 1/2'?

Page 27	Sutherland-Hodgman Polygon Clipping 2/2
Source cue	- Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region
Professor-style explanation	The slide 'Sutherland-Hodgman Polygon Clipping 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Sutherland-Hodgman Polygon Clipping 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Iterative algorithm paradigm / - Clipping a polygon takes place against a half-plane / - Clipping is performed sequentially for each edge of the clipping region / - Input / - Convex clipping region
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Sutherland-Hodgman Polygon Clipping 2/2'?

Page 28	Four Cases
Source cue	- Depending on which case occurs, following points added to output list / - Case 1: p / - Case 2: Intersection point i / - Case 3: -- / - Case 4: Intersection point i and p

Professor-style explanation	The slide 'Four Cases' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Depending on which case occurs, following points added to output list / - Case 1: p / - Case 2: Intersection point i / - Case 3: -- / - Case 4: Intersection point i and p. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Four Cases. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Depending on which case occurs, following points added to output list / - Case 1: p / - Case 2: Intersection point i / - Case 3: -- / - Case 4: Intersection point i and p
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Four Cases' into a causal sentence instead of repeating the slide title?

Page 29	Proceeding
Source cue	- Polygon to be clipped is defined by / - Vertices: $[v_1, v_2, \dots, v_n]$ / 1 2 n / - Contour: $[v_1, v_2, \dots, v_n, v_1]$ / 1 2 n 1
Professor-style explanation	The slide 'Proceeding' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Polygon to be clipped is defined by / - Vertices: $[v_1, v_2, \dots, v_n]$ / 1 2 n / - Contour: $[v_1, v_2, \dots, v_n, v_1]$ / 1 2 n 1. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Proceeding. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Polygon to be clipped is defined by / - Vertices: $[v_1, v_2, \dots, v_n]$ / 1 2 n / - Contour: $[v_1, v_2, \dots, v_n, v_1]$ / 1 2 n 1
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?

Page 30	Pipeline Version
Source cue	- Speed c can be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware

Professor-style explanation	The slide 'Pipeline Version' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Speed c an be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Pipeline Version. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Speed c an be optimized through pipeline version / - Input: sequence of polygon vertices processed by "Pipeline of Clippers" / - Passing on to the next level once a new vertex has been identified / - No cache is needed during clipping / - Suitable for implementation by hardware
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Pipeline Version'?

Page 31	Pseudo Code
Source cue	<pre>public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) {</pre>
Professor-style explanation	The slide 'Pseudo Code' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) {. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Pseudo Code. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: public VertexList shClipper(VertexList inputVertices, Edge clipE) { / VertexList outputVertices; // output list of vertices / int N = inputVertices.size(); // number of vertices to be processed / Vertex s = inputVertices.get(N-1); // get last vertex / for (int j = 0; j < N; j++) {

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Pseudo Code' into a causal sentence instead of repeating the slide title?
Page 32	Inside Test
Source cue	- Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top
Professor-style explanation	The slide 'Inside Test' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Inside Test. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Inside test for rectangular clip regions / boolean inside(Vertex v, Edge e) { // e is edge of a clip rectangle / if (e.v1.x > e.v0.x) // bottom / if (v.y >= e.v0.y) return true; / else if (e.v1.x < e.v0.x) // top
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Inside Test' into a causal sentence instead of repeating the slide title?
Page 33	Intersection Computation
Source cue	- For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y;
Professor-style explanation	The slide 'Intersection Computation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Intersection Computation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - For rectangular clip regions / Vertex intersect(Vertex p, Vertex s, Edge e) { / Vertex i; / if (e.v0.y == e.v1.y) { // horizontal edge / i.y = e.v0.y;

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Computation' into a causal sentence instead of repeating the slide title?
Page 34	5.4 Weiler-Atherton Polygon Clipping
Source cue	Clip arbitrary polygons against arbitrary clip polygons
Professor-style explanation	The slide '5.4 Weiler-Atherton Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: Clip arbitrary polygons against arbitrary clip polygons. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.4 Weiler-Atherton Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: Clip arbitrary polygons against arbitrary clip polygons
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.4 Weiler-Atherton Polygon Clipping'?
Page 35	Weiler-Atherton Polygon Clipping
Source cue	- Basic idea: edges of the clip polygon A and the polygon B to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray)
Professor-style explanation	The slide 'Weiler-Atherton Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Basic idea: edges of the clip polygon A and the polygon B to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray). The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about Weiler-Atherton Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Basic idea: edges of the clip polygon A and the polygon B to be clipped / divide the plane into disjoint regions / - Following four region types are distinguished / 1. Only-in-A (light blue) / 2. Only-in-B (gray)
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Polygon Clipping'?
Page 36	Example Run
Source cue	- Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of A and B / - Double all edge segments, and label each segment / according to the adjacent region
Professor-style explanation	The slide 'Example Run' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of A and B / - Double all edge segments, and label each segment / according to the adjacent region. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Example Run. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Proceeding best illustrated by example run / - Given case: two partially overlapping polygons / - Insert all intersections of A and B / - Double all edge segments, and label each segment / according to the adjacent region
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example Run' into a causal sentence instead of repeating the slide title?
Page 37	Intersection Types
Source cue	- There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal)

Professor-style explanation	The slide 'Intersection Types' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Intersection Types. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - There are two intersection types that are distinguished / - Transversal intersections / - Two edges of the two polygons intersect at one point / - Procedure: intersection is inserted in vertex list of both polygons / - Segment overlaps (non-transversal)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Types' into a causal sentence instead of repeating the slide title?

Page 38	Contour Processing
Source cue	- In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of A / and insert the contour edges of B one after the other
Professor-style explanation	The slide 'Contour Processing' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of A / and insert the contour edges of B one after the other. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Contour Processing. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - In order to find the regions-enclosing contours, / one must eliminate overlaps of the contours / - Transversal intersections / - Take the contour edges of A / and insert the contour edges of B one after the other
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Contour Processing' into a causal sentence instead of repeating the slide title?

Page 39	Evaluation
Source cue	- Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion
Professor-style explanation	The slide 'Evaluation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Evaluation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Extension to multiple polygons / - Efficiently possible through iterative application / - Alternatively, intersection points can be generated / for all polygons in in the first step / - Conclusion
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Evaluation' into a causal sentence instead of repeating the slide title?
Page 40	5.5 Greiner-Hormann Polygon Clipping
Source cue	Exploitation of the winding number to clip arbitrary polygons
Professor-style explanation	The slide '5.5 Greiner-Hormann Polygon Clipping' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: Exploitation of the winding number to clip arbitrary polygons. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 5.5 Greiner-Hormann Polygon Clipping. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: Exploitation of the winding number to clip arbitrary polygons
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '5.5 Greiner-Hormann Polygon Clipping'?
Page 41	Greiner-Hormann

Source cue	- Basic Idea: clipped region is made up of all edges of S that lie in C, / combined with all edges of C that lie in S / [Greiner & Hormann, TOG 1998]
Professor-style explanation	The slide 'Greiner-Hormann' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Basic Idea: clipped region is made up of all edges of S that lie in C, / combined with all edges of C that lie in S / [Greiner & Hormann, TOG 1998]. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Greiner-Hormann. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Basic Idea: clipped region is made up of all edges of S that lie in C, / combined with all edges of C that lie in S / [Greiner & Hormann, TOG 1998]
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Greiner-Hormann'?
Page 42	Chalk Wagon Metaphor
Source cue	- Using a chalk wagon metaphor / - Drive along all edges of A with a chalk wagon / that is only open when the current position is within B / - Drive along all edges of B with a chalk wagon / that is only open when the current position is within A
Professor-style explanation	The slide 'Chalk Wagon Metaphor' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Using a chalk wagon metaphor / - Drive along all edges of A with a chalk wagon / that is only open when the current position is within B / - Drive along all edges of B with a chalk wagon / that is only open when the current position is within A. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Chalk Wagon Metaphor. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Using a chalk wagon metaphor / - Drive along all edges of A with a chalk wagon / that is only open when the current position is within B / - Drive along all edges of B with a chalk wagon / that is only open when the current position is within A

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Chalk Wagon Metaphor' into a causal sentence instead of repeating the slide title?
Page 43	Contour Detection
Source cue	- The edges of polygon A within polygon B can thus be found as follows / - Insert in A all intersection points with B / - Start at any vertex v of A / - If v is within B, the edge flag is turned on, otherwise not / - Traverse edges of A and invert edge marker flag when intersecting with B
Professor-style explanation	The slide 'Contour Detection' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - The edges of polygon A within polygon B can thus be found as follows / - Insert in A all intersection points with B / - Start at any vertex v of A / - If v is within B, the edge flag is turned on, otherwise not / - Traverse edges of A and invert edge marker flag when intersecting with B. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Contour Detection. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - The edges of polygon A within polygon B can thus be found as follows / - Insert in A all intersection points with B / - Start at any vertex v of A / - If v is within B, the edge flag is turned on, otherwise not / - Traverse edges of A and invert edge marker flag when intersecting with B
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Contour Detection' into a causal sentence instead of repeating the slide title?
Page 44	Set Operations
Source cue	[Greiner & Hormann, TOG 1998]
Professor-style explanation	The slide 'Set Operations' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: [Greiner & Hormann, TOG 1998]. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about Set Operations. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: [Greiner & Hormann, TOG 1998]
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Set Operations'?

Page 45	Data Structure 1/2
Source cue	- Polygons exist as a closed, double-linked list / $P, P, P, \dots, P = P / 0 \ 1 \ 2 \ n \ 0$ / - The following segments are included / $P \ P, P \ P, \dots, P \ P = P \ P$
Professor-style explanation	The slide 'Data Structure 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Polygons exist as a closed, double-linked list / $P, P, P, \dots, P = P / 0 \ 1 \ 2 \ n \ 0$ / - The following segments are included / $P \ P, P \ P, \dots, P \ P = P \ P$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Data Structure 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Polygons exist as a closed, double-linked list / $P, P, P, \dots, P = P / 0 \ 1 \ 2 \ n \ 0$ / - The following segments are included / $P \ P, P \ P, \dots, P \ P = P \ P$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Data Structure 1/2' into a causal sentence instead of repeating the slide title?

Page 46	Data Structure 2/2
Source cue	[Greiner & Hormann, TOG 1998]
Professor-style explanation	The slide 'Data Structure 2/2' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: [Greiner & Hormann, TOG 1998]. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Data Structure 2/2. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: [Greiner & Hormann, TOG 1998]

Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Data Structure 2/2'?
Page 47	Proceeding
Source cue	- Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule)
Professor-style explanation	The slide 'Proceeding' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Proceeding. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Create data structure / - Create doubly linked list with node data structure / - Add the intersections with the corresponding alpha values to the list / - Traverse the polygon to find entry and exit points / (use winding number rule)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?
Page 48	Pseudo Code
Source cue	while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current)
Professor-style explanation	The slide 'Pseudo Code' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Pseudo Code. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: while unprocessed intersections in subject polygon / current = first unprocessed intersecting / point of subject polygon / newPolygon() / newVertex(current)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Pseudo Code' into a causal sentence instead of repeating the slide title?

Page 49	Winding Number 1/3
Source cue	- Winding number of a point A and a closed curve γ / indicates the number of times a ray that traverses the entire curve from A / winds around A / $\omega \gamma$, $A = d\phi$ / - Counterclockwise: +1
Professor-style explanation	The slide 'Winding Number 1/3' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Winding number of a point A and a closed curve γ / indicates the number of times a ray that traverses the entire curve from A / winds around A / $\omega \gamma$, $A = d\phi$ / - Counterclockwise: +1. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Winding Number 1/3. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Winding number of a point A and a closed curve γ / indicates the number of times a ray that traverses the entire curve from A / winds around A / $\omega \gamma$, $A = d\phi$ / - Counterclockwise: +1
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Winding Number 1/3' works per object, per image region, per ray, or per fragment?

Page 50	Winding Number 2/3
Source cue	- The winding number of a point A has the following useful properties / 1. If A or the curve γ is continuously changed so that A maintains a / positive distance to γ , the winding number will not change / For curve γ , the winding number is constant in each region formed by γ / 2. When A moves and γ crosses once, winding number changes by exactly 1

Professor-style explanation	The slide 'Winding Number 2/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - The winding number of a point A has the following useful properties / 1. If A or the curve γ is continuously changed so that A maintains a / positive distance to γ , the winding number will not change / For curve γ , the winding number is constant in each region formed by γ / 2. When A moves and γ crosses once, winding number changes by exactly 1. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Winding Number 2/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - The winding number of a point A has the following useful properties / 1. If A or the curve γ is continuously changed so that A maintains a / positive distance to γ , the winding number will not change / For curve γ , the winding number is constant in each region formed by γ / 2. When A moves and γ crosses once, winding number changes by exactly 1
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Winding Number 2/3' into a causal sentence instead of repeating the slide title?

Page 51	Winding Number 3/3
Source cue	- From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside / - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]
Professor-style explanation	The slide 'Winding Number 3/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside / - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Winding Number 3/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - From the second property follows / - A path that cuts a curve (= the clipping polygon) just once, / runs either from inside to outside, or from outside to inside / - Clipping operation can be performed by computing / [Greiner & Hormann, TOG 1998]

Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Winding Number 3/3'?
Page 52	Special Cases
Source cue	- Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation / - One polygon is contained in the other polygon / - Polygons have no overlaps / - Problem: One point lies on one edge
Professor-style explanation	The slide 'Special Cases' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation / - One polygon is contained in the other polygon / - Polygons have no overlaps / - Problem: One point lies on one edge. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Special Cases. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Simple cases can be solved after unsuccessful intersection calculation / directly via winding number operation / - One polygon is contained in the other polygon / - Polygons have no overlaps / - Problem: One point lies on one edge
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Special Cases' into a causal sentence instead of repeating the slide title?
Page 53	Evaluation
Source cue	- Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,
Professor-style explanation	The slide 'Evaluation' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about Evaluation. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Allows efficient clipping of arbitrary 2D polygons / - Self overlaps and holes allowed / - Idea based on definition of winding number / - Very simple and efficient implementation possible / - Much easier than Weiler-Atherton,
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Evaluation'?
Page 54	by exploiting binary outcodes
Source cue	- Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon
Professor-style explanation	The slide 'by exploiting binary outcodes' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about by exploiting binary outcodes. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Cohen-Sutherland enables line clipping against rectangle / by exploiting binary outcodes / - Cyrus-Beck line clipping enables line clipping against convex polygon / by exploiting parametric representations / - Sutherland-Hodgman polygon clipping enables polygon against convex polygon
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'by exploiting binary outcodes'?
Page 55	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?

Page 56	Addison-Wesley 2013.
Source cue	- Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016.
Professor-style explanation	The slide 'Addison-Wesley 2013.' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Addison-Wesley 2013.. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Text Books / - J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", / Addison-Wesley 2013. / - P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th / Edition)", AK Peters 2016.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Addison-Wesley 2013.' into a causal sentence instead of repeating the slide title?

05.1 Cohen-Sutherland Line Clipping

Source location: Section 5.1

Commentary

Cohen-Sutherland assigns outcodes to line endpoints based on which side of the clipping window they lie on. These codes allow quick accept, quick reject, or partial clipping. The algorithm is efficient because many cases are decided before computing intersections.

The exam-friendly idea is the logic of region codes: if both endpoints are inside, keep the segment. If the bitwise AND of the outcodes is nonzero, both endpoints share an outside region and the segment can be rejected. Otherwise, compute an intersection and continue.

Mental model

Outcodes are a cheap classification before doing intersection math.

Check yourself

What does a nonzero bitwise AND of two endpoint outcodes imply?

05.2 Cyrus-Beck Line Clipping

Source location: Section 5.2

Commentary

Cyrus-Beck treats the line parametrically and clips against convex boundaries. Instead of repeatedly using region codes, it finds entering and leaving parameter values along the line.

The key is to reason about the line parameter t . A clipped line segment is not a new unrelated line; it is a restricted interval of the original parametric line. Boundary tests shrink that interval.

Mental model

Clipping a parametric line means narrowing the valid t interval.

Check yourself

Why does Cyrus-Beck naturally require convex clipping regions?

05.3 Sutherland-Hodgman Polygon Clipping

Source location: Section 5.3

Commentary

Sutherland-Hodgman clips a polygon against one boundary at a time. For each edge of the polygon, it decides whether vertices are inside or outside and emits zero, one, or two vertices depending on transitions across the boundary.

This algorithm is easiest to understand as a stream processor. Feed in a polygon, clip it against the left boundary, feed the result into the right boundary, and so on. Each boundary can add intersection vertices.

Mental model

Polygon clipping is repeated edge-by-edge filtering plus intersection insertion.

Check yourself

When can polygon clipping increase the number of vertices?

05.4 Weiler-Atherton and Greiner-Hormann

Source location: Sections 5.4-5.5

Commentary

These algorithms address more complex polygon clipping scenarios, especially when polygon relationships are not as simple as convex-window clipping. They organize intersections and traversal rules to produce correct output boundaries.

For preparation, focus on why simpler algorithms are not enough. Complex polygon intersection can require following alternating boundaries of subject and clipping polygons. That is a topology problem, not only a line-intersection problem.

Mental model

Complex polygon clipping is about traversing boundary networks after intersections are known.

Check yourself

Why are intersection ordering and traversal rules important for polygon clipping?

End-of-lecture summary

If you remember only one thing from Lecture 05, remember this: This lecture focuses on algorithms that remove or trim geometry against boundaries. Clipping is a geometric operation before rasterization, not a visibility test between overlapping objects.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 06 - Rasterization

This lecture explains how continuous primitives become discrete fragments on a grid. Rasterization is the bridge from geometric descriptions to sample-based image generation.

Source chunk	course_text_parts/03_lectures/06_rasterization.txt
Extracted slide pages	61
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Rasterization 1/2
Source cue	- Pixels ... / - ... are picture elements A Pixel Is NotA Little Square, / A Pixel Is NotA Little Square, / - ... are the smallest elements of a raster image / A Pixel Is NotA Little Square!

Professor-style explanation	The slide 'Rasterization 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Pixels ... / - ... are picture elements A Pixel Is NotA Little Square, / A Pixel Is NotA Little Square, / - ... are the smallest elements of a raster image / A Pixel Is NotA Little Square!. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Rasterization 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Pixels ... / - ... are picture elements A Pixel Is NotA Little Square, / A Pixel Is NotA Little Square, / - ... are the smallest elements of a raster image / A Pixel Is NotA Little Square!
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization 1/2'?

Page 3	Rasterization 2/2
Source cue	- Pixel-based displays require discretization of primitives into pixels / - Discretization is done by rasterization algorithms / - Rasterization is time-critical / - Avoid expensive operations / - Optimize for hardware implementation
Professor-style explanation	The slide 'Rasterization 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Pixel-based displays require discretization of primitives into pixels / - Discretization is done by rasterization algorithms / - Rasterization is time-critical / - Avoid expensive operations / - Optimize for hardware implementation. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Rasterization 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Pixel-based displays require discretization of primitives into pixels / - Discretization is done by rasterization algorithms / - Rasterization is time-critical / - Avoid expensive operations / - Optimize for hardware implementation

Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization 2/2'?
Page 4	6.1 Line Rasterization
Source cue	6.2 Triangle Edge Rasterization / 6.3 Region Filling / 6.4 Scanline-Based Triangle Rasterization / 6.5 Block-Based Triangle Rasterization
Professor-style explanation	The slide '6.1 Line Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: 6.2 Triangle Edge Rasterization / 6.3 Region Filling / 6.4 Scanline-Based Triangle Rasterization / 6.5 Block-Based Triangle Rasterization. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 6.1 Line Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: 6.2 Triangle Edge Rasterization / 6.3 Region Filling / 6.4 Scanline-Based Triangle Rasterization / 6.5 Block-Based Triangle Rasterization
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.1 Line Rasterization'?
Page 5	6.1 Line Rasterization
Source cue	Drawing a one pixel thick line
Professor-style explanation	The slide '6.1 Line Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Drawing a one pixel thick line. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about 6.1 Line Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Drawing a one pixel thick line
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.1 Line Rasterization'?

Page 6	Line Rasterization
Source cue	- Given two 2D endpoints in integer coordinates, / determine pixels depicting 1-pixel wide line segment between endpoints / - Trivial cases / - Horizontal line segments / - Vertical line segments
Professor-style explanation	The slide 'Line Rasterization' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Given two 2D endpoints in integer coordinates, / determine pixels depicting 1-pixel wide line segment between endpoints / - Trivial cases / - Horizontal line segments / - Vertical line segments. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Line Rasterization. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Given two 2D endpoints in integer coordinates, / determine pixels depicting 1-pixel wide line segment between endpoints / - Trivial cases / - Horizontal line segments / - Vertical line segments
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Line Rasterization'?

Page 7	Quality Criteria
Source cue	- For all line segments with a slope $ m \leq 1$, / one pixel should be drawn per column / - For all line segments with a slope $ m \geq 1$, / one pixel per line should be drawn / $ m > 1$

Professor-style explanation	The slide 'Quality Criteria' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - For all line segments with a slope $ m \leq 1$, / one pixel should be drawn per column / - For all line segments with a slope $ m \geq 1$, / one pixel per line should be drawn / $ m > 1$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Quality Criteria. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - For all line segments with a slope $ m \leq 1$, / one pixel should be drawn per column / - For all line segments with a slope $ m \geq 1$, / one pixel per line should be drawn / $ m > 1$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Quality Criteria'?

Page 8	Naïve Algorithm 1/2
Source cue	- Utilization of the Cartesian normal form of the straight line equation / $y = m \cdot x + B$ / - Assumptions (without restriction of generality) / - Line segments lie between (x, y) and $(x, y) / 0 \ 0 \ 1 \ 1$
Professor-style explanation	The slide 'Naïve Algorithm 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Utilization of the Cartesian normal form of the straight line equation / $y = m \cdot x + B$ / - Assumptions (without restriction of generality) / - Line segments lie between (x, y) and $(x, y) / 0 \ 0 \ 1 \ 1$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Naïve Algorithm 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Utilization of the Cartesian normal form of the straight line equation / $y = m \cdot x + B$ / - Assumptions (without restriction of generality) / - Line segments lie between (x, y) and $(x, y) / 0 \ 0 \ 1 \ 1$

Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Naïve Algorithm 1/2'?
Page 9	Naïve Algorithm 2/2
Source cue	- Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$)
Professor-style explanation	The slide 'Naïve Algorithm 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Naïve Algorithm 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Naïve Algorithm 2/2' into a causal sentence instead of repeating the slide title?
Page 10	Digital Difference Analyzer (DDA) 1/3
Source cue	- Reuse the last computed y to avoid multiplications in the loop / $x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + m)$ / void drawLine(int x0, int y0, int x1, int y1) { / float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope / float y = y0; // Start y at y0
Professor-style explanation	The slide 'Digital Difference Analyzer (DDA) 1/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Reuse the last computed y to avoid multiplications in the loop / $x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + m)$ / void drawLine(int x0, int y0, int x1, int y1) { / float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope / float y = y0; // Start y at y0. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Digital Difference Analyzer (DDA) 1/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Reuse the last computed y to avoid multiplications in the loop / $x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + m)$ / <code>void drawLine(int x0, int y0, int x1, int y1) { / float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope / float y = y0; // Start y at y0</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 1/3' into a causal sentence instead of repeating the slide title?

Page 11	Digital Difference Analyzer (DDA) 2/3
Source cue	- Algorithm can be extended to relax endpoint order / <code>void drawLine(int x0, int y0, int x1, int y1) { / int sx = (x0 < x1 ? 1 : -1); // Determine x direction / float sm = sx * float(y1 - y0) / float(x1 - x0); // Calculate slope multiplied by step / float y = y0; // Start y at y0</code>
Professor-style explanation	The slide 'Digital Difference Analyzer (DDA) 2/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Algorithm can be extended to relax endpoint order / <code>void drawLine(int x0, int y0, int x1, int y1) { / int sx = (x0 < x1 ? 1 : -1); // Determine x direction / float sm = sx * float(y1 - y0) / float(x1 - x0); // Calculate slope multiplied by step / float y = y0; // Start y at y0</code> . The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Digital Difference Analyzer (DDA) 2/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Algorithm can be extended to relax endpoint order / <code>void drawLine(int x0, int y0, int x1, int y1) { / int sx = (x0 < x1 ? 1 : -1); // Determine x direction / float sm = sx * float(y1 - y0) / float(x1 - x0); // Calculate slope multiplied by step / float y = y0; // Start y at y0</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 2/3' into a causal sentence instead of repeating the slide title?

Page 12	Digital Difference Analyzer (DDA) 3/3
Source cue	- Downsides / - Does not work for $x = x / 0$ / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$)

Professor-style explanation	The slide 'Digital Difference Analyzer (DDA) 3/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Digital Difference Analyzer (DDA) 3/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 3/3' into a causal sentence instead of repeating the slide title?

Page 13	Midpoint-Line Algorithm 1/2
Source cue	- Assumption 1: $0 \leq m \leq 1$ / (without loss of generality, as otherwise mirroring on axes/diagonals) / - Lower left end point $P = (x, y)$, / 0 0 0 / upper right end point $P = (x, y)$
Professor-style explanation	The slide 'Midpoint-Line Algorithm 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Assumption 1: $0 \leq m \leq 1$ / (without loss of generality, as otherwise mirroring on axes/diagonals) / - Lower left end point $P = (x, y)$, / 0 0 0 / upper right end point $P = (x, y)$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Midpoint-Line Algorithm 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Assumption 1: $0 \leq m \leq 1$ / (without loss of generality, as otherwise mirroring on axes/diagonals) / - Lower left end point $P = (x, y)$, / 0 0 0 / upper right end point $P = (x, y)$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Midpoint-Line Algorithm 1/2'?

Page 14	Midpoint-Line Algorithm 2/2
---------	-----------------------------

Source cue	- Observation / - E is closer to the line if M is above the line / - NE is closer to the line if M is below the line / - Vertical distance between pixel to chose / and the actual line segment is always < 0.5
Professor-style explanation	The slide 'Midpoint-Line Algorithm 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Observation / - E is closer to the line if M is above the line / - NE is closer to the line if M is below the line / - Vertical distance between pixel to chose / and the actual line segment is always < 0.5 . The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Midpoint-Line Algorithm 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Observation / - E is closer to the line if M is above the line / - NE is closer to the line if M is below the line / - Vertical distance between pixel to chose / and the actual line segment is always < 0.5
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Midpoint-Line Algorithm 2/2'?
Page 15	Implicit Function
Source cue	- Line equation usually given as explicit function $f(x) / f x = y = m \cdot x + B = \cdot x + B$, with $dx = x - x$ and $dy = y - y / 1 \text{ o } 1 \text{ o}$ / - Line equation can also be written as implicit function $F(x, y) / F x, y = a \cdot x + b \cdot y + c = 0$
Professor-style explanation	The slide 'Implicit Function' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Line equation usually given as explicit function $f(x) / f x = y = m \cdot x + B = \cdot x + B$, with $dx = x - x$ and $dy = y - y / 1 \text{ o } 1 \text{ o}$ / - Line equation can also be written as implicit function $F(x, y) / F x, y = a \cdot x + b \cdot y + c = 0$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Implicit Function. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Line equation usually given as explicit function $f(x) / f x = y = m \cdot x + B = \cdot x + B$, with $dx = x - x$ and $dy = y - y / 1 o 1 o$ / - Line equation can also be written as implicit function $F(x, y) / F x, y = a \cdot x + b \cdot y + c = 0$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Implicit Function'?

Page 16	Decision Variable d
Source cue	- Sign of $F(M)$ is sufficient to decide which point is chosen / - Define decision variable d as $d = F M = F(x + 1, y +) / p p$ / - If $d > 0$, select NE (M below the line) / - If $d < 0$, select E (M above the line)
Professor-style explanation	The slide 'Decision Variable d' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Sign of $F(M)$ is sufficient to decide which point is chosen / - Define decision variable d as $d = F M = F(x + 1, y +) / p p$ / - If $d > 0$, select NE (M below the line) / - If $d < 0$, select E (M above the line). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Decision Variable d. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Sign of $F(M)$ is sufficient to decide which point is chosen / - Define decision variable d as $d = F M = F(x + 1, y +) / p p$ / - If $d > 0$, select NE (M below the line) / - If $d < 0$, select E (M above the line)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Decision Variable d' into a causal sentence instead of repeating the slide title?

Page 17	Update of d for E
Source cue	- Previous decision variable was given by $1 1 / d = F x + 1, y + = a \cdot x + 1 + b \cdot y + + c / old p p p p / 2 2$
Professor-style explanation	The slide 'Update of d for E' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Previous decision variable was given by $1 1 / d = F x + 1, y + = a \cdot x + 1 + b \cdot y + + c / old p p p p / 2 2$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Update of d for E. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Previous decision variable was given by $1/d = Fx + 1$, $y = a \cdot x + 1 + b \cdot y + c$ / old p p p p / 2 2
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Update of d for E' into a causal sentence instead of repeating the slide title?

Page 18	Update of d for NE
Source cue	- Previous decision variable was given by $1/d = Fx + 1$, $y = a \cdot x + 1 + b \cdot y + c$ / old p p p p / 2 2
Professor-style explanation	The slide 'Update of d for NE' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Previous decision variable was given by $1/d = Fx + 1$, $y = a \cdot x + 1 + b \cdot y + c$ / old p p p p / 2 2. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Update of d for NE. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Previous decision variable was given by $1/d = Fx + 1$, $y = a \cdot x + 1 + b \cdot y + c$ / old p p p p / 2 2
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Update of d for NE' into a causal sentence instead of repeating the slide title?

Page 19	Proceeding
Source cue	- In each step, the algorithm selects between two pixels NE and E / based on sign of decision variable d / - Depending on the choice, decision variable is updated / by adding difference Δ_{NE} or Δ_E
Professor-style explanation	The slide 'Proceeding' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - In each step, the algorithm selects between two pixels NE and E / based on sign of decision variable d / - Depending on the choice, decision variable is updated / by adding difference Δ_{NE} or Δ_E . The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Proceeding. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - In each step, the algorithm selects between two pixels NE and E / based on sign of decision variable d / - Depending on the choice, decision variable is updated / by adding difference ΔNE or ΔE
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?
Page 20	Initializing d
Source cue	- First midpoint M is located at $(x + 1, y +) / 0 0 / 1 1 b / F x + 1, y + = a \cdot x + 1 + b \cdot y + + c = a \cdot x + b \cdot y + c + a + / 0 0 0 0 0 0$
Professor-style explanation	The slide 'Initializing d' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - First midpoint M is located at $(x + 1, y +) / 0 0 / 1 1 b / F x + 1, y + = a \cdot x + 1 + b \cdot y + + c = a \cdot x + b \cdot y + c + a + / 0 0 0 0 0 0$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Initializing d. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - First midpoint M is located at $(x + 1, y +) / 0 0 / 1 1 b / F x + 1, y + = a \cdot x + 1 + b \cdot y + + c = a \cdot x + b \cdot y + c + a + / 0 0 0 0 0 0$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Initializing d' into a causal sentence instead of repeating the slide title?
Page 21	Using d
Source cue	- Midpoint line algorithms uses d as follows / $d = 2 \cdot dy - dx$ / start / $\Delta NE = 2 \cdot dy - dx$ / $\Delta E = 2 \cdot dy$

Professor-style explanation	The slide 'Using d' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Midpoint line algorithms uses d as follows / $d = 2 \cdot dy - dx$ / start / $\Delta NE = 2 \cdot dy - dx$ / $\Delta E = 2 \cdot dy$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Using d. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Midpoint line algorithms uses d as follows / $d = 2 \cdot dy - dx$ / start / $\Delta NE = 2 \cdot dy - dx$ / $\Delta E = 2 \cdot dy$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Using d'?
Page 22	Pseudo Code - Simple Version
Source cue	<pre>void midPointLine(int x0, int y0, int x1, int y1) { / int dx = x1 - x0; // Calculate delta x / int dy = y1 - y0; // Calculate delta y / int d = 2 * dy - dx; // Initial decision variable / int dE = 2 * dy; // Increment used for move to E</pre>
Professor-style explanation	The slide 'Pseudo Code - Simple Version' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: void midPointLine(int x0, int y0, int x1, int y1) { / int dx = x1 - x0; // Calculate delta x / int dy = y1 - y0; // Calculate delta y / int d = 2 * dy - dx; // Initial decision variable / int dE = 2 * dy; // Increment used for move to E. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Pseudo Code - Simple Version. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: void midPointLine(int x0, int y0, int x1, int y1) { / int dx = x1 - x0; // Calculate delta x / int dy = y1 - y0; // Calculate delta y / int d = 2 * dy - dx; // Initial decision variable / int dE = 2 * dy; // Increment used for move to E
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Pseudo Code - Simple Version' into a causal sentence instead of repeating the slide title?
Page 23	Example

Source cue	- Draw line from (5, 8) to (9, 11) / - Order of values for d / - 2, 0, 6, 4 / - Order of chosen pixels / - NE, E, NE, NE
Professor-style explanation	The slide 'Example' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Draw line from (5, 8) to (9, 11) / - Order of values for d / - 2, 0, 6, 4 / - Order of chosen pixels / - NE, E, NE, NE. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Example. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Draw line from (5, 8) to (9, 11) / - Order of values for d / - 2, 0, 6, 4 / - Order of chosen pixels / - NE, E, NE, NE
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Example'?
Page 24	Evaluation
Source cue	- Necessary arithmetic operations / - As only integer values are used, only integer operations are necessary / - Additions/subtractions only and multiplication by 2 / (left shift by 1 bit position) / - No multiplications in the loop
Professor-style explanation	The slide 'Evaluation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Necessary arithmetic operations / - As only integer values are used, only integer operations are necessary / - Additions/subtractions only and multiplication by 2 / (left shift by 1 bit position) / - No multiplications in the loop. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Evaluation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Necessary arithmetic operations / - As only integer values are used, only integer operations are necessary / - Additions/subtractions only and multiplication by 2 / (left shift by 1 bit position) / - No multiplications in the loop
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Evaluation' into a causal sentence instead of repeating the slide title?
Page 25	Pseudo Code - Reversible
Source cue	<pre>void drawLine(int x0, int y0, int x1, int y1) { / int dx = abs(x1 - x0); // Calculate absolute delta x / int sx = (x0 < x1 ? 1 : -1); // Set step x based on direction / int dy = abs(y1 - y0); // Calculate absolute delta y / int sy = (y0 < y1 ? 1 : -1); // Set step y based on direction</pre>
Professor-style explanation	The slide 'Pseudo Code - Reversible' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: <code>void drawLine(int x0, int y0, int x1, int y1) { / int dx = abs(x1 - x0); // Calculate absolute delta x / int sx = (x0 < x1 ? 1 : -1); // Set step x based on direction / int dy = abs(y1 - y0); // Calculate absolute delta y / int sy = (y0 < y1 ? 1 : -1); // Set step y based on direction</code> . The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Pseudo Code - Reversible. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: <code>void drawLine(int x0, int y0, int x1, int y1) { / int dx = abs(x1 - x0); // Calculate absolute delta x / int sx = (x0 < x1 ? 1 : -1); // Set step x based on direction / int dy = abs(y1 - y0); // Calculate absolute delta y / int sy = (y0 < y1 ? 1 : -1); // Set step y based on direction</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Pseudo Code - Reversible' into a causal sentence instead of repeating the slide title?
Page 26	6.2 Triangle Edge Rasterization
Source cue	Drawing a one pixel thick triangle edge
Professor-style explanation	The slide '6.2 Triangle Edge Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Drawing a one pixel thick triangle edge. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 6.2 Triangle Edge Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Drawing a one pixel thick triangle edge

Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.2 Triangle Edge Rasterization'?
Page 27	Triangle Edges 1/2
Source cue	- Using midpoint line algorithm for triangle edge rasterization / - Provides pixels closest to triangle edge / - Does not consider if pixel lies inside or outside triangle / $\Rightarrow$ Edge pixels can penetrate into interior of adjacent triangles
Professor-style explanation	The slide 'Triangle Edges 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Using midpoint line algorithm for triangle edge rasterization / - Provides pixels closest to triangle edge / - Does not consider if pixel lies inside or outside triangle / $\Rightarrow$ Edge pixels can penetrate into interior of adjacent triangles. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Triangle Edges 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Using midpoint line algorithm for triangle edge rasterization / - Provides pixels closest to triangle edge / - Does not consider if pixel lies inside or outside triangle / $\Rightarrow$ Edge pixels can penetrate into interior of adjacent triangles
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Edges 1/2'?
Page 28	Triangle Edges 2/2
Source cue	- Midpoint line algorithm must consider inside vs. outside pixels / - Adaptation of the midpoint line algorithm / - Restriction to pixels strictly inside triangle / - Must be applied to line rasterization also for consistency reasons / - Definition of triangle inside:

Professor-style explanation	The slide 'Triangle Edges 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Midpoint line algorithm must consider inside vs. outside pixels - Adaptation of the midpoint line algorithm - Restriction to pixels strictly inside triangle - Must be applied to line rasterization also for consistency reasons - Definition of triangle inside The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Triangle Edges 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Midpoint line algorithm must consider inside vs. outside pixels - Adaptation of the midpoint line algorithm - Restriction to pixels strictly inside triangle - Must be applied to line rasterization also for consistency reasons - Definition of triangle inside:
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Edges 2/2'?

Page 29	Edge Rasterization 1/3
Source cue	- Triangle edge rasterization can exploit edge coherence - Two edges of triangle intersect a scanline - Same edges intersected by scanline i likely also intersect scanline $i + 1$ - Due to coherence, intersections can be calculated incrementally - For each increase of $y \rightarrow y + 1$
Professor-style explanation	The slide 'Edge Rasterization 1/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Triangle edge rasterization can exploit edge coherence - Two edges of triangle intersect a scanline - Same edges intersected by scanline i likely also intersect scanline $i + 1$ - Due to coherence, intersections can be calculated incrementally - For each increase of $y \rightarrow y + 1$ The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Edge Rasterization 1/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Triangle edge rasterization can exploit edge coherence / - Two edges of triangle intersect a scanline / - Same edges intersected by scanline i likely also intersect scanline $i + 1$ / - Due to coherence, intersections can be calculated incrementally / - For each increase of $y \rightarrow y + 1$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 1/3'?

Page 30	Edge Rasterization 2/3
Source cue	- Integer arithmetic can be used for (left) edges with slopes > 1 / - Consider $dx dy = (x_{max} - x_{min}) / (y_{max} - y_{min})$ / - x is only changed by $dx dy$ / $\Rightarrow x$ can be represented as a fraction with denominator $(y_{max} - y_{min})$ / - Let a be the integer part
Professor-style explanation	The slide 'Edge Rasterization 2/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Integer arithmetic can be used for (left) edges with slopes > 1 / - Consider $dx dy = (x_{max} - x_{min}) / (y_{max} - y_{min})$ / - x is only changed by $dx dy$ / $\Rightarrow x$ can be represented as a fraction with denominator $(y_{max} - y_{min})$ / - Let a be the integer part. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Edge Rasterization 2/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Integer arithmetic can be used for (left) edges with slopes > 1 / - Consider $dx dy = (x_{max} - x_{min}) / (y_{max} - y_{min})$ / - x is only changed by $dx dy$ / $\Rightarrow x$ can be represented as a fraction with denominator $(y_{max} - y_{min})$ / - Let a be the integer part
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 2/3'?

Page 31	Edge Rasterization 3/3
Source cue	<pre>void leftEdgeScan(int x0, int y0, int x1, int y1) { // slope > 1 / int d = y1 - y0; // denominator (Nenner) / int n = x1 - x0; // numerator (Zähler) / int a = x0; // integer part of x / int b = d; // fraction part of x plus d</pre>

Professor-style explanation	The slide 'Edge Rasterization 3/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: <code>void leftEdgeScan(int x0, int y0, int x1, int y1) { // slope > 1 / int d = y1 - y0; // denominator (Nenner) / int n = x1 - x0; // numerator (Zähler) / int a = x0; // integer part of x / int b = d; // fraction part of x plus d</code> . The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Edge Rasterization 3/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: <code>void leftEdgeScan(int x0, int y0, int x1, int y1) { // slope > 1 / int d = y1 - y0; // denominator (Nenner) / int n = x1 - x0; // numerator (Zähler) / int a = x0; // integer part of x / int b = d; // fraction part of x plus d</code>
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 3/3'?

Page 32	6.3 Region Filling
Source cue	Homogeneous coloring of regions
Professor-style explanation	The slide '6.3 Region Filling' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Homogeneous coloring of regions. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 6.3 Region Filling. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Homogeneous coloring of regions
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.3 Region Filling'?

Page 33	Region Types 1/2
---------	------------------

Source cue	- 4-connected / - All pixels of region can be reached by following movements / - Left, right, bottom, top / - 8-connected / - All pixels of region can be reached by following movements
Professor-style explanation	The slide 'Region Types 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - 4-connected / - All pixels of region can be reached by following movements / - Left, right, bottom, top / - 8-connected / - All pixels of region can be reached by following movements. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Region Types 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - 4-connected / - All pixels of region can be reached by following movements / - Left, right, bottom, top / - 8-connected / - All pixels of region can be reached by following movements
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Region Types 1/2' into a causal sentence instead of repeating the slide title?

Page 34	Region Types 2/2
Source cue	- Examples / 4-connected 8-connected 8-connected
Professor-style explanation	The slide 'Region Types 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Examples / 4-connected 8-connected 8-connected. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Region Types 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Examples / 4-connected 8-connected 8-connected
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Region Types 2/2' into a causal sentence instead of repeating the slide title?

Page 35	Region Spezifikation
Source cue	- Color-based / - Starting at pixel P, determine largest contiguous region / which pixels have a specific color / - Color assignment done by flood fill algorithms / - Border-based

Professor-style explanation	The slide 'Region Spezifikation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Color-based / - Starting at pixel P, determine largest contiguous region / which pixels have a specific color / - Color assignment done by flood fill algorithms / - Border-based. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Region Spezifikation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Color-based / - Starting at pixel P, determine largest contiguous region / which pixels have a specific color / - Color assignment done by flood fill algorithms / - Border-based
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Region Spezifikation' into a causal sentence instead of repeating the slide title?

Page 36	Flood Fill Algorithm
Source cue	<pre>void floodFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / if (oldC == readPixel(x, y)) { // Check if the current color matches oldC / writePixel(x, y, newC); // Change the color of the pixel</pre>
Professor-style explanation	The slide 'Flood Fill Algorithm' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: void floodFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / if (oldC == readPixel(x, y)) { // Check if the current color matches oldC / writePixel(x, y, newC); // Change the color of the pixel. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Flood Fill Algorithm. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: void floodFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / if (oldC == readPixel(x, y)) { // Check if the current color matches oldC / writePixel(x, y, newC); // Change the color of the pixel
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Flood Fill Algorithm' into a causal sentence instead of repeating the slide title?
Page 37	Boundary Fill Algorithm
Source cue	<code>void boundaryFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / Color c = readPixel(x, y); / if (c == oldC) return; // Return if pixel is a boundary pixel</code>
Professor-style explanation	The slide 'Boundary Fill Algorithm' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: <code>void boundaryFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / Color c = readPixel(x, y); / if (c == oldC) return; // Return if pixel is a boundary pixel</code> . The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Boundary Fill Algorithm. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: <code>void boundaryFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / Color c = readPixel(x, y); / if (c == oldC) return; // Return if pixel is a boundary pixel</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Boundary Fill Algorithm' into a causal sentence instead of repeating the slide title?
Page 38	Fill Algorithms
Source cue	- High number of recursions can lead to stack overflow / - Use smarter data structures, i.e., queues and stacks / Queue Stack
Professor-style explanation	The slide 'Fill Algorithms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - High number of recursions can lead to stack overflow / - Use smarter data structures, i.e., queues and stacks / Queue Stack. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Fill Algorithms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - High number of recursions can lead to stack overflow / - Use smarter data structures, i.e., queues and stacks / Queue Stack

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Fill Algorithms' into a causal sentence instead of repeating the slide title?
Page 39	6.4 Scanline-Based Triangle Rasterization
Source cue	Segment-based rasterization of the simplest type of polygon
Professor-style explanation	The slide '6.4 Scanline-Based Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Segment-based rasterization of the simplest type of polygon. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 6.4 Scanline-Based Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Segment-based rasterization of the simplest type of polygon
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.4 Scanline-Based Triangle Rasterization'?
Page 40	Triangle Rasterization
Source cue	- Triangle rasterization / - Rasterize filled triangles is main task in real-time rendering / - Realization through graphics hardware desired / - General polygons split into triangles through tessellation / - Rasterization requires three steps
Professor-style explanation	The slide 'Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Triangle rasterization / - Rasterize filled triangles is main task in real-time rendering / - Realization through graphics hardware desired / - General polygons split into triangles through tessellation / - Rasterization requires three steps. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Triangle rasterization / - Rasterize filled triangles is main task in real-time rendering / - Realization through graphics hardware desired / - General polygons split into triangles through tessellation / - Rasterization requires three steps
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Rasterization'?

Page 41	Scanline-Based Rasterization
Source cue	- Rasterization through scanline conversion exploits spatial coherence / - Pixels of a scanline are colored similarly / - Adjacent scanlines span similar pixels / - Adjacent scanlines are colored similarly / - Scanline segments can be calculated
Professor-style explanation	The slide 'Scanline-Based Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Rasterization through scanline conversion exploits spatial coherence / - Pixels of a scanline are colored similarly / - Adjacent scanlines span similar pixels / - Adjacent scanlines are colored similarly / - Scanline segments can be calculated. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Scanline-Based Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Rasterization through scanline conversion exploits spatial coherence / - Pixels of a scanline are colored similarly / - Adjacent scanlines span similar pixels / - Adjacent scanlines are colored similarly / - Scanline segments can be calculated
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Scanline-Based Rasterization'?

Page 42	Algorithm Overview
Source cue	- Observation: triangle can be split into top and bottom half / - For each half two edge pixels and one segment exist / - Segments differ only slightly, depending on edge slope / - Procedure / - Sort triangle vertices based on y-coordinate

Professor-style explanation	The slide 'Algorithm Overview' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Observation: triangle can be split into top an bottom half / - For each half two edge pixels and one segment exist / - Segments differ only slightly, depending on edge slope / - Procedure / - Sort triangle vertices based on y-coordinate. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Algorithm Overview. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Observation: triangle can be split into top an bottom half / - For each half two edge pixels and one segment exist / - Segments differ only slightly, depending on edge slope / - Procedure / - Sort triangle vertices based on y-coordinate
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Algorithm Overview'?

Page 43	Slope Calculation
Source cue	- Calculate slope for each triangle edge: $1/A = dx/dy$ / - $\Delta = (x - x)/(y - y)$ / 02 2 0 2 0 / - $\Delta = (x - x)/(y - y)$ / 01 1 0 1 0
Professor-style explanation	The slide 'Slope Calculation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Calculate slope for each triangle edge: $1/A = dx/dy$ / - $\Delta = (x - x)/(y - y)$ / 02 2 0 2 0 / - $\Delta = (x - x)/(y - y)$ / 01 1 0 1 0. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Slope Calculation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Calculate slope for each triangle edge: $1/A = dx/dy$ / - $\Delta = (x - x)/(y - y)$ / 02 2 0 2 0 / - $\Delta = (x - x)/(y - y)$ / 01 1 0 1 0
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.

Check yourself	Can you explain which samples/fragments are generated by 'Slope Calculation'?
Page 44	Pseudo Code
Source cue	<pre>void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) { / float XLeft = x0; // Start position for left x / float XRight = x0; // Start position for right x / int Y = y0; // Start position for y / float DeltaLeft = delta02; // Delta for left x for bottom half</pre>
Professor-style explanation	The slide 'Pseudo Code' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) { / float XLeft = x0; // Start position for left x / float XRight = x0; // Start position for right x / int Y = y0; // Start position for y / float DeltaLeft = delta02; // Delta for left x for bottom half. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Pseudo Code. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) { / float XLeft = x0; // Start position for left x / float XRight = x0; // Start position for right x / int Y = y0; // Start position for y / float DeltaLeft = delta02; // Delta for left x for bottom half
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Pseudo Code'?
Page 45	Segment Rasterization
Source cue	- Each segment generates fragments / - Rendering pipeline processes generated fragments / (x , y) / (e.g., depth testing, alpha blending) right s / (x , y)

Professor-style explanation	The slide 'Segment Rasterization' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: - Each segment generates fragments / - Rendering pipeline processes generated fragments / (x, y) / (e.g., depth testing, alpha blending) right $s / (x, y)$. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.
Technical commentary	This slide is about Segment Rasterization. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: - Each segment generates fragments / - Rendering pipeline processes generated fragments / (x, y) / (e.g., depth testing, alpha blending) right $s / (x, y)$
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'Segment Rasterization' in the rendering pipeline?

Page 46	Vertex Attribute Interpolation
Source cue	- During rasterization vertex attributes (e.g., color, depth) / must be interpolated over triangle / (x, y, z) / right s right / - Bilinear interpolation can be used
Professor-style explanation	The slide 'Vertex Attribute Interpolation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - During rasterization vertex attributes (e.g., color, depth) / must be interpolated over triangle / (x, y, z) / right s right / - Bilinear interpolation can be used. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Vertex Attribute Interpolation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - During rasterization vertex attributes (e.g., color, depth) / must be interpolated over triangle / (x, y, z) / right s right / - Bilinear interpolation can be used
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.

Check yourself	Can you explain which samples/fragments are generated by 'Vertex Attribute Interpolation'?
Page 47	Incremental Interpolation 1/3
Source cue	- Spatial coherence allows for an incremental attribute interpolation / - Example: consider triangle's plane equation for calculating z increments / - $Ax + By + Cz + D = 0$ / - With plane coefficients A, B, C: $A, B, C = V - V \times (V - V) / 2 \ 0 \ 1 \ 2$
Professor-style explanation	The slide 'Incremental Interpolation 1/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Spatial coherence allows for an incremental attribute interpolation / - Example: consider triangle's plane equation for calculating z increments / - $Ax + By + Cz + D = 0$ / - With plane coefficients A, B, C: $A, B, C = V - V \times (V - V) / 2 \ 0 \ 1 \ 2$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Incremental Interpolation 1/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Spatial coherence allows for an incremental attribute interpolation / - Example: consider triangle's plane equation for calculating z increments / - $Ax + By + Cz + D = 0$ / - With plane coefficients A, B, C: $A, B, C = V - V \times (V - V) / 2 \ 0 \ 1 \ 2$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Incremental Interpolation 1/3'?
Page 48	Incremental Interpolation 2/3
Source cue	- Increments $dzdx$ and $dzdy$ are used to incrementally modify given z value, / i.e., when moving from (x, y) to (x + 1, y) or (x, y + 1) / - Case 1: next fragment in scanline (horizontal change by +1) / - $x \rightarrow x \Rightarrow z \blacksquare z + dzdx / i \ i+1 \ i$

Professor-style explanation	The slide 'Incremental Interpolation 2/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Increments $dzdx$ and $dzdy$ are used to incrementally modify given z value, / i.e., when moving from (x, y) to $(x + 1, y)$ or $(x, y + 1)$ / - Case 1: next fragment in scanline (horizontal change by +1) / - $x \rightarrow x \Rightarrow z \blacksquare z + dzdx / i + 1$ i. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Incremental Interpolation 2/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Increments $dzdx$ and $dzdy$ are used to incrementally modify given z value, / i.e., when moving from (x, y) to $(x + 1, y)$ or $(x, y + 1)$ / - Case 1: next fragment in scanline (horizontal change by +1) / - $x \rightarrow x \Rightarrow z \blacksquare z + dzdx / i + 1$ i
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Incremental Interpolation 2/3'?

Page 49	Incremental Interpolation 3/3
Source cue	- All vertex attributes can be interpolated incrementally analogous to depth / - Colors (r, g, b, a) / - Texture coordinates (u, v) / - ... / - For each attribute θ define
Professor-style explanation	The slide 'Incremental Interpolation 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - All vertex attributes can be interpolated incrementally analogous to depth / - Colors (r, g, b, a) / - Texture coordinates (u, v) / - ... / - For each attribute θ define. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Incremental Interpolation 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - All vertex attributes can be interpolated incrementally analogous to depth / - Colors (r, g, b, a) / - Texture coordinates (u, v) / - ... / - For each attribute θ define
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Incremental Interpolation 3/3'?

Page 50	6.5 Tile-Based Triangle Rasterization
Source cue	Parallel tile-based triangle rasterization
Professor-style explanation	The slide '6.5 Tile-Based Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Parallel tile-based triangle rasterization. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 6.5 Tile-Based Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Parallel tile-based triangle rasterization
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.5 Tile-Based Triangle Rasterization'?

Page 51	Tile-Based Triangle Rasterization
Source cue	- Modern GPUs have a block-wise structure, / where one processor is responsible for one screen tile of adjacent pixels / - Segment-based triangled rasterization is difficult to parallelize / - Tile-based rasterization is a better fit with modern GPUs / - Advantage

Professor-style explanation	The slide 'Tile-Based Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Modern GPUs have a block-wise structure, / where one processor is responsible for one screen tile of adjacent pixels / - Segment-based triangled rasterization is difficult to parallelize / - Tile-based rasterization is a better fit with modern GPUs / - Advantage. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Tile-Based Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Modern GPUs have a block-wise structure, / where one processor is responsible for one screen tile of adjacent pixels / - Segment-based triangled rasterization is difficult to parallelize / - Tile-based rasterization is a better fit with modern GPUs / - Advantage
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Tile-Based Triangle Rasterization'?

Page 52	Edge-Based Triangle Representation 1/2
Source cue	- Edge-based area representation is possible through implicit edge functions / - Implicit edge function can be defined based on two edge vertices / $V = (x, y)$ and $V = (x, y) / 0 \ 0 \ 0 \ 1 \ 1 \ 1$ / - For a given coordinate (x, y) , the implicit edge function is defined as
Professor-style explanation	The slide 'Edge-Based Triangle Representation 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Edge-based area representation is possible through implicit edge functions / - Implicit edge function can be defined based on two edge vertices / $V = (x, y)$ and $V = (x, y) / 0 \ 0 \ 0 \ 1 \ 1 \ 1$ / - For a given coordinate (x, y) , the implicit edge function is defined as. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Edge-Based Triangle Representation 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Edge-based area representation is possible through implicit edge functions / - Implicit edge function can be defined based on two edge vertices / $V = (x, y)$ and $V = (x, y)$ / $0 \ 0 \ 0 \ 1 \ 1 \ 1$ / - For a given coordinate (x, y) , the implicit edge function is defined as
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Edge-Based Triangle Representation 1/2'?

Page 53	Edge-Based Triangle Representation 2/2
Source cue	- To represent a triangle, three implicit edge functions are combined / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0$ / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 1 \ 2 \ 2 \ 1 \ 2 \ 1 \ 1 \ 2 \ 2 \ 1$
Professor-style explanation	The slide 'Edge-Based Triangle Representation 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - To represent a triangle, three implicit edge functions are combined / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0$ / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 1 \ 2 \ 2 \ 1 \ 2 \ 1 \ 1 \ 2 \ 2 \ 1$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Edge-Based Triangle Representation 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - To represent a triangle, three implicit edge functions are combined / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0$ / - $E_x, y = y - y \cdot x - x - x \cdot y + B \cdot x - x / 1 \ 2 \ 2 \ 1 \ 2 \ 1 \ 1 \ 2 \ 2 \ 1$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge-Based Triangle Representation 2/2'?

Page 54	Rasterization Procedure
Source cue	1. Determine tiles which are (partially) covering triangle / (e.g., easiest approximation would be bounding box) / 2. For each covering tile / - For each pixel / - If all implicit edge functions evaluate to < 0 2

Professor-style explanation	The slide 'Rasterization Procedure' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: 1. Determine tiles which are (partially) covering triangle / (e.g., easiest approximation would be bounding box) / 2. For each covering tile / - For each pixel / - If all implicit edge functions evaluate to < 0 2. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Rasterization Procedure. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: 1. Determine tiles which are (partially) covering triangle / (e.g., easiest approximation would be bounding box) / 2. For each covering tile / - For each pixel / - If all implicit edge functions evaluate to < 0 2
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization Procedure'?

Page 55	between adjacent triangles
Source cue	- Pixel-based displays require the rasterization of primitives / - Rasterization fidelity is critical for image quality / - Rasterization is time-critical and should be optimized for hardware / - Focus on triangles due to speed and ease of tessellation of other polygons / - Segment-based triangle rasterization avoids overlap
Professor-style explanation	The slide 'between adjacent triangles' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Pixel-based displays require the rasterization of primitives / - Rasterization fidelity is critical for image quality / - Rasterization is time-critical and should be optimized for hardware / - Focus on triangles due to speed and ease of tessellation of other polygons / - Segment-based triangle rasterization avoids overlap. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about between adjacent triangles. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Pixel-based displays require the rasterization of primitives / - Rasterization fidelity is critical for image quality / - Rasterization is time-critical and should be optimized for hardware / - Focus on triangles due to speed and ease of tessellation of other polygons / - Segment-based triangle rasterization avoids overlap
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'between adjacent triangles'?

Page 56	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?

Page 57	Practice, Addison-Wesley.
Source cue	- Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Research Publications / - Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter.

Professor-style explanation	The slide 'Practice, Addison-Wesley.' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Research Publications / - Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Practice, Addison-Wesley.. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Research Publications / - Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Practice, Addison-Wesley.' into a causal sentence instead of repeating the slide title?

Page 58	Slide for Future: Sustainability in CG (@ EG25)
Source cue	also @ IEEE VR
Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: also @ IEEE VR. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: also @ IEEE VR
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?

Page 59	Slide for Future: Sustainability in CG (@ EG25)
Source cue	Unoptimized Optimized Unoptimized Optimized / Inverse Simulation of Radiative Thermal Transport / Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn

Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Unoptimized Optimized Unoptimized Optimized / Inverse Simulation of Radiative Thermal Transport / Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Unoptimized Optimized Unoptimized Optimized / Inverse Simulation of Radiative Thermal Transport / Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?
Page 60	Slide for Future: Sustainability in CG (@ EG25)
Source cue	Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints / Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi
Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints / Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints / Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?
Page 61	Slide for Future: Sustainability in CG (@ EG25)

Source cue	Stress-Aligned Hexahedral Lattice Structures / Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann
Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Stress-Aligned Hexahedral Lattice Structures / Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Stress-Aligned Hexahedral Lattice Structures / Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?

06.1 Line Rasterization

Source location: Section 6.1

Commentary

A mathematical line has infinitely many points, but a raster display has a finite grid. Line rasterization decides which grid cells or samples best approximate the ideal line. The algorithm must balance accuracy, speed, and consistency.

Incremental algorithms avoid recomputing expensive formulas for every pixel. The important idea is to step along one axis and update an error term that decides when to step along the other axis.

Mental model

Rasterizing a line means approximating a continuous path by grid decisions.

Check yourself

Why are incremental error updates useful in line rasterization?

06.2 Triangle Edge Rasterization

Source location: Section 6.2

Commentary

Triangles are the core primitive in real-time rendering. Edge functions or half-space tests decide whether a sample lies inside the triangle. This allows the rasterizer to test coverage systematically.

Coverage is not final visibility. A covered sample becomes a fragment candidate. Depth testing, stencil testing, blending, and other operations still decide whether the framebuffer changes.

Mental model

Triangle rasterization answers the question: which samples are inside this projected triangle?

Check yourself

What later stage can reject a fragment after triangle coverage succeeds?

06.3 Region Filling

Source location: Section 6.3

Commentary

Region filling generalizes the idea of determining interior samples. The algorithm must identify connected areas or spans that should receive color. This connects rasterization to scan conversion and image-space reasoning.

The practical issue is avoiding gaps, double fills, or inconsistent boundary handling. Small choices at edges can become visible artifacts when many primitives meet.

Mental model

Filling is about turning boundary descriptions into consistent interior samples.

Check yourself

Why can inconsistent edge rules create cracks between adjacent primitives?

06.4 Scanline-Based Triangle Rasterization

Source location: Section 6.4

Commentary

Scanline rasterization processes horizontal rows of the image. For each row that crosses a triangle, the algorithm finds the covered interval and fills the span. Attribute interpolation can be updated along edges and across each span.

The method is intuitive because it follows the memory layout of many images. It also demonstrates why interpolation is tied to rasterization: once you know a sample location inside the primitive, you can interpolate depth, color, normals, or texture coordinates.

Mental model

Each scanline asks: where does this triangle enter and leave this row?

Check yourself

Which attributes might be interpolated while filling triangle spans?

06.5 Tile-Based Triangle Rasterization

Commentary

Tile- or block-based approaches group pixels into small regions. This can improve locality and parallel work distribution. Modern GPUs often reason in blocks or tiles internally because rendering is massively parallel.

For the exam, connect this to efficiency. The mathematical goal is still coverage, but the implementation is organized to use hardware resources well.

Mental model

Block-based rasterization keeps the same coverage problem but changes the work organization.

Check yourself

Why is block organization attractive for parallel graphics hardware?

End-of-lecture summary

If you remember only one thing from Lecture 06, remember this: This lecture explains how continuous primitives become discrete fragments on a grid. Rasterization is the bridge from geometric descriptions to sample-based image generation.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 07 - Visibility Determination

This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones should affect the image.

Source chunk	course_text_parts/03_lectures/07_visibility-determination.txt
Extracted slide pages	69
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Visibility Determination
Source cue	- Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects

Professor-style explanation	The slide 'Visibility Determination' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Visibility Determination. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Visibility Determination' works per object, per image region, per ray, or per fragment?

Page 3	Object-Based Algorithms
Source cue	- Geometry-based analysis of scene objects before rasterization / - Compare each scene object against all other scene objects / - Determine visible parts based on virtual camera / - Rasterize the visible scene parts / For each object in sceneObjects
Professor-style explanation	The slide 'Object-Based Algorithms' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Geometry-based analysis of scene objects before rasterization / - Compare each scene object against all other scene objects / - Determine visible parts based on virtual camera / - Rasterize the visible scene parts / For each object in sceneObjects. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Object-Based Algorithms. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Geometry-based analysis of scene objects before rasterization / - Compare each scene object against all other scene objects / - Determine visible parts based on virtual camera / - Rasterize the visible scene parts / For each object in sceneObjects

Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Object-Based Algorithms'?
Page 4	Image-Based Algorithms
Source cue	- Pixel-based analysis of scene objects after rasterization / - Rasterize scene objects based on image resolution / - Investigate for each pixel which scene object is visible / For each pixel in outputRaster / // Determine which scene object is visible at this pixel
Professor-style explanation	The slide 'Image-Based Algorithms' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Pixel-based analysis of scene objects after rasterization / - Rasterize scene objects based on image resolution / - Investigate for each pixel which scene object is visible / For each pixel in outputRaster / // Determine which scene object is visible at this pixel. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Image-Based Algorithms. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Pixel-based analysis of scene objects after rasterization / - Rasterize scene objects based on image resolution / - Investigate for each pixel which scene object is visible / For each pixel in outputRaster / // Determine which scene object is visible at this pixel
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Image-Based Algorithms'?
Page 5	7.1 Object-Based Algorithms
Source cue	7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting

Professor-style explanation	The slide '7.1 Object-Based Algorithms' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: 7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about 7.1 Object-Based Algorithms. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: 7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '7.1 Object-Based Algorithms' works per object, per image region, per ray, or per fragment?
Page 6	7.1 Object-Based Algorithms
Source cue	Exploiting sorting and clipping
Professor-style explanation	The slide '7.1 Object-Based Algorithms' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: Exploiting sorting and clipping. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about 7.1 Object-Based Algorithms. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: Exploiting sorting and clipping
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '7.1 Object-Based Algorithms'?
Page 7	Painter's Algorithm 1/3
Source cue	Bob Ross - Grandeur of Summer

Professor-style explanation	The slide 'Painter's Algorithm 1/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Bob Ross - Grandeur of Summer. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Painter's Algorithm 1/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Bob Ross - Grandeur of Summer
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Painter's Algorithm 1/3' into a causal sentence instead of repeating the slide title?
Page 8	Painter's Algorithm 2/3
Source cue	- First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order
Professor-style explanation	The slide 'Painter's Algorithm 2/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Painter's Algorithm 2/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Painter's Algorithm 2/3' into a causal sentence instead of repeating the slide title?
Page 9	Painter's Algorithm 3/3
Source cue	- Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects

Professor-style explanation	The slide 'Painter's Algorithm 3/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Painter's Algorithm 3/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Painter's Algorithm 3/3' into a causal sentence instead of repeating the slide title?

Page 10	Weiler-Atherton Algorithm 1/3
Source cue	- Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list L in front-to-back order
Professor-style explanation	The slide 'Weiler-Atherton Algorithm 1/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list L in front-to-back order. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Weiler-Atherton Algorithm 1/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list L in front-to-back order
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 1/3'?

Page 11	Weiler-Atherton Algorithm 2/3
---------	-------------------------------

Source cue	- Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts
Professor-style explanation	The slide 'Weiler-Atherton Algorithm 2/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Weiler-Atherton Algorithm 2/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 2/3'?
Page 12	Weiler-Atherton Algorithm 3/3
Source cue	- Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles
Professor-style explanation	The slide 'Weiler-Atherton Algorithm 3/3' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about Weiler-Atherton Algorithm 3/3. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 3/3'?

Page 13	Back Face Culling 1/2
Source cue	- Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object
Professor-style explanation	The slide 'Back Face Culling 1/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Back Face Culling 1/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Back Face Culling 1/2' works per object, per image region, per ray, or per fragment?
Page 14	Back Face Culling 2/2
Source cue	- Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process

Professor-style explanation	The slide 'Back Face Culling 2/2' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: - Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process. The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.
Technical commentary	This slide is about Back Face Culling 2/2. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: - Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'Back Face Culling 2/2' in the rendering pipeline?

Page 15	Robert's Algorithm
Source cue	- First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling
Professor-style explanation	The slide 'Robert's Algorithm' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Robert's Algorithm. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Robert's Algorithm'?

Page 16	Assessment Robert's Algorithm
Source cue	- Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings
Professor-style explanation	The slide 'Assessment Robert's Algorithm' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Assessment Robert's Algorithm. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Assessment Robert's Algorithm'?

Page 17	7.2 Binary Space Partitioning
Source cue	Hierarchical scene bisection

Professor-style explanation	The slide '7.2 Binary Space Partitioning' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Hierarchical scene bisection. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about 7.2 Binary Space Partitioning. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Hierarchical scene bisection
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '7.2 Binary Space Partitioning' into a causal sentence instead of repeating the slide title?

Page 18	Spatial Data Structures
Source cue	- Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes
Professor-style explanation	The slide 'Spatial Data Structures' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Spatial Data Structures. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Spatial Data Structures' into a causal sentence instead of repeating the slide title?

Page 19	BSP Trees 1/2
---------	---------------

Source cue	- BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility
Professor-style explanation	The slide 'BSP Trees 1/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about BSP Trees 1/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Trees 1/2' works per object, per image region, per ray, or per fragment?
Page 20	BSP Trees 2/2
Source cue	- Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C
Professor-style explanation	The slide 'BSP Trees 2/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about BSP Trees 2/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Trees 2/2' works per object, per image region, per ray, or per fragment?

Page 21	Node Data Structure
Source cue	- Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane
Professor-style explanation	The slide 'Node Data Structure' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Node Data Structure. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Node Data Structure' works per object, per image region, per ray, or per fragment?

Page 22	BSP Tree Construction
Source cue	- Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node

Professor-style explanation	The slide 'BSP Tree Construction' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about BSP Tree Construction. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Construction' works per object, per image region, per ray, or per fragment?

Page 23	BSP Tree Construction - Example
Source cue	1 2 / 3 4 / A B E / 3 1
Professor-style explanation	The slide 'BSP Tree Construction - Example' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: 1 2 / 3 4 / A B E / 3 1. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about BSP Tree Construction - Example. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: 1 2 / 3 4 / A B E / 3 1
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Construction - Example' works per object, per image region, per ray, or per fragment?

Page 24	BSP Tree Construction - Pseudo Code
---------	-------------------------------------

Source cue	Function buildBSPTree(BSPNode node, List of Polygons list) // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry
Professor-style explanation	The slide 'BSP Tree Construction - Pseudo Code' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: Function buildBSPTree(BSPNode node, List of Polygons list) // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about BSP Tree Construction - Pseudo Code. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: Function buildBSPTree(BSPNode node, List of Polygons list) // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Construction - Pseudo Code' works per object, per image region, per ray, or per fragment?

Page 25	Visibility Determination
Source cue	- Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order"
Professor-style explanation	The slide 'Visibility Determination' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order". The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about Visibility Determination. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order"
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Visibility Determination' works per object, per image region, per ray, or per fragment?
Page 26	Visibility Determination - Pseudo Code
Source cue	Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) / // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then / // Eye-point is on the back side of the plane
Professor-style explanation	The slide 'Visibility Determination - Pseudo Code' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) / // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then / // Eye-point is on the back side of the plane. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Visibility Determination - Pseudo Code. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) / // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then / // Eye-point is on the back side of the plane
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Visibility Determination - Pseudo Code' works per object, per image region, per ray, or per fragment?
Page 27	BSP Tree Traversal - Example
Source cue	B F / 4 1 2 / F B F B / 3 4 / F B F B

Professor-style explanation	The slide 'BSP Tree Traversal - Example' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: B F / 4 1 2 / F B F B / 3 4 / F B F B. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about BSP Tree Traversal - Example. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: B F / 4 1 2 / F B F B / 3 4 / F B F B
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Traversal - Example' works per object, per image region, per ray, or per fragment?

Page 28	Splitting Planes Selection
Source cue	- Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics
Professor-style explanation	The slide 'Splitting Planes Selection' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Splitting Planes Selection. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Splitting Planes Selection' into a causal sentence instead of repeating the slide title?

Page 29	Traversal Optimization
---------	------------------------

Source cue	- Sub-trees can be skipped during traversal if content is outside view frustum - Testing is done by checking 8 corner points of 3D view frustum - If there are two points on different sides, then the subtree is traversed - If all points are in one subspace, then the other subspace does not have to be traversed (view-frustum culling)
Professor-style explanation	The slide 'Traversal Optimization' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Sub-trees can be skipped during traversal if content is outside view frustum - Testing is done by checking 8 corner points of 3D view frustum - If there are two points on different sides, then the subtree is traversed - If all points are in one subspace, then the other subspace does not have to be traversed (view-frustum culling). The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Traversal Optimization. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Sub-trees can be skipped during traversal if content is outside view frustum - Testing is done by checking 8 corner points of 3D view frustum - If there are two points on different sides, then the subtree is traversed - If all points are in one subspace, then the other subspace does not have to be traversed (view-frustum culling)
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Traversal Optimization' works per object, per image region, per ray, or per fragment?

Page 30	Assessment BSP Trees
Source cue	- Advantages - Simple implementation - Do not need a depth buffer - Applicable to static and dynamic scenes (static scene geometry precalculated, dynamic objects inserted)
Professor-style explanation	The slide 'Assessment BSP Trees' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Advantages - Simple implementation - Do not need a depth buffer - Applicable to static and dynamic scenes (static scene geometry precalculated, dynamic objects inserted). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Assessment BSP Trees. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Advantages / - Simple implementation / - Do not need a depth buffer / - Applicable to static and dynamic scenes / (static scene geometry precalculated, dynamic objects inserted)
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Assessment BSP Trees'?

Page 31	7.3 Warnock Algorithm
Source cue	Image-based divide-and-conquer strategy
Professor-style explanation	The slide '7.3 Warnock Algorithm' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: Image-based divide-and-conquer strategy. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about 7.3 Warnock Algorithm. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: Image-based divide-and-conquer strategy
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '7.3 Warnock Algorithm' works per object, per image region, per ray, or per fragment?

Page 32	Warnock Algorithm
Source cue	- Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions

Professor-style explanation	The slide 'Warnock Algorithm' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Warnock Algorithm. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Warnock Algorithm' works per object, per image region, per ray, or per fragment?

Page 33	Region-Based Visibility Determination
Source cue	R P / Case 1 / - Given viewport region R and set of polygons P, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in P are outside of R R P i P i
Professor-style explanation	The slide 'Region-Based Visibility Determination' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: R P / Case 1 / - Given viewport region R and set of polygons P, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in P are outside of R R P i P i. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Region-Based Visibility Determination. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: R P / Case 1 / - Given viewport region R and set of polygons P, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in P are outside of R R P i P i

Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Region-Based Visibility Determination' works per object, per image region, per ray, or per fragment?
Page 34	Procedure
Source cue	- Start with entire viewport region R / - Subdivide R into four equal subregions / - Terminate if subregion size matches pixel size / - Classify subregions based on trivial cases / - If subregion matches trivial cases: resolve
Professor-style explanation	The slide 'Procedure' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: - Start with entire viewport region R / - Subdivide R into four equal subregions / - Terminate if subregion size matches pixel size / - Classify subregions based on trivial cases / - If subregion matches trivial cases: resolve. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Procedure. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: - Start with entire viewport region R / - Subdivide R into four equal subregions / - Terminate if subregion size matches pixel size / - Classify subregions based on trivial cases / - If subregion matches trivial cases: resolve
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Procedure' changes positions before rasterization?
Page 35	Pseudo Code
Source cue	Procedure warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then / // Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else

Professor-style explanation	The slide 'Pseudo Code' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: Procedure warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then / // Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Pseudo Code. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: Procedure warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then / // Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Pseudo Code' works per object, per image region, per ray, or per fragment?

Page 36	Assessment Warnock Algorithm
Source cue	- Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives
Professor-style explanation	The slide 'Assessment Warnock Algorithm' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Assessment Warnock Algorithm. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.

Check yourself	Can you decide whether 'Assessment Warnock Algorithm' works per object, per image region, per ray, or per fragment?
Page 37	7.4 Depth Buffer Algorithm
Source cue	Exploiting image storage for visibility determination
Professor-style explanation	The slide '7.4 Depth Buffer Algorithm' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Exploiting image storage for visibility determination. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 7.4 Depth Buffer Algorithm. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Exploiting image storage for visibility determination
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '7.4 Depth Buffer Algorithm'?
Page 38	Depth Buffer Algorithm 1/3
Source cue	- Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization
Professor-style explanation	The slide 'Depth Buffer Algorithm 1/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Depth Buffer Algorithm 1/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 1/3'?

Page 39	Depth Buffer Algorithm 2/3
Source cue	- Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an <code>OcclusionRepresentation OR</code> / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by <code>OR</code> Then
Professor-style explanation	The slide 'Depth Buffer Algorithm 2/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an <code>OcclusionRepresentation OR</code> / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by <code>OR</code> Then. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Depth Buffer Algorithm 2/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an <code>OcclusionRepresentation OR</code> / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by <code>OR</code> Then
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 2/3'?

Page 40	Depth Buffer Algorithm 3/3
Source cue	- Depth buffer algorithm performs depth testing for each fragment / - Depth buffer is initialized with background value (e.g., $z = 1$) / - For every scene object / - Rasterize scene object / - For each fragment

Professor-style explanation	The slide 'Depth Buffer Algorithm 3/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Depth buffer algorithm performs depth testing for each fragment / - Depth buffer is initialized with background value (e.g., $z = 1$) / - For every scene object / - Rasterize scene object / - For each fragment. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Depth Buffer Algorithm 3/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Depth buffer algorithm performs depth testing for each fragment / - Depth buffer is initialized with background value (e.g., $z = 1$) / - For every scene object / - Rasterize scene object / - For each fragment
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 3/3'?
Page 41	Pseudo Code - Depth Buffer Test
Source cue	Function zBufferHSR(List of Geometry list) // Initialize z-buffer and color buffer for each pixel on the screen / For x from 0 to Width / For y from 0 to Height / Set depthbuffer[x][y] to 1.0 // Maximum depth value
Professor-style explanation	The slide 'Pseudo Code - Depth Buffer Test' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Function zBufferHSR(List of Geometry list) // Initialize z-buffer and color buffer for each pixel on the screen / For x from 0 to Width / For y from 0 to Height / Set depthbuffer[x][y] to 1.0 // Maximum depth value. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Pseudo Code - Depth Buffer Test. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Function <code>zBufferHSR(List of Geometry list)</code> // Initialize z-buffer and color buffer for each pixel on the screen / For x from 0 to Width / For y from 0 to Height / Set <code>depthbuffer[x][y]</code> to 1.0 // Maximum depth value
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Pseudo Code - Depth Buffer Test'?

Page 42	OpenGL Depth Buffer
Source cue	- OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code>
Professor-style explanation	The slide 'OpenGL Depth Buffer' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code> . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Depth Buffer. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Depth Buffer'?

Page 43	Depth Buffer Precision
Source cue	- Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog

Professor-style explanation	The slide 'Depth Buffer Precision' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Depth Buffer Precision. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Precision'?
Page 44	Z-Fighting 1/2
Source cue	- Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene
Professor-style explanation	The slide 'Z-Fighting 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Z-Fighting 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Z-Fighting 1/2'?
Page 45	Z-Fighting 2/2
Source cue	- Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties
Professor-style explanation	The slide 'Z-Fighting 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Z-Fighting 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Z-Fighting 2/2'?
Page 46	Assessment Depth Buffering
Source cue	- Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries

Professor-style explanation	The slide 'Assessment Depth Buffering' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Assessment Depth Buffering. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Assessment Depth Buffering'?

Page 47	7.5 Depth Buffer Extensions
Source cue	How to render semi-transparent scene objects, and how to generate halos
Professor-style explanation	The slide '7.5 Depth Buffer Extensions' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: How to render semi-transparent scene objects, and how to generate halos. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 7.5 Depth Buffer Extensions. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: How to render semi-transparent scene objects, and how to generate halos
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '7.5 Depth Buffer Extensions'?

Page 48	Semi-Transparent Objects 1/2
Source cue	- Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function <code>draw(Vector scene)</code> / // Clear the framebuffer to prepare for new drawing
Professor-style explanation	The slide 'Semi-Transparent Objects 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function <code>draw(Vector scene)</code> / // Clear the framebuffer to prepare for new drawing. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Semi-Transparent Objects 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function <code>draw(Vector scene)</code> / // Clear the framebuffer to prepare for new drawing
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Semi-Transparent Objects 1/2'?
Page 49	Semi-Transparent Objects 2/2
Source cue	- Issue: correct transparency without presorting of scene objects not possible
Professor-style explanation	The slide 'Semi-Transparent Objects 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Issue: correct transparency without presorting of scene objects not possible. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Semi-Transparent Objects 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Issue: correct transparency without presorting of scene objects not possible

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Semi-Transparent Objects 2/2' into a causal sentence instead of repeating the slide title?
Page 50	Order-Independent Transparency 1/4
Source cue	- Order-independent transparency exploits depth peeling to avoid sorting / - Depth peeling peels away depth layers one-by-one in front-to-back order / - Extract nearest fragments with smallest z values in first rendering pass / - Extract next further fragments in each subsequent rendering pass / - The nth rendering pass delivers the fragments to the nth depth plane
Professor-style explanation	The slide 'Order-Independent Transparency 1/4' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Order-independent transparency exploits depth peeling to avoid sorting / - Depth peeling peels away depth layers one-by-one in front-to-back order / - Extract nearest fragments with smallest z values in first rendering pass / - Extract next further fragments in each subsequent rendering pass / - The nth rendering pass delivers the fragments to the nth depth plane. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Order-Independent Transparency 1/4. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Order-independent transparency exploits depth peeling to avoid sorting / - Depth peeling peels away depth layers one-by-one in front-to-back order / - Extract nearest fragments with smallest z values in first rendering pass / - Extract next further fragments in each subsequent rendering pass / - The nth rendering pass delivers the fragments to the nth depth plane
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Order-Independent Transparency 1/4' works per object, per image region, per ray, or per fragment?
Page 51	Order-Independent Transparency 2/4
Source cue	- Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1

Professor-style explanation	The slide 'Order-Independent Transparency 2/4' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Order-Independent Transparency 2/4. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Order-Independent Transparency 2/4' works per object, per image region, per ray, or per fragment?

Page 52	Order-Independent Transparency 3/4
Source cue	- 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass
Professor-style explanation	The slide 'Order-Independent Transparency 3/4' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Order-Independent Transparency 3/4. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Order-Independent Transparency 3/4'?

Page 53	Order-Independent Transparency 4/4
Source cue	1 layer 2 layers / 3 layers 4 layers
Professor-style explanation	The slide 'Order-Independent Transparency 4/4' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: 1 layer 2 layers / 3 layers 4 layers. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Order-Independent Transparency 4/4. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: 1 layer 2 layers / 3 layers 4 layers
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Order-Independent Transparency 4/4' into a causal sentence instead of repeating the slide title?
Page 54	Generating Haloes 1/2
Source cue	- Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines
Professor-style explanation	The slide 'Generating Haloes 1/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Generating Haloes 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.

Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Generating Haloes 1/2'?
Page 55	Generating Haloes 2/2
Source cue	- Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin"
Professor-style explanation	The slide 'Generating Haloes 2/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: - Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin". The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Generating Haloes 2/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin"
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Generating Haloes 2/2'?
Page 56	7.6 Ray Casting
Source cue	Shooting rays to determine visibility
Professor-style explanation	The slide '7.6 Ray Casting' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: Shooting rays to determine visibility. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about 7.6 Ray Casting. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: Shooting rays to determine visibility
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '7.6 Ray Casting' works per object, per image region, per ray, or per fragment?

Page 57	Ray Casting Principle
Source cue	- Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection)
Professor-style explanation	The slide 'Ray Casting Principle' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection). The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Ray Casting Principle. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection)
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Ray Casting Principle' works per object, per image region, per ray, or per fragment?

Page 58	Ray Initialization 1/3
Source cue	- Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $t / s - e / - p t = e + t s / s - e$

Professor-style explanation	The slide 'Ray Initialization 1/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $t / s - e / - p$ $t = e + t s / s - e$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Ray Initialization 1/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $t / s - e / - p$ $t = e + t s / s - e$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Ray Initialization 1/3'?

Page 59	Ray Initialization 2/3
Source cue	- How can we find s ? / - Assumption: we have a simplified camera model / - Positioned in e / - Orthonormal base $\{u, v, w\}$ / - Orthonormal: orthogonal + normalized
Professor-style explanation	The slide 'Ray Initialization 2/3' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - How can we find s ? / - Assumption: we have a simplified camera model / - Positioned in e / - Orthonormal base $\{u, v, w\}$ / - Orthonormal: orthogonal + normalized. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Ray Initialization 2/3. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - How can we find s ? / - Assumption: we have a simplified camera model / - Positioned in e / - Orthonormal base $\{u, v, w\}$ / - Orthonormal: orthogonal + normalized
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Ray Initialization 2/3' works per object, per image region, per ray, or per fragment?

Page 60	Ray Initialization 3/3
Source cue	- Definition of the pixel grid with $n \times n$ pixels / $x\ y$ / - Size of the pixel raster is $r - l \times t - b$ / - Pixel spacing is / - x-coordinate: $(r - l)/n$
Professor-style explanation	The slide 'Ray Initialization 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Definition of the pixel grid with $n \times n$ pixels / $x\ y$ / - Size of the pixel raster is $r - l \times t - b$ / - Pixel spacing is / - x-coordinate: $(r - l)/n$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Ray Initialization 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Definition of the pixel grid with $n \times n$ pixels / $x\ y$ / - Size of the pixel raster is $r - l \times t - b$ / - Pixel spacing is / - x-coordinate: $(r - l)/n$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Ray Initialization 3/3'?
Page 61	Intersection Calculation
Source cue	- Find smallest positive t for which $p(t)$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest t
Professor-style explanation	The slide 'Intersection Calculation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Find smallest positive t for which $p(t)$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest t . The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Intersection Calculation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Find smallest positive t for which $p(t)$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest t

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Calculation' into a causal sentence instead of repeating the slide title?
Page 62	Implicit Surface Intersection Calculation
Source cue	- Find set of all points with $f(x, y, z) = 0$ / - Examples: plane, sphere, quadrics, ... / $n = (A, B, C)$ / $c = (x, y, z)$ / $c \cdot c \cdot c$
Professor-style explanation	The slide 'Implicit Surface Intersection Calculation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Find set of all points with $f(x, y, z) = 0$ / - Examples: plane, sphere, quadrics, ... / $n = (A, B, C)$ / $c = (x, y, z)$ / $c \cdot c \cdot c$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Implicit Surface Intersection Calculation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Find set of all points with $f(x, y, z) = 0$ / - Examples: plane, sphere, quadrics, ... / $n = (A, B, C)$ / $c = (x, y, z)$ / $c \cdot c \cdot c$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Implicit Surface Intersection Calculation' into a causal sentence instead of repeating the slide title?
Page 63	Intersection Calculation - Triangle
Source cue	- Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates
Professor-style explanation	The slide 'Intersection Calculation - Triangle' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Intersection Calculation - Triangle. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Intersection Calculation - Triangle'?

Page 64	Pseudo Code - Analytic Intersection Computation
Source cue	Function castRay(List sceneObjects, Window win, Camera cam) // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y)
Professor-style explanation	The slide 'Pseudo Code - Analytic Intersection Computation' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: Function castRay(List sceneObjects, Window win, Camera cam) // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y). The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Pseudo Code - Analytic Intersection Computation. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: Function castRay(List sceneObjects, Window win, Camera cam) // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y)
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Pseudo Code - Analytic Intersection Computation' works per object, per image region, per ray, or per fragment?
Page 65	Pseudo Code - Iterative Intersection Calculation (Ray Marching)

Source cue	- Ray march in ϵ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter / // Loop to incrementally trace the ray through the scene
Professor-style explanation	The slide 'Pseudo Code - Iterative Intersection Calculation (Ray Marching)' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Ray march in ϵ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter / // Loop to incrementally trace the ray through the scene. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Pseudo Code - Iterative Intersection Calculation (Ray Marching). The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Ray march in ϵ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter / // Loop to incrementally trace the ray through the scene
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Pseudo Code - Iterative Intersection Calculation (Ray Marching)' works per object, per image region, per ray, or per fragment?
Page 66	Assessment Ray Casting
Source cue	- Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages

Professor-style explanation	The slide 'Assessment Ray Casting' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Assessment Ray Casting. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Assessment Ray Casting' works per object, per image region, per ray, or per fragment?

Page 67	but are crucial for accurate visibility determination
Source cue	- Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable
Professor-style explanation	The slide 'but are crucial for accurate visibility determination' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. On the slide, the concrete items are: - Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about but are crucial for accurate visibility determination. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. Concrete items shown: - Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'but are crucial for accurate visibility determination'?
Page 68	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 69	Practice, Addison-Wesley.
Source cue	- Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,

Professor-style explanation	The slide 'Practice, Addison-Wesley.' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Practice, Addison-Wesley.. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Practice, Addison-Wesley.' into a causal sentence instead of repeating the slide title?

07.1 Object-Based Algorithms

Source location: Section 7.1

Commentary

Object-based visibility algorithms reason about geometry before or while comparing surfaces. They can sort, split, or reject objects based on spatial relations. This is different from simply letting every fragment fight in the depth buffer.

The value of object-based reasoning is reducing work or establishing correct order. The difficulty is that geometry can overlap in complicated ways, so simple global ordering is not always possible.

Mental model

Object-based visibility tries to solve visibility with geometry before final pixels are written.

Check yourself

Why can intersecting polygons make simple depth sorting fail?

07.2 Binary Space Partitioning

Source location: Section 7.2

Commentary

A BSP tree recursively divides space with planes. Once built, it can help traverse geometry in a view-dependent order. This is useful for visibility and ordering because the tree encodes spatial relationships.

The cost is preprocessing and possible splitting of geometry. BSP is a good example of trading memory and setup work for faster or more structured visibility decisions later.

Mental model

A BSP tree stores a recursive answer to 'which side of this plane is the geometry on?'

Check yourself

Why can building a BSP tree require splitting polygons?

07.3 Warnock Algorithm

Source location: Section 7.3

Commentary

Warnock's algorithm works in image space by subdividing regions until visibility is simple enough to decide. If a region is ambiguous, split it. If it is simple, fill it.

This illustrates a general graphics strategy: recursively reduce a hard global problem into smaller local problems. The algorithm is less central in modern OpenGL practice than the depth buffer, but it helps contrast image-space and object-space approaches.

Mental model

When a region is too complicated, subdivide until the answer becomes simple.

Check yourself

What makes Warnock's algorithm image-space rather than object-space?

07.4 Depth Buffer Algorithm

Source location: Section 7.4

Commentary

The depth buffer stores the closest accepted depth at each sample or pixel. For each fragment, compare its depth to the stored depth. If it passes, update the color and depth; otherwise discard it.

The strength of the depth buffer is simplicity and hardware efficiency. The weakness is precision: depth values are finite, and projection distributes precision unevenly. Bad near/far settings can create z-fighting.

Mental model

The depth buffer is a per-sample competition for closest visible fragment.

Check yourself

Why should the near plane not be unnecessarily close to the camera?

07.5 Depth Extensions and Ray Casting

Source location: Sections 7.5-7.6

Commentary

Depth buffer extensions refine or adapt depth-based visibility, for example by handling precision, transparency, or multiple layers more carefully. Standard depth testing alone is not a complete solution for every visibility situation.

Ray casting takes the opposite direction from rasterization: for each image sample, cast a ray into the scene and find the closest intersection. This naturally solves primary visibility but requires intersection work instead of raster coverage work.

Mental model

Rasterization pushes primitives to pixels; ray casting pulls visibility from pixels into the scene.

Check yourself

Why is transparency harder than opaque nearest-surface visibility?

End-of-lecture summary

If you remember only one thing from Lecture 07, remember this: This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones should affect the image.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 08 - Local Illumination

This lecture explains how visible surface points receive color from local light, material, normal, and view information. It is about shading a point, not solving all global light transport.

Source chunk	course_text_parts/03_lectures/08_local-illumination.txt
Extracted slide pages	62
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Local Illumination
Source cue	- Illumination is the process of simulating light interactions in a virtual scene / - Requires light and material properties / - Achieved through illumination model / - Essential for scene perception / without illumination with illumination

Professor-style explanation	The slide 'Local Illumination' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Illumination is the process of simulating light interactions in a virtual scene / - Requires light and material properties / - Achieved through illumination model / - Essential for scene perception / without illumination with illumination. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Local Illumination. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Illumination is the process of simulating light interactions in a virtual scene / - Requires light and material properties / - Achieved through illumination model / - Essential for scene perception / without illumination with illumination
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Local Illumination'?

Page 3	8.1 Physics of Light
Source cue	8.2 Light Sources / 8.3 Material Models / 8.4 Phong Illumination Model / 8.5 Shading
Professor-style explanation	The slide '8.1 Physics of Light' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: 8.2 Light Sources / 8.3 Material Models / 8.4 Phong Illumination Model / 8.5 Shading. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about 8.1 Physics of Light. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: 8.2 Light Sources / 8.3 Material Models / 8.4 Phong Illumination Model / 8.5 Shading
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '8.1 Physics of Light'?

Page 4	8.1 Physics of Light
Source cue	Understanding light in the real world
Professor-style explanation	The slide '8.1 Physics of Light' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Understanding light in the real world. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about 8.1 Physics of Light. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Understanding light in the real world
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '8.1 Physics of Light'?
Page 5	Electromagnetic Radiation
Source cue	- Electromagnetic radiation is defined by a spectrum of wavelengths / Wave length λ / - Visible light is subset of electromagnetic spectrum between 400 and 700nm, / which humans perceive as colors
Professor-style explanation	The slide 'Electromagnetic Radiation' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Electromagnetic radiation is defined by a spectrum of wavelengths / Wave length λ / - Visible light is subset of electromagnetic spectrum between 400 and 700nm, / which humans perceive as colors. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Electromagnetic Radiation. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Electromagnetic radiation is defined by a spectrum of wavelengths / Wave length λ / - Visible light is subset of electromagnetic spectrum between 400 and 700nm, / which humans perceive as colors
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.

Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Electromagnetic Radiation'?
Page 6	Visible Light
Source cue	1000 nm / Infrared / Radio / 700 nm / TV Red
Professor-style explanation	The slide 'Visible Light' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: 1000 nm / Infrared / Radio / 700 nm / TV Red. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Visible Light. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: 1000 nm / Infrared / Radio / 700 nm / TV Red
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Visible Light'?
Page 7	Visible Light
Source cue	1000 nm / Infrared / Radio / 700 nm / TV Red
Professor-style explanation	The slide 'Visible Light' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: 1000 nm / Infrared / Radio / 700 nm / TV Red. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Visible Light. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: 1000 nm / Infrared / Radio / 700 nm / TV Red
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Visible Light'?

Page 8	Light Spectra
Source cue	- Different light sources have different light spectra / - Intensity differs based on wavelength in visible spectrum / [http://www.micro.magnet.fsu.edu/optics/lightandcolor/sources.html]
Professor-style explanation	The slide 'Light Spectra' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Different light sources have different light spectra / - Intensity differs based on wavelength in visible spectrum / [http://www.micro.magnet.fsu.edu/optics/lightandcolor/sources.html]. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Light Spectra. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Different light sources have different light spectra / - Intensity differs based on wavelength in visible spectrum / [http://www.micro.magnet.fsu.edu/optics/lightandcolor/sources.html]
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Spectra'?
Page 9	Black-Body Radiation
Source cue	- A continuous spectrum emitted by a black body at a certain temperature / Steel / > 1570 K / 820 K / Stars
Professor-style explanation	The slide 'Black-Body Radiation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - A continuous spectrum emitted by a black body at a certain temperature / Steel / > 1570 K / 820 K / Stars. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Black-Body Radiation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - A continuous spectrum emitted by a black body at a certain temperature / Steel / > 1570 K / 820 K / Stars
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Black-Body Radiation' into a causal sentence instead of repeating the slide title?
Page 10	Black-Body Radiation
Source cue	Stars / https://www.tec-science.com/thermodynamics/temperature/black-body-radiation/ 23/06/2026 10
Professor-style explanation	The slide 'Black-Body Radiation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Stars / https://www.tec-science.com/thermodynamics/temperature/black-body-radiation/ 23/06/2026 10. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Black-Body Radiation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Stars / https://www.tec-science.com/thermodynamics/temperature/black-body-radiation/ 23/06/2026 10
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Black-Body Radiation' into a causal sentence instead of repeating the slide title?
Page 11	Spectral Density Function (SDF)
Source cue	- Light spectra are represented as spectral density functions / - Spectral density functions provide the light intensity (=energy) per wavelength / 400 Wavelength λ (nm) 700 / ygrenE / 400 Wavelength λ (nm) 700
Professor-style explanation	The slide 'Spectral Density Function (SDF)' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Light spectra are represented as spectral density functions / - Spectral density functions provide the light intensity (=energy) per wavelength / 400 Wavelength λ (nm) 700 / ygrenE / 400 Wavelength λ (nm) 700. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Spectral Density Function (SDF). The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Light spectra are represented as spectral density functions / - Spectral density functions provide the light intensity (=energy) per wavelength / 400 Wavelength λ (nm) 700 / ygrenE / 400 Wavelength λ (nm) 700

Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Spectral Density Function (SDF)'?
Page 12	Light Perception
Source cue	- Humans perceive light spectra / as differently colored lights / - Often described using three / main properties (HSL/HSV model) / - Hue - primary color tone
Professor-style explanation	The slide 'Light Perception' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Humans perceive light spectra / as differently colored lights / - Often described using three / main properties (HSL/HSV model) / - Hue - primary color tone. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Light Perception. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Humans perceive light spectra / as differently colored lights / - Often described using three / main properties (HSL/HSV model) / - Hue - primary color tone
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Perception'?
Page 13	Interpreting SDFs
Source cue	- Shape of an SDF depicts how humans perceive represented light / - Hue: dominant wavelength / - Saturation: % of energy in dom. wavelength / - Luminance: total energy (SDF integral) / 400 Wavelength λ (nm) 700

Professor-style explanation	The slide 'Interpreting SDFs' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Shape of an SDF depicts how humans perceive represented light - Hue: dominant wavelength - Saturation: % of energy in dom. wavelength - Luminance: total energy (SDF integral) / 400 Wavelength λ (nm) 700. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Interpreting SDFs. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Shape of an SDF depicts how humans perceive represented light - Hue: dominant wavelength - Saturation: % of energy in dom. wavelength - Luminance: total energy (SDF integral) / 400 Wavelength λ (nm) 700
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Interpreting SDFs'?
Page 14	Representing SDFs
Source cue	- In spectral rendering SDFs are discretized as arrays - Cell number represents differentiable wavelength ranges - Cell values represent energy per wavelength range - Much more frequently SDFs are represented as three-channel color - HSL values are derived from SDF
Professor-style explanation	The slide 'Representing SDFs' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - In spectral rendering SDFs are discretized as arrays - Cell number represents differentiable wavelength ranges - Cell values represent energy per wavelength range - Much more frequently SDFs are represented as three-channel color - HSL values are derived from SDF. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about Representing SDFs. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - In spectral rendering SDFs are discretized as arrays / - Cell number represents differentiable wavelength ranges / - Cell values represent energy per wavelength range / - Much more frequently SDFs are represented as three-channel color / - HSL values are derived from SDF
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Representing SDFs' works per object, per image region, per ray, or per fragment?

Page 15	8.2 Light Sources
Source cue	How to represent light sources in computer graphics
Professor-style explanation	The slide '8.2 Light Sources' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: How to represent light sources in computer graphics. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about 8.2 Light Sources. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: How to represent light sources in computer graphics
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '8.2 Light Sources'?

Page 16	Light Sources
Source cue	- All light sources have a RGB color property representing their SDF / - Light sources are also defined through their location, orientation and extent / within a virtual scene / - Different light source types omit different spatial properties / - Light sources have no associated geometry

Professor-style explanation	The slide 'Light Sources' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - All light sources have a RGB color property representing their SDF / - Light sources are also defined through their location, orientation and extent / within a virtual scene / - Different light source types omit different spatial properties / - Light sources have no associated geometry. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Light Sources. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - All light sources have a RGB color property representing their SDF / - Light sources are also defined through their location, orientation and extent / within a virtual scene / - Different light source types omit different spatial properties / - Light sources have no associated geometry
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Sources'?

Page 17	Point Light Sources
Source cue	- Point light sources $L = P, C$ have a geometric center $P = (x, y, z)$ / and radiate light with color $C = (R, G, B)$ evenly in all directions
Professor-style explanation	The slide 'Point Light Sources' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Point light sources $L = P, C$ have a geometric center $P = (x, y, z)$ / and radiate light with color $C = (R, G, B)$ evenly in all directions. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Point Light Sources. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Point light sources $L = P, C$ have a geometric center $P = (x, y, z)$ / and radiate light with color $C = (R, G, B)$ evenly in all directions
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.

Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Point Light Sources'?
Page 18	Cone Light Sources
Source cue	- Special point light source with geometric center / - Radiation of light restricted to a cone-shaped area with its tip in the center / - Specified by cut-off angle and exponents for the decrease / of brightness with distance to the center / (spot light exponent)
Professor-style explanation	The slide 'Cone Light Sources' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Special point light source with geometric center / - Radiation of light restricted to a cone-shaped area with its tip in the center / - Specified by cut-off angle and exponents for the decrease / of brightness with distance to the center / (spot light exponent). The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Cone Light Sources. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Special point light source with geometric center / - Radiation of light restricted to a cone-shaped area with its tip in the center / - Specified by cut-off angle and exponents for the decrease / of brightness with distance to the center / (spot light exponent)
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Cone Light Sources'?
Page 19	Light Attenuation
Source cue	- Light attenuation is inversely proportional to the square / of the distance d between the light source and the object / - Attenuation factor / $f = \min(1, \frac{1}{att_c + c \cdot d + c \cdot d^2})$

Professor-style explanation	The slide 'Light Attenuation' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Light attenuation is inversely proportional to the square / of the distance d between the light source and the object / - Attenuation factor $f = \min(1, \frac{1}{att \cdot c + c \cdot d + c \cdot d^2})$. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Light Attenuation. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Light attenuation is inversely proportional to the square / of the distance d between the light source and the object / - Attenuation factor $f = \min(1, \frac{1}{att \cdot c + c \cdot d + c \cdot d^2})$
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Attenuation'?

Page 20	Area Light Sources
Source cue	- Light source with geometric center and geometric extension / - Mostly of planar shape / - Transmission of light over the entire surface analogous to a point light source
Professor-style explanation	The slide 'Area Light Sources' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Light source with geometric center and geometric extension / - Mostly of planar shape / - Transmission of light over the entire surface analogous to a point light source. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Area Light Sources. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Light source with geometric center and geometric extension / - Mostly of planar shape / - Transmission of light over the entire surface analogous to a point light source

Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Area Light Sources'?
Page 21	Directed Light
Source cue	- Infinity-far away imaginary light source / - Light rays run parallel and evenly throughout the scene, / direction is specified as a vector / - Light intensity does not diminish / with increasing distance
Professor-style explanation	The slide 'Directed Light' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Infinity-far away imaginary light source / - Light rays run parallel and evenly throughout the scene, / direction is specified as a vector / - Light intensity does not diminish / with increasing distance. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Directed Light. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Infinity-far away imaginary light source / - Light rays run parallel and evenly throughout the scene, / direction is specified as a vector / - Light intensity does not diminish / with increasing distance
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Directed Light' works per object, per image region, per ray, or per fragment?
Page 22	Local vs. Global Illumination
Source cue	- Simple case: Local lighting / - Consideration of direct illumination by light sources / (without obstacles or reflection) / - Simplified light models approximate / real lighting effect

Professor-style explanation	The slide 'Local vs. Global Illumination' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Simple case: Local lighting / - Consideration of direct illumination by light sources / (without obstacles or reflection) / - Simplified light models approximate / real lighting effect. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Local vs. Global Illumination. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Simple case: Local lighting / - Consideration of direct illumination by light sources / (without obstacles or reflection) / - Simplified light models approximate / real lighting effect
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Local vs. Global Illumination'?

Page 23	Ambient Light
Source cue	- Undirected light rays, evenly distributed / - Light intensity constant, regardless of distance / - Simulates general room brightness (basic brightness)
Professor-style explanation	The slide 'Ambient Light' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: - Undirected light rays, evenly distributed / - Light intensity constant, regardless of distance / - Simulates general room brightness (basic brightness). The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Ambient Light. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: - Undirected light rays, evenly distributed / - Light intensity constant, regardless of distance / - Simulates general room brightness (basic brightness)
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.

Check yourself	Can you decide whether 'Ambient Light' works per object, per image region, per ray, or per fragment?
Page 24	8.3 Material Models
Source cue	How to represent materials in computer graphics
Professor-style explanation	The slide '8.3 Material Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: How to represent materials in computer graphics. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about 8.3 Material Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: How to represent materials in computer graphics
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by '8.3 Material Models' before rendering begins?
Page 25	Material Models
Source cue	- Material models capture the different properties of materials / - Materials can have a wide variety of appearances / - Different groups of materials can be classified / based on appearance / - Metal - shiny and specular appearance
Professor-style explanation	The slide 'Material Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Material models capture the different properties of materials / - Materials can have a wide variety of appearances / - Different groups of materials can be classified / based on appearance / - Metal - shiny and specular appearance. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Material Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Material models capture the different properties of materials / - Materials can have a wide variety of appearances / - Different groups of materials can be classified / based on appearance / - Metal - shiny and specular appearance

Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Material Models' before rendering begins?
Page 26	Materials vs. Textures
Source cue	- Material models specify the material properties for an infinitesimal surface / - No material variance with respect to location is supported / (i.e., structures such as wood grain cannot be represented) / - Textures are able to support material variance with respect to location
Professor-style explanation	The slide 'Materials vs. Textures' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Material models specify the material properties for an infinitesimal surface / - No material variance with respect to location is supported / (i.e., structures such as wood grain cannot be represented) / - Textures are able to support material variance with respect to location. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Materials vs. Textures. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Material models specify the material properties for an infinitesimal surface / - No material variance with respect to location is supported / (i.e., structures such as wood grain cannot be represented) / - Textures are able to support material variance with respect to location
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Materials vs. Textures'?
Page 27	Light Material Interaction 1/3
Source cue	- Material appearance is influenced by three wavelength-dependent effects / - Reflection - light energy bounces off / - Absorption - light energy is reduced / - Transmission - light passes through the object (refraction occurs) / R = reflected energy

Professor-style explanation	The slide 'Light Material Interaction 1/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Material appearance is influenced by three wavelength-dependent effects / - Reflection - light energy bounces off / - Absorption - light energy is reduced / - Transmission - light passes through the object (refraction occurs) / R = reflected energy. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Light Material Interaction 1/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Material appearance is influenced by three wavelength-dependent effects / - Reflection - light energy bounces off / - Absorption - light energy is reduced / - Transmission - light passes through the object (refraction occurs) / R = reflected energy
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Material Interaction 1/3'?
Page 28	Light Material Interaction 2/3
Source cue	- Material properties determine weight of the individual effects / opaque colored material opaque black material / $A > 0$, $T = 0$, $R > 0$ $A = I$, $T = 0$, $R = 0$ / perfect mirror material transparent material / $A = 0$, $T = 0$, $R = I$ $A \sim 0$, $R \sim 0$, $T \sim I$
Professor-style explanation	The slide 'Light Material Interaction 2/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Material properties determine weight of the individual effects / opaque colored material opaque black material / $A > 0$, $T = 0$, $R > 0$ $A = I$, $T = 0$, $R = 0$ / perfect mirror material transparent material / $A = 0$, $T = 0$, $R = I$ $A \sim 0$, $R \sim 0$, $T \sim I$. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Light Material Interaction 2/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Material properties determine weight of the individual effects / opaque colored material opaque black material / $A > 0$, $T = 0$, $R > 0$ $A = I$, $T = 0$, $R = 0$ / perfect mirror material transparent material / $A = 0$, $T = 0$, $R = I$ $A \approx 0$, $R \approx 0$, $T \approx I$
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Material Interaction 2/3'?

Page 29	Light Material Interaction 3/3
Source cue	- Light-material interaction described by the Spectral Response Function (SRF) / 400 Wavelength λ (nm) 700 / ygrenE / 100% / All pass Filter (ideal, real)
Professor-style explanation	The slide 'Light Material Interaction 3/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Light-material interaction described by the Spectral Response Function (SRF) / 400 Wavelength λ (nm) 700 / ygrenE / 100% / All pass Filter (ideal, real). The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Light Material Interaction 3/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Light-material interaction described by the Spectral Response Function (SRF) / 400 Wavelength λ (nm) 700 / ygrenE / 100% / All pass Filter (ideal, real)
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Light Material Interaction 3/3'?

Page 30	Reflection = SDF x SRF
Source cue	- Light-material interaction results are the product of the SDF and SRF / $100\% \times 100\%$ / SDF SRF / 0% 0% / 400 Wavelength λ (nm) 700 400 Wavelength λ (nm) 700

Professor-style explanation	The slide 'Reflection = SDF x SRF' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Light-material interaction results are the product of the SDF and SRF / $100\% \times 100\%$ / SDF SRF / 0% 0% / 400 Wavelength λ (nm) 700 400 Wavelength λ (nm) 700. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Reflection = SDF x SRF. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Light-material interaction results are the product of the SDF and SRF / $100\% \times 100\%$ / SDF SRF / 0% 0% / 400 Wavelength λ (nm) 700 400 Wavelength λ (nm) 700
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Reflection = SDF x SRF'?

Page 31	Reflection Modeling 1/2
Source cue	- All light sources can be described by their Spectral Density Functions / - Natural light: sun, fire, ... / - Artificial light: light bulb, laser, neon tube, LED, ... / - All material properties can be described by Spectral Response Functions / - Glass, Water: SRF for transmission
Professor-style explanation	The slide 'Reflection Modeling 1/2' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - All light sources can be described by their Spectral Density Functions / - Natural light: sun, fire, ... / - Artificial light: light bulb, laser, neon tube, LED, ... / - All material properties can be described by Spectral Response Functions / - Glass, Water: SRF for transmission. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Reflection Modeling 1/2. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - All light sources can be described by their Spectral Density Functions / - Natural light: sun, fire, ... / - Artificial light: light bulb, laser, neon tube, LED, ... / - All material properties can be described by Spectral Response Functions / - Glass, Water: SRF for transmission

Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Reflection Modeling 1/2' before rendering begins?
Page 32	Reflection Modeling 2/2
Source cue	- Synthetic and natural objects are in general not self-emitting / - Light is reflected at its surfaces, i.e., re-emitted into the scene / - Reflection is dependent on surface properties / - Rough surface: scattering of light / - Smooth surface: reflection of light
Professor-style explanation	The slide 'Reflection Modeling 2/2' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Synthetic and natural objects are in general not self-emitting / - Light is reflected at its surfaces, i.e., re-emitted into the scene / - Reflection is dependent on surface properties / - Rough surface: scattering of light / - Smooth surface: reflection of light. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Reflection Modeling 2/2. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Synthetic and natural objects are in general not self-emitting / - Light is reflected at its surfaces, i.e., re-emitted into the scene / - Reflection is dependent on surface properties / - Rough surface: scattering of light / - Smooth surface: reflection of light
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Reflection Modeling 2/2' before rendering begins?
Page 33	8.4 Phong Illumination Model
Source cue	Determining the color of vertices
Professor-style explanation	The slide '8.4 Phong Illumination Model' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Determining the color of vertices. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about 8.4 Phong Illumination Model. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Determining the color of vertices
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '8.4 Phong Illumination Model'?

Page 34	Empirical Observations
Source cue	- Light reflections are / - Dependent on the position of the observer / "... position of the observer..." / - Dependent on the material properties of the surface / "... the specular properties of the object." Photo
Professor-style explanation	The slide 'Empirical Observations' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Light reflections are / - Dependent on the position of the observer / "... position of the observer..." / - Dependent on the material properties of the surface / "... the specular properties of the object." Photo. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Empirical Observations. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Light reflections are / - Dependent on the position of the observer / "... position of the observer..." / - Dependent on the material properties of the surface / "... the specular properties of the object." Photo
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Empirical Observations'?

Page 35	Illumination Models
Source cue	- Illumination models calculate the color of scene objects / based on the following properties / - Location - Position $P = (x, y, z)$ of the selected point P / - Orientation - normal direction $N = (n_x, n_y, n_z)$ of surface in $P / x y z$

Professor-style explanation	The slide 'Illumination Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. On the slide, the concrete items are: - Illumination models calculate the color of scene objects / based on the following properties / - Location - Position $P = (x, y, z)$ of the selected point P / - Orientation - normal direction $N = (n_x, n_y, n_z)$ of surface in $P / x y z$. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Illumination Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. Concrete items shown: - Illumination models calculate the color of scene objects / based on the following properties / - Location - Position $P = (x, y, z)$ of the selected point P / - Orientation - normal direction $N = (n_x, n_y, n_z)$ of surface in $P / x y z$
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Illumination Models' before rendering begins?

Page 36	yaR
Source cue	thgiL / Diffuse Intensity 1/3 / - Simulation of light reflections on diffuse surfaces / - Model assumption / - Diffuse light reflections are non-directional (scattering)
Professor-style explanation	The slide 'yaR' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: thgiL / Diffuse Intensity 1/3 / - Simulation of light reflections on diffuse surfaces / - Model assumption / - Diffuse light reflections are non-directional (scattering). The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about yaR. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: thgiL / Diffuse Intensity 1/3 / - Simulation of light reflections on diffuse surfaces / - Model assumption / - Diffuse light reflections are non-directional (scattering)
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.

Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'yaR'?
Page 37	Diffuse Intensity 2/3
Source cue	- Equation $I = I_k \cdot n \cdot L \cdot \max(0,)$ / diffuse d d / - k diffuse material properties (surface dependent) / - L diffuse light intensity (light source dependent)
Professor-style explanation	The slide 'Diffuse Intensity 2/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Equation $I = I_k \cdot n \cdot L \cdot \max(0,)$ / diffuse d d / - k diffuse material properties (surface dependent) / - L diffuse light intensity (light source dependent). The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Diffuse Intensity 2/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Equation $I = I_k \cdot n \cdot L \cdot \max(0,)$ / diffuse d d / - k diffuse material properties (surface dependent) / - L diffuse light intensity (light source dependent)
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Diffuse Intensity 2/3'?
Page 38	Diffuse Intensity 3/3
Source cue	$I = k \cdot L \cdot \max(0, \blacksquare \cdot n)$ / diffuse d d / https://bit.ly/2XFhbcM / Extrahierte Tabellen: / [Tabelle 1]
Professor-style explanation	The slide 'Diffuse Intensity 3/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: $I = k \cdot L \cdot \max(0, \blacksquare \cdot n)$ / diffuse d d / https://bit.ly/2XFhbcM / Extrahierte Tabellen: / [Tabelle 1]. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Diffuse Intensity 3/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: $I = k \cdot L \cdot \max(0, \blacksquare \cdot n)$ / diffuse d d / https://bit.ly/2XFhbcM / Extrahierte Tabellen: / [Tabelle 1]
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Diffuse Intensity 3/3' into a causal sentence instead of repeating the slide title?
Page 39	Ambient Intensity 1/2
Source cue	- Simulation of the general room brightness / - Indirect light incidence or multiple reflections / - Model assumptions / - Constant influence on all scene objects / - No influence of position or alignment
Professor-style explanation	The slide 'Ambient Intensity 1/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Simulation of the general room brightness / - Indirect light incidence or multiple reflections / - Model assumptions / - Constant influence on all scene objects / - No influence of position or alignment. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Ambient Intensity 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Simulation of the general room brightness / - Indirect light incidence or multiple reflections / - Model assumptions / - Constant influence on all scene objects / - No influence of position or alignment
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Ambient Intensity 1/2'?
Page 40	Ambient Intensity 2/2
Source cue	$I = k \cdot L / \text{ambient } a$ / https://bit.ly/2XGn61n / Extrahierte Tabellen: / [Tabelle 1]
Professor-style explanation	The slide 'Ambient Intensity 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: $I = k \cdot L / \text{ambient } a$ / https://bit.ly/2XGn61n / Extrahierte Tabellen: / [Tabelle 1]. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Ambient Intensity 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: $I = k \cdot L / \text{ambient } a$ / https://bit.ly/2XGn61n / Extrahierte Tabellen: / [Tabelle 1]

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Ambient Intensity 2/2' into a causal sentence instead of repeating the slide title?
Page 41	Specular Intensity 1/3
Source cue	- Simulation of light reflections on specular surfaces / - Model assumption / - Reflection of light are directed (reflection) / - Observations / - Reflections of light are observer-dependent
Professor-style explanation	The slide 'Specular Intensity 1/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Simulation of light reflections on specular surfaces / - Model assumption / - Reflection of light are directed (reflection) / - Observations / - Reflections of light are observer-dependent. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Specular Intensity 1/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Simulation of light reflections on specular surfaces / - Model assumption / - Reflection of light are directed (reflection) / - Observations / - Reflections of light are observer-dependent
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Specular Intensity 1/3'?
Page 42	Reflection Vector r
Source cue	- Calculation of the reflection vector / - Incidence angle corresponds to exit angle / - Equation $r = 2n \cdot n - I$
Professor-style explanation	The slide 'Reflection Vector r ' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Calculation of the reflection vector / - Incidence angle corresponds to exit angle / - Equation $r = 2n \cdot n - I$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Reflection Vector $\mathbf{r}$. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Calculation of the reflection vector / - Incidence angle corresponds to exit angle / - Equation $\ \mathbf{r}\ / \ \mathbf{r}\ = 2\mathbf{n} \cdot \mathbf{n} \cdot \ \mathbf{l}\ - \ \mathbf{l}\ $
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Reflection Vector $\mathbf{r}$ ' into a causal sentence instead of repeating the slide title?

Page 43	Specular Intensity 2/3
Source cue	- Equation $I = k \mathbf{r} \cdot \mathbf{v} \cdot L \cdot \max(0, \dots)$ / specular s / - k specular material properties (surface dependant) / - L specular light intensity (light source dependent)
Professor-style explanation	The slide 'Specular Intensity 2/3' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Equation $I = k \mathbf{r} \cdot \mathbf{v} \cdot L \cdot \max(0, \dots)$ / specular s / - k specular material properties (surface dependant) / - L specular light intensity (light source dependent). The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Specular Intensity 2/3. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Equation $I = k \mathbf{r} \cdot \mathbf{v} \cdot L \cdot \max(0, \dots)$ / specular s / - k specular material properties (surface dependant) / - L specular light intensity (light source dependent)
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Specular Intensity 2/3'?

Page 44	Specular Exponent p
Source cue	- p influences fall off of specular highlight ($p \in [1, M]$ mit $M \approx 100$) / $p = 1$ / $\max(0, \mathbf{r} \cdot \mathbf{v})^1$ / $p = 5$ / $\max(0, \mathbf{r} \cdot \mathbf{v})^5$

Professor-style explanation	The slide 'Specular Exponent p' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - p influences fall off of specular highlight ($p \in [1, M]$ mit $M \approx 100$) / $p = 1 / \max(0, \mathbf{r} \cdot \mathbf{v})^1$ / $p = 5 / \max(0, \mathbf{r} \cdot \mathbf{v})^5$. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Specular Exponent p. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - p influences fall off of specular highlight ($p \in [1, M]$ mit $M \approx 100$) / $p = 1 / \max(0, \mathbf{r} \cdot \mathbf{v})^1$ / $p = 5 / \max(0, \mathbf{r} \cdot \mathbf{v})^5$
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Specular Exponent p'?
Page 45	Specular Intensity 3/3
Source cue	$\mathbf{r} = 2 \cdot \mathbf{n} \cdot \mathbf{n} \cdot \mathbf{l} - \mathbf{l} / \text{https://bit.ly/2XbPc7s}$ $I = k \cdot L \cdot \max(0, \mathbf{r} \cdot \mathbf{v})^p$ / specular s s / Extrahierte Tabellen: / [Tabelle 1]
Professor-style explanation	The slide 'Specular Intensity 3/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: $\mathbf{r} = 2 \cdot \mathbf{n} \cdot \mathbf{n} \cdot \mathbf{l} - \mathbf{l} / \text{https://bit.ly/2XbPc7s}$ $I = k \cdot L \cdot \max(0, \mathbf{r} \cdot \mathbf{v})^p$ / specular s s / Extrahierte Tabellen: / [Tabelle 1]. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Specular Intensity 3/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: $\mathbf{r} = 2 \cdot \mathbf{n} \cdot \mathbf{n} \cdot \mathbf{l} - \mathbf{l} / \text{https://bit.ly/2XbPc7s}$ $I = k \cdot L \cdot \max(0, \mathbf{r} \cdot \mathbf{v})^p$ / specular s s / Extrahierte Tabellen: / [Tabelle 1]
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Specular Intensity 3/3' into a causal sentence instead of repeating the slide title?
Page 46	Blinn-Phong Illumination Model 1/2

Source cue	- Alternative simulation of specular reflections / - Replace mirror direction r with Halfway-Vector h / - $h = \frac{l + v}{2}$ / - h is halfway between l and v / - Equation
Professor-style explanation	The slide 'Blinn-Phong Illumination Model 1/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Alternative simulation of specular reflections / - Replace mirror direction r with Halfway-Vector h / - $h = \frac{l + v}{2}$ / - h is halfway between l and v / - Equation. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Blinn-Phong Illumination Model 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Alternative simulation of specular reflections / - Replace mirror direction r with Halfway-Vector h / - $h = \frac{l + v}{2}$ / - h is halfway between l and v / - Equation
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Blinn-Phong Illumination Model 1/2'?
Page 47	Blinn-Phong Illumination Model 2/2
Source cue	$h = \frac{l + v}{2}$ / https://bit.ly/2ICDwBD $I = k \cdot L \cdot \max(0, h \cdot n)^p$ / specular $s s$ / Extrahierte Tabellen: / [Tabelle 1]
Professor-style explanation	The slide 'Blinn-Phong Illumination Model 2/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: $h = \frac{l + v}{2}$ / https://bit.ly/2ICDwBD $I = k \cdot L \cdot \max(0, h \cdot n)^p$ / specular $s s$ / Extrahierte Tabellen: / [Tabelle 1]. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Blinn-Phong Illumination Model 2/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: $h = \frac{l + v}{2}$ / https://bit.ly/2ICDwBD $I = k \cdot L \cdot \max(0, h \cdot n)^p$ / specular $s s$ / Extrahierte Tabellen: / [Tabelle 1]

Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Blinn-Phong Illumination Model 2/2'?
Page 48	Phong Illumination Model
Source cue	- Additive combination of all factors $I = k \cdot L / \text{ambient } a / I = k \cdot L \cdot \max(0, n \cdot l) / \text{diffuse } d$
Professor-style explanation	The slide 'Phong Illumination Model' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Additive combination of all factors $I = k \cdot L / \text{ambient } a / I = k \cdot L \cdot \max(0, n \cdot l) / \text{diffuse } d$. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Phong Illumination Model. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Additive combination of all factors $I = k \cdot L / \text{ambient } a / I = k \cdot L \cdot \max(0, n \cdot l) / \text{diffuse } d$
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Phong Illumination Model'?
Page 49	Phong Illumination Model
Source cue	- Additive combination of all factors $I = k \cdot L / \text{ambient } a / I = k \cdot L \cdot \max(0, n \cdot l) / \text{diffuse } d$
Professor-style explanation	The slide 'Phong Illumination Model' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Additive combination of all factors $I = k \cdot L / \text{ambient } a / I = k \cdot L \cdot \max(0, n \cdot l) / \text{diffuse } d$. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Phong Illumination Model. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Additive combination of all factors / $I = k \cdot L + \text{ambient } a$ / $I = k \cdot L \cdot \max(0, n \cdot l) + \text{diffuse } d$
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Phong Illumination Model'?

Page 50	8.5 Shading
Source cue	Coloring triangles when using vertex-based illumination
Professor-style explanation	The slide '8.5 Shading' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Coloring triangles when using vertex-based illumination. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 8.5 Shading. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Coloring triangles when using vertex-based illumination
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '8.5 Shading'?

Page 51	Shading
Source cue	- Illumination model usually applied in the fragment stage / - Vertex-based illumination also possible to save compute / - Requires to spread the illumination information of the vertices / over the triangle / Flat Gouroud Phong

Professor-style explanation	The slide 'Shading' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Illumination model usually applied in the fragment stage / - Vertex-based illumination also possible to save compute / - Requires to spread the illumination information of the vertices / over the triangle / Flat Gouroud Phong. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Shading. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Illumination model usually applied in the fragment stage / - Vertex-based illumination also possible to save compute / - Requires to spread the illumination information of the vertices / over the triangle / Flat Gouroud Phong
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Shading'?

Page 52	Shading Computation
Source cue	- Tasks of shading procedures / - Colorize surfaces based on colors calculated by illumination model / - Shading is closely linked to the rasterization process and integrated into it / - Distribute lighting and shading / - Calculate light intensity for surface points
Professor-style explanation	The slide 'Shading Computation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Tasks of shading procedures / - Colorize surfaces based on colors calculated by illumination model / - Shading is closely linked to the rasterization process and integrated into it / - Distribute lighting and shading / - Calculate light intensity for surface points. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Shading Computation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Tasks of shading procedures / - Colorize surfaces based on colors calculated by illumination model / - Shading is closely linked to the rasterization process and integrated into it / - Distribute lighting and shading / - Calculate light intensity for surface points

Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Shading Computation'?
Page 53	Flat Shading
Source cue	- Operation / - Calling the lighting model for a single point of a surface / (e.g., polygon center) / - Shading of the entire polygon in the determined intensity / - Useful if the following assumptions apply
Professor-style explanation	The slide 'Flat Shading' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Operation / - Calling the lighting model for a single point of a surface / (e.g., polygon center) / - Shading of the entire polygon in the determined intensity / - Useful if the following assumptions apply. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Flat Shading. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Operation / - Calling the lighting model for a single point of a surface / (e.g., polygon center) / - Shading of the entire polygon in the determined intensity / - Useful if the following assumptions apply
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Flat Shading'?
Page 54	Gouraud Shading 1/5
Source cue	- Shading method for polygons based on the interpolation / of intensity values at the polygon vertices / - Requirements / - Area decomposition in polygons (usually triangles) / - Normals for polygon corners (vertex normals)

Professor-style explanation	The slide 'Gouraud Shading 1/5' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Shading method for polygons based on the interpolation of intensity values at the polygon vertices - Requirements - Area decomposition in polygons (usually triangles) - Normals for polygon corners (vertex normals). The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Gouraud Shading 1/5. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Shading method for polygons based on the interpolation of intensity values at the polygon vertices - Requirements - Area decomposition in polygons (usually triangles) - Normals for polygon corners (vertex normals)
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Gouraud Shading 1/5'?

Page 55	Gouraud Shading 2/5
Source cue	- Proceeding - Compute intensity values I for polygon corners v_i - Linear interpolation of the intensity values along the edges - Linear interpolation of the intensity values along the scanlines
Professor-style explanation	The slide 'Gouraud Shading 2/5' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Proceeding - Compute intensity values I for polygon corners v_i - Linear interpolation of the intensity values along the edges - Linear interpolation of the intensity values along the scanlines. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Gouraud Shading 2/5. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Proceeding - Compute intensity values I for polygon corners v_i - Linear interpolation of the intensity values along the edges - Linear interpolation of the intensity values along the scanlines

Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Gouraud Shading 2/5'?
Page 56	Gouraud Shading 3/5
Source cue	- Shading at conceptually smooth transitions / - Geometrically "hard" edges instead of conceptually "soft" edges / - Gouraud shading can create visually smooth transitions / through smooth shading / - Example: P / P smooth transition, P / P hard transition
Professor-style explanation	The slide 'Gouraud Shading 3/5' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Shading at conceptually smooth transitions / - Geometrically "hard" edges instead of conceptually "soft" edges / - Gouraud shading can create visually smooth transitions / through smooth shading / - Example: P / P smooth transition, P / P hard transition. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Gouraud Shading 3/5. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Shading at conceptually smooth transitions / - Geometrically "hard" edges instead of conceptually "soft" edges / - Gouraud shading can create visually smooth transitions / through smooth shading / - Example: P / P smooth transition, P / P hard transition
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Gouraud Shading 3/5'?
Page 57	Gouraud Shading 4/5
Source cue	- interpolation problems / - The shading inside a triangle results from interpolation / - Intensity fluctuations in the interior are not taken into account / - Intensity fluctuations can only be considered by finer tessellation / - Example: Spotlight shines into the interior of a triangle

Professor-style explanation	The slide 'Gouraud Shading 4/5' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - interpolation problems / - The shading inside a triangle results from interpolation / - Intensity fluctuations in the interior are not taken into account / - Intensity fluctuations can only be considered by finer tessellation / - Example: Spotlight shines into the interior of a triangle. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Gouraud Shading 4/5. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - interpolation problems / - The shading inside a triangle results from interpolation / - Intensity fluctuations in the interior are not taken into account / - Intensity fluctuations can only be considered by finer tessellation / - Example: Spotlight shines into the interior of a triangle
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Gouraud Shading 4/5'?

Page 58	Gouraud Shading 5/5
Source cue	- Interpolation problems / - Contiguous polygons that do not fully share the same edges / will be shaded differently
Professor-style explanation	The slide 'Gouraud Shading 5/5' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Interpolation problems / - Contiguous polygons that do not fully share the same edges / will be shaded differently. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Gouraud Shading 5/5. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Interpolation problems / - Contiguous polygons that do not fully share the same edges / will be shaded differently
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.

Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Gouraud Shading 5/5'?
Page 59	Phong Shading 1/2
Source cue	- Interpolation of the corner normals instead of the corner intensities / - Proceeding / - Calculation of the corner normals / - Interpolation of the normals between the corners / - Interpolation between the endpoints of a scanline
Professor-style explanation	The slide 'Phong Shading 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Interpolation of the corner normals instead of the corner intensities / - Proceeding / - Calculation of the corner normals / - Interpolation of the normals between the corners / - Interpolation between the endpoints of a scanline. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Phong Shading 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Interpolation of the corner normals instead of the corner intensities / - Proceeding / - Calculation of the corner normals / - Interpolation of the normals between the corners / - Interpolation between the endpoints of a scanline
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Phong Shading 1/2'?
Page 60	Phong Shading 2/2
Source cue	- Advantages / - Pixel-based (i.e., image-precise) evaluation of the light sources / - Highly focused highlights at corners are not interpolated over the edge / - Highlights inside a triangle can be displayed / - Disadvantage

Professor-style explanation	The slide 'Phong Shading 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Advantages / - Pixel-based (i.e., image-precise) evaluation of the light sources / - Highly focused highlights at corners are not interpolated over the edge / - Highlights inside a triangle can be displayed / - Disadvantage. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Phong Shading 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Advantages / - Pixel-based (i.e., image-precise) evaluation of the light sources / - Highly focused highlights at corners are not interpolated over the edge / - Highlights inside a triangle can be displayed / - Disadvantage
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Phong Shading 2/2'?
Page 61	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 62	Lesen und Ausprobieren

Source cue	- Zum Lesen - Zum Ausprobieren / Marschner, Steve und Peter Shirley: / - Simulation von drei / Fundamentals of Computer Graphics, / ausgewählten Materialien
Professor-style explanation	The slide 'Lesen und Ausprobieren' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: - Zum Lesen - Zum Ausprobieren / Marschner, Steve und Peter Shirley: / - Simulation von drei / Fundamentals of Computer Graphics, / ausgewählten Materialien. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Lesen und Ausprobieren. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Zum Lesen - Zum Ausprobieren / Marschner, Steve und Peter Shirley: / - Simulation von drei / Fundamentals of Computer Graphics, / ausgewählten Materialien
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Lesen und Ausprobieren'?

08.1 Physics of Light

Source location: Section 8.1

Commentary

The physics slides provide intuition for light as energy interacting with surfaces. For this course, the goal is not full physical simulation but a usable model for real-time rendering.

Important terms are reflection, absorption, wavelength/color, intensity, and direction. These terms become shader inputs or material parameters later. The simplified model must still preserve the relation between light direction, surface orientation, and observed brightness.

Mental model

Shading is an approximation of light-surface interaction that is cheap enough for rendering.

Check yourself

Why does surface orientation affect diffuse brightness?

08.2 Light Sources

Source location: Section 8.2

Commentary

Light source models define where light comes from and how its direction and intensity are computed. Directional lights approximate distant sources with parallel rays. Point lights emit from a position and often include attenuation. Spotlights add a directional cone.

In shader code, the type of light determines how you compute the light vector. A directional light can use a fixed direction; a point light requires subtracting the surface position from the light position.

Mental model

Different light types mainly change how the light direction and intensity are computed at the surface point.

Check yourself

What is the difference between a directional light vector and a point light vector?

08.3 Material Models

Source location: Section 8.3

Commentary

A material model tells the shader how a surface responds to light. Diffuse material scatters light broadly; specular material creates view-dependent highlights; ambient terms approximate background illumination.

Material coefficients are not arbitrary decoration. They scale the contribution of each lighting term. If the specular coefficient is zero, the surface should not show a specular highlight under that model.

Mental model

Light asks what arrives; material answers how the surface reacts.

Check yourself

Which material parameter would you adjust to reduce shiny highlights?

08.4 Phong Illumination Model

Source location: Section 8.4

Commentary

The Phong illumination model combines ambient, diffuse, and specular terms. Diffuse lighting uses the angle between normal and light direction. Specular lighting also depends on the viewer because highlights move with the view direction.

The formula is less important than the decomposition. Ambient is a rough constant approximation. Diffuse is orientation-dependent matte reflection. Specular is view-dependent shininess. A correct answer should name the vectors and normalize them.

Mental model

Phong-style lighting is a sum of simple terms, each modeling a different visual effect.

Check yourself

Why does the specular term depend on the view direction?

08.5 Shading

Source location: Section 8.5

Commentary

Shading decides where the illumination calculation is evaluated and how results are interpolated. Flat shading uses one value per primitive. Gouraud shading evaluates at vertices and interpolates colors. Phong shading interpolates normals and evaluates lighting per fragment.

Per-fragment shading is more expensive but can represent highlights more accurately. This connects directly to the pipeline: interpolation during rasterization provides the data used by the fragment shader.

Mental model

The shading method decides what is computed at vertices and what is computed at fragments.

Check yourself

Why can Gouraud shading miss a small specular highlight?

End-of-lecture summary

If you remember only one thing from Lecture 08, remember this: This lecture explains how visible surface points receive color from local light, material, normal, and view information. It is about shading a point, not solving all global light transport.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 09 - Texturing

This lecture treats textures as sampled data for shading, not merely images glued onto objects. It connects UV coordinates, sampling, filtering, mipmaps, OpenGL texture state, and advanced texture uses.

Source chunk	course_text_parts/03_lectures/09_texturing.txt
Extracted slide pages	133
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Textures: Sampled Data for Shading
Source cue	■ Textures are sampled data fields used during rendering / ■ Most common case: 2D image data mapped onto 3D geometry / ■ Texture values can represent color, transparency, normals, roughness, height, masks, or arbitrary / lookup data

Professor-style explanation	The slide 'Textures: Sampled Data for Shading' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Textures are sampled data fields used during rendering / ■ Most common case: 2D image data mapped onto 3D geometry / ■ Texture values can represent color, transparency, normals, roughness, height, masks, or arbitrary / lookup data. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Textures: Sampled Data for Shading. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Textures are sampled data fields used during rendering / ■ Most common case: 2D image data mapped onto 3D geometry / ■ Texture values can represent color, transparency, normals, roughness, height, masks, or arbitrary / lookup data
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Textures: Sampled Data for Shading'?

Page 3	Texturing Example
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Texturing Example' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Texturing Example. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Texturing Example' into a causal sentence instead of repeating the slide title?

Page 4	Texture Mapping
--------	-----------------

Source cue	■ Texture mapping / • Application of texture defined in multidimensional texture space to object defined in 3D space / • Realized through per-fragment operations / • A texture lookup turns texture coordinates into data used by the shader
Professor-style explanation	The slide 'Texture Mapping' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Texture mapping / • Application of texture defined in multidimensional texture space to object defined in 3D space / • Realized through per-fragment operations / • A texture lookup turns texture coordinates into data used by the shader. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Texture Mapping. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Texture mapping / • Application of texture defined in multidimensional texture space to object defined in 3D space / • Realized through per-fragment operations / • A texture lookup turns texture coordinates into data used by the shader
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texture Mapping'?

Page 5	Why Textures Matter
Source cue	■ Geometry defines shape / ■ Textures define appearance detail / ■ This separates visual complexity from geometric complexity / ■ Instead of modeling every scratch, brick, pore, label, or color variation, we store it in texture data / ■ Textures are efficient
Professor-style explanation	The slide 'Why Textures Matter' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Geometry defines shape / ■ Textures define appearance detail / ■ This separates visual complexity from geometric complexity / ■ Instead of modeling every scratch, brick, pore, label, or color variation, we store it in texture data / ■ Textures are efficient. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.

Technical commentary	This slide is about Why Textures Matter. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Geometry defines shape / ■ Textures define appearance detail / ■ This separates visual complexity from geometric complexity / ■ Instead of modeling every scratch, brick, pore, label, or color variation, we store it in texture data / ■ Textures are efficient
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Why Textures Matter'?

Page 6	Textures Are More Than Color Images
Source cue	■ Modern rendering usually uses several texture maps per object / ■ Each map controls one part of the material or shading model / ■ Textures are therefore general-purpose GPU data sources / ■ Common texture maps / • Base color / albedo
Professor-style explanation	The slide 'Textures Are More Than Color Images' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Modern rendering usually uses several texture maps per object / ■ Each map controls one part of the material or shading model / ■ Textures are therefore general-purpose GPU data sources / ■ Common texture maps / • Base color / albedo. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Textures Are More Than Color Images. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Modern rendering usually uses several texture maps per object / ■ Each map controls one part of the material or shading model / ■ Textures are therefore general-purpose GPU data sources / ■ Common texture maps / • Base color / albedo
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Textures Are More Than Color Images'?

Page 7	Texturing Demo
Source cue	Preset Plain Object / Rotate / Base color / albedo / Normal / Roughness

Professor-style explanation	The slide 'Texturing Demo' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Preset Plain Object / Rotate / Base color / albedo / Normal / Roughness. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Texturing Demo. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Preset Plain Object / Rotate / Base color / albedo / Normal / Roughness
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Texturing Demo'?

Page 8	Overview
Source cue	9.1 Texture Objects, Texels, Formats, and Color Space / 9.2 Texture Coordinates, UV Mapping, and Wrapping / 9.3 Sampling, Filtering, Mipmaps, and Anisotropy / 9.4 Modern OpenGL Texture Pipeline / 9.5 Normal Mapping
Professor-style explanation	The slide 'Overview' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: 9.1 Texture Objects, Texels, Formats, and Color Space / 9.2 Texture Coordinates, UV Mapping, and Wrapping / 9.3 Sampling, Filtering, Mipmaps, and Anisotropy / 9.4 Modern OpenGL Texture Pipeline / 9.5 Normal Mapping. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Overview. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: 9.1 Texture Objects, Texels, Formats, and Color Space / 9.2 Texture Coordinates, UV Mapping, and Wrapping / 9.3 Sampling, Filtering, Mipmaps, and Anisotropy / 9.4 Modern OpenGL Texture Pipeline / 9.5 Normal Mapping
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.

Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Overview'?
Page 9	9.1 Texture Objects, Texels, Formats, and Color Space
Source cue	What texture data stores and how the GPU interprets it
Professor-style explanation	The slide '9.1 Texture Objects, Texels, Formats, and Color Space' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: What texture data stores and how the GPU interprets it. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about 9.1 Texture Objects, Texels, Formats, and Color Space. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: What texture data stores and how the GPU interprets it
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in '9.1 Texture Objects, Texels, Formats, and Color Space'?
Page 10	From Image File to GPU Texture Object
Source cue	■ Texture data starts as image file or generated array / ■ During loading, data is uploaded to GPU memory / ■ On the GPU, data becomes a texture object / ■ The shader accesses it through a sampler / ■ Pipeline
Professor-style explanation	The slide 'From Image File to GPU Texture Object' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Texture data starts as image file or generated array / ■ During loading, data is uploaded to GPU memory / ■ On the GPU, data becomes a texture object / ■ The shader accesses it through a sampler / ■ Pipeline. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about From Image File to GPU Texture Object. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Texture data starts as image file or generated array / ■ During loading, data is uploaded to GPU memory / ■ On the GPU, data becomes a texture object / ■ The shader accesses it through a sampler / ■ Pipeline
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'From Image File to GPU Texture Object'?

Page 11	Texels, Channels, and Meaning
Source cue	■ Texel: smallest sample element of a texture / ■ In 2D, a texel is analogous to a pixel / ■ Texels store numeric channel values / ■ Common channel layouts / • R, RG, RGB, RGBA
Professor-style explanation	The slide 'Texels, Channels, and Meaning' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Texel: smallest sample element of a texture / ■ In 2D, a texel is analogous to a pixel / ■ Texels store numeric channel values / ■ Common channel layouts / • R, RG, RGB, RGBA. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Texels, Channels, and Meaning. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Texel: smallest sample element of a texture / ■ In 2D, a texel is analogous to a pixel / ■ Texels store numeric channel values / ■ Common channel layouts / • R, RG, RGB, RGBA
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Texels, Channels, and Meaning'?

Page 12	Texture Targets and Dimensionality
Source cue	■ Texture target describes the sampled data domain / ■ Different targets use different coordinate types / ■ 1D texture / • Coordinate / • Lookup tables or transfer functions

Professor-style explanation	The slide 'Texture Targets and Dimensionality' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Texture target describes the sampled data domain / ■ Different targets use different coordinate types / ■ 1D texture / • Coordinate / • Lookup tables or transfer functions. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Targets and Dimensionality. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Texture target describes the sampled data domain / ■ Different targets use different coordinate types / ■ 1D texture / • Coordinate / • Lookup tables or transfer functions
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Targets and Dimensionality'?

Page 13	Internal Texture Formats
Source cue	■ Internal format defines GPU texel storage / ■ It specifies channels, precision, and representation / ■ Examples / • GL_R8: one 8-bit normalized channel / • GL_RGBA8: four 8-bit normalized channels
Professor-style explanation	The slide 'Internal Texture Formats' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Internal format defines GPU texel storage / ■ It specifies channels, precision, and representation / ■ Examples / • GL_R8: one 8-bit normalized channel / • GL_RGBA8: four 8-bit normalized channels. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Internal Texture Formats. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Internal format defines GPU texel storage / ■ It specifies channels, precision, and representation / ■ Examples / • GL_R8: one 8-bit normalized channel / • GL_RGBA8: four 8-bit normalized channels

Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Internal Texture Formats'?
Page 14	Numeric Texture Representations
Source cue	■ Texel values can be interpreted differently / ■ Normalized integer formats / • Stored as integers / • Sampled as normalized floats / 255 → 1.0
Professor-style explanation	The slide 'Numeric Texture Representations' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Texel values can be interpreted differently / ■ Normalized integer formats / • Stored as integers / • Sampled as normalized floats / 255 → 1.0. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Numeric Texture Representations. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Texel values can be interpreted differently / ■ Normalized integer formats / • Stored as integers / • Sampled as normalized floats / 255 → 1.0
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Numeric Texture Representations'?
Page 15	Color Textures vs. Data Textures
Source cue	■ Some textures store perceptual color / ■ Other textures store numeric shader data / ■ Color textures / • Base color / albedo / • Emissive color
Professor-style explanation	The slide 'Color Textures vs. Data Textures' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Some textures store perceptual color / ■ Other textures store numeric shader data / ■ Color textures / • Base color / albedo / • Emissive color. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Color Textures vs. Data Textures. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Some textures store perceptual color / ■ Other textures store numeric shader data / ■ Color textures / • Base color / albedo / • Emissive color
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Color Textures vs. Data Textures'?

Page 16	2D Texture Access
Source cue	■ For each fragment, texturecoordinates are interpolated / ■ During texture access texel values are bilinearly interpolated
Professor-style explanation	The slide '2D Texture Access' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ For each fragment, texturecoordinates are interpolated / ■ During texture access texel values are bilinearly interpolated. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 2D Texture Access. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ For each fragment, texturecoordinates are interpolated / ■ During texture access texel values are bilinearly interpolated
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '2D Texture Access'?

Page 17	Bilinear Interpolation
Source cue	■ Example: linear interpolation in 2D (=bilinear) / $f = f(s, t)$ / ■ Interpolation requires three computations (Notation: i_j) / $s-s / f = (1 - \alpha) \cdot f + \alpha \cdot f_\alpha =$

Professor-style explanation	The slide 'Bilinear Interpolation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Example: linear interpolation in 2D (=bilinear) / $f = f(s, t)$ / ■ Interpolation requires three computations (Notation: $ij\ ij\ ij$) / $s-s / f = (1 - \alpha) \cdot f + \alpha \cdot f \alpha =$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Bilinear Interpolation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Example: linear interpolation in 2D (=bilinear) / $f = f(s, t)$ / ■ Interpolation requires three computations (Notation: $ij\ ij\ ij$) / $s-s / f = (1 - \alpha) \cdot f + \alpha \cdot f \alpha =$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Bilinear Interpolation' into a causal sentence instead of repeating the slide title?
Page 18	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 19	Texture Memory Footprint
Source cue	■ Texture memory depends on resolution and format / $\text{memory} = \text{width} \cdot \text{height} \cdot \text{bytes per texel}$ / ■ Example memory costs / • RGBA8: 4 MiB / • RGBA8: 16 MiB

Professor-style explanation	The slide 'Texture Memory Footprint' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. On the slide, the concrete items are: ■ Texture memory depends on resolution and format / $\text{memory} = \text{width} \cdot \text{height} \cdot \text{bytes per texel}$ / ■ Example memory costs / • RGBA8: 4 MiB / • RGBA8: 16 MiB. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.
Technical commentary	This slide is about Texture Memory Footprint. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. Concrete items shown: ■ Texture memory depends on resolution and format / $\text{memory} = \text{width} \cdot \text{height} \cdot \text{bytes per texel}$ / ■ Example memory costs / • RGBA8: 4 MiB / • RGBA8: 16 MiB
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'Texture Memory Footprint'?

Page 20	Texture Compression and Asset Pipelines
Source cue	■ File formats and GPU formats are different concepts / ■ File formats / • PNG, JPEG, OpenEXR / • Used for storage and authoring / ■ GPU formats
Professor-style explanation	The slide 'Texture Compression and Asset Pipelines' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ File formats and GPU formats are different concepts / ■ File formats / • PNG, JPEG, OpenEXR / • Used for storage and authoring / ■ GPU formats. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Texture Compression and Asset Pipelines. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ File formats and GPU formats are different concepts / ■ File formats / • PNG, JPEG, OpenEXR / • Used for storage and authoring / ■ GPU formats
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.

Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Texture Compression and Asset Pipelines'?
Page 21	Steps Involved in Texturing
Source cue	■ Provide textures (e.g., load from file) / ■ Configure texture mapping(e.g., border handling, texture coordinates, texture filtering) / ■ Rendering / • Fetch texture values (usually by exploiting interpolation) / • Modify fragment color based on fetched texel value
Professor-style explanation	The slide 'Steps Involved in Texturing' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Provide textures (e.g., load from file) / ■ Configure texture mapping(e.g., border handling, texture coordinates, texture filtering) / ■ Rendering / • Fetch texture values (usually by exploiting interpolation) / • Modify fragment color based on fetched texel value. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Steps Involved in Texturing. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Provide textures (e.g., load from file) / ■ Configure texture mapping(e.g., border handling, texture coordinates, texture filtering) / ■ Rendering / • Fetch texture values (usually by exploiting interpolation) / • Modify fragment color based on fetched texel value
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Steps Involved in Texturing'?
Page 22	9.2 Texture Coordinates, UV Mapping, and Wrapping
Source cue	Aligning texture space with geometry

Professor-style explanation	The slide '9.2 Texture Coordinates, UV Mapping, and Wrapping' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Aligning texture space with geometry. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 9.2 Texture Coordinates, UV Mapping, and Wrapping. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Aligning texture space with geometry
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '9.2 Texture Coordinates, UV Mapping, and Wrapping'?

Page 23	Texture Coordinate System 1/2
Source cue	■ Object space: 3D vertices in model coordinate system / (x, y, z) / • Coordinate representation: / ■ Texture space: 1D, 2D, 3D space of texture data / (s, t, r, q)
Professor-style explanation	The slide 'Texture Coordinate System 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Object space: 3D vertices in model coordinate system / (x, y, z) / • Coordinate representation: / ■ Texture space: 1D, 2D, 3D space of texture data / (s, t, r, q). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Coordinate System 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Object space: 3D vertices in model coordinate system / (x, y, z) / • Coordinate representation: / ■ Texture space: 1D, 2D, 3D space of texture data / (s, t, r, q)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate System 1/2'?

Page 24	Texture Coordinate System 2/2
Source cue	$[0, 1]^2$ / ■ Texture coordinate system normalized to for 2D textures (analog for 1D and 3D textures) / s, t, r, q $s, t, r, q \in [0, 1]$ / • Coordinate axes denoted by with / (s)
Professor-style explanation	The slide 'Texture Coordinate System 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: $[0, 1]^2$ / ■ Texture coordinate system normalized to for 2D textures (analog for 1D and 3D textures) / s, t, r, q $s, t, r, q \in [0, 1]$ / • Coordinate axes denoted by with / (s). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Coordinate System 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $[0, 1]^2$ / ■ Texture coordinate system normalized to for 2D textures (analog for 1D and 3D textures) / s, t, r, q $s, t, r, q \in [0, 1]$ / • Coordinate axes denoted by with / (s)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate System 2/2'?
Page 25	Texture Coordinate Specification
Source cue	■ 2D texturing / $p = (x, y, z)$ $p = (s, t)$ / • For every vertex g of a polygon, texture coordinates t are specified / • Texture coordinates t are obtained from parametrization algorithms / • Texture coordinates t are linearly interpolated over a polygon (analogous to Gouraud shading)

Professor-style explanation	The slide 'Texture Coordinate Specification' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ 2D texturing / $p = (x, y, z)$ $p = (s, t)$ / • For every vertex g of a polygon, texture coordinates t are specified / • Texture coordinates t are obtained from parametrization algorithms / • Texture coordinates t are linearly interpolated over a polygon (analogous to Gouraud shading). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Coordinate Specification. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ 2D texturing / $p = (x, y, z)$ $p = (s, t)$ / • For every vertex g of a polygon, texture coordinates t are specified / • Texture coordinates t are obtained from parametrization algorithms / • Texture coordinates t are linearly interpolated over a polygon (analogous to Gouraud shading)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate Specification'?
Page 26	Disc Parametrization
Source cue	r xy / ■ Circular disc with radius r , centered in the plane / $p = (x, y, z) = (a \cdot \cos(\theta), a \cdot \sin(\theta), 0)$ $\theta \in [0, 2\pi]$ $a \in [0, r]$ / • g with and / $p = (s, t)$ $s = \theta$ $t = a$
Professor-style explanation	The slide 'Disc Parametrization' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: r xy / ■ Circular disc with radius r , centered in the plane / $p = (x, y, z) = (a \cdot \cos(\theta), a \cdot \sin(\theta), 0)$ $\theta \in [0, 2\pi]$ $a \in [0, r]$ / • g with and / $p = (s, t)$ $s = \theta$ $t = a$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Disc Parametrization. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: r xy / ■ Circular disc with radius r , centered in the plane / $p = (x, y, z) = (a \cdot \cos(\theta), a \cdot \sin(\theta), 0)$ $\theta \in [0, 2\pi]$ $a \in [0, r]$ / • g with and / $p = (s, t)$ $s = \theta$ $t = a$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Disc Parametrization' into a causal sentence instead of repeating the slide title?
Page 27	Cylinder Parametrization
Source cue	$xy \ r = 1 \ h \ z$ / ■ Cylinder based in plane with radius and height along axis (surface is topologically / two-dimensional and overlap-free) / $p = (x, y, z) = (r \cdot \cos(\theta), r \cdot \sin(\theta), z) \ \theta \in [0, 2\pi] \ z \in [0, h]$ / • g with and
Professor-style explanation	The slide 'Cylinder Parametrization' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: $xy \ r = 1 \ h \ z$ / ■ Cylinder based in plane with radius and height along axis (surface is topologically / two-dimensional and overlap-free) / $p = (x, y, z) = (r \cdot \cos(\theta), r \cdot \sin(\theta), z) \ \theta \in [0, 2\pi] \ z \in [0, h]$ / • g with and. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Cylinder Parametrization. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: $xy \ r = 1 \ h \ z$ / ■ Cylinder based in plane with radius and height along axis (surface is topologically / two-dimensional and overlap-free) / $p = (x, y, z) = (r \cdot \cos(\theta), r \cdot \sin(\theta), z) \ \theta \in [0, 2\pi] \ z \in [0, h]$ / • g with and
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Cylinder Parametrization' into a causal sentence instead of repeating the slide title?
Page 28	Two-Part Mapping 1/2
Source cue	■ Texture coordinates for complex geometries are derived from intermediate objects (i.e., geometries / with inherent parametrization) / ■ Proceeding / • Calculate texture coordinates for intermediate object / $p = (x, y, z) \ p = (s, t) \ p$
Professor-style explanation	The slide 'Two-Part Mapping 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Texture coordinates for complex geometries are derived from intermediate objects (i.e., geometries / with inherent parametrization) / ■ Proceeding / • Calculate texture coordinates for intermediate object / $p = (x, y, z) \ p = (s, t) \ p$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Two-Part Mapping 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Texture coordinates for complex geometries are derived from intermediate objects (i.e., geometries / with inherent parametrization) / ■ Proceeding / • Calculate texture coordinates for intermediate object / $p = (x, y, z)$ $p = (s, t)$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Two-Part Mapping 1/2'?

Page 29	Two-Part Mapping 2/2
Source cue	■ Two-part mapping has no restrictions wrt. objects to be textured / ■ Intermediate object must be appropriate for object to be textured
Professor-style explanation	The slide 'Two-Part Mapping 2/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Two-part mapping has no restrictions wrt. objects to be textured / ■ Intermediate object must be appropriate for object to be textured. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Two-Part Mapping 2/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Two-part mapping has no restrictions wrt. objects to be textured / ■ Intermediate object must be appropriate for object to be textured
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Two-Part Mapping 2/2'?

Page 30	Two-Part Mapping Example
Source cue	■ Object: non-convex polyhedron / ■ Intermediate object: cylinder / • 1 : Cast rays from object center through vertices to intersect cylinder (projection) / • 2 : Cylinder parametrization / $F(p) = F(F(p)) = (s, t)$

Professor-style explanation	The slide 'Two-Part Mapping Example' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Object: non-convex polyhedron / ■ Intermediate object: cylinder / • 1 : Cast rays from object center through vertices to intersect cylinder (projection) / • 2 : Cylinder parametrization / $F(p) = F(F(p)) = (s, t)$. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Two-Part Mapping Example. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Object: non-convex polyhedron / ■ Intermediate object: cylinder / • 1 : Cast rays from object center through vertices to intersect cylinder (projection) / • 2 : Cylinder parametrization / $F(p) = F(F(p)) = (s, t)$
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Two-Part Mapping Example' works per object, per image region, per ray, or per fragment?

Page 31	Projection Calculation Variations
Source cue	O O / ■ G center through vertices to I / O O / ■ G along normal to I / O O
Professor-style explanation	The slide 'Projection Calculation Variations' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: O O / ■ G center through vertices to I / O O / ■ G along normal to I / O O. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection Calculation Variations. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: O O / ■ G center through vertices to I / O O / ■ G along normal to I / O O
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Calculation Variations' changes positions before rasterization?

Page 32	Texture Coordinate Transformation
Source cue	■ Texture coordinates can be transformed through transformation matrix / • No restriction regarding matrix coefficients (4x4 matrix) / ■ Applications / • Shifting texture coordinates / • Scaling texture coordinates
Professor-style explanation	The slide 'Texture Coordinate Transformation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Texture coordinates can be transformed through transformation matrix / • No restriction regarding matrix coefficients (4x4 matrix) / ■ Applications / • Shifting texture coordinates / • Scaling texture coordinates. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Coordinate Transformation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Texture coordinates can be transformed through transformation matrix / • No restriction regarding matrix coefficients (4x4 matrix) / ■ Applications / • Shifting texture coordinates / • Scaling texture coordinates
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Coordinate Transformation'?
Page 33	Demo - Texture Coordinate Transformation
Source cue	Stage Transformed Texture / scaleS 1 / scaleT 1 / offsetS 0 / offsetT 0
Professor-style explanation	The slide 'Demo - Texture Coordinate Transformation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: Stage Transformed Texture / scaleS 1 / scaleT 1 / offsetS 0 / offsetT 0. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Demo - Texture Coordinate Transformation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Stage Transformed Texture / scaleS 1 / scaleT 1 / offsetS 0 / offsetT 0
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Demo - Texture Coordinate Transformation'?

Page 34	Height Field Texturing Example
Source cue	■ Height field represented as triangular mesh / (x, y) (s, t) / • Use normalized coordinates as texture coordinates (can be used to map aerial / image) / z r
Professor-style explanation	The slide 'Height Field Texturing Example' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Height field represented as triangular mesh / (x, y) (s, t) / • Use normalized coordinates as texture coordinates (can be used to map aerial / image) / z r. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Height Field Texturing Example. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Height field represented as triangular mesh / (x, y) (s, t) / • Use normalized coordinates as texture coordinates (can be used to map aerial / image) / z r
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Height Field Texturing Example'?

Page 35	9.3 Sampling, Filtering, Mipmaps, and Anisotropy
Source cue	Reconstructing texture values under magnification and minification

Professor-style explanation	The slide '9.3 Sampling, Filtering, Mipmaps, and Anisotropy' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: Reconstructing texture values under magnification and minification. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about 9.3 Sampling, Filtering, Mipmaps, and Anisotropy. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: Reconstructing texture values under magnification and minification
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in '9.3 Sampling, Filtering, Mipmaps, and Anisotropy'?

Page 36	Forward vs. Inverse Mapping
Source cue	■ Forward mapping - from texture to screen space / • Object's surface parameterization / • Projection transformation / ■ Inverse mapping - from screen to texture space / • Find corresponding pre-image/footprint of each pixel in texture
Professor-style explanation	The slide 'Forward vs. Inverse Mapping' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Forward mapping - from texture to screen space / • Object's surface parameterization / • Projection transformation / ■ Inverse mapping - from screen to texture space / • Find corresponding pre-image/footprint of each pixel in texture. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Forward vs. Inverse Mapping. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Forward mapping - from texture to screen space / • Object's surface parameterization / • Projection transformation / ■ Inverse mapping - from screen to texture space / • Find corresponding pre-image/footprint of each pixel in texture

Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Forward vs. Inverse Mapping'?
Page 37	Forward Mapping
Source cue	■ Maps each texel to screen space / • Uniform sampling of texture does not guarantee uniform sampling in screen space / ■ Forward mapping used for following cases / • Texture-to-screen mapping difficult to invert / • Texture image does not fit into memory
Professor-style explanation	The slide 'Forward Mapping' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Maps each texel to screen space / • Uniform sampling of texture does not guarantee uniform sampling in screen space / ■ Forward mapping used for following cases / • Texture-to-screen mapping difficult to invert / • Texture image does not fit into memory. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Forward Mapping. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Maps each texel to screen space / • Uniform sampling of texture does not guarantee uniform sampling in screen space / ■ Forward mapping used for following cases / • Texture-to-screen mapping difficult to invert / • Texture image does not fit into memory
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Forward Mapping'?
Page 38	Inverse Mapping 1/2
Source cue	■ For each pixel pre-image in texture space is found and its area is integrated over / ■ Inverse mapping can be used when / • Texture image fits into memory / • Forward mapping can be inverted / 1 for y in range(height): # screen 'height'

Professor-style explanation	The slide 'Inverse Mapping 1/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ For each pixel pre-image in texture space is found and its area is integrated over / ■ Inverse mapping can be used when / • Texture image fits into memory / • Forward mapping can be inverted / 1 for y in range(height): # screen 'height'. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Inverse Mapping 1/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ For each pixel pre-image in texture space is found and its area is integrated over / ■ Inverse mapping can be used when / • Texture image fits into memory / • Forward mapping can be inverted / 1 for y in range(height): # screen 'height'
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Inverse Mapping 1/2'?

Page 39	Inverse Mapping 2/2
Source cue	■ Pre-image of square screen pixel intersecting curved surface is curvilinear quadrilateral in texture / space / ■ Curvilinear quadrilateral often approximated / • By general quadrilateral / • By parallelogram
Professor-style explanation	The slide 'Inverse Mapping 2/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Pre-image of square screen pixel intersecting curved surface is curvilinear quadrilateral in texture / space / ■ Curvilinear quadrilateral often approximated / • By general quadrilateral / • By parallelogram. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Inverse Mapping 2/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Pre-image of square screen pixel intersecting curved surface is curvilinear quadrilateral in texture / space / ■ Curvilinear quadrilateral often approximated / • By general quadrilateral / • By parallelogram

Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Inverse Mapping 2/2'?
Page 40	Magnification vs. Minification
Source cue	■ Magnification / • Map few texels onto many pixels / • Two filtering approaches / • Nearest: Take the nearest texel / • Bilinear interpolation: Interpolation between 4 nearest texels
Professor-style explanation	The slide 'Magnification vs. Minification' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Magnification / • Map few texels onto many pixels / • Two filtering approaches / • Nearest: Take the nearest texel / • Bilinear interpolation: Interpolation between 4 nearest texels. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Magnification vs. Minification. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Magnification / • Map few texels onto many pixels / • Two filtering approaches / • Nearest: Take the nearest texel / • Bilinear interpolation: Interpolation between 4 nearest texels
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Magnification vs. Minification'?
Page 41	Nearest vs. Linear Interpolation
Source cue	■ Nearest neighbor interpolation / (s, t) / • For coordinates select nearest integer texel / (int(round(s · texdim)), int(round(t · texdim))) / x y

Professor-style explanation	The slide 'Nearest vs. Linear Interpolation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Nearest neighbor interpolation / (s, t) / • For coordinates select nearest integer texel / (int(round(s · texdim)), int(round(t · texdim))) / x y. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Nearest vs. Linear Interpolation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Nearest neighbor interpolation / (s, t) / • For coordinates select nearest integer texel / (int(round(s · texdim)), int(round(t · texdim))) / x y
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Nearest vs. Linear Interpolation'?

Page 42	Minification Filtering Requirements
Source cue	■ Preimage integration with insufficient filtering results in aliasing / artifacts
Professor-style explanation	The slide 'Minification Filtering Requirements' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Preimage integration with insufficient filtering results in aliasing / artifacts. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Minification Filtering Requirements. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Preimage integration with insufficient filtering results in aliasing / artifacts
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Minification Filtering Requirements'?

Page 43	Demo - Magnification Filtering: Nearest vs Linear
Source cue	Stage Split Comparison / magFilter GL_NEAREST / zoom 12 / textureResolution 8x8 / showTexelGrid
Professor-style explanation	The slide 'Demo - Magnification Filtering: Nearest vs Linear' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: Stage Split Comparison / magFilter GL_NEAREST / zoom 12 / textureResolution 8x8 / showTexelGrid. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Demo - Magnification Filtering: Nearest vs Linear. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: Stage Split Comparison / magFilter GL_NEAREST / zoom 12 / textureResolution 8x8 / showTexelGrid
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Magnification Filtering: Nearest vs Linear'?
Page 44	MIP Mapping (multum in parvo = 'much in little')
Source cue	■ Efficient scheme for sampling texture values / ■ Representation of a texture at different resolution levels(levels of Detail, LoDs) / ■ Exploiting a texture pyramid (here: 2D, analog for 1D and 3D) / • Each pyramid level contains filtered texture of half resolution of previous level / • Each layer contains only quarter of texels of previous layer
Professor-style explanation	The slide 'MIP Mapping (multum in parvo = 'much in little')' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Efficient scheme for sampling texture values / ■ Representation of a texture at different resolution levels(levels of Detail, LoDs) / ■ Exploiting a texture pyramid (here: 2D, analog for 1D and 3D) / • Each pyramid level contains filtered texture of half resolution of previous level / • Each layer contains only quarter of texels of previous layer. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.

Technical commentary	This slide is about MIP Mapping (multum in parvo = 'much in little'). The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Efficient scheme for sampling texture values / ■ Representation of a texture at different resolution levels(levels of Detail, LoDs) / ■ Exploiting a texture pyramid (here: 2D, analog for 1D and 3D) / • Each pyramid level contains filtered texture of half resolution of previous level / • Each layer contains only quarter of texels of previous layer
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'MIP Mapping (multum in parvo = 'much in little')'?

Page 45	MIP Map Pyramid 1/2
Source cue	■ MIP Map-Level 0 : Texture in original size / $2N-1$ / ■ MIP Map-Level 1: Texture size / ■ ... / $N/20$
Professor-style explanation	The slide 'MIP Map Pyramid 1/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ MIP Map-Level 0 : Texture in original size / $2N-1$ / ■ MIP Map-Level 1: Texture size / ■ ... / $N/20$. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about MIP Map Pyramid 1/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ MIP Map-Level 0 : Texture in original size / $2N-1$ / ■ MIP Map-Level 1: Texture size / ■ ... / $N/20$
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'MIP Map Pyramid 1/2'?

Page 46	MIP Map Pyramid 2/2
Source cue	■ Example MIP Map / ■ Self-Contained MIP Map for efficient storage

Professor-style explanation	The slide 'MIP Map Pyramid 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Example MIP Map / ■ Self-Contained MIP Map for efficient storage. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about MIP Map Pyramid 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Example MIP Map / ■ Self-Contained MIP Map for efficient storage
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'MIP Map Pyramid 2/2' into a causal sentence instead of repeating the slide title?
Page 47	MIP Mapping Memory
Source cue	■ 33% more memory is needed
Professor-style explanation	The slide 'MIP Mapping Memory' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. On the slide, the concrete items are: ■ 33% more memory is needed. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.
Technical commentary	This slide is about MIP Mapping Memory. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. Concrete items shown: ■ 33% more memory is needed
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in 'MIP Mapping Memory'?
Page 48	MIP Mapping Procedure
Source cue	$\lambda \in [0, N]$ / ■ Selection of the resolution levels based on continuous LoD level for a given fragment / ■ Procedure / • LoD level calculation / • Intra-level interpolation: interpolation within two levels closest to

Professor-style explanation	The slide 'MIP Mapping Procedure' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: $\lambda \in [0, N]$ / ■ Selection of the resolution levels based on continuous LoD level for a given fragment / ■ Procedure / • LoD level calculation / • Intra-level interpolation: interpolation within two levels closest to. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about MIP Mapping Procedure. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: $\lambda \in [0, N]$ / ■ Selection of the resolution levels based on continuous LoD level for a given fragment / ■ Procedure / • LoD level calculation / • Intra-level interpolation: interpolation within two levels closest to
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'MIP Mapping Procedure' into a causal sentence instead of repeating the slide title?

Page 49	LoD Calculation 1/2
Source cue	■ Assumptions / $[0, 1]^2$ / • 2D texture is parametrized with texture coordinates / $M \cdot M = 2N$ / • 2D texture has texels with (power-of-two texture)
Professor-style explanation	The slide 'LoD Calculation 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Assumptions / $[0, 1]^2$ / • 2D texture is parametrized with texture coordinates / $M \cdot M = 2N$ / • 2D texture has texels with (power-of-two texture). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about LoD Calculation 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Assumptions / $[0, 1]^2$ / • 2D texture is parametrized with texture coordinates / $M \cdot M = 2N$ / • 2D texture has texels with (power-of-two texture)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'LoD Calculation 1/2'?

Page 50	LoD Calculation 2/2
Source cue	■ Approximation of the preimage by square / ■ Calculation of the MIP map level / • Area of approximating square is / $b = A$ / • Length of square's side is
Professor-style explanation	The slide 'LoD Calculation 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Approximation of the preimage by square / ■ Calculation of the MIP map level / • Area of approximating square is / $b = A$ / • Length of square's side is. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about LoD Calculation 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Approximation of the preimage by square / ■ Calculation of the MIP map level / • Area of approximating square is / $b = A$ / • Length of square's side is
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'LoD Calculation 2/2' into a causal sentence instead of repeating the slide title?
Page 51	Demo - Minification, Mipmaps, and Mipmap Level Visualization
Source cue	Stage Minified Floor / minFilter GL_LINEAR_MIPMAP_LINEAR / mipmapsEnabled / lodBias 0 / visualizeMipLevels
Professor-style explanation	The slide 'Demo - Minification, Mipmaps, and Mipmap Level Visualization' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: Stage Minified Floor / minFilter GL_LINEAR_MIPMAP_LINEAR / mipmapsEnabled / lodBias 0 / visualizeMipLevels. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Demo - Minification, Mipmaps, and Mipmap Level Visualization. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: Stage Minified Floor / minFilter GL_LINEAR_MIPMAP_LINEAR / mipmapsEnabled / lodBias 0 / visualizeMipLevels
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.

Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Minification, Mipmaps, and Mipmap Level Visualization'?
Page 52	Anisotropic Filtering
Source cue	■ Approximation of preimage through several squares
Professor-style explanation	The slide 'Anisotropic Filtering' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Approximation of preimage through several squares. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Anisotropic Filtering. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Approximation of preimage through several squares
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Anisotropic Filtering'?
Page 53	Demo - Anisotropic Filtering
Source cue	Stage 1x vs Selected Anisotropy / anisotropy 16x / minFilter GL_LINEAR_MIPMAP_LINEAR / textureFrequency 34 / cameraAngle 10
Professor-style explanation	The slide 'Demo - Anisotropic Filtering' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: Stage 1x vs Selected Anisotropy / anisotropy 16x / minFilter GL_LINEAR_MIPMAP_LINEAR / textureFrequency 34 / cameraAngle 10. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Demo - Anisotropic Filtering. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: Stage 1x vs Selected Anisotropy / anisotropy 16x / minFilter GL_LINEAR_MIPMAP_LINEAR / textureFrequency 34 / cameraAngle 10

Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Demo - Anisotropic Filtering' changes positions before rasterization?
Page 54	9.4 Modern OpenGL Texture Pipeline
Source cue	Texture objects, sampler uniforms, and shader-based access
Professor-style explanation	The slide '9.4 Modern OpenGL Texture Pipeline' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Texture objects, sampler uniforms, and shader-based access. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 9.4 Modern OpenGL Texture Pipeline. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Texture objects, sampler uniforms, and shader-based access
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '9.4 Modern OpenGL Texture Pipeline'?
Page 55	OpenGL Texturing
Source cue	■ Texturing procedure / • Fetch interpolated texel value from texture / • Use texel value to modulate fragment color or depth / ■ Configuration / • Per texture
Professor-style explanation	The slide 'OpenGL Texturing' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Texturing procedure / • Fetch interpolated texel value from texture / • Use texel value to modulate fragment color or depth / ■ Configuration / • Per texture. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about OpenGL Texturing. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Texturing procedure / • Fetch interpolated texel value from texture / • Use texel value to modulate fragment color or depth / ■ Configuration / • Per texture
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Texturing'?

Page 56	OpenGL Minification & Magnification 1/3
Source cue	■ OpenGL exposes two texture filtering parameters: / • GL_TEXTURE_MIN_FILTER / • GL_TEXTURE_MAG_FILTER / ■ Texture minification (GL_TEXTURE_MIN_FILTER) options / • GL_NEAREST - nearest neighbor interpolation
Professor-style explanation	The slide 'OpenGL Minification & Magnification 1/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ OpenGL exposes two texture filtering parameters: / • GL_TEXTURE_MIN_FILTER / • GL_TEXTURE_MAG_FILTER / ■ Texture minification (GL_TEXTURE_MIN_FILTER) options / • GL_NEAREST - nearest neighbor interpolation. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Minification & Magnification 1/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ OpenGL exposes two texture filtering parameters: / • GL_TEXTURE_MIN_FILTER / • GL_TEXTURE_MAG_FILTER / ■ Texture minification (GL_TEXTURE_MIN_FILTER) options / • GL_NEAREST - nearest neighbor interpolation
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 1/3'?

Page 57	OpenGL Minification & Magnification 2/3
Source cue	■ OpenGL bilinear filtering example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...

Professor-style explanation	The slide 'OpenGL Minification & Magnification 2/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ OpenGL bilinear filtering example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Minification & Magnification 2/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ OpenGL bilinear filtering example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 2/3'?

Page 58	OpenGL Minification & Magnification 3/3
Source cue	■ OpenGL mipmapping example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...
Professor-style explanation	The slide 'OpenGL Minification & Magnification 3/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ OpenGL mipmapping example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 . The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about OpenGL Minification & Magnification 3/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ OpenGL mipmapping example / 1 glEnable(GL_TEXTURE_2D); / 2 glGenTextures(1, &textureID); / 3 glBindTexture(GL_TEXTURE_2D, textureID); / 4 ...

Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Minification & Magnification 3/3'?
Page 59	Demo - Texture Units and Sampler Uniforms
Source cue	Stage Combined Sampling / textureUnit0 Checker / textureUnit1 Brick / textureUnit2 UV Test / baseSamplerUnit 0
Professor-style explanation	The slide 'Demo - Texture Units and Sampler Uniforms' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: Stage Combined Sampling / textureUnit0 Checker / textureUnit1 Brick / textureUnit2 UV Test / baseSamplerUnit 0. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Demo - Texture Units and Sampler Uniforms. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: Stage Combined Sampling / textureUnit0 Checker / textureUnit1 Brick / textureUnit2 UV Test / baseSamplerUnit 0
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Texture Units and Sampler Uniforms'?
Page 60	Texture Wrapping 1/4
Source cue	■ Texture wrapping specifies border handling behavior when accessing texture outside the / [0, 1]D / normalized texture value range / ■ Texture wrapping mode is part of texture specification, and specified separately for each texture / dimension

Professor-style explanation	The slide 'Texture Wrapping 1/4' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Texture wrapping specifies border handling behavior when accessing texture outside the $[0, 1]^D$ / normalized texture value range / ■ Texture wrapping mode is part of texture specification, and specified separately for each texture / dimension. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Texture Wrapping 1/4. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Texture wrapping specifies border handling behavior when accessing texture outside the $[0, 1]^D$ / normalized texture value range / ■ Texture wrapping mode is part of texture specification, and specified separately for each texture / dimension
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Texture Wrapping 1/4'?
Page 61	Texture Wrapping 2/4
Source cue	■ GL_CLAMP_TO_EDGE / • Texture coordinates below 0 become 0 / • Texture coordinates above 1 become 1 / • Results in constant continuation of texture / ■ GL_CLAMP_TO_BORDER
Professor-style explanation	The slide 'Texture Wrapping 2/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ GL_CLAMP_TO_EDGE / • Texture coordinates below 0 become 0 / • Texture coordinates above 1 become 1 / • Results in constant continuation of texture / ■ GL_CLAMP_TO_BORDER. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Texture Wrapping 2/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ GL_CLAMP_TO_EDGE / • Texture coordinates below 0 become 0 / • Texture coordinates above 1 become 1 / • Results in constant continuation of texture / ■ GL_CLAMP_TO_BORDER
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Wrapping 2/4'?
Page 62	Texture Wrapping 3/4
Source cue	■ GL_REPEAT / • Texture coordinates ignore digits before decimal point, and use digits after decimal point as / texture coordinates / • Results in periodic continuation of normalized texture space / ■ GL_MIRRORED_REPEAT
Professor-style explanation	The slide 'Texture Wrapping 3/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ GL_REPEAT / • Texture coordinates ignore digits before decimal point, and use digits after decimal point as / texture coordinates / • Results in periodic continuation of normalized texture space / ■ GL_MIRRORED_REPEAT. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Texture Wrapping 3/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ GL_REPEAT / • Texture coordinates ignore digits before decimal point, and use digits after decimal point as / texture coordinates / • Results in periodic continuation of normalized texture space / ■ GL_MIRRORED_REPEAT
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Texture Wrapping 3/4'?
Page 63	Texture Wrapping 4/4
Source cue	■ OpenGL code snippets / 1 glTexParameteri(textureID, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT); / 2 glTexParameteri(textureID, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT); / 1 const GLfloat borderColor[] = { 1.0f, 1.0f, 0.0f, 1.0f }; / 3 glTexParameteri(textureID, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER);

Professor-style explanation	The slide 'Texture Wrapping 4/4' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ OpenGL code snippets / 1 <pre>glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT); / 2 glTextureParameteri(textureID, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT); / 1 const GLfloat borderColor[] = { 1.0f, 1.0f, 0.0f, 1.0f }; / 3 glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER).</pre> The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Texture Wrapping 4/4. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ OpenGL code snippets / 1 <pre>glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT); / 2 glTextureParameteri(textureID, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT); / 1 const GLfloat borderColor[] = { 1.0f, 1.0f, 0.0f, 1.0f }; / 3 glTextureParameteri(textureID, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER);</pre>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texture Wrapping 4/4'?
Page 64	Texel Value Usage
Source cue	■ Older OpenGL versions support several modes for modulating fragment colors by texel colors / (today still relevant to reimplement in shader) - is current fragment with calculated color values / C A T C A / F and alpha values F - is current texture with color values T and alpha values T (or / L C A C

Professor-style explanation	The slide 'Texel Value Usage' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Older OpenGL versions support several modes for modulating fragment colors by texel colors / (today still relevant to reimplement in shader) - is current fragment with calculated color values / C A T C A / F and alpha values F - is current texture with color values T and alpha values T (or / L C A C. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Texel Value Usage. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Older OpenGL versions support several modes for modulating fragment colors by texel colors / (today still relevant to reimplement in shader) - is current fragment with calculated color values / C A T C A / F and alpha values F - is current texture with color values T and alpha values T (or / L C A C
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texel Value Usage'?
Page 65	Texturing in GLSL Shaders 1/3
Source cue	■ Textures are bound to texture units / ■ Sampler uniforms select the texture unit used by the shader / 1 glBindTextureUnit(0, texture2D); / 2 glBindTextureUnit(1, texture3D); / 4 glUseProgram(p);
Professor-style explanation	The slide 'Texturing in GLSL Shaders 1/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Textures are bound to texture units / ■ Sampler uniforms select the texture unit used by the shader / 1 glBindTextureUnit(0, texture2D); / 2 glBindTextureUnit(1, texture3D); / 4 glUseProgram(p). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Texturing in GLSL Shaders 1/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Textures are bound to texture units / ■ Sampler uniforms select the texture unit used by the shader / 1 glBindTextureUnit(0, texture2D); / 2 glBindTextureUnit(1, texture3D); / 4 glUseProgram(p);
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 1/3'?
Page 66	Texturing in GLSL Shaders 2/3
Source cue	■ Vertex Shader / 1 layout(location = 0) in vec3 position; / 2 layout(location = 1) in vec3 texCoord; / 4 out vec3 vTexCoord; / 6 uniform mat4 modelViewProjectionMatrix;
Professor-style explanation	The slide 'Texturing in GLSL Shaders 2/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Vertex Shader / 1 layout(location = 0) in vec3 position; / 2 layout(location = 1) in vec3 texCoord; / 4 out vec3 vTexCoord; / 6 uniform mat4 modelViewProjectionMatrix. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Texturing in GLSL Shaders 2/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Vertex Shader / 1 layout(location = 0) in vec3 position; / 2 layout(location = 1) in vec3 texCoord; / 4 out vec3 vTexCoord; / 6 uniform mat4 modelViewProjectionMatrix;
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 2/3'?
Page 67	Texturing in GLSL Shaders 3/3
Source cue	■ Fragment Shader / 1 in vec3 vTexCoord; / 3 out vec4 fragmentColor; / 5 uniform sampler2D texture2D_; / 6 uniform sampler3D texture3D_;

Professor-style explanation	The slide 'Texturing in GLSL Shaders 3/3' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Fragment Shader / 1 in vec3 vTexCoord; / 3 out vec4 fragmentColor; / 5 uniform sampler2D texture2D_; / 6 uniform sampler3D texture3D_. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Texturing in GLSL Shaders 3/3. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Fragment Shader / 1 in vec3 vTexCoord; / 3 out vec4 fragmentColor; / 5 uniform sampler2D texture2D_; / 6 uniform sampler3D texture3D_;
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Texturing in GLSL Shaders 3/3'?

Page 68	9.5 Normal Mapping
Source cue	Encoding surface detail in texture space
Professor-style explanation	The slide '9.5 Normal Mapping' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Encoding surface detail in texture space. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about 9.5 Normal Mapping. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Encoding surface detail in texture space
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '9.5 Normal Mapping'?

Page 69	Motivation
---------	------------

Source cue	■ Render content dichotomy / • Shape: Triangles are used to generate geometric details / k k k / • Appearance: Textures are used to generate material variations (a , d , s) / ■ Consequences
Professor-style explanation	The slide 'Motivation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: ■ Render content dichotomy / • Shape: Triangles are used to generate geometric details / k k k / • Appearance: Textures are used to generate material variations (a , d , s) / ■ Consequences. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Motivation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: ■ Render content dichotomy / • Shape: Triangles are used to generate geometric details / k k k / • Appearance: Textures are used to generate material variations (a , d , s) / ■ Consequences
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Motivation'?
Page 70	Mesh Compression
Source cue	[source]
Professor-style explanation	The slide 'Mesh Compression' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: [source]. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Mesh Compression. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: [source]
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Mesh Compression' into a causal sentence instead of repeating the slide title?
Page 71	Normal Mapping 1/2

Source cue	■ Breaks dichotomy by representing geometric details in textures, as texels represent variations in / surface normals / ■ Normal maps accompany diffuse maps with same resolution Material details and geometric / details match
Professor-style explanation	The slide 'Normal Mapping 1/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Breaks dichotomy by representing geometric details in textures, as texels represent variations in / surface normals / ■ Normal maps accompany diffuse maps with same resolution Material details and geometric / details match. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Normal Mapping 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Breaks dichotomy by representing geometric details in textures, as texels represent variations in / surface normals / ■ Normal maps accompany diffuse maps with same resolution Material details and geometric / details match
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Mapping 1/2'?
Page 72	Normal Mapping 2/2
Source cue	k_n / ■ During illumination computation d is fetched from the diffuse map, while i is fetched from the / normal map / $I = k \cdot L \cdot k \cdot L \cdot \max(0, n \cdot l) + k \cdot L \cdot \max(0, v \cdot r) \cdot p / a + d \cdot d \cdot s$
Professor-style explanation	The slide 'Normal Mapping 2/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: k_n / ■ During illumination computation d is fetched from the diffuse map, while i is fetched from the / normal map / $I = k \cdot L \cdot k \cdot L \cdot \max(0, n \cdot l) + k \cdot L \cdot \max(0, v \cdot r) \cdot p / a + d \cdot d \cdot s$. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Normal Mapping 2/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: $k_n \cdot n \cdot l$ / $k_s \cdot \max(0, v \cdot r)$ / $k_a + d$ / s During illumination computation d is fetched from the diffuse map, while s is fetched from the / normal map / $l = k \cdot L \cdot k \cdot L \cdot \max(0, n \cdot l) + k \cdot L \cdot \max(0, v \cdot r)$ / $a + d$ / s
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Mapping 2/2'?

Page 73	Normal Maps
Source cue	■ Normal vector components can be positive or negative Components need to be normalized to / $[0, 1]$ $[-1, 1]$ / before storage, and rescaled to after texture fetch / $n' := (n) + 0.5$ / Before storage:
Professor-style explanation	The slide 'Normal Maps' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Normal vector components can be positive or negative Components need to be normalized to / $[0, 1]$ $[-1, 1]$ / before storage, and rescaled to after texture fetch / $n' := (n) + 0.5$ / Before storage. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Normal Maps. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Normal vector components can be positive or negative Components need to be normalized to / $[0, 1]$ $[-1, 1]$ / before storage, and rescaled to after texture fetch / $n' := (n) + 0.5$ / Before storage:
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Normal Maps'?

Page 74	Tangent Space
Source cue	■ Normal Map vectors align along axis and need to be transformed into view coordinate system / before illumination computation / ■ Tangent coordinate system spanned by / Normal t / t s

Professor-style explanation	The slide 'Tangent Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Normal Map vectors align along axis and need to be transformed into view coordinate system / before illumination computation / ■ Tangent coordinate system spanned by / Normal $t / t s$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Tangent Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Normal Map vectors align along axis and need to be transformed into view coordinate system / before illumination computation / ■ Tangent coordinate system spanned by / Normal $t / t s$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Tangent Space'?
Page 75	TBN Matrix
Source cue	■ TBN matrix (Tangent, Bitangent, and Normal) is an orthonormal matrix, which transforms tangent / space normal to view space normal / ■ TBN matrix construction (similar to camera transformation) / • TBN matrix contains three perpendicular vectors / • First column vector: tangent
Professor-style explanation	The slide 'TBN Matrix' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ TBN matrix (Tangent, Bitangent, and Normal) is an orthonormal matrix, which transforms tangent / space normal to view space normal / ■ TBN matrix construction (similar to camera transformation) / • TBN matrix contains three perpendicular vectors / • First column vector: tangent. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about TBN Matrix. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ TBN matrix (Tangent, Bitangent, and Normal) is an orthonormal matrix, which transforms tangent / space normal to view space normal / ■ TBN matrix construction (similar to camera transformation) / • TBN matrix contains three perpendicular vectors / • First column vector: tangent
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'TBN Matrix'?
Page 76	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 77	Tangent Vector Derivation 1/2
Source cue	■ ■ / t b / Observation: triangle edges can be expressed based on basis given by and / ■ ■ / e ■ = $\Delta u \cdot t + \Delta v \cdot b$
Professor-style explanation	The slide 'Tangent Vector Derivation 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: ■ ■ / t b / Observation: triangle edges can be expressed based on basis given by and / ■ ■ / e ■ = $\Delta u \cdot t + \Delta v \cdot b$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Tangent Vector Derivation 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: $\begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} \Delta u \\ \Delta v \\ e \end{bmatrix} = \Delta u \cdot t + \Delta v \cdot b$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Tangent Vector Derivation 1/2'?

Page 78	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?

Page 79	Tangent Vector Derivation 2/2
Source cue	$\begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} \Delta u \\ \Delta v \\ e \end{bmatrix} = \Delta u \cdot t + \Delta v \cdot b$ / We can solve this matrix for $\begin{bmatrix} x \\ y \\ z \end{bmatrix}$ and $\begin{bmatrix} \Delta u \\ \Delta v \\ e \end{bmatrix}$ / $\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & x & 0 & y & 0 & z \end{bmatrix}$
Professor-style explanation	The slide 'Tangent Vector Derivation 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: $\begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} \Delta u \\ \Delta v \\ e \end{bmatrix} = \Delta u \cdot t + \Delta v \cdot b$ / We can solve this matrix for $\begin{bmatrix} x \\ y \\ z \end{bmatrix}$ and $\begin{bmatrix} \Delta u \\ \Delta v \\ e \end{bmatrix}$ / $\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & x & 0 & y & 0 & z \end{bmatrix}$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Tangent Vector Derivation 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} / t \ b /$ We can solve this matrix for $\Delta u \ \Delta v \ e \ e / x \ y \ z$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Tangent Vector Derivation 2/2'?

Page 80	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?

Page 81	Observations
Source cue	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} / t \ b \ n \Rightarrow t \ b /$ and are perpendicular to each other as well as to t only one, or n , needs to be computed, while the other can be derived through cross product $/ \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
Professor-style explanation	The slide 'Observations' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} / t \ b \ n \Rightarrow t \ b /$ and are perpendicular to each other as well as to t only one, or n , needs to be computed, while the other can be derived through cross product $/ \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Observations. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: $\mathbf{t} \cdot \mathbf{b} \cdot \mathbf{n} \Rightarrow \mathbf{t} \cdot \mathbf{b}$ / and are perpendicular to each other as well as to $\mathbf{t}$ only one, or , needs to be / computed, while the other can be derived through cross product / $\mathbf{b} \times \mathbf{n}$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Observations' into a causal sentence instead of repeating the slide title?

Page 82	Variations
Source cue	■ Different variations of normal maps exist / • Bump maps - specify displacement of surface points in normal direction / - Vector offset bump maps - offset vector added to normal / • Vector rotation bump maps - normal rotated given angle
Professor-style explanation	The slide 'Variations' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Different variations of normal maps exist / • Bump maps - specify displacement of surface points in normal direction / - Vector offset bump maps - offset vector added to normal / • Vector rotation bump maps - normal rotated given angle. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Variations. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Different variations of normal maps exist / • Bump maps - specify displacement of surface points in normal direction / - Vector offset bump maps - offset vector added to normal / • Vector rotation bump maps - normal rotated given angle
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Variations'?

Page 83	9.6 Environment Mapping
Source cue	Sampling directional environment data for reflections and lighting

Professor-style explanation	The slide '9.6 Environment Mapping' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Sampling directional environment data for reflections and lighting. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about 9.6 Environment Mapping. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Sampling directional environment data for reflections and lighting
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '9.6 Environment Mapping'?

Page 84	Environment Mapping Motivation
Source cue	■ Many reflective and glossy materials show light from their surroundings / ■ Explicitly tracing all reflection rays is expensive in interactive rendering / ■ Environment maps provide a compact representation of incident radiance / ■ Typical use cases • mirror-like reflections
Professor-style explanation	The slide 'Environment Mapping Motivation' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Many reflective and glossy materials show light from their surroundings / ■ Explicitly tracing all reflection rays is expensive in interactive rendering / ■ Environment maps provide a compact representation of incident radiance / ■ Typical use cases / • mirror-like reflections. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Environment Mapping Motivation. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Many reflective and glossy materials show light from their surroundings / ■ Explicitly tracing all reflection rays is expensive in interactive rendering / ■ Environment maps provide a compact representation of incident radiance / ■ Typical use cases / • mirror-like reflections
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.

Check yourself	Can you decide whether 'Environment Mapping Motivation' works per object, per image region, per ray, or per fragment?
Page 85	Environment Mapping Process
Source cue	■ Environment mapping simulates reflections by sampling an image of the surrounding scene ■ The environment is treated as if it were infinitely far away ■ The reflected view direction determines which part of the environment is visible [Need for Speed no Limits]
Professor-style explanation	The slide 'Environment Mapping Process' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Environment mapping simulates reflections by sampling an image of the surrounding scene ■ The environment is treated as if it were infinitely far away ■ The reflected view direction determines which part of the environment is visible [Need for Speed no Limits]. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Environment Mapping Process. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Environment mapping simulates reflections by sampling an image of the surrounding scene ■ The environment is treated as if it were infinitely far away ■ The reflected view direction determines which part of the environment is visible [Need for Speed no Limits]
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Process'?
Page 86	Environment Mapping Assumptions
Source cue	■ The environment is assumed to be far away • translation of the object does not change the sampled environment ■ The reflected object does not reflect itself ■ Nearby dynamic objects are usually missing ■ The approximation works best for

Professor-style explanation	The slide 'Environment Mapping Assumptions' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ The environment is assumed to be far away / • translation of the object does not change the sampled environment / ■ The reflected object does not reflect itself / ■ Nearby dynamic objects are usually missing / ■ The approximation works best for. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Environment Mapping Assumptions. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ The environment is assumed to be far away / • translation of the object does not change the sampled environment / ■ The reflected object does not reflect itself / ■ Nearby dynamic objects are usually missing / ■ The approximation works best for
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Assumptions'?
Page 87	Environment Mapping 2/2
Source cue	$V \cdot N$ / ■ At the shaded surface point, the view vector is reflected about the normal / ■ The resulting reflection vector is used to access the environment map / ■ is calculated analog to the Phong illumination model / $R = 2 \cdot (V \cdot N) \cdot N - V$
Professor-style explanation	The slide 'Environment Mapping 2/2' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: $V \cdot N$ / ■ At the shaded surface point, the view vector is reflected about the normal / ■ The resulting reflection vector is used to access the environment map / ■ is calculated analog to the Phong illumination model / $R = 2 \cdot (V \cdot N) \cdot N - V$. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Environment Mapping 2/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: $V \cdot N$ / ■ At the shaded surface point, the view vector is reflected about the normal / ■ The resulting reflection vector is used to access the environment map / ■ is calculated analog to the Phong illumination model / $R = 2 \cdot (V \cdot N) \cdot N - V$
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Environment Mapping 2/2'?

Page 88	Demo - Cube-map Reflection Vector
Source cue	Stage Reflection Vector / object Sphere / showSkybox / reflectionMode Reflection / reflectivity 1
Professor-style explanation	The slide 'Demo - Cube-map Reflection Vector' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Stage Reflection Vector / object Sphere / showSkybox / reflectionMode Reflection / reflectivity 1. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Demo - Cube-map Reflection Vector. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Stage Reflection Vector / object Sphere / showSkybox / reflectionMode Reflection / reflectivity 1
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Demo - Cube-map Reflection Vector' into a causal sentence instead of repeating the slide title?

Page 89	Environment Mapping Types 1/2
Source cue	■ Different parameterizations store directional data in different ways / ■ They differ with respect to / • distortion / • filtering behavior / • hardware support

Professor-style explanation	The slide 'Environment Mapping Types 1/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Different parameterizations store directional data in different ways / ■ They differ with respect to / • distortion / • filtering behavior / • hardware support. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Environment Mapping Types 1/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Different parameterizations store directional data in different ways / ■ They differ with respect to / • distortion / • filtering behavior / • hardware support
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Types 1/2'?
Page 90	Environment Mapping Types 2/2
Source cue	■ Spherical environment maps / • compact representation in one 2D texture / • strong distortion near the border / • difficult filtering and interpolation behavior / ■ Dual-paraboloid maps
Professor-style explanation	The slide 'Environment Mapping Types 2/2' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Spherical environment maps / • compact representation in one 2D texture / • strong distortion near the border / • difficult filtering and interpolation behavior / ■ Dual-paraboloid maps. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Environment Mapping Types 2/2. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Spherical environment maps / • compact representation in one 2D texture / • strong distortion near the border / • difficult filtering and interpolation behavior / ■ Dual-paraboloid maps

Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Environment Mapping Types 2/2'?
Page 91	Spherical Mapping 1/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Spherical Mapping 1/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Spherical Mapping 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Spherical Mapping 1/2' into a causal sentence instead of repeating the slide title?
Page 92	Spherical Mapping 2/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Spherical Mapping 2/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Spherical Mapping 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Spherical Mapping 2/2' into a causal sentence instead of repeating the slide title?
Page 93	Spherical Environment Mapping

Source cue	■ A sphere map stores the environment as seen in a perfectly reflective sphere / ■ It was useful before cube maps became widely supported in graphics hardware / ■ It is mainly relevant today as historical context / ■ Important limitations / • non-uniform distortion
Professor-style explanation	The slide 'Spherical Environment Mapping' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ A sphere map stores the environment as seen in a perfectly reflective sphere / ■ It was useful before cube maps became widely supported in graphics hardware / ■ It is mainly relevant today as historical context / ■ Important limitations / • non-uniform distortion. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Spherical Environment Mapping. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ A sphere map stores the environment as seen in a perfectly reflective sphere / ■ It was useful before cube maps became widely supported in graphics hardware / ■ It is mainly relevant today as historical context / ■ Important limitations / • non-uniform distortion
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping'?
Page 94	Sphere Map Distortion
Source cue	■ Sphere-map texels represent differently sized regions of the environment / ■ Distortion is especially problematic in the outer region of the map / ■ Hardware performs linear texture filtering, not spherical interpolation / ■ For modern real-time graphics, this is one reason cube maps are preferred

Professor-style explanation	The slide 'Sphere Map Distortion' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Sphere-map texels represent differently sized regions of the environment / ■ Distortion is especially problematic in the outer region of the map / ■ Hardware performs linear texture filtering, not spherical interpolation / ■ For modern real-time graphics, this is one reason cube maps are preferred. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Sphere Map Distortion. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Sphere-map texels represent differently sized regions of the environment / ■ Distortion is especially problematic in the outer region of the map / ■ Hardware performs linear texture filtering, not spherical interpolation / ■ For modern real-time graphics, this is one reason cube maps are preferred
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Sphere Map Distortion'?

Page 95	Demo - Environment Map Distortion
Source cue	Stage Debug Distortion / object Sphere / environmentContent Debug / mapType Spherical map / mappingType Reflection
Professor-style explanation	The slide 'Demo - Environment Map Distortion' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: Stage Debug Distortion / object Sphere / environmentContent Debug / mapType Spherical map / mappingType Reflection. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Demo - Environment Map Distortion. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: Stage Debug Distortion / object Sphere / environmentContent Debug / mapType Spherical map / mappingType Reflection

Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Demo - Environment Map Distortion'?
Page 96	Sphere Map Interpolation
Source cue	■ Sphere maps represent non-linear images / ■ Two sphere map texels should be interpolated along an arc / • Graphics hardware does not support spherical interpolation / • Interpolation in the inner area is little distorted / • Interpolation in the outer area is highly distorted
Professor-style explanation	The slide 'Sphere Map Interpolation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Sphere maps represent non-linear images / ■ Two sphere map texels should be interpolated along an arc / • Graphics hardware does not support spherical interpolation / • Interpolation in the inner area is little distorted / • Interpolation in the outer area is highly distorted. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Sphere Map Interpolation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Sphere maps represent non-linear images / ■ Two sphere map texels should be interpolated along an arc / • Graphics hardware does not support spherical interpolation / • Interpolation in the inner area is little distorted / • Interpolation in the outer area is highly distorted
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Sphere Map Interpolation' into a causal sentence instead of repeating the slide title?
Page 97	Sphere Map Coordinates
Source cue	■ Sphere map texture coordinates are calculated in camera coordinates / (0, 0, 0) / • Camera in origin / - V -z +x +y / • View direction along the -axis (axis to the right, axis up)

Professor-style explanation	The slide 'Sphere Map Coordinates' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Sphere map texture coordinates are calculated in camera coordinates / $(0, 0, 0) / \bullet$ Camera in origin / $V -z +x +y / \bullet$ View direction along the -axis (axis to the right, axis up). The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Sphere Map Coordinates. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Sphere map texture coordinates are calculated in camera coordinates / $(0, 0, 0) / \bullet$ Camera in origin / $V -z +x +y / \bullet$ View direction along the -axis (axis to the right, axis up)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Sphere Map Coordinates'?

Page 98	Cube Environment Mapping
Source cue	■ Cube maps are represented by 6 square textures forming the sides of a cube / ■ The cube is centered around the shading point or camera / ■ Each face stores the environment as seen through one side of the cube / ■ A 3D direction vector is used to select the face and lookup position
Professor-style explanation	The slide 'Cube Environment Mapping' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: ■ Cube maps are represented by 6 square textures forming the sides of a cube / ■ The cube is centered around the shading point or camera / ■ Each face stores the environment as seen through one side of the cube / ■ A 3D direction vector is used to select the face and lookup position. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Cube Environment Mapping. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: ■ Cube maps are represented by 6 square textures forming the sides of a cube / ■ The cube is centered around the shading point or camera / ■ Each face stores the environment as seen through one side of the cube / ■ A 3D direction vector is used to select the face and lookup position
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Cube Environment Mapping' changes positions before rasterization?

Page 99	Cube Mapping Example 1/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Cube Mapping Example 1/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Cube Mapping Example 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Cube Mapping Example 1/2' into a causal sentence instead of repeating the slide title?

Page 100	Cube Mapping Example 2/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Cube Mapping Example 2/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Cube Mapping Example 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Cube Mapping Example 2/2' into a causal sentence instead of repeating the slide title?
Page 101	Cube Mapping Distortion
Source cue	■ Cube maps still distort the environment / ■ The distortion is distributed more uniformly than in sphere maps / ■ There are no pole singularities / ■ The main artifacts occur at cube-face boundaries
Professor-style explanation	The slide 'Cube Mapping Distortion' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Cube maps still distort the environment / ■ The distortion is distributed more uniformly than in sphere maps / ■ There are no pole singularities / ■ The main artifacts occur at cube-face boundaries. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Cube Mapping Distortion. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Cube maps still distort the environment / ■ The distortion is distributed more uniformly than in sphere maps / ■ There are no pole singularities / ■ The main artifacts occur at cube-face boundaries
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Cube Mapping Distortion'?
Page 102	Cube Map Texture Coordinates
Source cue	$R = (R_x, R_y, R_z)$ / ■ The reflection vector $x\ y\ z$ is used as a 3D lookup direction / ■ The cube-map face is selected by the major axis of R / • largest absolute component determines the face / ■ The remaining two components determine the 2D coordinates on that face

Professor-style explanation	The slide 'Cube Map Texture Coordinates' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: $R = (R_x, R_y, R_z)$ / ■ The reflection vector x, y, z is used as a 3D lookup direction / ■ The cube-map face is selected by the major axis of / • largest absolute component determines the face / ■ The remaining two components determine the 2D coordinates on that face. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Cube Map Texture Coordinates. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $R = (R_x, R_y, R_z)$ / ■ The reflection vector x, y, z is used as a 3D lookup direction / ■ The cube-map face is selected by the major axis of / • largest absolute component determines the face / ■ The remaining two components determine the 2D coordinates on that face
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Cube Map Texture Coordinates'?
Page 103	Environment Mapping in OpenGL
Source cue	■ In modern OpenGL, reflections are usually stored in a cube map / • six square images form the faces of a virtual cube / • reflection vector is used as a 3D texture coordinate / ■ Typical use cases / • reflective objects
Professor-style explanation	The slide 'Environment Mapping in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ In modern OpenGL, reflections are usually stored in a cube map / • six square images form the faces of a virtual cube / • reflection vector is used as a 3D texture coordinate / ■ Typical use cases / • reflective objects. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Environment Mapping in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ In modern OpenGL, reflections are usually stored in a cube map / • six square images form the faces of a virtual cube / • reflection vector is used as a 3D texture coordinate / ■ Typical use cases / • reflective objects
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Environment Mapping in OpenGL'?

Page 104	Cube Map Texture Object
Source cue	■ Cube maps are OpenGL texture objects with target GL_TEXTURE_CUBE_MAP / ■ Each face is addressed by a separate cube-map face target / ■ Use immutable storage for fixed size, format, and mipmap count / 1 GLuint envMap; / 2 glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &envMap);
Professor-style explanation	The slide 'Cube Map Texture Object' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Cube maps are OpenGL texture objects with target GL_TEXTURE_CUBE_MAP / ■ Each face is addressed by a separate cube-map face target / ■ Use immutable storage for fixed size, format, and mipmap count / 1 GLuint envMap; / 2 glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &envMap). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Cube Map Texture Object. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Cube maps are OpenGL texture objects with target GL_TEXTURE_CUBE_MAP / ■ Each face is addressed by a separate cube-map face target / ■ Use immutable storage for fixed size, format, and mipmap count / 1 GLuint envMap; / 2 glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &envMap);
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Cube Map Texture Object'?
Page 105	Sampling Direction

Source cue	■ A cube map is sampled with a 3D direction vector / ■ The largest absolute component of the direction selects the cube face / ■ The remaining two components determine the 2D position on that face / ■ For reflection mapping, use the reflected view vector / 1 vec3 V = normalize(cameraPosition - worldPosition);
Professor-style explanation	The slide 'Sampling Direction' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: ■ A cube map is sampled with a 3D direction vector / ■ The largest absolute component of the direction selects the cube face / ■ The remaining two components determine the 2D position on that face / ■ For reflection mapping, use the reflected view vector / 1 vec3 V = normalize(cameraPosition - worldPosition). The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Sampling Direction. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: ■ A cube map is sampled with a 3D direction vector / ■ The largest absolute component of the direction selects the cube face / ■ The remaining two components determine the 2D position on that face / ■ For reflection mapping, use the reflected view vector / 1 vec3 V = normalize(cameraPosition - worldPosition);
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Sampling Direction' changes positions before rasterization?
Page 106	Environment Mapping in GLSL 1/2
Source cue	■ Use samplerCube instead of sampler2D / ■ Store positions and normals in world space / ■ Keep the cube map fixed in world space / • otherwise the reflection appears glued to the camera / 1 // Fragment shader

Professor-style explanation	The slide 'Environment Mapping in GLSL 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Use samplerCube instead of sampler2D / ■ Store positions and normals in world space / ■ Keep the cube map fixed in world space / • otherwise the reflection appears glued to the camera / 1 // Fragment shader. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Environment Mapping in GLSL 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Use samplerCube instead of sampler2D / ■ Store positions and normals in world space / ■ Keep the cube map fixed in world space / • otherwise the reflection appears glued to the camera / 1 // Fragment shader
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Environment Mapping in GLSL 1/2'?
Page 107	Environment Mapping in GLSL 2/2
Source cue	<pre>1 void main() / 2 { / 3 vec3 V = normalize(cameraPosition - vWorldPosition); / 4 vec3 N = normalize(vWorldNormal); / 5 vec3 R = reflect(-V, N);</pre>
Professor-style explanation	The slide 'Environment Mapping in GLSL 2/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: 1 void main() / 2 { / 3 vec3 V = normalize(cameraPosition - vWorldPosition); / 4 vec3 N = normalize(vWorldNormal); / 5 vec3 R = reflect(-V, N). The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Environment Mapping in GLSL 2/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: 1 void main() / 2 { / 3 vec3 V = normalize(cameraPosition - vWorldPosition); / 4 vec3 N = normalize(vWorldNormal); / 5 vec3 R = reflect(-V, N);
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Environment Mapping in GLSL 2/2' changes positions before rasterization?
Page 108	Practical Cube-Map Issues
Source cue	■ Cube-map reflections are an approximation / • environment is assumed to be far away / • object does not reflect itself / • other moving objects are usually missing / ■ Important implementation details
Professor-style explanation	The slide 'Practical Cube-Map Issues' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Cube-map reflections are an approximation / • environment is assumed to be far away / • object does not reflect itself / • other moving objects are usually missing / ■ Important implementation details. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Practical Cube-Map Issues. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Cube-map reflections are an approximation / • environment is assumed to be far away / • object does not reflect itself / • other moving objects are usually missing / ■ Important implementation details
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Practical Cube-Map Issues'?
Page 109	1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);
Source cue	3 glTextureParameteri(envMap, / Glass Utah teapot rendering; image: Ronak19, CC / BY-SA 4.0. / Extrahierte Tabellen: / [Tabelle 1]

Professor-style explanation	The slide '1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: 3 glTextureParameteri(envMap, / Glass Utah teapot rendering; image: Ronak19, CC / BY-SA 4.0. / Extrahierte Tabellen: / [Tabelle 1]. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about 1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: 3 glTextureParameteri(envMap, / Glass Utah teapot rendering; image: Ronak19, CC / BY-SA 4.0. / Extrahierte Tabellen: / [Tabelle 1]
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in '1 glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);'?
Page 110	Cube Mapping Pros and Cons
Source cue	■ Advantages / • direct hardware support / • limited distortion / • no pole singularities / • natural representation for directional data
Professor-style explanation	The slide 'Cube Mapping Pros and Cons' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Advantages / • direct hardware support / • limited distortion / • no pole singularities / • natural representation for directional data. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Cube Mapping Pros and Cons. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Advantages / • direct hardware support / • limited distortion / • no pole singularities / • natural representation for directional data
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Cube Mapping Pros and Cons' into a causal sentence instead of repeating the slide title?
Page 111	Spherical Environment Mapping Assumptions

Source cue	■ Environment assumed far from object to be textured / ■ Self-reflection not taken into account / • Object to be textured assumed a convex shape(even if it is not geometrically convex) / ■ No other moving objects in the environment / • Interreflection is not taken into account
Professor-style explanation	The slide 'Spherical Environment Mapping Assumptions' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Environment assumed far from object to be textured / ■ Self-reflection not taken into account / • Object to be textured assumed a convex shape(even if it is not geometrically convex) / ■ No other moving objects in the environment / • Interreflection is not taken into account. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Spherical Environment Mapping Assumptions. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Environment assumed far from object to be textured / ■ Self-reflection not taken into account / • Object to be textured assumed a convex shape(even if it is not geometrically convex) / ■ No other moving objects in the environment / • Interreflection is not taken into account
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping Assumptions'?
Page 112	Spherical Environment Mapping Conclusions
Source cue	■ Advantages / • Contains panoramic image of an environment in a single texture / • Filtering and LoD formation takes place within a single texture / ■ Disadvantages / • Singularity along the outer disc border
Professor-style explanation	The slide 'Spherical Environment Mapping Conclusions' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Advantages / • Contains panoramic image of an environment in a single texture / • Filtering and LoD formation takes place within a single texture / ■ Disadvantages / • Singularity along the outer disc border. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.

Technical commentary	This slide is about Spherical Environment Mapping Conclusions. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Advantages / • Contains panoramic image of an environment in a single texture / • Filtering and LoD formation takes place within a single texture / ■ Disadvantages / • Singularity along the outer disc border
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Spherical Environment Mapping Conclusions'?

Page 113	9.7 3D Textures and Volume Rendering
Source cue	Texturing volumetric data, solid materials, and participating media
Professor-style explanation	The slide '9.7 3D Textures and Volume Rendering' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Texturing volumetric data, solid materials, and participating media. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about 9.7 3D Textures and Volume Rendering. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Texturing volumetric data, solid materials, and participating media
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '9.7 3D Textures and Volume Rendering'?

Page 114	3D Texturing: Relevant Topics
Source cue	■ Data model / • Regular voxel grids, channels, formats, memory footprint / ■ Mapping model / (s, t, r) / • Texture coordinates and object-to-texture transformations

Professor-style explanation	The slide '3D Texturing: Relevant Topics' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ Data model / • Regular voxel grids, channels, formats, memory footprint / ■ Mapping model / (s, t, r) / • Texture coordinates and object-to-texture transformations. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3D Texturing: Relevant Topics. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ Data model / • Regular voxel grids, channels, formats, memory footprint / ■ Mapping model / (s, t, r) / • Texture coordinates and object-to-texture transformations
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3D Texturing: Relevant Topics'?
Page 115	3D Texture as a Voxel Grid
Source cue	■ A 3D texture stores samples on a regular three-dimensional grid / ■ A sample is often called a voxel / ■ Each voxel can store scalar, vector, or multi-channel data / • Density, temperature, material ID / • Color and opacity
Professor-style explanation	The slide '3D Texture as a Voxel Grid' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ A 3D texture stores samples on a regular three-dimensional grid / ■ A sample is often called a voxel / ■ Each voxel can store scalar, vector, or multi-channel data / • Density, temperature, material ID / • Color and opacity. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about 3D Texture as a Voxel Grid. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ A 3D texture stores samples on a regular three-dimensional grid / ■ A sample is often called a voxel / ■ Each voxel can store scalar, vector, or multi-channel data / • Density, temperature, material ID / • Color and opacity
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '3D Texture as a Voxel Grid'?

Page 116	Object Space to Texture Space
Source cue	■ 3D texture coordinates are usually derived from object-space position / ■ For a volume-aligned bounding box / $p - p / \text{object min} / (s, t, r) =$
Professor-style explanation	The slide 'Object Space to Texture Space' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ 3D texture coordinates are usually derived from object-space position / ■ For a volume-aligned bounding box / $p - p / \text{object min} / (s, t, r) =$. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Object Space to Texture Space. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ 3D texture coordinates are usually derived from object-space position / ■ For a volume-aligned bounding box / $p - p / \text{object min} / (s, t, r) =$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Object Space to Texture Space'?

Page 117	3D Texture Access
Source cue	■ For each fragment, texture coordinates are interpolated / ■ A 3D lookup fetches the neighboring voxel values / ■ Linear filtering in 3D performs trilinear interpolation

Professor-style explanation	The slide '3D Texture Access' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ For each fragment, texture coordinates are interpolated / ■ A 3D lookup fetches the neighboring voxel values / ■ Linear filtering in 3D performs trilinear interpolation. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 3D Texture Access. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ For each fragment, texture coordinates are interpolated / ■ A 3D lookup fetches the neighboring voxel values / ■ Linear filtering in 3D performs trilinear interpolation
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3D Texture Access'?

Page 118	Trilinear Interpolation
Source cue	■ Trilinear interpolation is linear interpolation along three axes / ■ It blends the eight voxel values surrounding the lookup position / ■ Conceptually / • interpolate along / • interpolate the results along
Professor-style explanation	The slide 'Trilinear Interpolation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Trilinear interpolation is linear interpolation along three axes / ■ It blends the eight voxel values surrounding the lookup position / ■ Conceptually / • interpolate along / • interpolate the results along. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Trilinear Interpolation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Trilinear interpolation is linear interpolation along three axes / ■ It blends the eight voxel values surrounding the lookup position / ■ Conceptually / • interpolate along / • interpolate the results along
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Trilinear Interpolation' into a causal sentence instead of repeating the slide title?

Page 119	$f = (1 - \alpha) \cdot f + \alpha \cdot f$
Source cue	00 000 100 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 01 001 101 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 10 010 110
Professor-style explanation	The slide ' $f = (1 - \alpha) \cdot f + \alpha \cdot f$ ' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: 00 000 100 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 01 001 101 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 10 010 110. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about $f = (1 - \alpha) \cdot f + \alpha \cdot f$. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: 00 000 100 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 01 001 101 / $f = (1 - \alpha) \cdot f + \alpha \cdot f$ / 10 010 110
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn ' $f = (1 - \alpha) \cdot f + \alpha \cdot f$ ' into a causal sentence instead of repeating the slide title?
Page 120	3D Texture Memory Footprint
Source cue	■ Memory grows cubically with resolution / memory = $w \cdot h \cdot d \cdot \text{bytes per voxel}$ / Resolution R8 R16F RGBA8 RGBA16F / 1 B/voxel 2 B/voxel 4 B/voxel 8 B/voxel / 2 MiB 4 MiB 8 MiB 16 MiB
Professor-style explanation	The slide '3D Texture Memory Footprint' explains raster images as concrete stored data. A raster image is a rectangular grid of pixels. Each pixel stores one or more channel values, such as red, green, blue, and sometimes alpha. Color depth tells us how many bits are available per pixel or per channel, and that immediately determines both the number of representable colors and the memory footprint of the image. On the slide, the concrete items are: ■ Memory grows cubically with resolution / memory = $w \cdot h \cdot d \cdot \text{bytes per voxel}$ / Resolution R8 R16F RGBA8 RGBA16F / 1 B/voxel 2 B/voxel 4 B/voxel 8 B/voxel / 2 MiB 4 MiB 8 MiB 16 MiB. The object relation is pixel count, bits per pixel, color range, and memory size. This is why a simple image-resolution question is also a performance question: more pixels and more bits mean more memory traffic, more storage, and more work for display or image-processing operations.
Technical commentary	This slide is about 3D Texture Memory Footprint. The slide describes image data as discrete samples: pixels, color channels, bit depth, memory layout, and the amount of storage needed for a raster image. Concrete items shown: ■ Memory grows cubically with resolution / memory = $w \cdot h \cdot d \cdot \text{bytes per voxel}$ / Resolution R8 R16F RGBA8 RGBA16F / 1 B/voxel 2 B/voxel 4 B/voxel 8 B/voxel / 2 MiB 4 MiB 8 MiB 16 MiB
Why it matters	Pixel-data slides connect visual output to memory size, bandwidth, precision, and image-processing cost.
Check yourself	Can you compute or explain the pixel count, color depth, or memory relation in '3D Texture Memory Footprint'?

Page 121	3D Mipmaps and Filtering
Source cue	■ 3D textures can use mipmaps analogously to 2D textures / ■ Each mip level halves width, height, and depth / $k + 1$ / ■ Level contains one eighth of the voxels of level / ■ Useful for
Professor-style explanation	The slide '3D Mipmaps and Filtering' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ 3D textures can use mipmaps analogously to 2D textures / ■ Each mip level halves width, height, and depth / $k + 1$ / ■ Level contains one eighth of the voxels of level / ■ Useful for. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about 3D Mipmaps and Filtering. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ 3D textures can use mipmaps analogously to 2D textures / ■ Each mip level halves width, height, and depth / $k + 1$ / ■ Level contains one eighth of the voxels of level / ■ Useful for
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '3D Mipmaps and Filtering' works per object, per image region, per ray, or per fragment?
Page 122	Solid Texturing
Source cue	■ 3D textures can define material throughout the interior of an object / ■ Surface fragments sample the material at their 3D position / ■ This avoids seams caused by 2D UV unwrapping / ■ Useful for materials with volumetric structure / • wood grain
Professor-style explanation	The slide 'Solid Texturing' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ 3D textures can define material throughout the interior of an object / ■ Surface fragments sample the material at their 3D position / ■ This avoids seams caused by 2D UV unwrapping / ■ Useful for materials with volumetric structure / • wood grain. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Solid Texturing. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ 3D textures can define material throughout the interior of an object / ■ Surface fragments sample the material at their 3D position / ■ This avoids seams caused by 2D UV unwrapping / ■ Useful for materials with volumetric structure / • wood grain
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Solid Texturing'?

Page 123	Volumetric Data as Texture Data
Source cue	■ Volume data is often represented as a scalar field / $\rho = f(x, y, z)$ / ■ Examples / • CT or MRI intensity / • smoke density
Professor-style explanation	The slide 'Volumetric Data as Texture Data' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Volume data is often represented as a scalar field / $\rho = f(x, y, z)$ / ■ Examples / • CT or MRI intensity / • smoke density. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Volumetric Data as Texture Data. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Volume data is often represented as a scalar field / $\rho = f(x, y, z)$ / ■ Examples / • CT or MRI intensity / • smoke density
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Volumetric Data as Texture Data'?

Page 124	Transfer Functions
Source cue	■ A transfer function maps data values to optical properties / $T(\rho) = (c, \alpha)$ / ■ Color emphasizes material classes or value ranges / ■ Opacity controls which structures become visible / ■ Common choices

Professor-style explanation	The slide 'Transfer Functions' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ A transfer function maps data values to optical properties / $T(p) = (c, \alpha)$ / ■ Color emphasizes material classes or value ranges / ■ Opacity controls which structures become visible / ■ Common choices. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Transfer Functions. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ A transfer function maps data values to optical properties / $T(p) = (c, \alpha)$ / ■ Color emphasizes material classes or value ranges / ■ Opacity controls which structures become visible / ■ Common choices
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Transfer Functions'?
Page 125	Volume Rendering with 3D Textures
Source cue	■ 3D grid data is stored in a 3D texture / ■ Slice-based volume rendering draws proxy geometry through the volume / ■ Slices are usually orthogonal to the view direction / ■ Each slice samples the 3D texture and applies the transfer function / ■ Slices are blended back to front or front to back
Professor-style explanation	The slide 'Volume Rendering with 3D Textures' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ 3D grid data is stored in a 3D texture / ■ Slice-based volume rendering draws proxy geometry through the volume / ■ Slices are usually orthogonal to the view direction / ■ Each slice samples the 3D texture and applies the transfer function / ■ Slices are blended back to front or front to back. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.

Technical commentary	This slide is about Volume Rendering with 3D Textures. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ 3D grid data is stored in a 3D texture ■ Slice-based volume rendering draws proxy geometry through the volume ■ Slices are usually orthogonal to the view direction ■ Each slice samples the 3D texture and applies the transfer function ■ Slices are blended back to front or front to back
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Volume Rendering with 3D Textures'?

Page 126	Front-to-Back Alpha Compositing
Source cue	■ Volume rendering approximates absorption and emission along a ray / $c \cdot \alpha$ ■ For a sample with color i and opacity α / $C = C + (1 - \alpha) \cdot \alpha \cdot c$ / out in in i
Professor-style explanation	The slide 'Front-to-Back Alpha Compositing' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Volume rendering approximates absorption and emission along a ray / $c \cdot \alpha$ ■ For a sample with color i and opacity α / $C = C + (1 - \alpha) \cdot \alpha \cdot c$ / out in in i The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Front-to-Back Alpha Compositing. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Volume rendering approximates absorption and emission along a ray / $c \cdot \alpha$ ■ For a sample with color i and opacity α / $C = C + (1 - \alpha) \cdot \alpha \cdot c$ / out in in i
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Front-to-Back Alpha Compositing' works per object, per image region, per ray, or per fragment?

Page 127	Ray Marching Volume Rendering
Source cue	■ Instead of drawing slices, cast one ray per fragment through the volume ■ Repeatedly sample the 3D texture along the ray ■ Apply the transfer function at every sample ■ Composite samples until the ray exits the volume or becomes opaque / 1 for (float $t = t_{\text{Entry}}$; $t < t_{\text{Exit}}$; $t += \text{stepSize}$) {

Professor-style explanation	The slide 'Ray Marching Volume Rendering' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Instead of drawing slices, cast one ray per fragment through the volume / ■ Repeatedly sample the 3D texture along the ray / ■ Apply the transfer function at every sample / ■ Composite samples until the ray exits the volume or becomes opaque / 1 for (float t = tEntry; t < tExit; t += stepSize) {. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Ray Marching Volume Rendering. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Instead of drawing slices, cast one ray per fragment through the volume / ■ Repeatedly sample the 3D texture along the ray / ■ Apply the transfer function at every sample / ■ Composite samples until the ray exits the volume or becomes opaque / 1 for (float t = tEntry; t < tExit; t += stepSize) {
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Ray Marching Volume Rendering' works per object, per image region, per ray, or per fragment?
Page 128	Gradients and Volume Lighting
Source cue	■ Gradients estimate local orientation in the scalar field / ■ Central differences approximate the gradient / $\rho(x + \Delta, y, z) - \rho(x - \Delta, y, z)$ / ■ ■ / $\nabla \rho \approx$ ■ $\rho(x, y + \Delta, z) - \rho(x, y - \Delta, z)$ ■
Professor-style explanation	The slide 'Gradients and Volume Lighting' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Gradients estimate local orientation in the scalar field / ■ Central differences approximate the gradient / $\rho(x + \Delta, y, z) - \rho(x - \Delta, y, z)$ / ■ ■ / $\nabla \rho \approx$ ■ $\rho(x, y + \Delta, z) - \rho(x, y - \Delta, z)$ ■. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Gradients and Volume Lighting. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Gradients estimate local orientation in the scalar field / ■ Central differences approximate the gradient / $\rho(x + \Delta, y, z) - \rho(x - \Delta, y, z)$ / ■ ■ / $\nabla \rho \approx$ ■ $\rho(x, y + \Delta, z) - \rho(x, y - \Delta, z)$ ■
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Gradients and Volume Lighting'?
Page 129	Quality and Performance Trade-Offs
Source cue	■ Step size / • smaller steps improve quality but increase cost / • larger steps are faster but can miss thin structures / ■ Opacity correction / • keeps appearance stable when step size changes
Professor-style explanation	The slide 'Quality and Performance Trade-Offs' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: ■ Step size / • smaller steps improve quality but increase cost / • larger steps are faster but can miss thin structures / ■ Opacity correction / • keeps appearance stable when step size changes. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Quality and Performance Trade-Offs. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: ■ Step size / • smaller steps improve quality but increase cost / • larger steps are faster but can miss thin structures / ■ Opacity correction / • keeps appearance stable when step size changes
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Quality and Performance Trade-Offs' into a causal sentence instead of repeating the slide title?
Page 130	3D Texturing Takeaways
Source cue	■ 3D textures generalize 2D texture lookup from surfaces to volumes / ■ They are useful for solid materials, volumetric effects, and sampled scientific data / ■ Trilinear interpolation provides smooth access to voxel grids / ■ Volume rendering turns sampled data into color and opacity along viewing rays / ■ Transfer functions and compositing determine what becomes visible

Professor-style explanation	The slide '3D Texturing Takeaways' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ 3D textures generalize 2D texture lookup from surfaces to volumes ■ They are useful for solid materials, volumetric effects, and sampled scientific data ■ Trilinear interpolation provides smooth access to voxel grids ■ Volume rendering turns sampled data into color and opacity along viewing rays ■ Transfer functions and compositing determine what becomes visible. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about 3D Texturing Takeaways. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ 3D textures generalize 2D texture lookup from surfaces to volumes ■ They are useful for solid materials, volumetric effects, and sampled scientific data ■ Trilinear interpolation provides smooth access to voxel grids ■ Volume rendering turns sampled data into color and opacity along viewing rays ■ Transfer functions and compositing determine what becomes visible
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '3D Texturing Takeaways' works per object, per image region, per ray, or per fragment?
Page 131	■ Textures are sampled GPU data sources used by shaders
Source cue	• Color, opacity, normals, roughness, height, masks, and volume density ■ Texture coordinates define where texture data is sampled / (s, t, r) • UV mapping, texture transformations, and 3D coordinates ■ Texture formats, channel semantics, and color space determine correct interpretation
Professor-style explanation	The slide '■ Textures are sampled GPU data sources used by shaders' collects the source material behind the chapter. References are not rendering objects themselves, but they identify the books, papers, or external resources from which the lecture's terminology and algorithms are drawn. On the slide, the concrete items are: • Color, opacity, normals, roughness, height, masks, and volume density ■ Texture coordinates define where texture data is sampled / (s, t, r) • UV mapping, texture transformations, and 3D coordinates ■ Texture formats, channel semantics, and color space determine correct interpretation. In practical terms, a reference slide marks the boundary of the chapter and tells us where the formal definitions, derivations, or extended examples can be found if a topic needs more depth than the lecture slides provide.

Technical commentary	This slide is about ■ Textures are sampled GPU data sources used by shaders. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: • Color, opacity, normals, roughness, height, masks, and volume density / ■ Texture coordinates define where texture data is sampled / (s, t, r) / • UV mapping, texture transformations, and 3D coordinates / ■ Texture formats, channel semantics, and color space determine correct interpretation
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic '■ Textures are sampled GPU data sources used by shaders' points to for deeper study?

Page 132	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?

Page 133	■ Text Books
Source cue	• J. Foley, A. van Dam, S. Fine"Computer Graphics: Principles and Practice (3About Edition)", / Addison-Wesley 2013. / • T. Akenine-Möller, E. Haines: Real-Time Rendering, Addison-Wesley. / • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Others

Professor-style explanation	The slide '■ Text Books' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: • J. Foley, A. van Dam, S. Fine"Computer Graphics: Principles and Practice (3About Edition)", / Addison-Wesley 2013. / • T. Akenine-Möller, E. Haines: Real-Time Rendering, Addison-Wesley. / • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Others. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about ■ Text Books. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: • J. Foley, A. van Dam, S. Fine"Computer Graphics: Principles and Practice (3About Edition)", / Addison-Wesley 2013. / • T. Akenine-Möller, E. Haines: Real-Time Rendering, Addison-Wesley. / • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Others
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '■ Text Books' into a causal sentence instead of repeating the slide title?

09.1 Texture Objects, Texels, Formats, and Color Space

Source location: Section 9.1

Commentary

A texture object stores sampled data on the GPU. A texel is one stored texture sample. The texture format decides what channels and numeric representation the samples use. Color space matters because color values may be stored nonlinearly, especially for sRGB images.

The key shift is to stop thinking of texture as only a picture. Textures can store color, normals, depth, lookup data, environment information, or volume data. The shader decides how the sampled values are interpreted.

Mental model

A texture is a GPU-accessible sampled data field.

Check yourself

Why can the same texture mechanism store both color maps and normal maps?

09.2 Texture Coordinates, UV Mapping, and Wrapping

Source location: Section 9.2

Commentary

Texture coordinates map surface points to locations in texture space. UV coordinates are usually interpolated across primitives during rasterization and then used by the fragment shader to sample a texture.

Wrapping rules decide what happens outside the normal coordinate range. Repeat, clamp, mirrored repeat, and border behavior are not visual afterthoughts; they define the sampling function outside the base domain.

Mental model

UVs are the address system that lets a fragment look up texture data.

Check yourself

What artifact might appear if a wrapping mode is wrong at the edge of a surface?

09.3 Sampling, Filtering, Mipmaps, and Anisotropy

Source location: Section 9.3

Commentary

Sampling turns continuous texture coordinates into values from discrete texels. Nearest filtering selects a nearby texel and can look blocky. Linear filtering blends nearby texels and looks smoother. Minification is harder because many texels may map to one pixel.

Mipmaps store prefiltered lower-resolution versions of a texture. They reduce aliasing and shimmer when textures are seen at small scale. Anisotropic filtering improves quality when a texture is viewed at a steep angle where footprint shape is elongated.

Mental model

Filtering reconstructs values; mipmaps choose an appropriate scale before reconstruction.

Check yourself

Why does a distant checkerboard shimmer without mipmapping?

09.4 Modern OpenGL Texture Pipeline

Source location: Section 9.4

Commentary

In modern OpenGL, textures are represented by texture objects, bound to texture units, connected to sampler uniforms, and accessed in shaders. The shader does not sample a filename; it samples a bound texture through a sampler.

Texture bugs often come from state mismatches: wrong active texture unit, wrong sampler uniform, missing mipmaps for a mipmap filter, wrong wrap mode, or wrong internal format.

Mental model

OpenGL texturing is a chain: object data -> texture unit -> sampler uniform -> shader lookup.

Check yourself

Why can a texture appear black when the shader code is mathematically correct?

09.5 Normal Mapping

Source location: Section 9.5

Commentary

Normal mapping stores normal directions in a texture so that lighting can vary at a finer scale than the geometry. The surface may have few triangles, but the shader receives detailed normals per fragment.

The hard part is coordinate space. Normal maps are often defined in tangent space, so the shader must transform or interpret them with the correct tangent, bitangent, and normal basis.

Mental model

Normal mapping changes the lighting normal, not the actual mesh silhouette.

Check yourself

Why does normal mapping not change the geometric outline of an object?

09.6 Environment Mapping and 3D Textures

Source location: Sections 9.6-9.7

Commentary

Environment mapping uses textures to represent surrounding illumination or reflections. A direction, not a surface UV alone, can be used to sample an environment map. This is a texture lookup driven by view/reflection geometry.

3D textures and volume rendering extend the same sampled-data idea into three dimensions. Instead of sampling a 2D image, the shader samples a volume. This is useful for data such as medical scans, density fields, or procedural volumetric effects.

Mental model

Textures are general sampled data; the coordinate dimensionality depends on the problem.

Check yourself

What coordinate type would you use to sample a 3D texture?

End-of-lecture summary

If you remember only one thing from Lecture 09, remember this: This lecture treats textures as sampled data for shading, not merely images glued onto objects. It connects UV coordinates, sampling, filtering, mipmaps, OpenGL texture state, and advanced texture uses.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.

Lecture 10 - Shadows

This lecture explains shadows as a visibility problem from the light source. A point is lit if the light can see it; it is shadowed if another object blocks that visibility.

Source chunk	course_text_parts/03_lectures/10_shadows.txt
Extracted slide pages	60
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Shadow Perception 1/3
Source cue	■ Visual perception of an environment is influenced by shadows / • Object object relationships / • Object space relationships / ■ Improved depth perception is made possible by shadows / ■ Shadows provide appealing visual effects

Professor-style explanation	The slide 'Shadow Perception 1/3' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Visual perception of an environment is influenced by shadows / • Object object relationships / • Object space relationships / ■ Improved depth perception is made possible by shadows / ■ Shadows provide appealing visual effects. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Shadow Perception 1/3. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Visual perception of an environment is influenced by shadows / • Object object relationships / • Object space relationships / ■ Improved depth perception is made possible by shadows / ■ Shadows provide appealing visual effects
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Shadow Perception 1/3' works per object, per image region, per ray, or per fragment?

Page 3	Shadow Perception 2/3
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Shadow Perception 2/3' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Shadow Perception 2/3. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Perception 2/3'?

Page 4	Shadow Perception 3/3
Source cue	■ KerstenD. Knill, D. C., Mamassian, P. and Bülthoffl. Illusory motion from shadows, Nature, 379 (31), / 1996.

Professor-style explanation	The slide 'Shadow Perception 3/3' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: ■ KerstenD. Knill, D. C., Mamassian, P. and Bühlhoffl. Illusory motion from shadows, Nature, 379 (31), / 1996. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.
Technical commentary	This slide is about Shadow Perception 3/3. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: ■ KerstenD. Knill, D. C., Mamassian, P. and Bühlhoffl. Illusory motion from shadows, Nature, 379 (31), / 1996.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Perception 3/3'?
Page 5	Shadow Atmosphere
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Shadow Atmosphere' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Shadow Atmosphere. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Atmosphere'?
Page 6	■ Shadows createa certain atmosphere
Source cue	■ The Lighthouse (2019)

Professor-style explanation	The slide '■ Shadows createa certain atmosphere' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ The Lighthouse (2019). The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about ■ Shadows createa certain atmosphere. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ The Lighthouse (2019)
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '■ Shadows createa certain atmosphere'?

Page 7	Shadow Atmosphere
Source cue	■ Shadows createa certain atmosphere / ■ The Lighthouse (2019) / ■ Kill Bill: Volume 1 (2003)
Professor-style explanation	The slide 'Shadow Atmosphere' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Shadows createa certain atmosphere / ■ The Lighthouse (2019) / ■ Kill Bill: Volume 1 (2003). The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Shadow Atmosphere. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Shadows createa certain atmosphere / ■ The Lighthouse (2019) / ■ Kill Bill: Volume 1 (2003)
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Shadow Atmosphere'?

Page 8	Overview
--------	----------

Source cue	10.1 Definitions / 10.2 Ground Plane Shadows / 10.3 Light Maps / 10.4 Shadow Volumes / 10.5 Shadow Maps
Professor-style explanation	The slide 'Overview' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: 10.1 Definitions / 10.2 Ground Plane Shadows / 10.3 Light Maps / 10.4 Shadow Volumes / 10.5 Shadow Maps. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Overview. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: 10.1 Definitions / 10.2 Ground Plane Shadows / 10.3 Light Maps / 10.4 Shadow Volumes / 10.5 Shadow Maps
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Overview'?
Page 9	10.1 Definitions
Source cue	About shadow-receiving and shadow-casting scene objects
Professor-style explanation	The slide '10.1 Definitions' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: About shadow-receiving and shadow-casting scene objects. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.
Technical commentary	This slide is about 10.1 Definitions. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: About shadow-receiving and shadow-casting scene objects
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in '10.1 Definitions'?
Page 10	Classification of scene objects

Source cue	■ Distinction between / • Shadow-casting scene objects(Occluder) / • Shadow-receiving scene objects (Receiver) / ■ Classify shadow-casting and shadow-receiving objects in a preprocess? / • Self-shadowing results from same object casting and receiving shadows
Professor-style explanation	The slide 'Classification of scene objects' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: ■ Distinction between / • Shadow-casting scene objects(Occluder) / • Shadow-receiving scene objects (Receiver) / ■ Classify shadow-casting and shadow-receiving objects in a preprocess? / • Self-shadowing results from same object casting and receiving shadows. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.
Technical commentary	This slide is about Classification of scene objects. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: ■ Distinction between / • Shadow-casting scene objects(Occluder) / • Shadow-receiving scene objects (Receiver) / ■ Classify shadow-casting and shadow-receiving objects in a preprocess? / • Self-shadowing results from same object casting and receiving shadows
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Classification of scene objects'?

Page 11	Umbra and Penumbra
Source cue	■ Point light sources / • Result in hard shadows / • High intensity contrast between shadowed and non-shadowed region / ■ Area light sources / • Result in soft shadows
Professor-style explanation	The slide 'Umbra and Penumbra' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Point light sources / • Result in hard shadows / • High intensity contrast between shadowed and non-shadowed region / ■ Area light sources / • Result in soft shadows. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.

Technical commentary	This slide is about Umbra and Penumbra. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Point light sources / • Result in hard shadows / • High intensity contrast between shadowed and non-shadowed region / ■ Area light sources / • Result in soft shadows
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Umbra and Penumbra'?

Page 12	Hard and Soft Shadows
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Hard and Soft Shadows' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Hard and Soft Shadows. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Hard and Soft Shadows'?

Page 13	Description of Shadows
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Description of Shadows' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Description of Shadows. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.

Check yourself	Can you explain what the light can or cannot see in 'Description of Shadows'?
Page 14	■ Shadows can be described differently
Source cue	■ Shadows can be modeled as / • Separate objects / • Surfaces not seen when looking into scene from light source / • Volumetric regions with reduced lighting
Professor-style explanation	The slide '■ Shadows can be described differently' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Shadows can be modeled as / • Separate objects / • Surfaces not seen when looking into scene from light source / • Volumetric regions with reduced lighting. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about ■ Shadows can be described differently. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Shadows can be modeled as / • Separate objects / • Surfaces not seen when looking into scene from light source / • Volumetric regions with reduced lighting
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '■ Shadows can be described differently'?
Page 15	Observations 1/2
Source cue	■ Possible shadow generation approaches / • Shadows as surfaces: exploit hidden-surface algorithms / • Shadows as projection: create shadows from A to B by projecting A onto B from light source / ■ Static vs. dynamic scenes / • Shadows are static in scenes with static lighting and scene objects (shadow computation)

Professor-style explanation	The slide 'Observations 1/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: ■ Possible shadow generation approaches / • Shadows as surfaces: exploit hidden-surface algorithms / • Shadows as projection: create shadows from A to B by projecting A onto B from light source / ■ Static vs. dynamic scenes / • Shadows are static in scenes with static lighting and scene objects (shadow computation. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Observations 1/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: ■ Possible shadow generation approaches / • Shadows as surfaces: exploit hidden-surface algorithms / • Shadows as projection: create shadows from A to B by projecting A onto B from light source / ■ Static vs. dynamic scenes / • Shadows are static in scenes with static lighting and scene objects (shadow computation
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Observations 1/2' changes positions before rasterization?
Page 16	Observations 2/2
Source cue	■ Shadow shape can be obtained by projection / • Trivial for projection on flat surfaces / • Algorithmically very complex for projection in general case / ■ Shadow intensity / • Shadows attenuate the incoming light used in illumination model

Professor-style explanation	The slide 'Observations 2/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: ■ Shadow shape can be obtained by projection / • Trivial for projection on flat surfaces / • Algorithmically very complex for projection in general case / ■ Shadow intensity / • Shadows attenuate the incoming light used in illumination model. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Observations 2/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: ■ Shadow shape can be obtained by projection / • Trivial for projection on flat surfaces / • Algorithmically very complex for projection in general case / ■ Shadow intensity / • Shadows attenuate the incoming light used in illumination model
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Observations 2/2' changes positions before rasterization?

Page 17	Shadow Calculation Methods
Source cue	■ Ground plane shadows / • Static shadow polygons / • Dynamic shadow polygons / ■ Projected shadows / ■ Light maps
Professor-style explanation	The slide 'Shadow Calculation Methods' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Ground plane shadows / • Static shadow polygons / • Dynamic shadow polygons / ■ Projected shadows / ■ Light maps. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Shadow Calculation Methods. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Ground plane shadows / • Static shadow polygons / • Dynamic shadow polygons / ■ Projected shadows / ■ Light maps

Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Shadow Calculation Methods'?
Page 18	10.2 Ground Plane Shadows
Source cue	Static and dynamic shadow polygons
Professor-style explanation	The slide '10.2 Ground Plane Shadows' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: Static and dynamic shadow polygons. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.
Technical commentary	This slide is about 10.2 Ground Plane Shadows. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: Static and dynamic shadow polygons
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in '10.2 Ground Plane Shadows'?
Page 19	Static Shadow Polygons
Source cue	■ Render dark polygons to represent shadow regions • Cast ray from light source through center of shadow-casting objects • Position dark polygon where ray intersects shadow-receiving object ■ Disadvantages • Static shapes cannot represent geometry of dynamic scene objects
Professor-style explanation	The slide 'Static Shadow Polygons' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Render dark polygons to represent shadow regions • Cast ray from light source through center of shadow-casting objects • Position dark polygon where ray intersects shadow-receiving object ■ Disadvantages • Static shapes cannot represent geometry of dynamic scene objects The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about Static Shadow Polygons. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Render dark polygons to represent shadow regions / • Cast ray from light source through center of shadow-casting objects / • Position dark polygon where ray intersects shadow-receiving object / ■ Disadvantages / • Static shapes cannot represent geometry of dynamic scene objects
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Static Shadow Polygons' works per object, per image region, per ray, or per fragment?

Page 20	Render Static Shadow Polygons 1/2
Source cue	■ Depth value identical with that of shadow-receiving polygon (care must be taken wrt. z-buffer / fighting) / ■ Depth value smaller than that of shadow-receiving polygon (shadow can be too large)
Professor-style explanation	The slide 'Render Static Shadow Polygons 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Depth value identical with that of shadow-receiving polygon (care must be taken wrt. z-buffer / fighting) / ■ Depth value smaller than that of shadow-receiving polygon (shadow can be too large). The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Render Static Shadow Polygons 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Depth value identical with that of shadow-receiving polygon (care must be taken wrt. z-buffer / fighting) / ■ Depth value smaller than that of shadow-receiving polygon (shadow can be too large)
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Render Static Shadow Polygons 1/2'?

Page 21	Render Static Shadow Polygons 2/2
Source cue	■ Disable depth test / • Cast ray to determine shadow visibility / • Show entire shadow polygon / ■ Shadow too large and possibly in wrong places / ■ Test corners for shadow visibility

Professor-style explanation	The slide 'Render Static Shadow Polygons 2/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Disable depth test / • Cast ray to determine shadow visibility / • Show entire shadow polygon / ■ Shadow too large and possibly in wrong places / ■ Test corners for shadow visibility. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Render Static Shadow Polygons 2/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Disable depth test / • Cast ray to determine shadow visibility / • Show entire shadow polygon / ■ Shadow too large and possibly in wrong places / ■ Test corners for shadow visibility
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Render Static Shadow Polygons 2/2' works per object, per image region, per ray, or per fragment?

Page 22	Static Shadow Polygons
Source cue	■ Good for car racing games (static scene geometry) / ■ With fast movements and proximity of casters and receivers, wrong shadows are often not detected / as such
Professor-style explanation	The slide 'Static Shadow Polygons' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: ■ Good for car racing games (static scene geometry) / ■ With fast movements and proximity of casters and receivers, wrong shadows are often not detected / as such. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.
Technical commentary	This slide is about Static Shadow Polygons. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: ■ Good for car racing games (static scene geometry) / ■ With fast movements and proximity of casters and receivers, wrong shadows are often not detected / as such
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Static Shadow Polygons'?

Page 23	Dynamic Shadow Polygons
Source cue	■ Project shadow-casting scene objects onto shadow-receiving object by modifying shape / ■ Accurate procedure for semi-transparent shadows, but only if shadows do not overlap / ■ Restrictions / • Only planar polygons on shadow-receiving scene objects possible / • No self-shadowing possible
Professor-style explanation	The slide 'Dynamic Shadow Polygons' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: ■ Project shadow-casting scene objects onto shadow-receiving object by modifying shape / ■ Accurate procedure for semi-transparent shadows, but only if shadows do not overlap / ■ Restrictions / • Only planar polygons on shadow-receiving scene objects possible / • No self-shadowing possible. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.
Technical commentary	This slide is about Dynamic Shadow Polygons. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: ■ Project shadow-casting scene objects onto shadow-receiving object by modifying shape / ■ Accurate procedure for semi-transparent shadows, but only if shadows do not overlap / ■ Restrictions / • Only planar polygons on shadow-receiving scene objects possible / • No self-shadowing possible
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Dynamic Shadow Polygons'?
Page 24	Shadow Projection
Source cue	S L / ■ Shadow point (= vertex of the shadow polygon) lies on a line between light source and vertex / of shadow-casting scene object / $S = P - \alpha \cdot L$ / $S_z = 0$ $\alpha =$
Professor-style explanation	The slide 'Shadow Projection' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: S L / ■ Shadow point (= vertex of the shadow polygon) lies on a line between light source and vertex / of shadow-casting scene object / $S = P - \alpha \cdot L$ / $S_z = 0$ $\alpha =$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Shadow Projection. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: $S \cdot L / \blacksquare$ Shadow point (= vertex of the shadow polygon) lies on a line between light source and vertex $/$ of shadow-casting scene object $/ S = P - \alpha \cdot L / S \cdot z = 0 \alpha =$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Shadow Projection'?

Page 25	Rendering Dynamic Shadow Polygons
Source cue	$\blacksquare$ Matrix transforms object into its shadow polygon $/ \blacksquare$ Render scene object with shadows $/ \bullet$ Render scene object $/ \bullet$ Multiply current model-view -matrix with $/ \bullet$ Render scene object in shadow color with b lending
Professor-style explanation	The slide 'Rendering Dynamic Shadow Polygons' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: $\blacksquare$ Matrix transforms object into its shadow polygon $/ \blacksquare$ Render scene object with shadows $/ \bullet$ Render scene object $/ \bullet$ Multiply current model-view -matrix with $/ \bullet$ Render scene object in shadow color with b lending. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Rendering Dynamic Shadow Polygons. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $\blacksquare$ Matrix transforms object into its shadow polygon $/ \blacksquare$ Render scene object with shadows $/ \bullet$ Render scene object $/ \bullet$ Multiply current model-view -matrix with $/ \bullet$ Render scene object in shadow color with b lending
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Rendering Dynamic Shadow Polygons'?

Page 26	10.3 Light Maps
Source cue	Structured lighting with textures

Professor-style explanation	The slide '10.3 Light Maps' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Structured lighting with textures. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about 10.3 Light Maps. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Structured lighting with textures
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to '10.3 Light Maps'?

Page 27	Light Maps
Source cue	■ Static shadows can be precomputed and stored in light maps ■ When precomputing light maps, shadows are calculated offline(e.g. with ray tracing) ■ Soft shadows in lightmaps • Interpolation of the lightmap creates soft shadow approximations • Light source sampling results in better results,sending multiple rays to different points of the
Professor-style explanation	The slide 'Light Maps' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Static shadows can be precomputed and stored in light maps ■ When precomputing light maps, shadows are calculated offline(e.g. with ray tracing) ■ Soft shadows in lightmaps • Interpolation of the lightmap creates soft shadow approximations • Light source sampling results in better results,sending multiple rays to different points of the. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Light Maps. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Static shadows can be precomputed and stored in light maps ■ When precomputing light maps, shadows are calculated offline(e.g. with ray tracing) ■ Soft shadows in lightmaps • Interpolation of the lightmap creates soft shadow approximations • Light source sampling results in better results,sending multiple rays to different points of the

Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Light Maps' works per object, per image region, per ray, or per fragment?
Page 28	Soft Shadows with Light Maps 1/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Soft Shadows with Light Maps 1/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Soft Shadows with Light Maps 1/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Soft Shadows with Light Maps 1/2'?
Page 29	Soft Shadows with Light Maps 2/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Soft Shadows with Light Maps 2/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Soft Shadows with Light Maps 2/2. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Soft Shadows with Light Maps 2/2'?

Page 30	10.4 Shadow Volumes
Source cue	Representation of shadows by geometry enclosing them
Professor-style explanation	The slide '10.4 Shadow Volumes' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: Representation of shadows by geometry enclosing them. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.
Technical commentary	This slide is about 10.4 Shadow Volumes. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: Representation of shadows by geometry enclosing them
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in '10.4 Shadow Volumes'?

Page 31	Shadow Volumes
Source cue	■ Shadows are considered as volume in space / ■ Originally developed by Crow in 1977 / ■ Extension by Brotman and Badler in 1984 / ■ Heidmann introduced the technology for efficient rendering of shadow volumes in 1991
Professor-style explanation	The slide 'Shadow Volumes' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: ■ Shadows are considered as volume in space / ■ Originally developed by Crow in 1977 / ■ Extension by Brotman and Badler in 1984 / ■ Heidmann introduced the technology for efficient rendering of shadow volumes in 1991. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.
Technical commentary	This slide is about Shadow Volumes. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: ■ Shadows are considered as volume in space / ■ Originally developed by Crow in 1977 / ■ Extension by Brotman and Badler in 1984 / ■ Heidmann introduced the technology for efficient rendering of shadow volumes in 1991
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Volumes'?

Page 32	Initial Considerations
Source cue	■ Shadow volume shape depends on shape and position of light source and shadow-caster / ■ All points inside shadow volume are shadowed / ■ Shadow volume generation / • Find silhouettes of shadow-caster regarding light source / • Extrudes silhouettes in light ray direction to create polygons
Professor-style explanation	The slide 'Initial Considerations' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Shadow volume shape depends on shape and position of light source and shadow-caster / ■ All points inside shadow volume are shadowed / ■ Shadow volume generation / • Find silhouettes of shadow-caster regarding light source / • Extrudes silhouettes in light ray direction to create polygons. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Initial Considerations. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Shadow volume shape depends on shape and position of light source and shadow-caster / ■ All points inside shadow volume are shadowed / ■ Shadow volume generation / • Find silhouettes of shadow-caster regarding light source / • Extrudes silhouettes in light ray direction to create polygons
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Initial Considerations' works per object, per image region, per ray, or per fragment?
Page 33	Shadow Volume Geometry
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Shadow Volume Geometry' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Shadow Volume Geometry. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.

Check yourself	Can you explain what the light can or cannot see in 'Shadow Volume Geometry'?
Page 34	Shadow Volume Realization
Source cue	■ Silhouettes edges can be found by analyzing normals / • Silhouette edge lies between front and back face (as seen from light source) / ■ Silhouette edges are stored as vertex list containing vertices / ■ For convex shadow-casters / L V
Professor-style explanation	The slide 'Shadow Volume Realization' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Silhouettes edges can be found by analyzing normals / • Silhouette edge lies between front and back face (as seen from light source) / ■ Silhouette edges are stored as vertex list containing vertices / ■ For convex shadow-casters / L V. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Shadow Volume Realization. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Silhouettes edges can be found by analyzing normals / • Silhouette edge lies between front and back face (as seen from light source) / ■ Silhouette edges are stored as vertex list containing vertices / ■ For convex shadow-casters / L V
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Shadow Volume Realization'?
Page 35	Shadow Volume Properties 1/2
Source cue	■ All points inside shadow volume are shadowed / ■ Shadowing can be determined for each view ray by intersection tests / • Assumption: Camera not in shadow volume / • At each intersection with front-face shadow polygon, intersection counter cnt is incremented / • At each intersection with back-face shadow polygon, intersection counter cnt is decremented

Professor-style explanation	The slide 'Shadow Volume Properties 1/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ All points inside shadow volume are shadowed / ■ Shadowing can be determined for each view ray by intersection tests / • Assumption: Camera not in shadow volume / • At each intersection with front-face shadow polygon, intersection counter cnt is incremented / • At each intersection with back-face shadow polygon, intersection counter cnt is decremented. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Shadow Volume Properties 1/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ All points inside shadow volume are shadowed / ■ Shadowing can be determined for each view ray by intersection tests / • Assumption: Camera not in shadow volume / • At each intersection with front-face shadow polygon, intersection counter cnt is incremented / • At each intersection with back-face shadow polygon, intersection counter cnt is decremented
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Shadow Volume Properties 1/2' works per object, per image region, per ray, or per fragment?
Page 36	Shadow Volume Properties 2/2
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Shadow Volume Properties 2/2' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Shadow Volume Properties 2/2. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Volume Properties 2/2'?

Page 37	Stencil Shadow Volume
Source cue	■ Generation of shadow volumes per frame / • Works well for scenes with few moving objects / • Expensive for complex shadow volumes / • Improvement by simplifying shadow-casting geometries / ■ Four rendering passes required
Professor-style explanation	The slide 'Stencil Shadow Volume' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: ■ Generation of shadow volumes per frame / • Works well for scenes with few moving objects / • Expensive for complex shadow volumes / • Improvement by simplifying shadow-casting geometries / ■ Four rendering passes required. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.
Technical commentary	This slide is about Stencil Shadow Volume. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: ■ Generation of shadow volumes per frame / • Works well for scenes with few moving objects / • Expensive for complex shadow volumes / • Improvement by simplifying shadow-casting geometries / ■ Four rendering passes required
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Stencil Shadow Volume'?

Page 38	Implementation
Source cue	■ Determine visible scene objects / • Render scene without writing in color buffer / ■ Represent shadow volume in stencil buffer / • Initialize stencil buffer with 0 / • Render front-face shadow polygons and increment stencil buffer by 1
Professor-style explanation	The slide 'Implementation' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Determine visible scene objects / • Render scene without writing in color buffer / ■ Represent shadow volume in stencil buffer / • Initialize stencil buffer with 0 / • Render front-face shadow polygons and increment stencil buffer by 1. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about Implementation. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Determine visible scene objects / • Render scene without writing in color buffer / ■ Represent shadow volume in stencil buffer / • Initialize stencil buffer with 0 / • Render front-face shadow polygons and increment stencil buffer by 1
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Implementation'?

Page 39	Special Case: COP in the shadows
Source cue	■ When COP lies in a shadow volume, the situation can be unclear / • Near - or far -plane can intersect shadow volume / ■ Possible solutions / s = 0 / • Test COP against all shadow volumes cnt
Professor-style explanation	The slide 'Special Case: COP in the shadows' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: ■ When COP lies in a shadow volume, the situation can be unclear / • Near - or far -plane can intersect shadow volume / ■ Possible solutions / s = 0 / • Test COP against all shadow volumes cnt. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.
Technical commentary	This slide is about Special Case: COP in the shadows. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: ■ When COP lies in a shadow volume, the situation can be unclear / • Near - or far -plane can intersect shadow volume / ■ Possible solutions / s = 0 / • Test COP against all shadow volumes cnt
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Special Case: COP in the shadows'?

Page 40	Shadow Volume Example
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.

Professor-style explanation	The slide 'Shadow Volume Example' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Shadow Volume Example. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Shadow Volume Example'?

Page 41	Shadow Volume Conclusions
Source cue	Advantages / • Shadow-casting and shadow-receiving objects can have arbitrary shapes / • Applicable for multiple light sources / • Level-of-detail adaptable through approximated geometries / • Shadow volumes for static pairs of objects and lights can be precalculated
Professor-style explanation	The slide 'Shadow Volume Conclusions' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: Advantages / • Shadow-casting and shadow-receiving objects can have arbitrary shapes / • Applicable for multiple light sources / • Level-of-detail adaptable through approximated geometries / • Shadow volumes for static pairs of objects and lights can be precalculated. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Shadow Volume Conclusions. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: Advantages / • Shadow-casting and shadow-receiving objects can have arbitrary shapes / • Applicable for multiple light sources / • Level-of-detail adaptable through approximated geometries / • Shadow volumes for static pairs of objects and lights can be precalculated
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Shadow Volume Conclusions'?

Page 42	10.5 Shadow Maps
---------	------------------

Source cue	Depth test from the point of view of the light source
Professor-style explanation	The slide '10.5 Shadow Maps' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: Depth test from the point of view of the light source. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about 10.5 Shadow Maps. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: Depth test from the point of view of the light source
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '10.5 Shadow Maps' works per object, per image region, per ray, or per fragment?

Page 43	Shadow Mapping
Source cue	■ Based on z-Buffer (contains depth relative to the light source) / ■ First used within Pixar's Renderman
Professor-style explanation	The slide 'Shadow Mapping' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: ■ Based on z-Buffer (contains depth relative to the light source) / ■ First used within Pixar's Renderman. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Shadow Mapping. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: ■ Based on z-Buffer (contains depth relative to the light source) / ■ First used within Pixar's Renderman
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Shadow Mapping'?

Page 44	Projected Shadows
---------	-------------------

Source cue	■ depth texture / ■ Rendering algorithm / • Render shadow-casting objects as seen from the light source into depth texture / • Project depth texture using projective texturing when rendering shadow-receiving objects / ■ Characteristics of the process
Professor-style explanation	The slide 'Projected Shadows' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ depth texture / ■ Rendering algorithm / • Render shadow-casting objects as seen from the light source into depth texture / • Project depth texture using projective texturing when rendering shadow-receiving objects / ■ Characteristics of the process. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Projected Shadows. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ depth texture / ■ Rendering algorithm / • Render shadow-casting objects as seen from the light source into depth texture / • Project depth texture using projective texturing when rendering shadow-receiving objects / ■ Characteristics of the process
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Projected Shadows' works per object, per image region, per ray, or per fragment?

Page 45	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 46	Depth Comparison
Source cue	■ For each vertex in world coordinates determine associated depth value in shadow map • Transform into camera's eye space • Use light source's 4x4 transformation matrix to transform into light's clip space • Compare depth value with associated depth value in shadow map
Professor-style explanation	The slide 'Depth Comparison' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: ■ For each vertex in world coordinates determine associated depth value in shadow map • Transform into camera's eye space • Use light source's 4x4 transformation matrix to transform into light's clip space • Compare depth value with associated depth value in shadow map. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Depth Comparison. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: ■ For each vertex in world coordinates determine associated depth value in shadow map • Transform into camera's eye space • Use light source's 4x4 transformation matrix to transform into light's clip space • Compare depth value with associated depth value in shadow map
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Depth Comparison'?
Page 47	Coordinate Systems
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Coordinate Systems' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.

Technical commentary	This slide is about Coordinate Systems. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Coordinate Systems'?

Page 48	Pseudo Code
Source cue	// generate shadow map w.r.t light source / generateShadowMap(); // render scene from COP / for all rasterized fragments do { / Transform fragment xyz into lights coordinate system
Professor-style explanation	The slide 'Pseudo Code' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: // generate shadow map w.r.t light source / generateShadowMap(); // render scene from COP / for all rasterized fragments do { / Transform fragment xyz into lights coordinate system. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Pseudo Code. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: // generate shadow map w.r.t light source / generateShadowMap(); // render scene from COP / for all rasterized fragments do { / Transform fragment xyz into lights coordinate system
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Pseudo Code'?

Page 49	Properties
Source cue	■ Shadow maps are static when scene and lighting are static / ■ Shadow quality depends on shadow map resolution / ■ Shadow mapping has three main problems / • View volume problem / • Bias problem

Professor-style explanation	The slide 'Properties' explains how a visible surface point receives color from local lighting. The surface normal gives the orientation of the surface. The light vector gives the direction from which illumination arrives. The view vector connects the surface point to the observer, and the material parameters decide how strongly the surface reacts with ambient, diffuse, or specular reflection. On the slide, the concrete items are: ■ Shadow maps are static when scene and lighting are static / ■ Shadow quality depends on shadow map resolution / ■ Shadow mapping has three main problems / • View volume problem / • Bias problem. The object-level relation is light source, surface point, normal, material response, and computed color. The visual behavior changes when any of these objects changes: a rotated normal changes diffuse brightness, a different view vector moves the specular highlight, and different material coefficients change the perceived surface type.
Technical commentary	This slide is about Properties. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: ■ Shadow maps are static when scene and lighting are static / ■ Shadow quality depends on shadow map resolution / ■ Shadow mapping has three main problems / • View volume problem / • Bias problem
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Properties'?

Page 50	View Volume Problem
Source cue	■ When vertex lies outside light source's view volume, required depth information is missing / ■ Possible remedies / • Use cubic shadow map (similar to cubic environment map) / • Use only spotlights
Professor-style explanation	The slide 'View Volume Problem' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ When vertex lies outside light source's view volume, required depth information is missing / ■ Possible remedies / • Use cubic shadow map (similar to cubic environment map) / • Use only spotlights. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about View Volume Problem. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ When vertex lies outside light source's view volume, required depth information is missing / ■ Possible remedies / • Use cubic shadow map (similar to cubic environment map) / • Use only spotlights

Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'View Volume Problem' works per object, per image region, per ray, or per fragment?
Page 51	Bias Problem 1/2
Source cue	Incorrect self-shadowing occurs for / all fragments with $\text{ShadowMap}(x',y') \approx z'$ / Can be corrected by / increasing depth precision of z-buffer / applying z-bias
Professor-style explanation	The slide 'Bias Problem 1/2' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. On the slide, the concrete items are: Incorrect self-shadowing occurs for / all fragments with $\text{ShadowMap}(x',y') \approx z'$ / Can be corrected by / increasing depth precision of z-buffer / applying z-bias. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about Bias Problem 1/2. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Incorrect self-shadowing occurs for / all fragments with $\text{ShadowMap}(x',y') \approx z'$ / Can be corrected by / increasing depth precision of z-buffer / applying z-bias
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Bias Problem 1/2'?
Page 52	Bias Problem 2/2
Source cue	It is difficult to choose a good z-bias / $\text{ShadowMap}(x',y') + \text{bias}$? / ■ good / ■ too low / ■ too high
Professor-style explanation	The slide 'Bias Problem 2/2' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: It is difficult to choose a good z-bias / $\text{ShadowMap}(x',y') + \text{bias}$? / ■ good / ■ too low / ■ too high. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.

Technical commentary	This slide is about Bias Problem 2/2. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: It is difficult to choose a good z-bias / $\text{ShadowMap}(x',y') + \text{bias} ?$ / ■ good / ■ too low / ■ too high
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in 'Bias Problem 2/2'?
Page 53	Aliasing Problem 1/3
Source cue	■ Aliasing occurs when shadow map is undersampled(i.e., a texel in shadow map covers several / pixels) / ■ Particularly bad when camera and light source are facing each other
Professor-style explanation	The slide 'Aliasing Problem 1/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. On the slide, the concrete items are: ■ Aliasing occurs when shadow map is undersampled(i.e., a texel in shadow map covers several / pixels) / ■ Particularly bad when camera and light source are facing each other. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Aliasing Problem 1/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. Concrete items shown: ■ Aliasing occurs when shadow map is undersampled(i.e., a texel in shadow map covers several / pixels) / ■ Particularly bad when camera and light source are facing each other
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Aliasing Problem 1/3' changes positions before rasterization?
Page 54	Aliasing Problem 2/3
Source cue	■ Filter (= weighted average formation over neighbors) of a depth value does not make sense / ■ Percentage Closest Filtering (PCF) / • Filter results of shadow tests (weighted average of comparison results) / • Bias selection is made even more difficult / ■ Choosing the filter kernel

Professor-style explanation	The slide 'Aliasing Problem 2/3' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: ■ Filter (= weighted average formation over neighbors) of a depth value does not make sense / ■ Percentage Closest Filtering (PCF) / • Filter results of shadow tests (weighted average of comparison results) / • Bias selection is made even more difficult / ■ Choosing the filter kernel. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Aliasing Problem 2/3. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: ■ Filter (= weighted average formation over neighbors) of a depth value does not make sense / ■ Percentage Closest Filtering (PCF) / • Filter results of shadow tests (weighted average of comparison results) / • Bias selection is made even more difficult / ■ Choosing the filter kernel
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Aliasing Problem 2/3' works per object, per image region, per ray, or per fragment?
Page 55	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 56	Aliasing Problem 3/3

Source cue	■ Averaging Filter / ■ Percentage Closest Filtering
Professor-style explanation	The slide 'Aliasing Problem 3/3' explains textures as sampled data fields used during shading. A texture is not only a picture; it is a structured array of texels that can store color, normals, depth, material values, environment data, or volume data. A fragment supplies texture coordinates, the texture object supplies stored samples, sampler state controls wrapping and filtering, and the shader decides what the fetched value means. On the slide, the concrete items are: ■ Averaging Filter / ■ Percentage Closest Filtering. The object-level relation is coordinate, texture memory, sampling rule, and shader interpretation. This relation explains why texture bugs often look visual but originate from data addressing, filtering state, mipmap completeness, or a mismatch between stored values and shader expectations.
Technical commentary	This slide is about Aliasing Problem 3/3. The texturing objects on the slide define sampled data, coordinates, filtering rules, mip levels, and shader interpretation of fetched values. Concrete items shown: ■ Averaging Filter / ■ Percentage Closest Filtering
Why it matters	Texture sampling is a major source of visual detail and a common source of artifacts.
Check yourself	Can you identify the sampled data, coordinate, and filtering/state issue in 'Aliasing Problem 3/3'?
Page 57	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 58	■ Shadows in Visual Perception
Source cue	• Enhance depth perception and object relationships / • Create appealing visual effects and atmosphere / ■ Shadow Definitions and Types / • Shadow-casting (Occluder) vs. shadow-receiving (Receiver) objects / • Self-shadowing: Same object casts and receives shadow

Professor-style explanation	The slide '■ Shadows in Visual Perception' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. On the slide, the concrete items are: • Enhance depth perception and object relationships / • Create appealing visual effects and atmosphere / ■ Shadow Definitions and Types / • Shadow-casting (Occluder) vs. shadow-receiving (Receiver) objects / • Self-shadowing: Same object casts and receives shadow. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about ■ Shadows in Visual Perception. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. Concrete items shown: • Enhance depth perception and object relationships / • Create appealing visual effects and atmosphere / ■ Shadow Definitions and Types / • Shadow-casting (Occluder) vs. shadow-receiving (Receiver) objects / • Self-shadowing: Same object casts and receives shadow
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '■ Shadows in Visual Perception' works per object, per image region, per ray, or per fragment?

Page 59	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?

Page 60	■ Text Books
---------	--------------

Source cue	• P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Research Publications / • Crow, F., Shadows Algorithms for Computers Graphics, SIGGRAPH 1977. / • Lance Williams: Casting curved shadows on curved surfaces, SIGGRAPH 1978.
Professor-style explanation	The slide '■ Text Books' explains a shadow as a visibility relation from the light source. A surface point is lit when the light can reach it directly, and it is shadowed when another object blocks the path from the light to that point. The central objects are therefore the light, the occluder, the receiver, and some representation of light-space visibility. Depending on the method, that representation can be projected geometry, a precomputed light map, a shadow volume, or a shadow map storing depth from the light. On the slide, the concrete items are: • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Research Publications / • Crow, F., Shadows Algorithms for Computers Graphics, SIGGRAPH 1977. / • Lance Williams: Casting curved shadows on curved surfaces, SIGGRAPH 1978. The object-level relation is light, blocker, receiver, stored visibility information, and the darkened region visible in the final camera image.
Technical commentary	This slide is about ■ Text Books. The shadow objects on the slide represent visibility from the light source: occluders block light, receivers show the result, and the algorithm stores or computes that relation. Concrete items shown: • P. Shirley, M. AshikhminS. Marschner: Fundamentals of Computer Graphics (3rd ed. Edition), AK / ■ Research Publications / • Crow, F., Shadows Algorithms for Computers Graphics, SIGGRAPH 1977. / • Lance Williams: Casting curved shadows on curved surfaces, SIGGRAPH 1978.
Why it matters	Shadow algorithms reuse visibility ideas, but from the light's point of view.
Check yourself	Can you explain what the light can or cannot see in '■ Text Books'?

10.1 Definitions

Source location: Section 10.1

Commentary

A shadow is not simply a dark texture. It is evidence that light visibility is blocked. Important distinctions include hard versus soft shadows, umbra versus penumbra, and geometric versus precomputed approaches.

For real-time graphics, shadows are usually approximated. The approximation must answer a visibility question cheaply enough for interactive rendering.

Mental model

Shadows are light-space visibility made visible in the camera image.

Check yourself

Why is shadow computation related to visibility determination?

10.2 Ground Plane Shadows

Source location: Section 10.2

Commentary

Ground plane shadows project an object onto a receiving plane. This is simple and can look convincing in restricted scenes, but it assumes a known planar receiver and does not handle general shadowing between arbitrary objects.

The method is useful pedagogically because it makes the geometric nature of shadows explicit: a light source, an occluder, and a receiver define where the shadow lands.

Mental model

A planar shadow is projected geometry on a known receiver.

Check yourself

What scene limitation makes ground plane shadows less general than shadow maps?

10.3 Light Maps

Source location: Section 10.3

Commentary

Light maps store precomputed lighting or shadow information in textures. They can be efficient at runtime, but they are limited when lights, objects, or geometry move dynamically.

This is another example of a recurring graphics tradeoff: precompute for speed, but lose flexibility. Real-time rendering often mixes precomputed and dynamic techniques.

Mental model

A light map spends storage and preprocessing to save runtime shading work.

Check yourself

Why are light maps less suitable for fully dynamic moving lights?

10.4 Shadow Volumes

Source location: Section 10.4

Commentary

Shadow volumes construct the volume of space blocked from a light by an occluder. The camera view can then determine whether visible points lie inside that volume. Stencil buffer techniques are often associated with this method.

The strength is geometric precision for hard shadows. The cost is handling silhouette edges, volume construction, and robust stencil operations. It is a good contrast to shadow maps, which are image-based from the light view.

Mental model

A shadow volume is the 3D region where the light cannot reach.

Check yourself

Why are silhouette edges important for constructing shadow volumes?

10.5 Shadow Maps

Source location: Section 10.5

Commentary

Shadow mapping renders the scene from the light's point of view and stores depth. During the camera pass, a surface point is transformed into light space and compared against the stored depth. If it is farther than what the light saw, it is in shadow.

This method is practical and widely used, but it has artifacts. Shadow acne comes from precision/self-comparison issues. Bias can reduce acne but too much bias causes detached shadows. Resolution, filtering, and light projection strongly affect quality.

Mental model

Shadow maps reuse the depth-buffer idea, but from the light's camera.

Check yourself

What exactly is stored in a shadow map?

End-of-lecture summary

If you remember only one thing from Lecture 10, remember this: This lecture explains shadows as a visibility problem from the light source. A point is lit if the light can see it; it is shadowed if another object blocks that visibility.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.